

Computing NEA: BASLIX

TERMINAL

Name: Daniel Dennis

Centre Number: [REDACTED]

Candidate ID: [REDACTED]



Table of Contents

Computing NEA: BASLIX TERMINAL	1
Analysis	3
Background information	3
Sponsor Interview	4
Research.....	6
Initial Modelling.....	18
Objectives.....	22
Documented Design	23
Overview	23
Hierarchy Chart.....	24
System Flowcharts.....	25
Class Diagrams	35
Data Flow Diagram.....	39
Key Modules.....	41
Key Algorithms.....	52
File Structures	74
Networking & APIs	75
Human Computer Interface.....	76
Hardware Selection	77
Library Usage	78
Technical Solution	81
Technical Overview	81
Training Resources	82
Prediction Tools	109
User Interface.....	144
Testing.....	225
Functionality	225
Evaluation	273
Objective Review	273
Sponsor Feedback	278
Potential Improvements.....	279
Conclusion.....	280
References	281

Analysis

Background information

Financial markets, including cryptocurrencies and commodities, play a major role in the global economy. Traditionally, trading in these markets was reliant on human decision making. However, over the recent decades, this role has increasingly shifted towards automated systems, ranging from simple rule-based algorithmic trading bots to more advanced artificial intelligence systems capable of analysing data on a much larger scale than what is feasible to human traders.

Predicting asset price movements is inherently complex. Financial markets are influenced by many factors, many of which are noisy, non-linear, and difficult to quantitatively analyse. As a result, large financial institutions employ teams of quantitative researchers to design strategies via statistical modelling and machine learning techniques. On the other hand, retail traders typically rely on methods such as technical indicators and qualitative analysis of news surrounding specific assets.

In recent years, the application of artificial intelligence has increased within financial markets, including the use of natural language processing (NLP) to extract sentiment from economic news and financial reports. Such models include FinBERT, which demonstrates that sentiment extracted from financial text can provide useful contextual information beyond historical price data alone. Despite this, many advanced techniques remain challenging to apply effectively, especially in dynamic and noisy environments such as those found in financial markets.

Project Identification

This project is an investigation into the application of machine learning techniques for algorithmic trading, with a particular focus on combining sentiment analysis, gradient-boosted time-series regression, and reinforcement learning within a unified research pipeline.

The unified research system is **BASLIX Terminal** (Bot Assisted System for Learning Investment eXecutions). This acronym, while not being perfectly accurate, was chosen for its distinctiveness and appeal.

Gold will be the chosen asset for initial development due to its sensitivity to macroeconomic sentiment, as well as the availability of relevant financial news. It is also less volatile, making it potentially easier to forecast. These characteristics make it suitable for experimentation and analysis of system behaviour.

The research system is designed to be modular and capable of using more assets than just gold. This allows models to be trained and evaluated across multiple financial markets, supporting comparative analysis of model performance in environments with differing volatility and structural characteristics, ranging from commodities to cryptocurrencies.

Through the construction and evaluation of this system, this project aims to investigate the practical challenges, limitations, and benefits of applying advanced machine learning techniques to financial time-series data under simulated trading environments, while maintaining a design that supports real-time inference without executing live trades.

Sponsor Interview

Sponsor: [REDACTED], PhD, Senior Clinical Scientist and researcher in applied machine learning and AI

Q. When applying reinforcement learning to real-world systems such as financial markets, what are the most common errors in designing the environment that undermine validity?

A. Common errors typically include lookahead bias or information leakage. There can also be unrealistic assumptions made about the environment, which in this case might include things like having zero transaction costs or infinite liquidity. A poorly defined state or action space which embeds future information can cause leakage, which invalidates the results. It is also important to avoid making the reward function proxy profit directly – it is much better reward specific behaviours that you want from the agent.

Q. How should the objectives of an RL based system be constrained to ensure that the investigation remains meaningful rather than overfitted to historical data?

A. Simple, risk-adjusted objectives should be used, for example drawdown-aware returns. Realistic constraints should also be put into place, which can be turnover, leverage, and transaction costs. When testing the model, it should be done across different regimes and out of sample.

Q. Financial time series data is non-stationary. From your experience, how does this affect both supervised models and reinforcement learning agents, and what mitigations are realistic at a small research scale?

A. Both supervised and RL models degrade as data distributions shift. RL agents may also over-adapt to obsolete dynamics, reducing its ability to generalise. At a small research scale practical mitigations can include rolling windows, frequent retraining, and use of simpler models.

Q. What indicators suggest that a model is learning genuine structure rather than exploiting patterns in the dataset?

A. Feature importance and policy behaviour should be tracked for stability. It is also a good sign when behaviours are learnt which line up with known domain mechanics. For an RL agent, loss should converge over time.

Q. In complex pipelines combining a range of features, where do hidden sources of leakage or bias most often occur, and how can this be mitigated?

A. Feature engineering which uses future aggregates is a common source of leakage. Improper temporal data splits can also cause make results invalid. Target labels must also be hidden from supervised models.

Q. Is there a point at which increasing system complexity actively reduces reliability, and how can this be identified experimentally?

A. Yes, this is the case when marginal performance gains vanish, but variance increases. If models are too sensitive to hyperparameters or random seeds, this is also a sign of excessive complexity.

Ablation studies should be conducted to see how performance varies when certain features are removed.

Q. What are the major stability challenges in reinforcement learning when training on historical financial data, and how should results be interpreted cautiously?

A. Signals can have high variance in the financial sector, making learning quite difficult. Policies can collapse and models can be overconfident in their predictions. Results should be treated as scenario-conditional, not necessarily deployable right away.

Q. When performance varies significantly across test periods, how should this be interpreted in an academic investigation?

A. This would indicate regime dependence, and not necessarily failure. This should be analysed specifically, not averaged away.

Q. Why can the evaluation of sequential decision models be more challenging than standard predictive models?

A. In sequential decision models' actions change future data distributions. Errors can compound over time, and there is no clear ground truth for the agent. Standard metrics can fail to capture long-term effects.

In this interview, many possible challenges were discussed which may be faced building this system. Lookahead bias, information leakage, and unrealistic assumptions should be avoided, and for the RL system specifically, the reward function should encourage desired behaviours rather than directly targeting profits. Models should account for the non-stationary nature of financial data by using rolling windows, frequent retraining, and being kept simple to allow for generalisation. Results should be cautiously interpreted, accounting for regime dependence if there is large variance in performance.

Research

To support the development of this system, research was conducted into the individual components of the system, including relevant machine learning techniques and data processing methods. Methods applicable via Python or C# were selected for primary research due to their suitability to implementing the required models and my existing proficiency with them.

In addition, existing solutions in algorithmic trading and financial modelling were reviewed to inform the design and integration of different components, providing useful insight into the best practices and limitations of such systems.

Existing Solutions

Bloomberg Terminal:

Bloomberg terminal is a professional platform used by institutions, providing access to real-time market data, and a vast array of analytical tools. Modern terminals make use of machine learning techniques to provide actionable insights for users. While the platform is highly sophisticated and expensive for small scale research teams (up to \$32,000), it demonstrates the industry standard for advanced systems that can integrate multiple data sources to inform trading decisions.

(Investopedia) (Bloomberg, 2023)

[IMAGE REDACTED FOR COPYRIGHT REASONS]

Strengths	Weaknesses
High quality data	Limited customisation
Industry grade tools	Expensive
Real-time data	High complexity

FinRL:

FinRL is an open-source framework for financial reinforcement learning. It trains an RL Agent in a virtual environment and has included backtesting environments for users to observe their strategies.¹ However, it does not use sentiment data, which is useful for capturing real-world events that have high relevance to commodities such as gold.

Strengths	Weaknesses
Open source	Lack of sentiment data
Active community	Lack of custom feature engineering
Backtesting support	No GUI

QuantConnect:

QuantConnect is an open-source, cloud-based platform for algorithmic trading and quantitative research. It provides an environment for research, backtesting, and live deployment of trading strategies using the open-source LEAN engine. The platform supports multiple asset classes and integrates with major brokers, with extensive historical and alternative data available for testing realistic trading conditions. Its target audience is developers and quantitative researchers rather than non-technical users, similarly to BASLIX Terminal.

Strengths	Weaknesses
Robust backtests with realistic market modelling	Steep learning curve
Multi-asset support and broker integration	Less intuitive GUI
Open-source engine with an active community	Advanced features can be costly

System Components Research

Research was conducted into the implementation of individual system components to ensure the pipeline operates effectively and efficiently. This included the investigation of suitable machine learning techniques, data preprocessing methods, and reinforcement learning frameworks.

Web Scraping:

While many APIs exist for obtaining the required data, they typically require payment to retrieve the quantities of data that are necessary for training regression models and RL Agents. Therefore, web scraping is a free alternative that can provide me with said information.

Python and C# are both languages that can be used effectively for web scraping. This will be especially useful as substantial amounts of data must be pulled to train any AI system.

¹ <https://arxiv.org/abs/2011.09607>

C# is the faster language and has tight integration with Microsoft tools. However, python is considered much easier to use and has many libraries that can make scraping easier. Parsing code is also very straightforward. To account for speed issues, *asyncio* can be used to increase performance.²

(Sibisi, 2023)

Rate limits and TOS of certain websites must also be taken into consideration. Some news sources may not allow scraping or block my requests if data is pulled too fast.

Due to its simplicity and overall faster development time, the scraper will be built in python.

The methodology used for web scraping can vary depending on the structure of the target website and the intended use of the data. One common approach involves rendering web pages in a browser-like environment and parsing the resulting HTML to extract relevant information. While this method closely reflects how data is presented to human users, it is computationally expensive and can place unnecessary load on the host server.

In practice, many modern websites retrieve their data dynamically using structured network endpoints that return information in formats such as JSON and XML. These endpoints are used by the website itself to display content to users and are accessible without authentication.

Compared to browser-based scraping, directly consuming structured data responses is significantly more efficient and reliable, as it avoids the overhead of rendering web pages and reduces the complexity of parsing unstructured HTML. This methodology is therefore more suitable for large-scale data collection and time-series analysis, particularly where historical data over long periods is required.

Sentiment Analysis:

Sentiment analysis involves categorically labelling written text into categories of varying sentiments. To save time for later systems, a pre-existing system will be used, as it is unnecessary to build a sentiment analysis model from scratch when development will take a significant amount of time, and thoroughly tested solutions already exist.

FinBERT is used quite often for the labelling of financial news. The original *FinBERT* was pre-trained on around 4.9 billion words of company reports and analyst calls.³ It has shown significant performance over generic *BERT* for financial sentiment, making it particularly suitable at this task. The outputs of *FinBERT* are probabilities of three distinct classes: positive, negative, or neutral. To ensure articles are categorised appropriately, other models such as *FinRoBERTa* will also be taken into consideration.

(Yi Yang, 2020)

Leveraging a pre-trained sentiment analysis model will be useful for quickly obtaining sentiment scores to run through a regression model with other indicators.

² <https://brightdata.com/blog/how-tos/c-sharp-vs-python-for-web-scraping>

³ <https://ar5iv.labs.arxiv.org/html/2006.08097>

Sentiment analysis models start by tokenising their input, which is to break text into smaller units that can be represented with vectors. During the training process, the model learns patterns in how tokens appear together in context, which gradually organises the vectors so that words used in similar contexts have similar representations. Once the model has learned these contextual relationships, it can use them to interpret the overall meaning of a sentence. Finally, a classification layer maps the contextualized token representations to sentiment categories, allowing the model to label sentences as positive, negative, or neutral based on patterns it observed during training.

Continuous values will be more useful for data analysis rather than discrete labels due to its granularity. For this reason, label probabilities will be used to generate a continuous score from -1 to 1. Continuous scores from labels can be achieved with a simple function:

$$\textit{Sentiment Score} = P(\textit{positive}) - P(\textit{negative})$$

Where P is the output probabilities from the sentiment analysis. It can also be fine-tuned, however, to see meaningful results, a large dataset is needed.

Other sentiment tools other than *FinBERT* include options such as *VADER* and *GPT-4*. While GPT-4 is an attractive tool for this, there is typically a limit on the amount of data you can run through it for free, and it is not open source. *FinBERT*, or some other readily available model (such as those found on *hugging-face*) are the most appropriate tools for this purpose.

If the model appears to be too overconfident in predictions, a different formula can be employed, to dampen based on neutrality:

$$\textit{Sentiment Score} = (P(\textit{positive}) - P(\textit{negative})) * (1 - \textit{neutral})$$

This adds an appropriate level of damping if the headline does not have an obvious sentiment. If sentiments appear to be skewed, a custom function can be designed to map model outputs to more appropriate scores.

Machine Learning in Price Forecasts:

To ensure the RL Agent has adequate information to work with, the sentiment data should be run through a regression model along with *technical indicators* to predict a price movement along with uncertainty. Python has a vast array of tools which are incredibly useful for making regression models, such as *scikit-learn*, *numpy*, *transformers*, etc. While C# was considered for its tools in *ML.NET*, python will be easier to integrate with the sentiment analysis system and overall has better tools for making regression models.

The regression model should ideally take technical indicators and sentiment as inputs and output a short-term gold price movement.

Within python, there are many tools which can be used. A common library used is *sklearn* for simple regression models.⁴ However, it does not come with uncertainty estimates by default for most basic models.

([scikit documentation, 2025](#))

Gradient-Boosted Decision Trees:

Gradient-Boosted Decision Tree (GBDT) models such as *XGBoost*⁵ and *LightGBM* were investigated, as they are known to perform well in financial machine learning tasks due to their handling of non-linearities and feature interactions. *XGBoost* does not output uncertainty as it is, however. On the other hand, *LightGBM* natively supports *quantile regression*⁶, making it useful for getting uncertainty estimates – an invaluable tool for future predictions which will be passed on to the RL Agent.

Functionally, *GBDTs* are an *ensemble learning* method that combines multiple *regression trees* to improve existing predictions. The process begins with an initial prediction, $F_0(x)$ where the function typically represents some kind of constant derived from the data, such as the mean of the target values. At each iteration m , the model calculates the *residuals*, which represents the difference between the actual target values y_i and the current predictions $F_{m-1}(x_i)$. In gradient boosting, these residuals are the negative gradients (pushing predictions closer to actuals) for a loss function $L(y, F(x))$, represented by the following formula:

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F(x)=F_{m-1}(x)}$$

A *regression tree* $h_m(x)$ is then trained to predict these *residuals*. Each tree consists of *decision nodes* that split the data based on feature values, using rules similar to “if” statements. In standard regression, each leaf of the tree predicts a single value, such as the mean of the residuals in that region. *Quantile regression*, as is used in *LightGBM*, allows for each leaf to predict a specific quantile of the target distribution, rather than the mean. For example, the 50th percentile (median) predicts a value where half of the observations fall below it, while the 90th percentile predicts a value such that 90% of observations are below it. This enables the model to provide uncertainty estimates or predictive ranges, which are especially useful in applications where knowing best-case and worst-case scenarios is important. The depth of the tree determines how many splits (decisions) are made. This should be carefully controlled to avoid overfitting.

([Aswathi, 2023](#))

⁴ https://scikit-learn.org/stable/supervised_learning.html

⁵ <https://xgboost.readthedocs.io/en/stable/index.html>

⁶ <https://lightgbm.readthedocs.io/en/latest/>

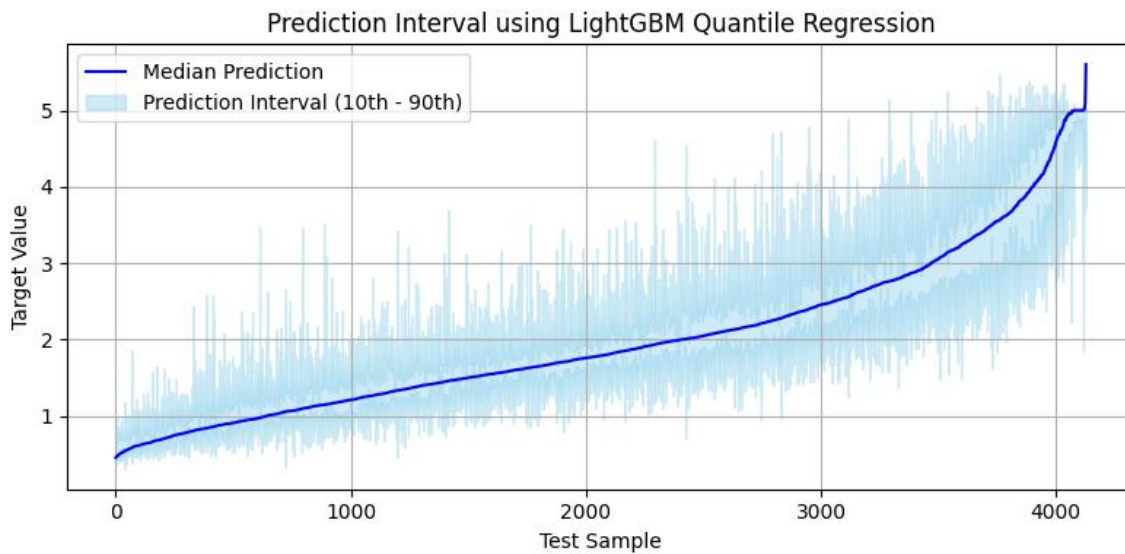


Image source: [GeeksForGeeks - LightGBM for Quantile Regression](#)

(XGBoost, 2022), (LightGBM, 2025)

Technical Indicators:

Feeding raw price data to a regression model is a bad idea. The data is noisy and does not give sufficient information to capture a trend. This is why technical indicators exist and can be calculated from the raw price data. Many such indicators exist, so it is essential to find the most relevant ones to reduce noise in the regression model.

Relative Strength Index (RSI) measures the speed and change of price movements, usually over 14 days. It outputs a value from 0 to 100 to identify overbought and oversold conditions. It is calculated with the following formulae:

$$RS = \text{Average Gain} / \text{Average Loss}$$

$$RSI = 100 - (100 / (1 + RS))$$

Exponential Moving Average (EMA) is a type of moving average that places greater weight on more recent data points. It is also used to calculate other indicators, such as *MACD*. It is expressed in the following formula:

$$EMA_{today} = (Value_{today} * (Smoothing / (1 + Days)) + EMA_{yesterday} * (1 - (Smoothing / (1 + Days)))$$

Moving Average Convergence Divergence (MACD) is used by investors to identify price trends and find entry points for buying and selling. It is best used with daily periods, which is what I will be trading on. It can be expressed with a simple formula:

$$MACD = 12\text{-Period EMA} - 26\text{-Period EMA}$$

Where the period is usually a day.

Bollinger Bands consist of three lines which are plotted onto a price chart. The middle band is the *Simple Moving Average*, usually calculated from the last 20 days. The upper/lower bands are calculated as follows, where the multiplier is usually 2:

$$Upper\ Band = Middle\ Band + (Standard\ Deviation * Multiplier)$$

$$Lower\ Band = Middle\ Band - (Standard\ Deviation * Multiplier)$$

Average True Range (ATR) is an indicator of volatility. It is especially useful for setting dynamic stop-losses, which the reinforcement agent might make use of. It is calculated with a series of formulae:

$$\frac{Previous\ ATR(n - 1) + TR}{n}$$

Where **n** is the number of periods, and **TR** is True Range

If a **Previous ATR** does not exist, you must use the following formula:

$$\left(\frac{1}{n}\right) \sum_i^n TR_i$$

Where **TR_i** is the true range of a particular day, and **n** is number of periods.

True Range is calculated with the following formula:

$$TR = Max[(H - L), |H - C_p|, |L - C_p|]$$

Where **H** is the day's high, **L** is the low and **C_p** is the closing price

A *Stochastic Oscillator* is a technical indicator used for momentum. It works by comparing a closing price to a range of its prices over time. This will be of use in generating overbought and oversold trading signals. It is calculated with the following formula:

$$\%K = \frac{C - L_n}{H_n - L_n} * 100$$

Where **C** is the most recent closing price, **L_n** is the lowest price over **n** periods, and **H_n** is the highest price over **n** periods.

The *Commodity Channel Index (CCI)* is an oscillator which is used to identify cyclical trends. Its usage has been on the rise with not just commodities, but also equities and currencies. The CCI was developed to determine overbought and oversold levels. This is done by measuring the relations between price and a moving average. It is calculated with the following formula:

$$CCI = \frac{\text{Typical Price} - \text{Simple Moving Average}}{0.015 * \text{Mean Deviation}}$$

Where typical price is (high + low + close) divided by 3. Values over 100 suggest overbought conditions, and below -100 suggest oversold.

The *Chande Momentum Oscillator* is a technical momentum indicator that identifies relative strength of weakness in a market. It is calculated with the following formula:

$$CMO = \frac{sH - sL}{sH + sL} * 100$$

Where **sH** is the number higher closes over **N** periods, and **sL** is the sum of lower closes over **N** periods.

The *Psychological Line* is a simple sentiment-based indicator, which measures the ratio of up periods to total periods.

$$PSY = \frac{n_{up}}{n_{tot}}$$

Source on technical indicators: [\(Investopedia\)](#)

Slopes are a simple rate of change measurement, and Divergences measure opposition between price action and a slope (which can be for an indicator). They were noted as important features in research on LightGBM for asset forecasting.

(Bisdoulis, 2024)

Reinforcement Learning:

Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make decisions by interacting with a simulated environment to maximise cumulative rewards. RL relies on trial and error, receiving feedback in the form of rewards and penalties. The core components include:

- *The Agent*
- *The Environment*
- *States*
- *Actions*
- *Rewards*
- *Policies*
- *Value Function (Q)*

Neural Networks are used in RL as function approximators. They help estimate the policy/value functions when dealing with large state spaces.

RL Algorithms include:

Deep Q-Network (DQN)

This makes use of a neural network to approximate the Q-value function (expected return of taking an action in a specific state). It applies the bellman equation to update Q-values:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Where α is the learning state, γ is the discount factor, r is the immediate reward, and s' is the next state. This encourages the agent to learn optimal long-term policies, which will be useful in getting the RL Agent to produce effective trading strategies.

Policy Gradient Methods

These optimise the policy directly by maximising the expected reward. This makes use of gradient ascent.

(Barto, 2018), (Schulman, 2017)

Soft Actor Critic (SAC)

This system combines elements of policy based and value-based methods. It is known to be effective at tasks involving continuous values. It consists of two neural networks: the actor and the critic. The actor decides what actions to take, while the critic evaluates these actions by estimating the expected return from taking a certain action, given the state.

*The SAC algorithm makes use of **entropy regularisation**, to balance the exploration of new actions with the exploitation of effective strategies. This adds a level of randomness, to prevent the agent from becoming focused on exploiting a suboptimal strategy. It makes use of the following formula:*

$$L(\pi) = E_{s \sim D} [\alpha \log \pi(a|s) - Q(s, a)]$$

Where α is the temperature parameter to control the balance between exploration and exploitation.

It also uses a Q-Value function (like DQN) to estimate the expected return from taking an action a in state s . This uses the soft Bellman equation:

$$Q(s, a) = r + \gamma E_{s'} [V(s')]$$

Where $V(s)$ is defined as:

$$V(s) = E_{a \sim \pi} [Q(s, a) - \alpha \log \pi(a|s)]$$

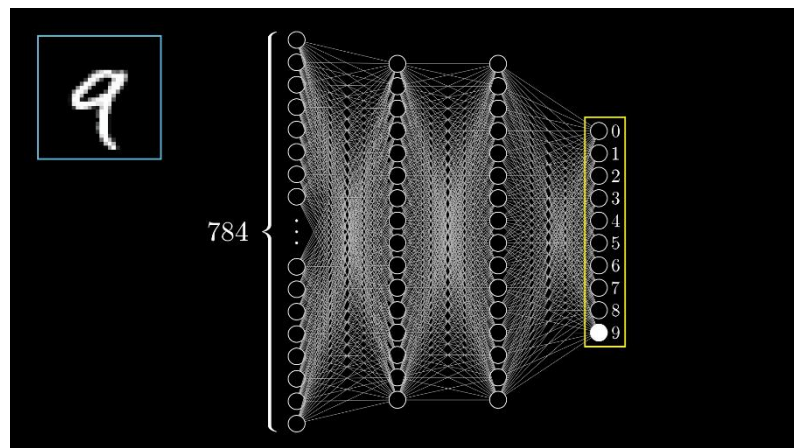
SAC is a more sample-efficient method and has much more stable learning due to the use of two Q-Value Networks to reduce overestimation bias. It also allows for exploration of more effective strategies due to entropy regularisation. However, it is also more complex than other methods, requiring careful tuning of hyperparameters. They also require a lot of computational power to train.

[\[Chen, 2024\]](#)

Again, for the sake of implementation and compatible tools, I have chosen python as the language for my reinforcement learning model. It has many reinforcement learning tools available, such as *Stable-Baselines3*, *TensorForce*, and *Ray/RLlib*. Another advantage of python is support for GPU acceleration as part of most deep learning frameworks, which will be incredibly useful considering how long it takes to train these models. I will be training the RL Agent locally on my laptop, utilising an 8GB RTX 4060 for accelerated training.

Neural Networks:

This is the brain of a reinforcement learning algorithm. It essentially acts as a function that maps a set of inputs to outputs. It consists of layers of *neurons*, each neuron computing a weighted sum of its inputs, adding a bias, and applying an *activation function*, which introduces nonlinearity, allowing for more complex problems to be solved (if it was linear, very basic policies would be learnt: if x, then y).

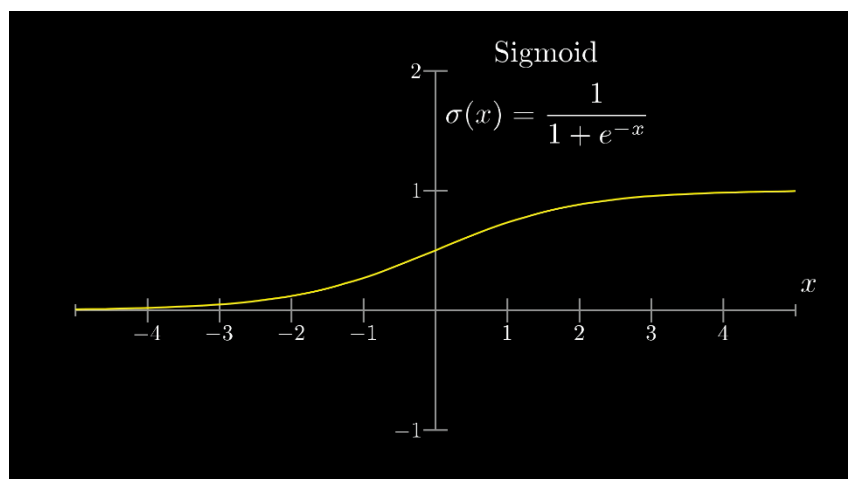


Mathematically, each neuron j in layer l (represented with a matrix) has its value calculated by the following formula:

$$z_j^{[l]} = \sum_i w_{ji}^{[l]} a_i^{[l-1]} + b_j^{[l]}$$

Where $z_j^{[l]}$ is the pre-activation value of neuron j in layer l , $w_{ji}^{[l]}$ is the weight of the connection from neuron i in the previous layer $[l-1]$ to neuron j in current layer $[l]$, $a_i^{[l-1]}$ is the activation of neuron i in the previous layer, and $b_j^{[l]}$ of the neuron in the current layer.

The pre-activation value is then passed through an **activation function**. Common activation functions include **sigmoid**, **tanh**, and **ReLU**.



Activations in one layer propagate through to the next layer. The layers between the input and output are known as hidden layers. While the developer defines the overall structure of the network, such as the number of layers and neurons, the weights and biases within these hidden layers are adjusted algorithmically during training. Each hidden layer learns patterns and relationships within the data, forming increasingly abstract representations as data passes through the network, before being mapped to an appropriate output layer. In the case of a DQN, the neural network is used to approximate the Q-Value function, predicting the expected future reward for each possible action. There will be a predicted Q-Value for all discrete actions, so the largest one is chosen as having the best future reward.

A neural network's goal is to make predictions $\mathbf{y}'$ as close as possible to the true label $\mathbf{y}$ as possible. To measure this, a loss function $L(\mathbf{y}, \mathbf{y}')$ is used. Common loss functions include the following:

Mean Squared Error

$$L(\mathbf{y}, \mathbf{y}') = \frac{1}{n} \sum_{i=1}^n (\mathbf{y}_i - \mathbf{y}'_i)^2$$

Cross-Entropy Loss

$$L(\mathbf{y}, \mathbf{y}') = - \sum_i \mathbf{y}_i \log(\mathbf{y}'_i)$$

Then, the network has its weights and biases updated to minimise loss. This is done using gradient descent:

$$\mathbf{w}_{ji}^{[l]} \leftarrow \mathbf{w}_{ji}^{[l]} - \eta \frac{\partial L}{\partial \mathbf{w}_{ji}^{[l]}}$$

$$\mathbf{b}_j^{[l]} \leftarrow \mathbf{b}_j^{[l]} - \eta \frac{\partial L}{\partial \mathbf{b}_j^{[l]}}$$

Where η is the learning rate.

More sophisticated variants of gradient descent exist. One example is *Adam* (*ADaptiveMoment estimation*). This might be more appropriate for this system, as it keeps a moving average of gradients $\mathbf{m}_t$ and gradients squared $\mathbf{v}_t$, which helps with noisy updates (likely to occur in a financial context). *AdamW* modifies *Adam* to include *decoupled weight decay*, which is better for regularisation. Instead of adding weight decay within the gradient term (as seen in Adam), it subtracts a fraction of the weights directly in each step:

Adam

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

AdamW

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \eta \lambda \theta_t$$

Given the noisy nature of training an RL Agent in financial markets, *AdamW* could prove to be beneficial for use as an optimiser. Its function is also built into most python libraries for neural networks.

Sources on neural networks and gradient descent: ([Sanderson, 2017](#)) ([Lunartech, 2025](#))

Initial Modelling

(Please note that all relevant formulae for system components are listed in the research section above)

There are **four** key systems which are necessary for this system's success, arranged in a pipeline based on their dependencies:



Data Retrieval

Retrieval of data will be necessary for all the systems to come, as they are all reliant on being provided with a large volume of accurate information.

This will either be done via a web scraper or API. The rest of the project is reliant on the data obtained, so this is the first system to be built. Historical prices are readily available via APIs such as *yfinance*, which is widely used in research for fetching OHLC data. News articles and sentiment typically will incur expenses to make use of an existing API. Web scraping is a viable alternative, though rate limits must be taken into consideration.

The scraper will be built in python with the aid of the *requests* library, as this is needed to send HTTP requests. The scraping methodology could entail either directly visiting the site and parsing HTML, which could require a library such as *BeautifulSoup* or *Selenium*, or directly accessing an API endpoint from the website in question.

After searching for a suitable website to extract data from, [REDACTED] was decided upon as the news source for natural language processing, as it has news written in a formal format that is understandable to sentiment analysis models (which are trained on similar data). Both sites are also freely accessible to the public. Rate limits should also be considered to avoid straining servers. Reverse engineering API endpoints on websites is also another possible approach, which would also be a lighter load on any servers accessed.

Sentiment Analysis

Sentiment Analysis will be a useful metric for the regression model to predict price movements.

FinRoBERTa, a transformer model fine-tuned on financial texts, is used for sentiment analysis due to its proven reliability in financial research and its training on formal market reports. Rather than using discrete sentiment labels, the model's output probabilities are converted into a continuous polarity score by taking the difference between positive and negative label probabilities. This score is further calibrated using temperature scaling to reduce overconfidence, and a custom nonlinear scaling function is applied to reshape sentiment magnitude and neutrality. Contextual biasing is introduced via a prefixed prompt to constrain sentiment interpretation to the desired market to analyse, producing a tuneable sentiment signal for downstream regression and reinforcement learning models.

Regression Model

The regression model should provide the Reinforcement Learning Agent with predictions for future price movements, allowing strategies to be more informed on market conditions and probable future prices.

For regression modelling, *LightGBM* will be used, which is known for its fast training speed and efficiency. As time-series data is also in use, combined with technical indicators and sentiment, *LightGBM* is a suitable tool for quickly making accurate models. This can be further accelerated with its GPU acceleration support, for which modern graphics cards such as the RTX 4060 can be used to train new models even faster. It is also good at capturing non-linear relationships, which is true of financial markets, which are often noisy. Furthermore, it has support for quantile regression, which allows the reinforcement learning agent to have an idea of the uncertainty in the predictions that the model makes. This is invaluable for risk management.

It will make use of the following technical indicators as inputs, calculated via the formulae listed in the research section (*see technical indicators*):

- *Relative Strength Index*
- *Moving Average Convergence Divergence*
- *Bollinger Bands*
- *Average True Range*
- *Stochastic Oscillator*
- *Commodity Channel Index*
- *Chande Momentum Oscillator*
- *Psychological Line*
- *Exponential Moving Averages*
- *Slopes*
- *Divergences*

Reinforcement Learning Model

This part of the system will be subject to change later, as it will require a lot of adjustments and fine-tuning, as well as testing of various strategies to find an appropriate approach.

The Reinforcement Learning Model will be the part that produces new trading strategies to maximise returns on gold using sentiment scores, regression predictions, and the previously mentioned technical indicators. It will run within a simulated trading environment, where it can make defined actions.

The *observation space* is information that the agent can see per timestep. It will consist of the following items:

- *Sentiment score, ranging -1 to 1*
- *Predicted price changes from LightGBM*
- *Technical Indicators*
- *A small window of raw prices*

The *action space* defines actions which the agent can take within the environment. It will consist of the following actions:

- *Buy*
- *Hold*
- *Close*

Reinforcement Learning also involves a *reward function*, which is the “reinforcement” part of the system. It is meant to “reward” the agent for doing well, depending on how the developer defines what “doing well” is. As per the sponsor’s advice, the reward will *not* be raw profit, and different behaviours will be experimented with to see what should be rewarded. The bellman equation will be used to determine q-values (as seen in the research section).

Actions are limited to once a day. This is to see long term profit and have accessible data for each time period (as news articles are usually updated daily). Methods like SAC may seem favourable due to their handling of continuous outputs for position sizing. However, this could cause problems when mapping continuous values to discrete actions, as minor changes in output can cause an entirely different outcome. For this reason, a *DQN* will be used, where the action space includes holding a position, closing a position, and several options for buying with varying sizes. These presets could also include dynamic changes such as scaling based on volatility. Training another RL Agent to optimise these would be needlessly complex.

User Interface

Once all other systems are functioning to a suitable level, a **user interface** will be implemented for ease of access. It will allow a user to quickly adjust parameters when training models, view backtest results, and potentially view actions the model would take on LIVE data.

Objectives

- 1. Create a machine learning research environment to investigate the use and effectiveness of Gradient-Boosted Decision Tree models and Deep Q-Networks for forecasting and decision making in financial markets**
- 2. Build infrastructure to dynamically pull required training data**
 - 2.1. Dynamically construct web API request URLs parametrically
 - 2.2. Detect and handle HTTP error conditions
 - 2.3. Filter articles on relevance
 - 2.4. Control frequency to respect rate limits
 - 2.5. Retrieve historical OHLC market data via an external financial API
- 3. Adapt and calibrate a pre-trained sentiment model for financial news analysis**
 - 3.1. Integrate a pre-trained transformer model for financial sentiment classification
 - 3.2. Convert discrete label probabilities into a continuous polarity score
 - 3.3. Apply contextual biasing to constrain sentiment interpretation
 - 3.4. Calibrate confidence via temperature scaling
 - 3.5. Apply a nonlinear scaling function to reshape sentiment magnitude and neutrality
 - 3.6. Expose configurable parameters for systematic experimentation
- 4. Process OHLC data into technical features for time-series data**
 - 4.1. Define parameters to calculate technical features
 - 4.2. Compute features to enhance the training data, including technical indicators
- 5. Dynamically construct a unified dataset for training RL and regression models**
 - 5.1. Include mean sentiment from individual days to produce a single score per day
 - 5.2. Integrate sentiment data, OHLC, and technical features into a single dataset
 - 5.3. Generate target labels for supervised learning
- 6. Implement a quantile LightGBM forecaster for time series prediction**
 - 6.1. Configure and expose LightGBM parameters
 - 6.2. Include the option to use conformal predictions
 - 6.3. Predict the target variable for different quantiles
- 7. Develop a 1D Convolutional Deep Q-Network with LightGBM informed features**
 - 7.1. Allow for customisation of observation space, environment, rewards, and actions
 - 7.2. Use a custom trading environment to track the agent's performance
 - 7.3. Calculate performance metrics such as returns and Sharpe ratio during and after training
 - 7.4. Sample past transitions via an experience replay buffer using a fixed-capacity circular queue
- 8. Create an interactive user interface for training and backtesting**
 - 8.1. Dataset management for training environment
 - 8.2. Backtesting environment for LightGBM and trained DQNs
 - 8.3. Training environment with LIVE view of statistics
 - 8.4. Dashboard to show model predictions on current-day data
 - 8.5. Display Directional Accuracy metrics for LightGBM, and trading performance metrics for trained DQNs

Documented Design

Overview

This project aims to develop a machine learning research environment for both LightGBM regression and Deep Q-Networks. All systems will be written in python for its AI support and GPU acceleration features.

The system has a modular design. Key elements include the DQN, Regression Model, Dataset Generator, Web Scraper, Sentiment Analysis, and GUI modules.

To provide models with sufficient training data, a web scraper is used to automatically pull financial news data, while the *yfinance* API is used to pull OHLC data. An internal API endpoint was located on an alternative website, allowing for article metadata to be fetched directly.

After news articles are gathered, *FinRoBERTa* is used to analyse the sentiment of news articles in memory. *FinRoBERTa* outputs probabilities for each sentiment label: Negative, Neutral, and Positive. These probabilities are then passed through a custom smoothing function designed to prevent overconfidence in predictions (which was a significant problem in early testing).

A separate module is used to fetch OHLC data via the *yfinance* API. This is then processed by an indicator calculator script, which generates relevant technical indicators.

A *LightGBM* regression model is used to guide the DQN's decisions by predicting percentage movements for specified future horizons (for example: 1-day movement, 3-day movement, and 7-day movement). It then makes use of *quantile regression* to create upper and lower bounds, allowing the DQN to make decisions based on risk.

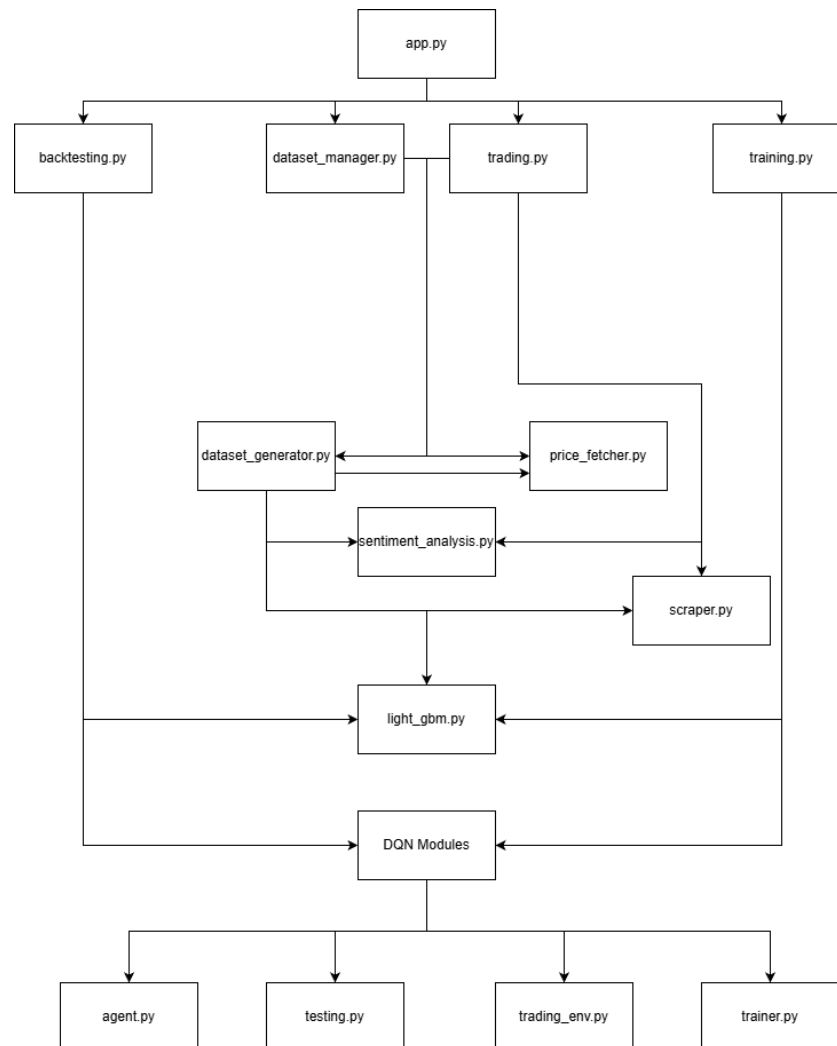
The dataset generator makes use of the aforementioned modules to dynamically create training data for *LightGBM* and the DQN. It takes the required date range as an input, and outputs a csv in the requested location containing all the necessary data. If a *LightGBM* model is available, predictions for specific future horizons and upper/lower quantiles are generated. To prevent overfitting in the regression model, this data is generated in a rolling window of a specified number of days. For the first window days, the model will simply be trained on the given data. Then, for the next window days, the model will generate a new set of predictions every day for percentage movements in price. This process is repeated to the end of the dataset.

The DQN uses the final dataset as training data. This is pulled over a large time period to provide it with a sufficient number of training episodes. Slices of a customisable length are randomly sampled from several years' worth of data. Each column in the dataset is transformed into a one-dimensional convolution so that the model can observe temporal patterns. The observation space includes this data along with the value of current positions being held, as well as the simulated bank balance of the agent. The action space will consist of (at a bare minimum) actions to BUY, HOLD, and CLOSE positions. Every day, the agent gets to decide on which action to take. The neural network predicts a Q-Value to find which action is most likely to generate the biggest reward. The final trained DQN should be able to suggest optimal trades to make at any given time.

The GUI will be built via the *streamlit* library in python for its simplicity, allowing for a clean user interface to be built in a relatively short amount of time.

Hierarchy Chart

This hierarchy chart shows which modules import others. As the DQN's general function is split among four scripts, they are linked together here for visual simplicity.



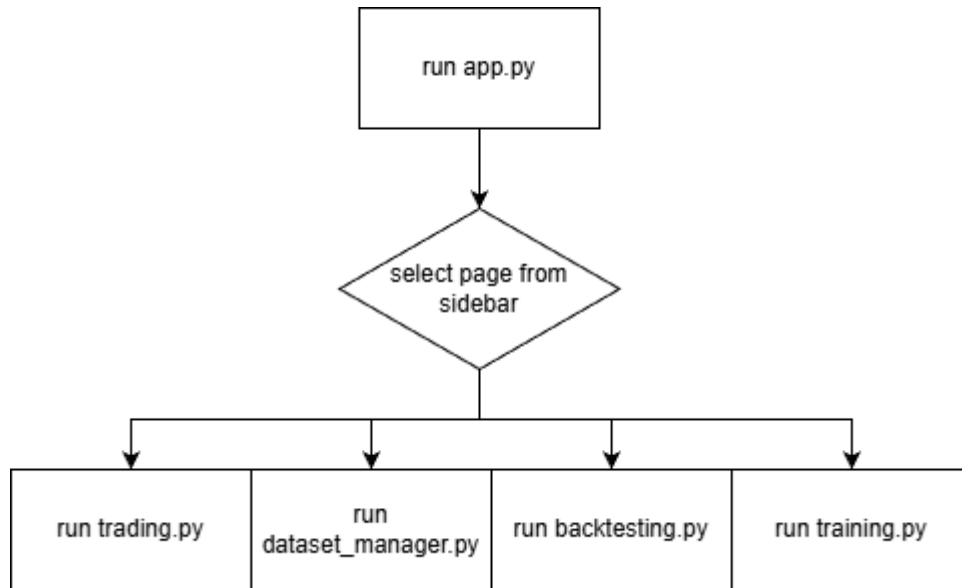
App.py imports all the web pages, which in turn manage the processing modules. Backtesting.py requires the DQN modules to load models and test them within the training environment. It also imports LightGBM for testing datasets. The dataset manager accesses the dataset generator for full generations, and the price fetcher when calculating indicators, in case an existing dataset is missing data. Trading.py is the module that displays data from the current market and therefore uses the dataset generator to provide models with their required features, as well as the price fetcher for recent data. It imports the scraper for a live news feed as well as sentiment_analysis.py to analyse the news. Training.py imports DQN modules, specifically trainer to train DQNs, as well as LightGBM to train new models. Dataset_generator.py imports the price fetcher, scraper, sentiment analysis modules, and LightGBM to provide rich features to models.

System Flowcharts

The flowcharts below show the actions that specific scripts take, and the order in which they can be run. More advanced logic will be on separate flowcharts continuing from the main one.

App.py

*This script is used to fulfil **Objective 8**.*



Each of the four components listed above have more detailed flowcharts explaining their processes in the pages below.

The main application has four tabs to organise functionality into different sections. Each page manages UI and which backend scripts to call.

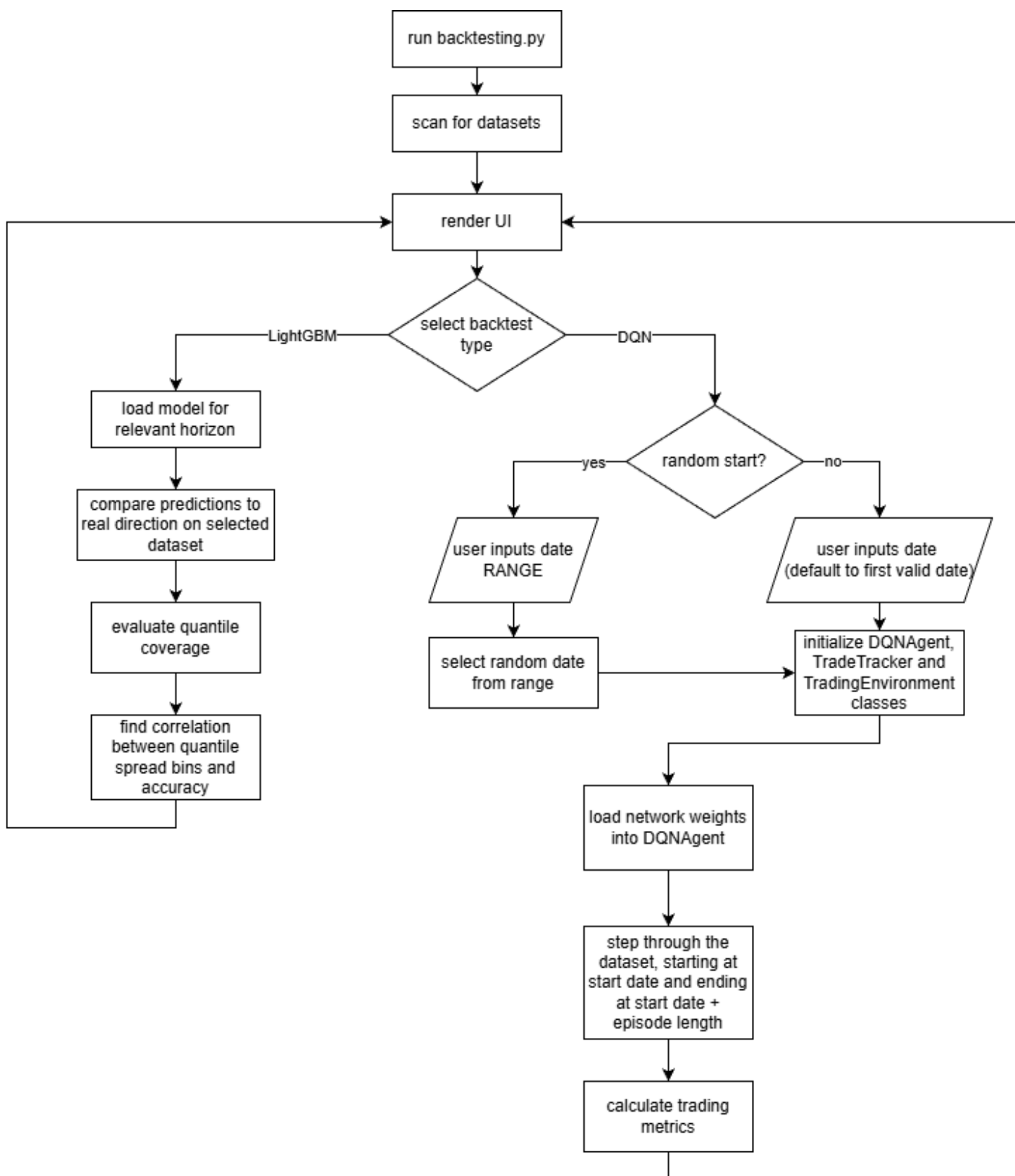
Given its declarative nature, much of the processes are reliant on waiting for user input.

Additionally, due to the large scale of this system, trivial steps have been abstracted in these diagrams for visual clarity. Simpler scripts do not have their own flowcharts, but their processes will be noted within the flowcharts of the scripts that call them.

User inputs are handled implicitly through Streamlit's reactive framework. Widget interactions automatically update session state variables without requiring explicit input acquisition steps. Flowcharts therefore represent user input as a single "build configuration" step when triggered by user actions (such as button presses) rather than showing individual input operations for each widget. Default values are applied automatically via predetermined widget parameters.

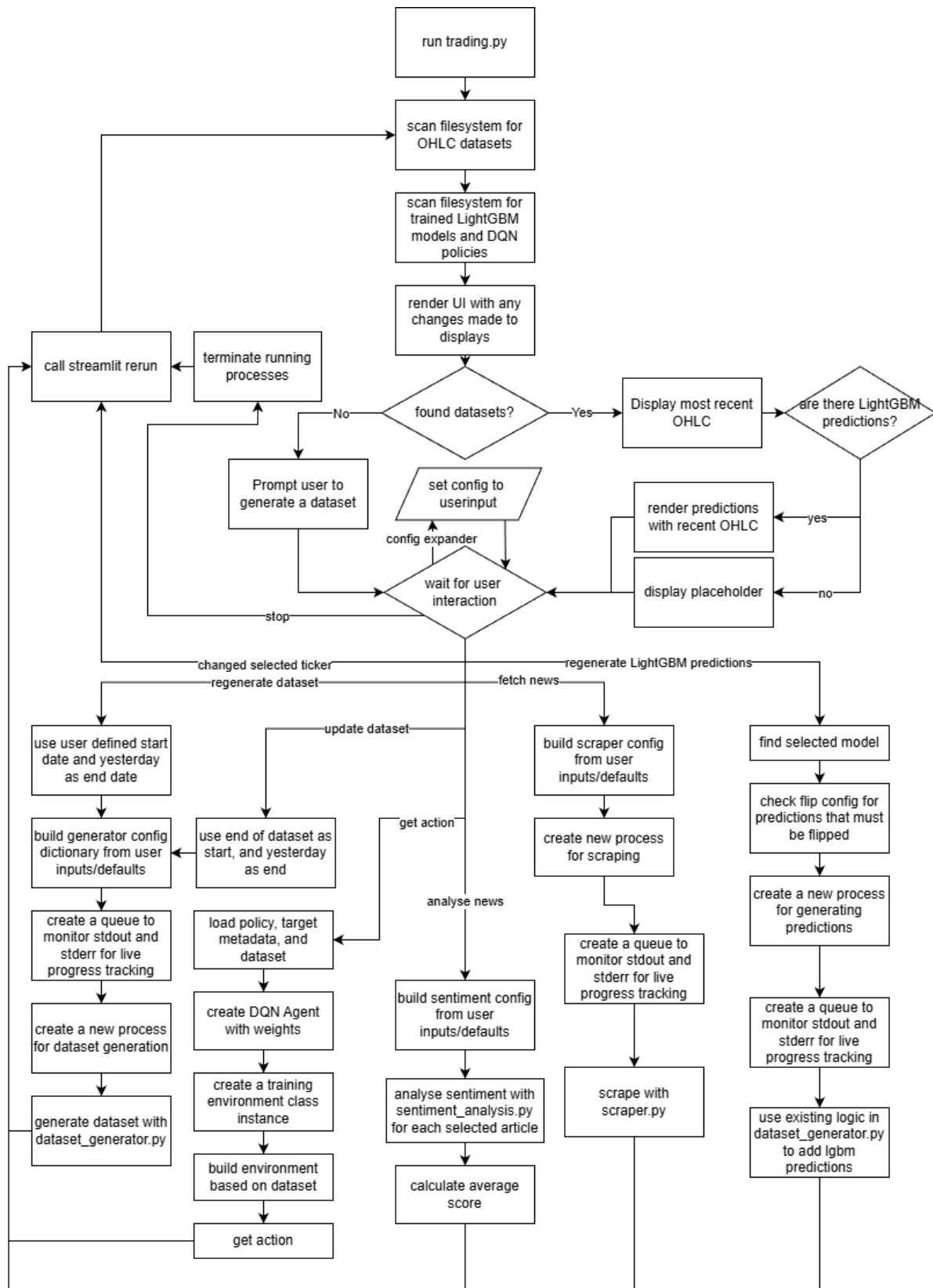
backtesting.py

This script is used to fulfil **Objectives 8.2 + 8.5.**



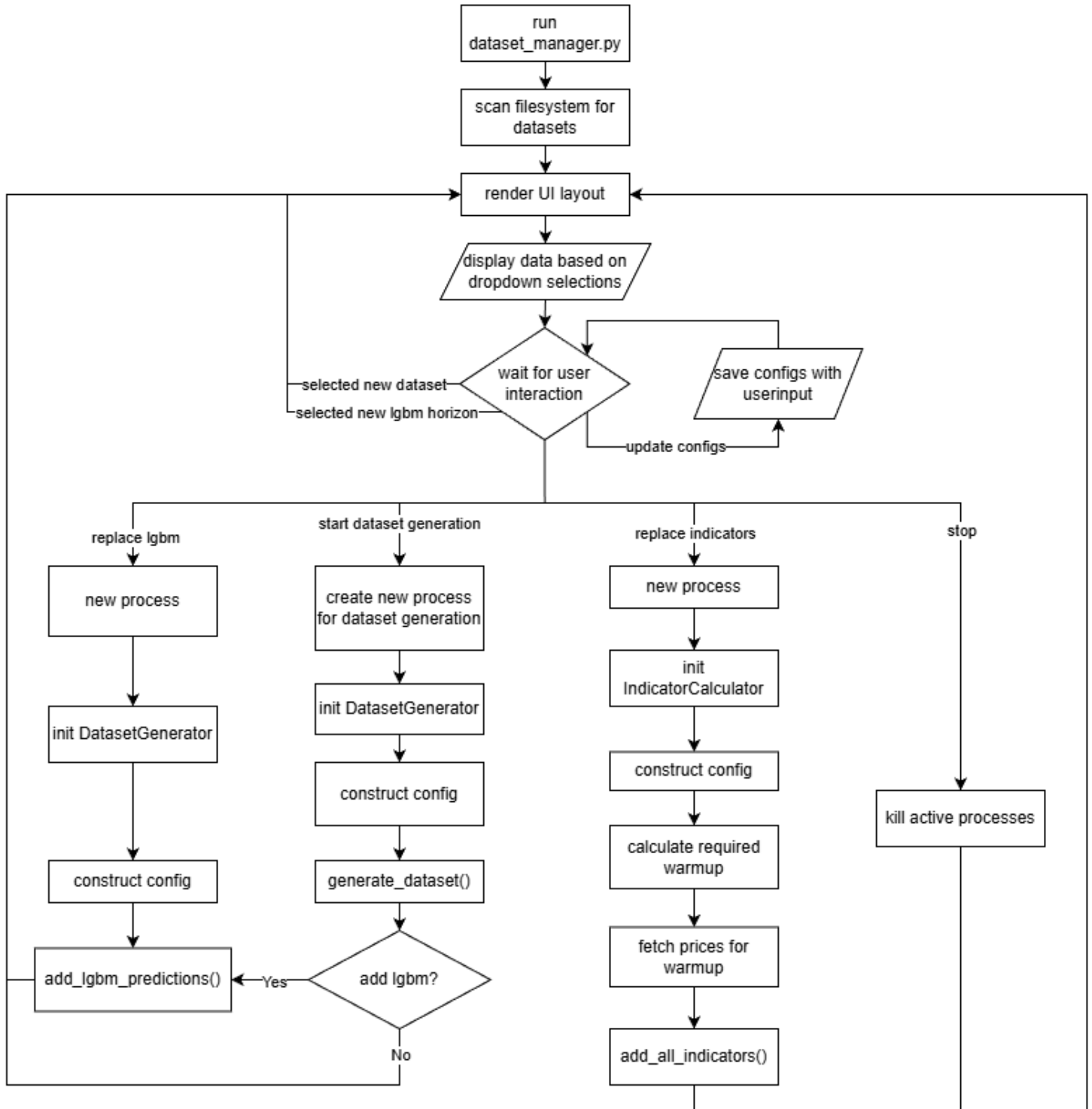
trading.py

This script is used to fulfil **Objective 8.4**.



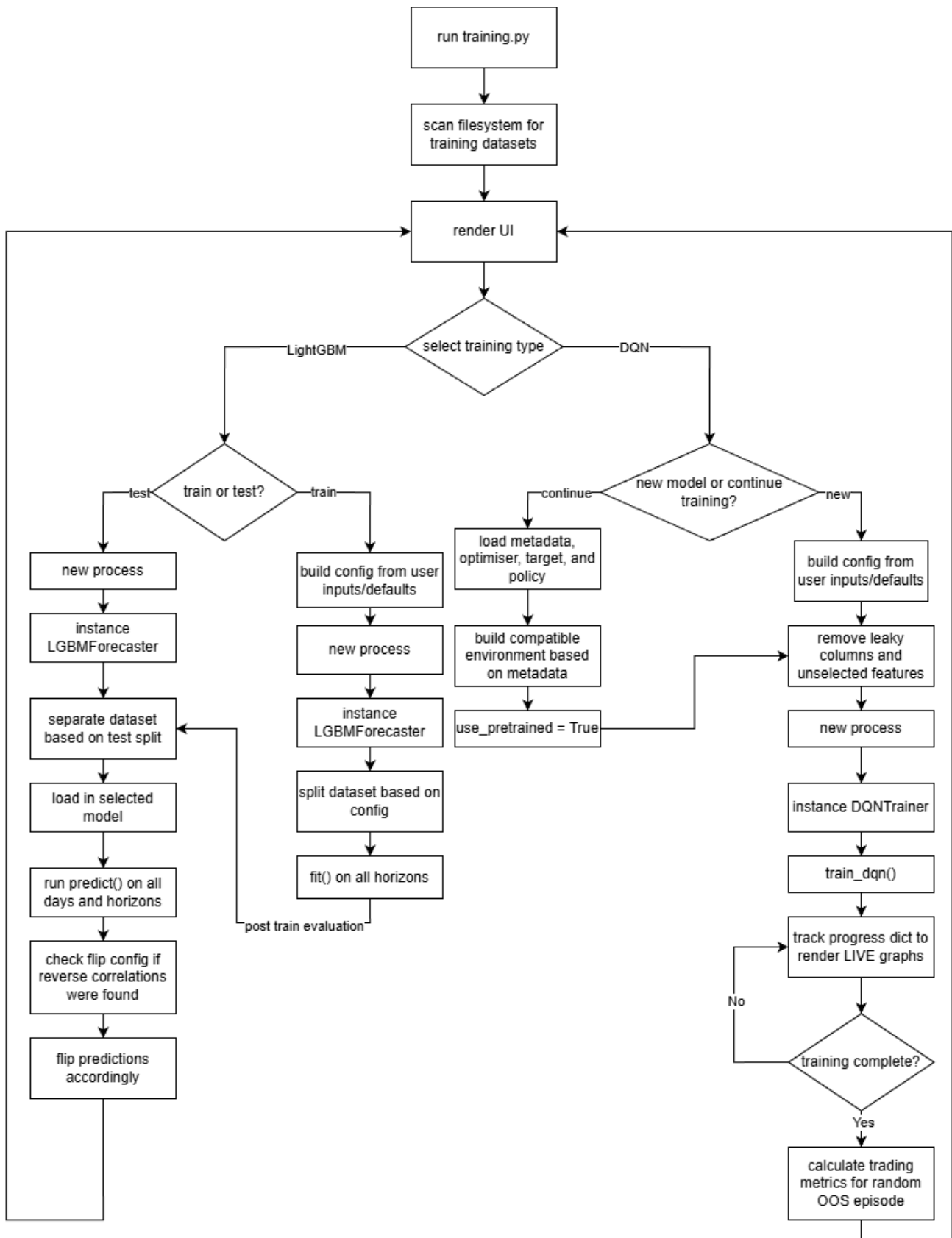
dataset_manager.py

This script is used to fulfil **Objective 8.1**.



training.py

This script is used to fulfil **Objective 8.3**.

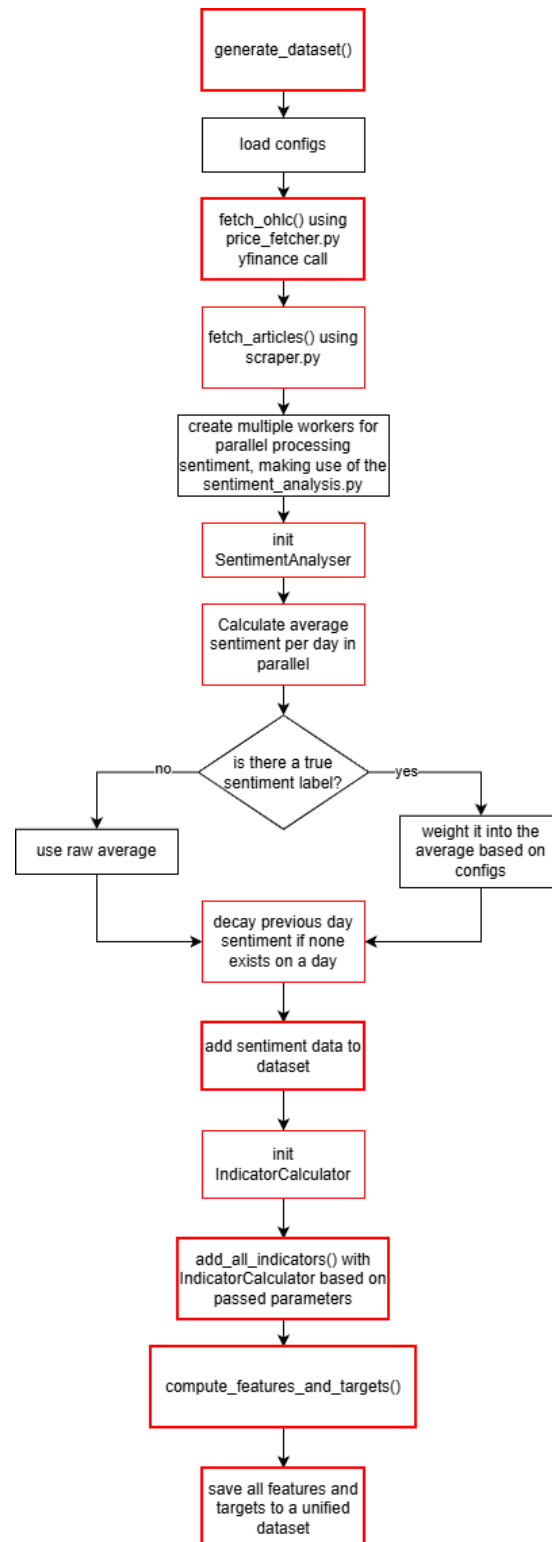


dataset_generator.py

The primary purpose of this script is to hold the *DatsetGenerator* class, which uses the *generate_dataset()* function, called in many other scripts.

*This function is used to meet **Objectives 4, 5, + 2.5.***

(steps directly achieving objectives are highlighted in red)

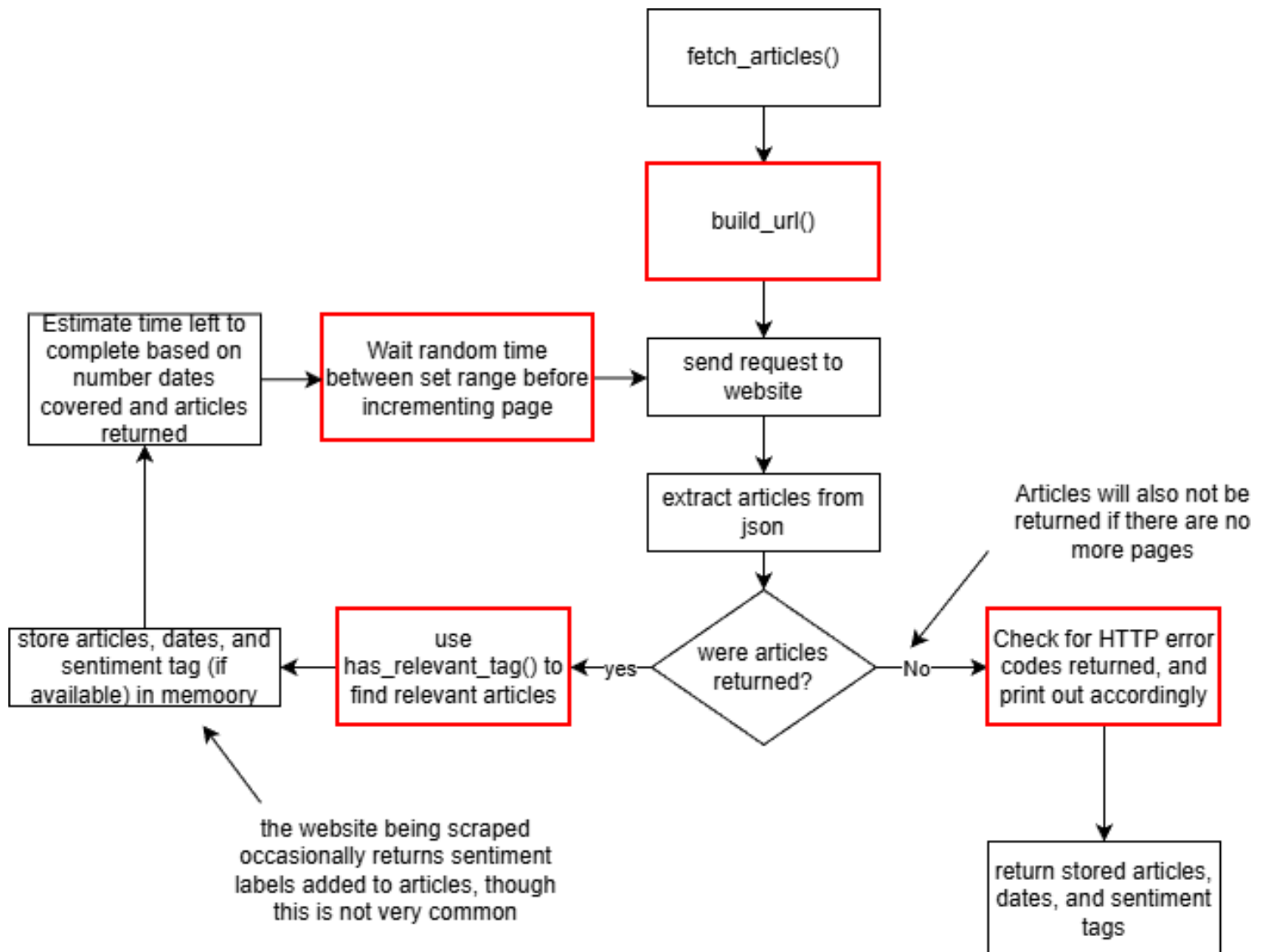


scraper.py

The primary purpose of this script is to hold the *ArticleScraper* class, which in turn has a *fetch_articles()* function called by other scripts.

*This function is used to fulfil **Objectives 2.1 – 2.4.***

(steps directly achieving objectives are highlighted in red)

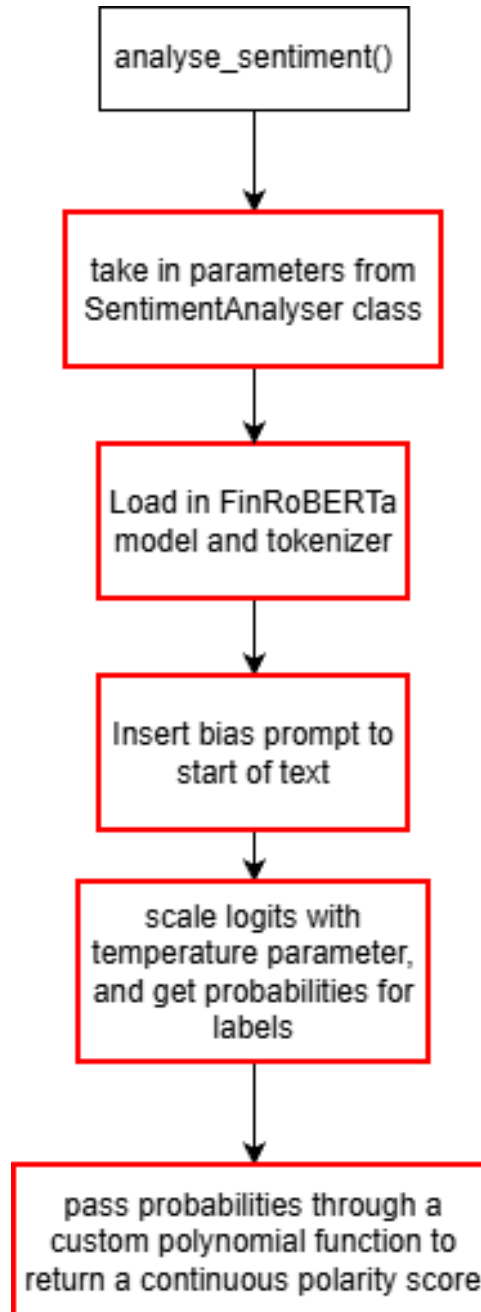


sentiment analysis.py

The primary purpose of this script is to hold the *SentimentAnalyser* class, which has an *analyse_sentiment()* function called by other scripts.

*This function is used to meet **Objective 3.***

(steps directly achieving objectives are highlighted in red)

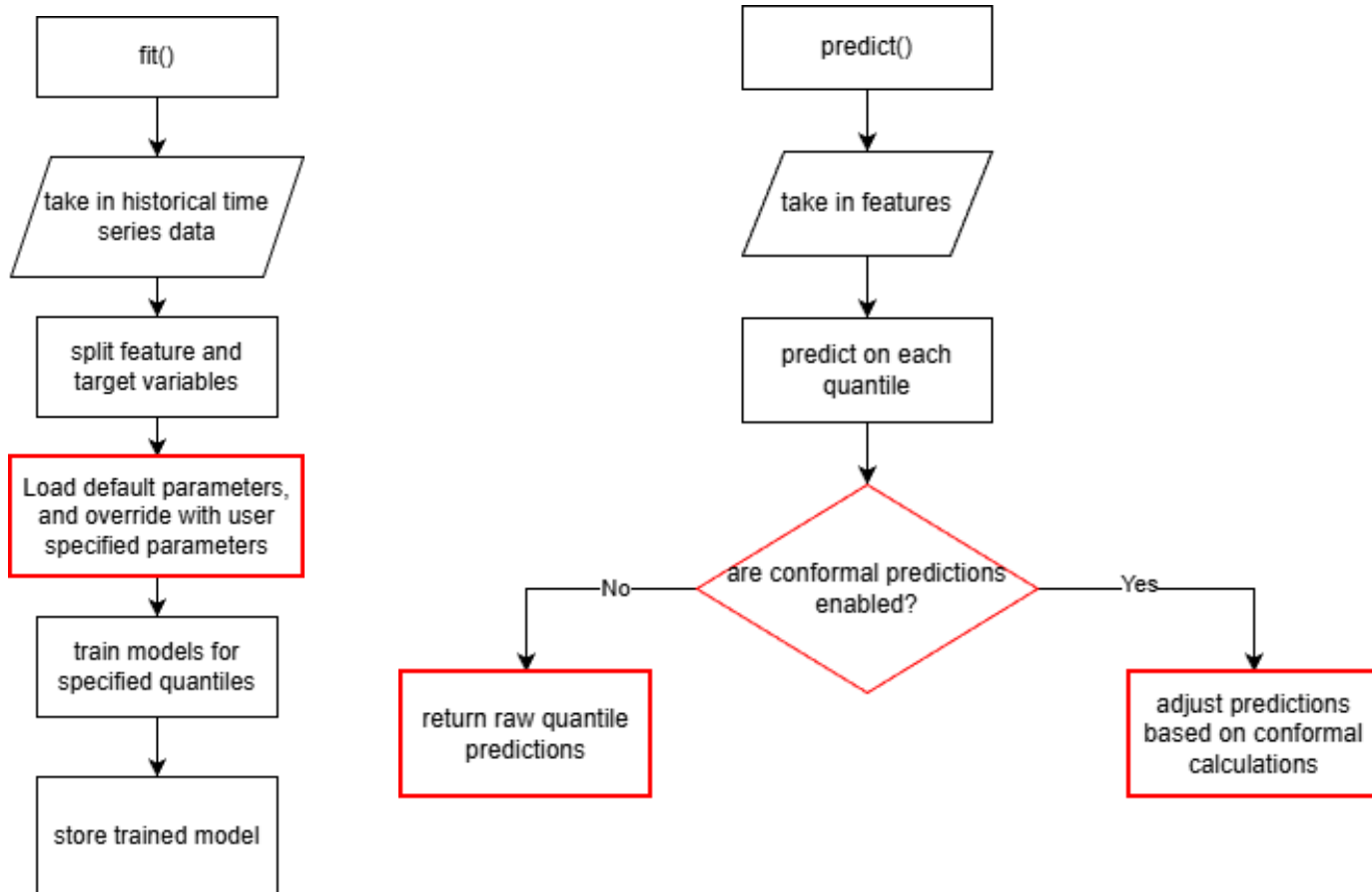


light_gbm.py

The primary purpose of this script is to hold the *LGBMForecaster* class, which has the *fit()* and *predict()* functions called by other scripts.

*These functions are used to meet **Objective 6.***

(steps directly achieving objectives are highlighted in red)

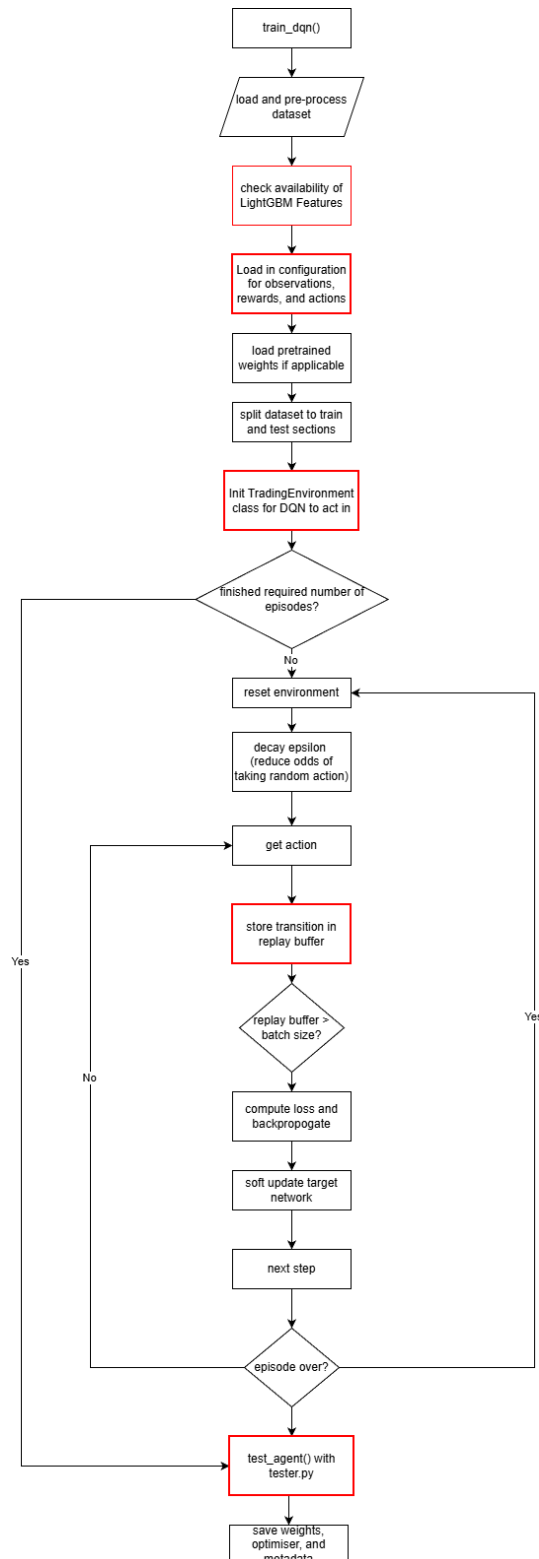


trainer.py

The primary purpose of this script is to hold the *DQNTrainer* class, which has a *train_dqn()* function called by other scripts.

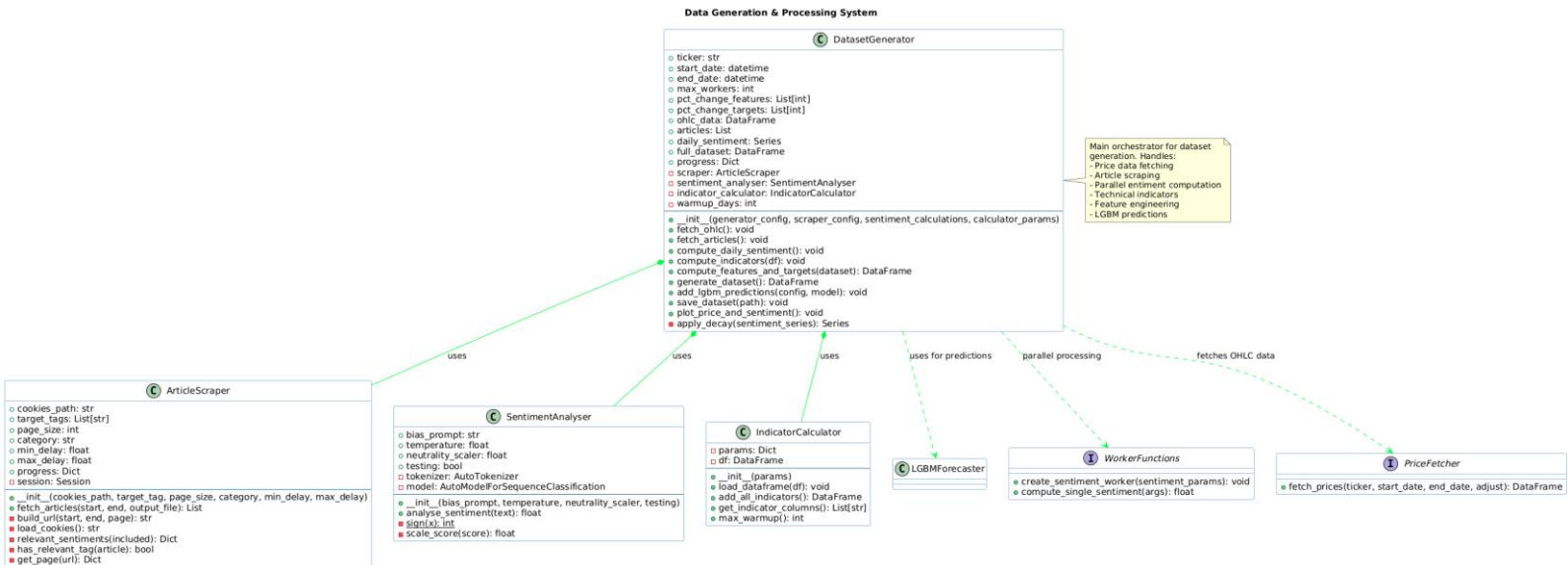
*This function is used to meet **Objective 7.***

(steps directly achieving objectives are highlighted in red)

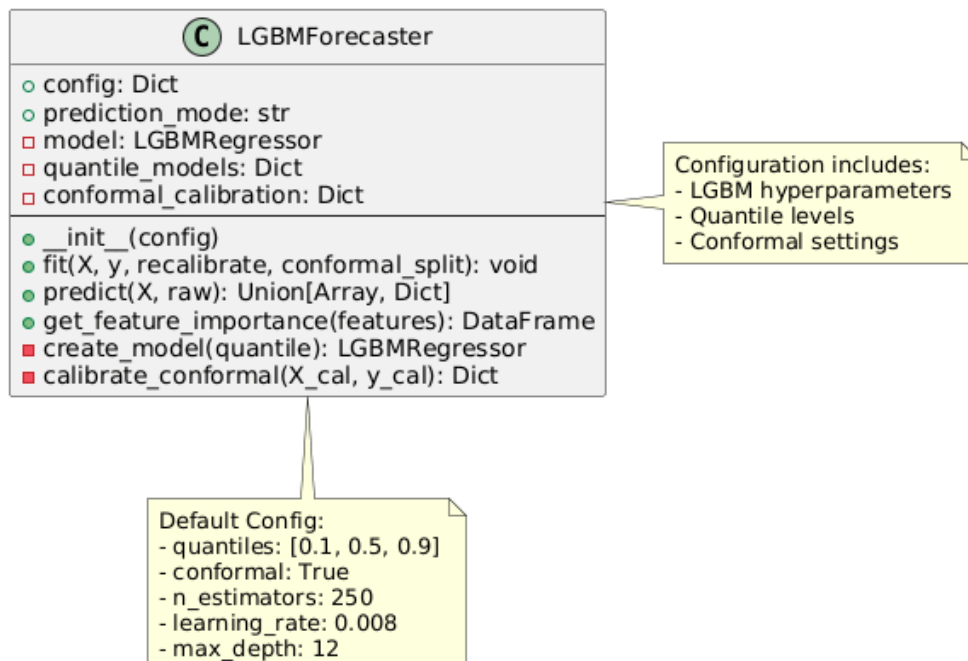


Class Diagrams

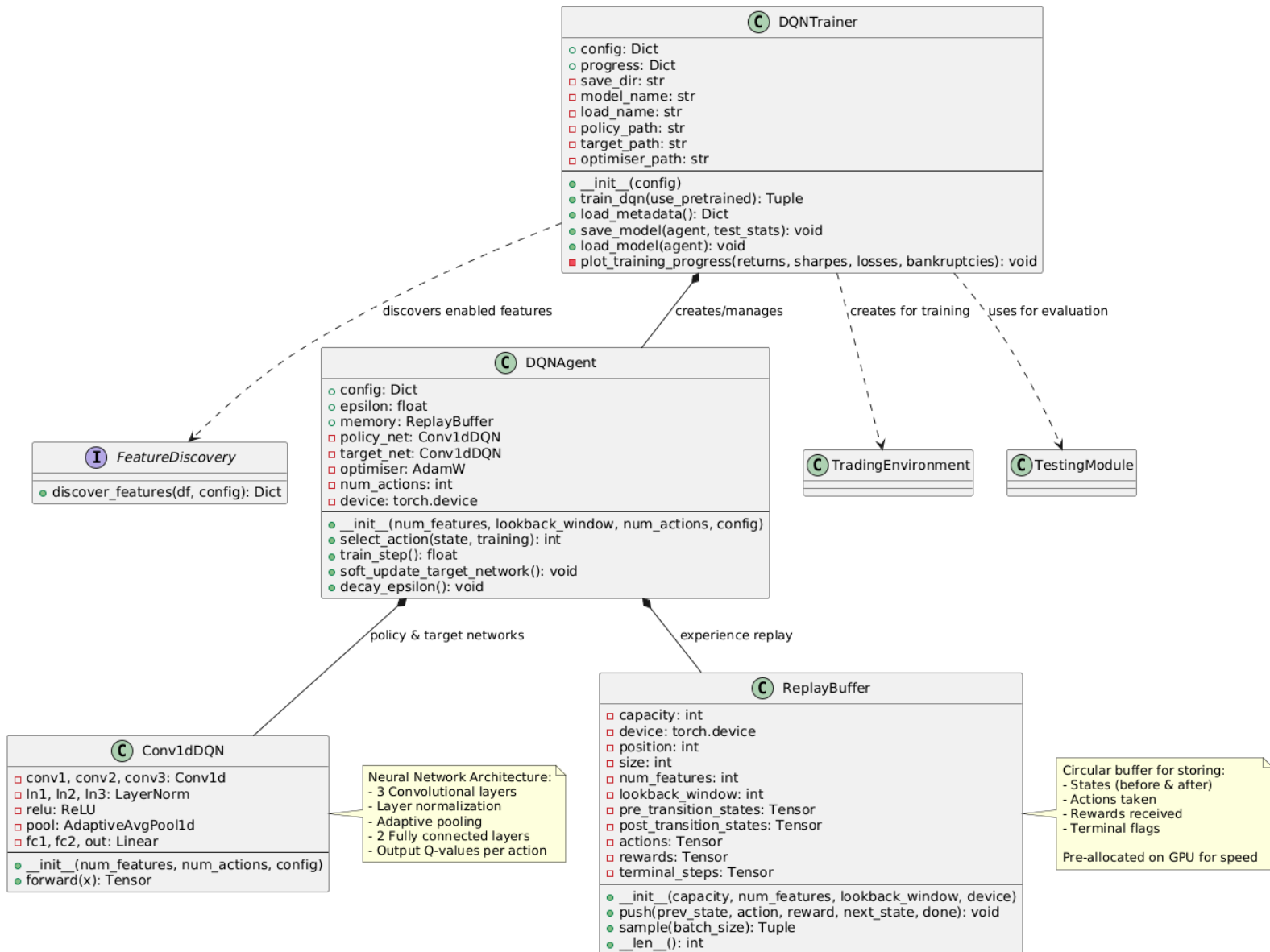
The diagrams below detail the key classes used within the project, along with their key variables and functions. Public access modifiers are denoted in green, and private in red. Many helper attributes and functions are kept public for the sake of granular control. This design trade-off is intentional, as the system operates in a research environment where the data is noisy and small parameter changes can have a significant impact on results.



LGBM Forecasting System



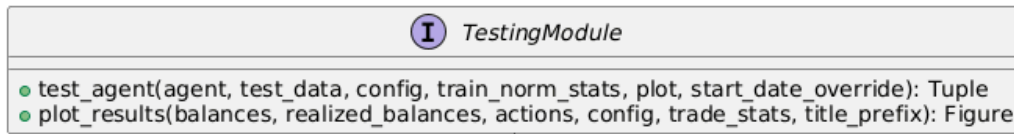
Deep Q-Network Trading System - Core Components



Trading Environment & Position Management



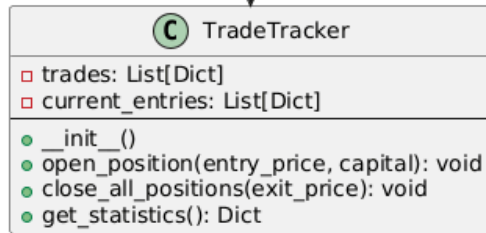
Testing & Utility Components



Testing utilities:

- Runs agent on OOS data
- Tracks trade statistics
- Generates performance plots
- Win rate, profit factor
- Sharpe ratio calculation
- Expanding window validation

uses



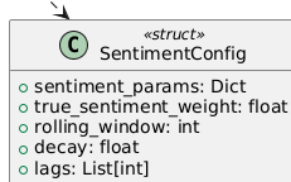
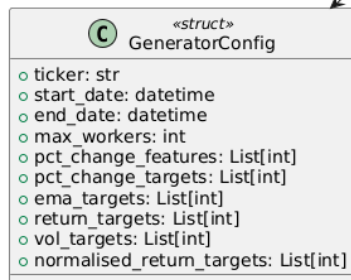
Configuration Objects



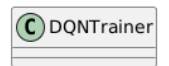
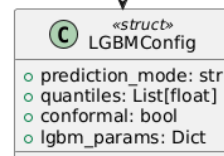
uses

uses

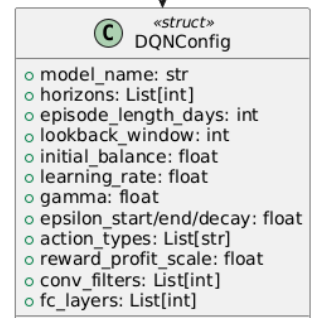
uses



uses

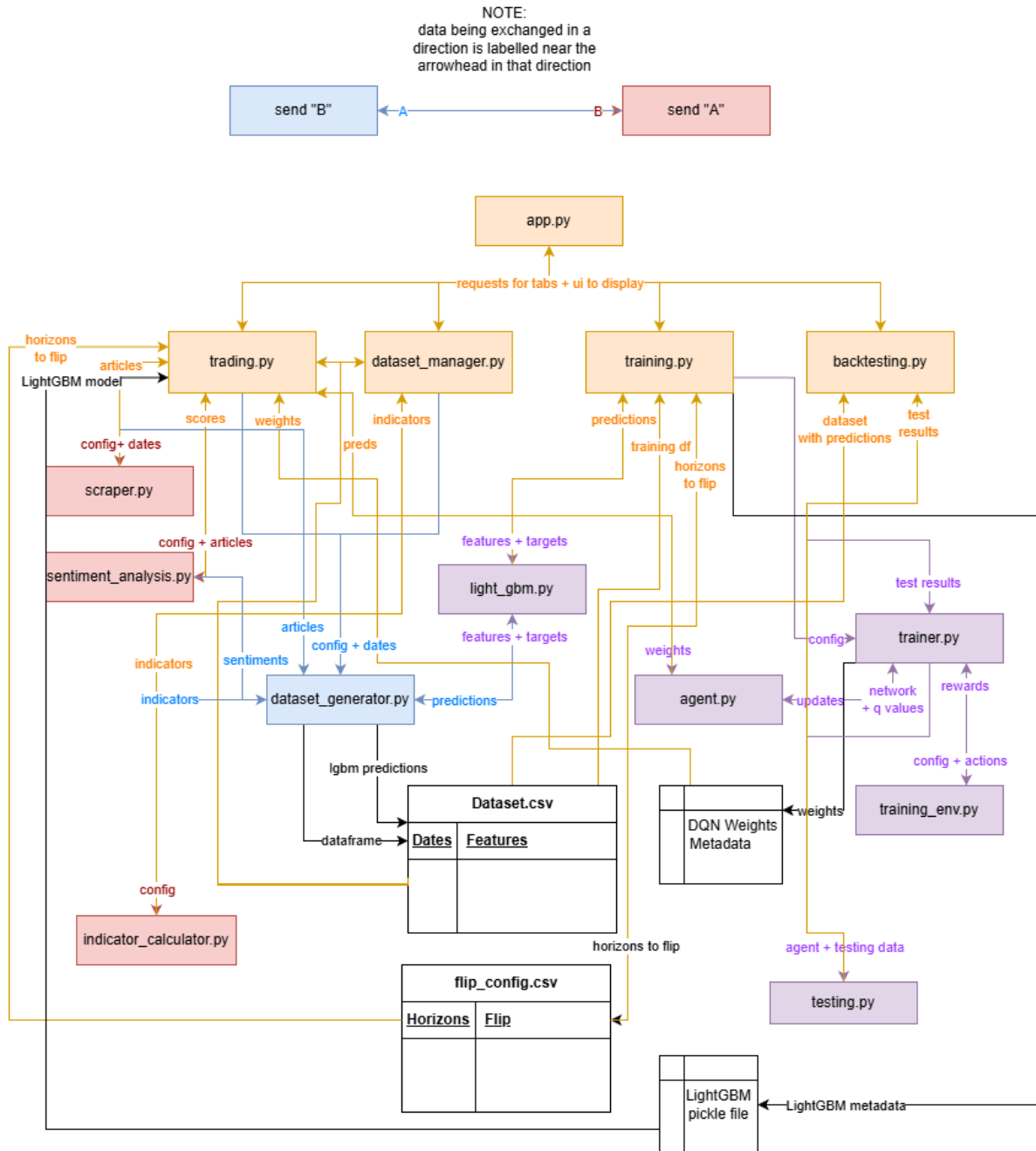


uses



Data Flow Diagram

This diagram shows the flow of data within the system, including which scripts process data, and what files store data.



At the frontend, the user simply makes a request to display a specific page. From that page, the user can control inputs and parameters being passed through to the processing modules. UI elements are shown in **orange**. Disk storage is shown in black. Files to be stored include the datasets for training, the “flip config,” which lists the horizons of LightGBM models that need to have their predictions reversed, as models can sometimes find reverse correlations. Trained DQN weights are saved to different files, which in this diagram has been abstracted to one. A trained DQN will include a saved policy network, target network, optimiser, and metadata which is used to determine how to use the model by other scripts. In **red** are components used for base feature generation. In **blue** is the dataset generation structure. Modules related to machine learning are all in **purple**.

App.py controls which page is displayed. From there, each page handles the processing components via worker processes.

Trading.py takes in a saved dataset of recent data from a csv file. After reading from a LightGBM pickle file, predictions from LightGBM are also taken in to append onto the end of the close data. Trading.py will first read the flip config csv saved by training.py, to determine which horizons have reverse correlations, and therefore will be more accurate to predict the opposite of what it says. Scraped articles can also be directly received from scraper.py, as well as sentiment scores from sentiment_analysis.py. trading.py can send configurations of parameters to each processing module to determine how the processing should take place. It also sends weights from trained DQNs, reads from the saved weights from training components, and uses those weights on the base agent to return a prediction, which is an optimal action according to the trained DQN.

Dataset_manager.py also works similarly, taking in saved csv datasets. It differs from trading.py by having direct access to the indicator calculator to replace indicators in datasets, and rather than individually accessing processing components, it calls the dataset generator to handle most tasks.

The dataset generator is a controller for many other processing scripts and can pass a config down to them. The scraper gives it articles which are processed using many sentiment analysis workers in parallel for speed. It uses the indicator calculator to generate indicators and statistical features. The dataset generator internally handles target generation. It also accessed LightGBM, sending features and targets and being returned predictions for different horizons to add to generated datasets in order to train DQN agents with LightGBM predictions.

Training.py takes in training dataframes to train models. For LightGBM it also takes in horizons to flip. It is also able to quickly test LightGBM models and therefore can both read and write to the flip config based on which models found reverse correlations. It takes in predictions from LightGBM to gauge the accuracy of them. Trained LightGBM models can have their object state saved to a pickle file, which effectively allows for the model to be reused later. It can also pass down a configuration to the DQN trainer script, which in turn manages the DQN training and saved the trained DQN into different pt (PyTorch) files for different parts of the DQN (such as the optimiser, target network, and policy). It also saved metadata as a json file so other scripts know how to use the DQN.

Backtesting.py takes in datasets with LightGBM predictions to evaluate the accuracy of LightGBM predictions previously added. It also takes in test results from testing.py after sending it the testing data with the agent's actions.

Trainer.py manages the DQN processes. It sends network updates to agent.py for improving the neural network, and the agent returns it the new q values from the network. It then uses this agent to interact with the training environment, which determines rewards based on the config from trainer.py. rewards are then returned to determine how the agent should be modified next. Once trained, the agent will be tested out of sample (OOS) via testing.py

Key Modules

The unified research environment for BASLIX Terminal contains many non-trivial processing modules with customised functionality, the details of which are listed in this section.

Scraper

The scraper will access an API endpoint on [REDACTED]. The request to fetch data can be made via the construction of a url:

[REDACTED URL]

This endpoint was discovered through inspection via chrome developer tools. Key parameters are highlighted in red. The other parameters need not be modified. “Fields” determines what the json file should include. A few more fields than necessary are being pulled in this instance, to provide the opportunity for further expansion for features, for example sentiment could be weighted somehow based on comment count. The include section also has a similar purpose. The category changes the type of assets being returned for news articles. For example, gold would fall under commodities, and bitcoin and crypto. The filter [since] and [until] parameters take in a *unix* timestamp, which is useful for pulling articles from relevant historical periods. Page size determines the amount of data returned in one request, which is important to control as excessively large pages may cause a timeout error. Page numbers allow for the algorithm to access information which could not fit on a single page and still falls within the filter specifications. For example, if the filters were set to pull data on commodities from 2010 to 2020, all that data would not be able to fit on a reasonably sized page, so the algorithm will increment step by step through each page.

Within the returned json, the tags for different tickers are included. Therefore, if you know the tag for a specific ticker, it is trivial to filter for articles from that tag only. Correlated tags can also be included, for example Gold Futures and Spot Gold.

To learn about web scraping, a YouTube tutorial⁷ was referred to and adapted for my purposes.

Sentiment Analysis

⁷ <https://www.youtube.com/watch?v=kJyOp12PgP4&t=2507s>,

Web Scraping APIs with Python for Data Science - Beginner to Advanced - Part 1

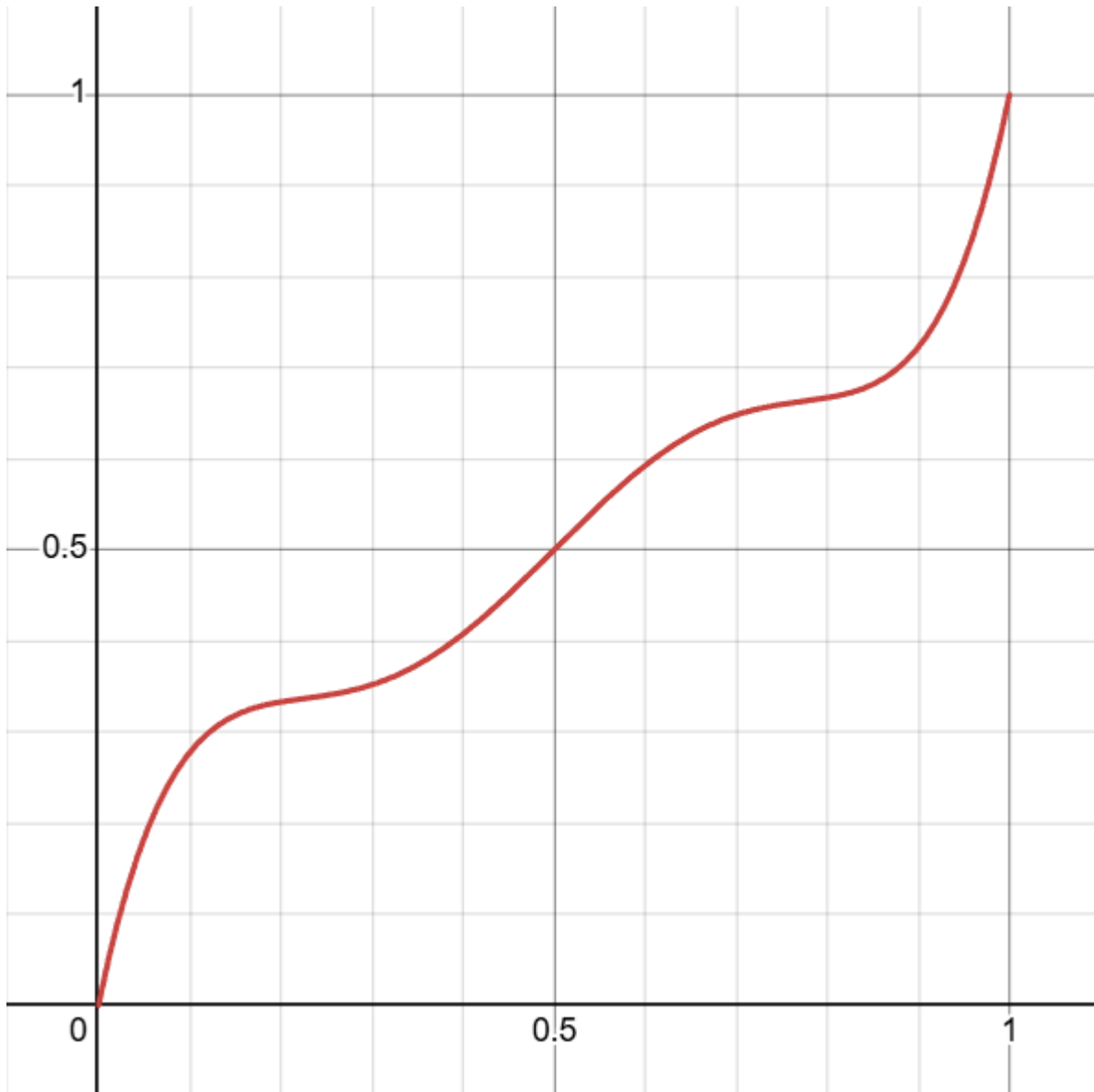
While a pretrained model is used for sentiment analysis, the outputs are post-processed and transformed before being used as features within the training environment. This is done to ensure that the sentiment values are numerically stable, less noisy, and more meaningful for downstream machine learning models.

A FinRoBERTa (financial-roberta-large-sentiment) model is used, as it is specifically trained on market-related text and therefore better suited to analysing financial headlines than a general-purpose sentiment model. Each input headline is prefixed with a bias prompt before tokenisation. An example for gold would be “Gold Market Outlook:”. This provides additional context to the model and helps keep the sentiment interpretation within the relevant market domain.

Transformer-based classification models output *logits*, which are unnormalised scores representing how strongly the model favours each class. These logits are typically passed through a *softmax* function to convert them into probabilities that sum to 1. Temperature scaling involves modifying the softmax operation by dividing the logits by a value known as the temperature. Lower temperatures will increase confidence, whereas higher temperatures will decrease confidence. As the use case for this sentiment analysis model is to output continuous scores, a lower confidence is preferable, so that the nuance from the less dominant labels can be accounted for.

To convert discrete label probabilities into a continuous score, the first step is to subtract negative sentiment from positive sentiment. However, this value is typically quite extreme, as even with significant temperature scaling, the outputs can be heavily skewed to extreme ends. Different temperature values were tested prior to designing the full module, and it was found that the model very rarely would output moderate confidence scores and the resulting sentiment would rather stay near -1, 0, or 1. Therefore, to capture nuance better, a custom neutrality scaler function was empirically derived via experimentation on Desmos. A function was designed such that inputs approaching extreme ends would be pushed closer to the middle, and values too close to 0 would be pushed into slightly higher confidence.

The derived function can be seen below:



Mathematically, the function used is:

$$f(x) = (x-0.5) - k(x-0.5)^3 + 4k(x-0.5)^5 + 0.5$$

where: $\{0 < x < 1\}$

However, this function is only applicable to positive scores. Therefore, the absolute value of a score is used for scaling, before having the original sign (negative or positive) applied to it again.

LightGBM

The forecasting model is built using LightGBM, a gradient boosting framework based on ensembles of decision trees. As a library, LightGBM provides an efficient implementation of gradient boosted trees, with advanced optimisations allowing models to learn

complex and non-linear relationships from input data while remaining computationally efficient.

LightGBM itself operates on tabular data and optimises a specified loss function, however it does not define how the model is trained over time, how uncertainty is handled, or how predictions are integrated into the pipeline. These aspects are explicitly designed and implemented as part of this project.

LightGBM was selected due to its ability to model non-linear relationships and handle many input features. These characteristics make it well suited to financial forecasting tasks, where relationships between indicators are often complex, non-linear, and subject to regime shifts.

Rather than producing only a single point estimate, the model is designed to operate in quantile prediction mode. While the LightGBM library provides built-in support for quantile regression as an objective function, the design decision to train separate models for multiple quantiles and treat their outputs as an uncertainty estimate is implemented at the system level. This allows the forecasting pipeline to estimate not only a central prediction (the q50 Median), but also upper and lower bounds that show forecast uncertainty. This is particularly important in volatile markets, where understanding the range of plausible outcomes is as important as predicting the expected value.

Although quantile regression provides uncertainty estimates, the resulting prediction intervals are not guaranteed to achieve correct statistical coverage in practice. This limitation becomes more severe in non-stationary time series, where changing market conditions can cause model confidence to drift over time. The LightGBM library does not address this issue.

To mitigate this, conformal prediction is implemented as an optional calibration on top of the LightGBM quantile models. After training the quantile models, a calibration slice is held out from the available training data. Predictions are generated for this calibration set, and residuals are calculated as the difference between the true target values and the predicted quantiles. For each quantile, the corresponding residual quantile is then computed and stored as a correction term. During inference, future predictions are adjusted using these correction terms, ensuring that each quantile forecast achieves the desired empirical coverage. This calibration process is designed and applied separately from the LightGBM library itself.

When incorporating LightGBM predictions into datasets, walk-forward windows are used. This approach is inspired by walk-forward testing which was specifically designed for time-series data where the temporal order of observations must be preserved. Rather than relying on a typical train-test split, the forecasting system simulates real-

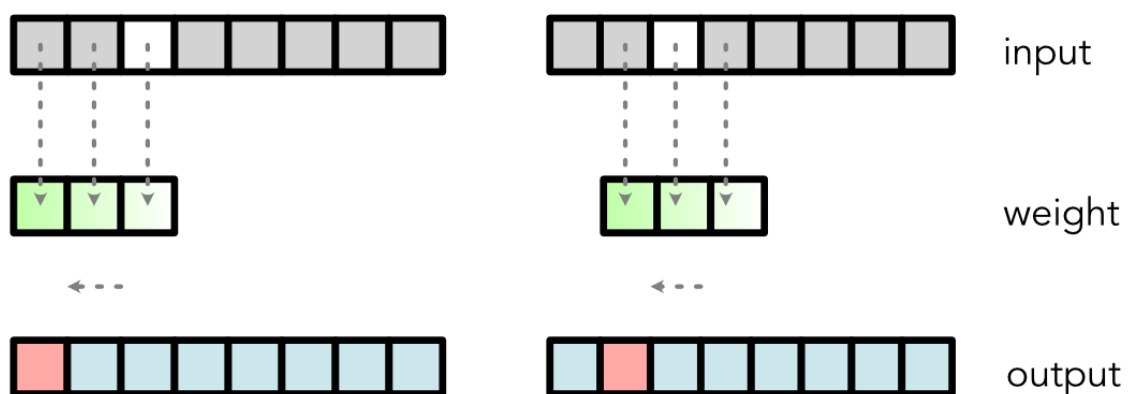
world deployment by training the model on a limited historical window and generating predictions for future timesteps without retraining for a defined period of time.

After a predefined number of steps, the model is retrained using an expanding window of historical data. The window is constrained by a minimum and maximum size, a design choice that ensures the model has sufficient data to learn meaningful patterns while remaining responsive to regime changes. This retraining strategy is implemented within the dataset generator and is not provided by LightGBM in itself. It is also effective at preventing data leakage, where future information influences the training process, leading to unrealistic performance estimates that would not be achievable in real-world deployment.

1D Convolutional Deep-Q Network

Convolutional Neural Networks (CNN)

Convolutional Neural Networks (CNNs) apply learnable filters (kernels) that slide across input data to detect patterns and features. While CNNs typically use 2D convolutions for image processing, 1D convolutions are particularly effective for sequential data like time series because they can identify temporal patterns across different time scales by sliding across the single time dimension. Unlike traditional feedforward networks that treat each timestep independently, convolutional layers preserve the sequential structure of financial data which allows the network to learn relationships between consecutive price movements, volume changes, and technical indicators.



Convolution kernels slide over the input to produce its output. Image source: <https://minitorch.github.io/module4/convolution/>

Network Architecture

The DQN architecture consists of three convolutional layers followed by two fully connected layers, with all hyperparameters configurable through the config dictionary

to allow for experimentation. The convolutional layers allow for patterns to be found, while the fully connected layers are used for complex decision making based on these patterns. While PyTorch's `nn.Module` serves as the base class for the network (standard practice for defining neural networks), the entire architecture design, layer configuration, and data flow logic has been custom-built specifically for this trading application.

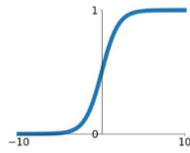
The first convolutional layer processes the raw input features with 64 filters and a kernel size of 3 as a default setting (this can be configured). Padding adds zeros at the boundaries of convolutions to prevent the output from shrinking. This is because when a 3-element kernel slides across a vector, the edges would have an insufficient number of neighbouring values at the ends, so padding ensures the output length matches the input length. This is followed by Layer Normalization and ReLU activation. Layer Normalization was chosen over the more common Batch Normalization because financial data distributions can shift between episodes, and Layer Normalization normalizes across features rather than across the batch, making it more robust to these non-stationary conditions.

The ReLU (Rectified Linear Unit) activation function is applied after each normalization layer. ReLU simply sets all negative values to zero while leaving positive values unchanged. This introduces non-linearity into the network, which is essential for learning complex patterns. Without activation functions, stacking multiple linear layers would simply produce another linear transformation, and the network would only learn linear relationships between features and Q-values. ReLU allows the network to learn non-linear trading strategies with conditional logic that cannot be expressed as a linear combination of inputs.

Activation Functions

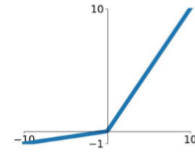
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



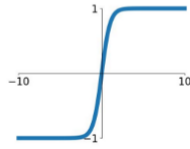
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

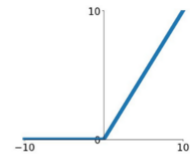


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

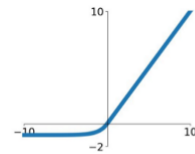
ReLU

$$\max(0, x)$$



ELU

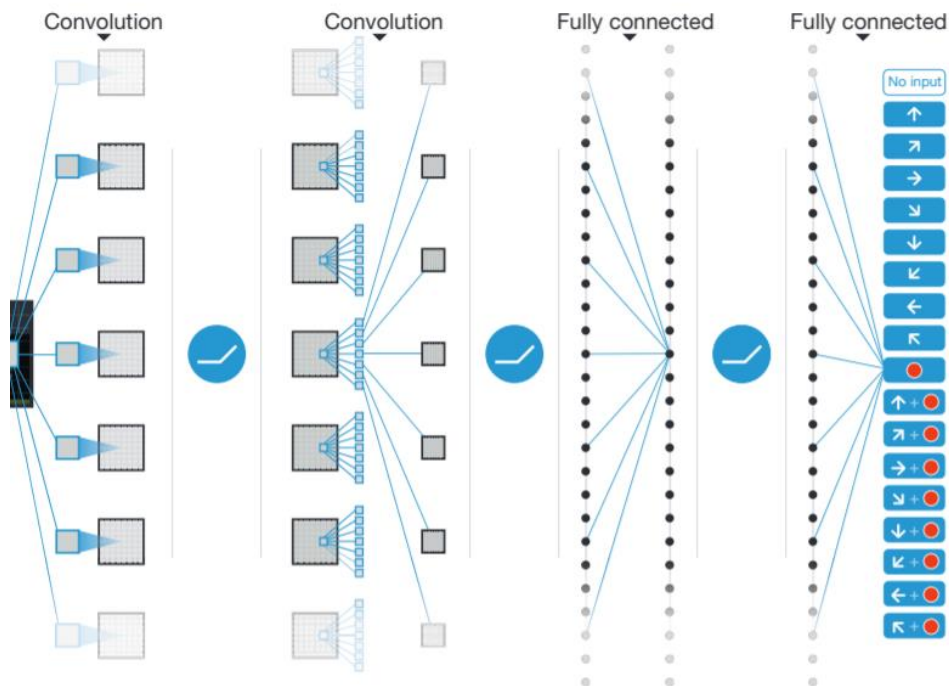
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Examples of other activation functions. Image source:

<https://electronics.stackexchange.com/questions/657902/what-are-the-activation-functions-that-can-be-generated-using-op-amps-and-filter>

The second and third convolutional layers use configurable filter counts (defaults 128 and 64), allowing for the network to progressively learn higher-level temporal patterns. Each convolution is followed by its own Layer Normalization and ReLU activation. After the convolutional layers, Adaptive Average Pooling (nn.AdaptiveAvgPool1d) is applied, which compresses the entire lookback window into a single value per filter. This provides a representation of the temporal patterns detected across all timesteps in a format understandable to the neural network. The pooled features are then passed through two fully connected layers (defaults 256 and 128) with ReLU activations before the output layer, which produces Q-values for each possible action.



Example of convolutional layers followed by fully connected layers in a different neural network application. This is a similar structure to my neural network. Image Source: <https://towardsdatascience.com/deep-q-network-with-pytorch-146bfa939dfe/>

While PyTorch provides the basic building blocks (convolution operations and activation functions), the specific architecture has been designed from scratch, including all layer sizes, the sequence of operations, and the forward pass logic. The library is simply used to execute the mathematical operations efficiently on the GPU (many of which are impossible to do in python, as languages such as C++ are needed to directly interact with the GPU memory).

The default configuration for the neural network can be easily modified by changing the config dictionary without touching any code in the neural network class itself.

Observation Space

The observation space (known as state representation in the mathematical context) will contain data from technical/statistical features, prices, and several custom build meta-features over several timesteps to provide the DQN with a training environment where it can effectively learn patterns to exploit when trading.

Pre-generated LightGBM predictions are separated into short-term and long-term horizons based on a configurable threshold (default 30 days). This separation allows the agent to differentiate between models predicting near-term price movements versus longer-term trends. For each horizon group, four derived features are calculated: mean prediction, standard deviation (disagreement between models), mean magnitude (strength of predictions), and sign consensus (directional agreement).

When multiple LightGBM models disagree (high standard deviation, low sign consensus), this signals uncertainty and should influence position sizing decisions. The LGBM columns are extracted, categorized by horizon, have statistical aggregations computed using NumPy operations (`np.mean`, `np.std`, `np.sign`), and the feature matrix is finally constructed using `np.column_stack` to combine all derived features.

Position-related information is appended across the entire lookback window. For each timestep in the lookback period, five features are included: whether positions are held, number of positions (normalized by maximum allowed), position exposure (as percentage of portfolio), unrealized P&L percentage, and price difference from average entry. By expanding these features across all timesteps rather than appending once at the end, the network can learn how the agent's positioning evolved over time (this could, for example, let the agent know if its portfolio is losing value, and at what rate). This temporal context will be important for the agent's ability to make informed decisions about when to hold versus close positions during testing.

Episode hit rate and percentage days remaining in the episode are also added as observations. The hit rate gives the agent real-time feedback on whether the LGBM models are currently performing well, while days remaining helps prevent the agent from opening risky positions near episode end when trends may not have time to materialize.

Replay Buffer

The replay buffer (essentially the memories of the agent) will be implemented as a custom `ReplayBuffer` class that stores past transitions for experience replay. This is a critical component of DQN training. Rather than storing transitions in a simple Python list, GPU memory should be pre-allocated using `torch.zeros()` to create fixed-size tensors for states, actions, rewards, and terminal flags. This design decision was made purely for computational efficiency, as transferring data between CPU and GPU memory is extremely slow. Allocating GPU blocks once at initialization and treating them as circular queues eliminates repeated memory transfers during training, significantly boosting training speed. The buffer capacity (default 30,000 transitions) determines how much experience the agent can remember. The circular queue tracks position using modulo arithmetic ($\text{position} = (\text{position} + 1) \% \text{capacity}$), allowing the oldest transitions to be automatically overwritten when capacity is reached. The `sample()` method uses `torch.randint()` to randomly select batch indices (default batch size 128) which is useful for breaking temporal correlations in the training data. Without randomization, the agent would learn from sequential experiences only and fail to generalize. While PyTorch provides the tensor data structures and random sampling function, the entire replay buffer architecture, circular queue logic, and sampling strategy was designed and will be implemented from scratch.

Training

The training process implements the Deep Q-Learning algorithm with custom enhancements. The main algorithm uses two networks: a policy network that is actively trained and a target network that provides stable Q-value estimates. This approach is standard in DQN research, but the specific implementation details are customised for the sake of this system.

During each training step, a random batch of transitions is sampled from the replay buffer. The policy network then computes Q-values for the states in these transitions, and `torch.gather()` is used to select the Q-values corresponding to the actions that were actually taken. After this, the target network computes the maximum Q-value for the next states, which is used in the Bellman equation (refer to research section under analysis). This equation combines immediate reward with discounted future reward, providing the learning target.

The AdamW optimizer (`optim.AdamW`) is used with a configurable learning rate (default 0.0002) rather than implementing gradient descent manually. While this might seem like a shortcut, implementing AdamW from scratch in Python would be impractical and inefficient. AdamW requires low-level CUDA operations written in C++ for efficient GPU execution, and python cannot access GPU memory and perform tensor operations at the speed required for reasonable training times.

Gradient clipping is then applied (`torch.nn.utils.clip_grad_norm_`) to prevent exploding gradients, which can be problematic when dealing with financial data's high volatility.

Rather than copying the policy network to the target network every N episodes (as in a standard DQN), a soft update strategy will be implemented. This gradually blends the target network toward the policy network (default tau 0.01), providing more stable learning.

Rewards

The reward function is an important part of the DQN and is critical to building a successful agent. As such I will make most parameters configurable for the sake of the investigation.

A custom indecision penalty system was designed to avoid the agent from learning to never trade. In early episodes, an agent might make losing trades, and realise taking actions cause punishment, and therefore decide to never trade. Simply adding a constant penalty for inaction might cause the agent to panic and trade erratically.

The solution is a progressive penalty that ramps up the longer the agent goes without trading. This gives the agent time to evaluate market conditions before the penalty becomes significant. Additionally, the penalty only applies when LGBM confidence exceeds a threshold (default 0.75). No penalty is applied during uncertain market conditions where inaction is genuinely the optimal decision.

Aside from this punishment, different rewards and punishments will be tested based on different behaviours that might be encouraged/discouraged. This will be configurable. Examples may include a reward for holding a rising position, or a quadratically scaled punishment for drawdown.

Normalisation

Neural networks require normalised inputs. However, this can subtly leak information under some conditions, though the effect is quite minor. During training, normalisation statistics (mean and standard deviation) are computed once using the entire training dataset. These statistics are then applied to normalise all data at the start. Since the agent will see the same episodes many times during training with added Gaussian noise for regularization, data leakage is not a concern. The agent needs consistent normalisation to learn stable patterns. The noise injection prevents overfitting by ensuring the agent never sees the exact same state twice.

During testing however, an expanding window normalization will be used to prevent any data leakage. A function can be used to recalculate normalization statistics using only data seen up to the current timestep. This ensures the agent cannot exploit future information during evaluation. The asymmetric normalization strategy is designed to ensure valid backtesting while maintaining training efficiency.

Configuration

Every hyperparameter is exposed in the DEFAULT_CONFIG dictionary, making the entire system highly configurable without code modification. This includes network architecture, fc_layers defaults, kernel_size, training parameters, reward shaping, and environment settings.

Key Algorithms

Listed are key algorithms, linked to the **objectives** which they fulfil. Details about what the code is doing can be found in the comments within the pseudocode.

Objective 2

Web Scraping Algorithm:

Fetches .json file from [REDACTED] API and processes it to return dates alongside articles relevant to the target. Also includes true sentiment if present.

```
1. FUNCTION fetch_articles(start_date, end_date, target_tags, page_size):
2.     since ← unix_timestamp(start_date)
3.     until ← unix_timestamp(end_date)
4.     page ← 1
5.     relevant_articles ← []
6.
7.     session ← create_http_session()
8.     session.headers ← load_authentication_cookies()
9.
10.    WHILE true DO:
11.        // 1.1.1: Dynamic URL construction
12.        url ← build_api_url(
13.            since=since,
14.            until=until,
15.            page=page,
16.            page_size=page_size,
17.            category=category
18.        )
19.
20.        response ← session.get(url)
21.
22.        // 1.1.2: HTTP error detection and handling
23.        IF response.status = 403 THEN:
24.            RETURN error("Access forbidden - IP blocked")
25.        ELSE IF response.status ≠ 200 THEN:
26.            RETURN error("HTTP error: " + response.status)
27.
28.        data ← parse_json(response.body)
29.        articles ← data["data"]
30.
31.        IF articles is empty THEN:
32.            BREAK
33.
34.        included ← data["included"]
35.        sentiments ← extract_sentiments(included, target_tags)
36.
37.        // 1.1.3: Filter on relevance
38.        FOR EACH article IN articles DO:
39.            IF has_target_tag(article, target_tags) THEN:
40.                title ← article.attributes.title
41.                date ← article.attributes.publishOn
42.                sentiment ← sentiments[article.id]
43.                relevant_articles.append((date, title, sentiment))
44.
45.        page ← page + 1
46.
47.        // 1.1.4: Rate limiting
48.        delay ← random_uniform(min_delay, max_delay)
49.        sleep(delay)
50.
51.    RETURN sort_by_date(relevant_articles)
52.
```

Tag-Based Article Filtering:

Filters to find only relevant articles based on the tag.

```
1. FUNCTION has_target_tag(article, target_tags):
2.   metadata ← article.relationships
3.
4.   FOR section IN ["primaryTickers", "secondaryTickers"] DO:
5.     tickers ← metadata[section].data
6.
7.     FOR EACH ticker IN tickers DO:
8.       IF ticker.id IN target_tags THEN:
9.         RETURN true
10.
11.   RETURN false
12.
```

OHLC Pulling:

Fetches OHLC via yfinance API.

```
1. FUNCTION fetch_prices(ticker, start_date, end_date, adjust):
2.   // Fetch via yfinance API
3.   data ← yfinance.download(
4.     ticker=ticker,
5.     start=start_date,
6.     end=end_date
7.   )
8.
9.   IF adjust = true THEN:
10.    // Use adjusted close prices for dividends/splits
11.    data["Close"] ← data["Adj Close"]
12.
13.    // Ensure datetime index
14.    data.index ← convert_to_datetime(data.index)
15.
16.   RETURN data // DataFrame with [Open, High, Low, Close, Volume]
17.
```

Objective 3

Sentiment Model with Temperature Scaling:

Leverages FinRoBERTa and applies temperature scaling to logits, as well as applying a custom scaling function for continuous polarity scores.

```
1. FUNCTION analyse_sentiment(text, bias_prompt, temperature): // 1.2.6: expose parameters
2.   // 1.2.3: Contextual biasing
3.   full_text ← bias_prompt + text
4.
5.   // 1.2.1: Pre-trained transformer integration
6.   inputs ← tokenizer(full_text, max_length=512, truncation=true)
7.
8.   WITH no_gradient_tracking DO:
9.     outputs ← transformer_model.forward(inputs)
10.    logits ← outputs.logits
11.
12.    // 1.2.4: Temperature scaling for calibration
```

```

13.     probs ← softmax(logits / temperature)
14.
15.     negative ← probs[0]
16.     neutral ← probs[1]
17.     positive ← probs[2]
18.
19.     // 1.2.2: Convert to continuous score
20.     raw_score ← positive - negative
21.
22.     // 1.2.5: Apply nonlinear scaling
23.     RETURN scale_score(raw_score, neutral)
24.

```

Nonlinear Neutrality Scaling Function:

Empirically derived formula for scaling, found by testing graph shapes on Desmos.

```

1. FUNCTION scale_score(score, neutral, neutrality_scaler): //1.2.5: Nonlinear scaling
2.     // Preserve sign
3.     sign ← sgn(score)
4.     x ← |score|
5.
6.     // Custom polynomial to reduce extreme values based on neutrality
7.     // Formula:  $f(x) = (x-0.5) - k(x-0.5)^3 + 4k(x-0.5)^5 + 0.5$ 
8.     // where  $k = \text{neutrality\_scaler}$ 
9.
10.    shifted ← x - 0.5
11.    k ← neutrality_scaler
12.
13.    scaled ← shifted - k × shifted3 + 4k × shifted5 + 0.5
14.
15.    RETURN sign × scaled
16.

```

Objective 4

Included below are well-known methods of calculating technical indicators, as well as some statistical features for machine learning.

Bollinger Bands:

Calculates an average of recent closing prices and uses the standard deviation to create upper and lower bounds around this average. This summarises price level and recent variation in a single feature, giving the model contextual information about how “unusual” the current price movement is.

```

1. FUNCTION calculate_bollinger_bands(close_prices, period, num_std):
2.     // Middle band is simple moving average
3.     middle_band ← []
4.     FOR i ← period TO length(close_prices) DO:
5.         window ← close_prices[i-period:i]
6.         middle_band[i] ← mean(window)
7.
8.     // Standard deviation calculation
9.     std_dev ← []
10.    FOR i ← period TO length(close_prices) DO:
11.        window ← close_prices[i-period:i]
12.        std_dev[i] ← standard_deviation(window)
13.

```

```

14.    // Upper and lower bands
15.    upper_band ← middle_band + (num_std × std_dev)
16.    lower_band ← middle_band - (num_std × std_dev)
17.
18.    RETURN (upper_band, middle_band, lower_band)
19.

```

Moving Average Convergence Divergence:

Calculates two exponential moving averages of different lengths and uses their difference to measure how recent prices are changing relative to longer-term trends. This captures the direction and strength of price movement over time, providing the model with information about whether prices are accelerating or slowing down.

```

1. FUNCTION calculate_macd(close_prices, fast_period, slow_period, signal_period):
2.    // Calculate EMAs
3.    ema_fast ← exponential_moving_average(close_prices, fast_period)
4.    ema_slow ← exponential_moving_average(close_prices, slow_period)
5.
6.    // MACD line
7.    macd_line ← ema_fast - ema_slow
8.
9.    // Signal line
10.   signal_line ← exponential_moving_average(macd_line, signal_period)
11.
12.   // Histogram
13.   histogram ← macd_line - signal_line
14.
15.   RETURN (macd_line, signal_line, histogram)
16.

```

Relative Strength Index:

Measures recent price changes by comparing average gains and losses over a fixed period and scales the result to a value between 0 and 100. This provides a normalised measure of short-term price momentum, allowing the model to distinguish between relatively strong and weak price movements.

```

1. FUNCTION calculate_rsi(close_prices, period):
2.    // Calculate price changes
3.    deltas ← []
4.    FOR i ← 1 TO length(close_prices) - 1 DO:
5.        deltas[i] ← close_prices[i] - close_prices[i-1]
6.
7.    // Separate gains and losses
8.    gains ← [max(0, d) FOR d IN deltas]
9.    losses ← [max(0, -d) FOR d IN deltas]
10.
11.   // Exponential moving averages
12.   avg_gain ← exponential_moving_average(gains, period)
13.   avg_loss ← exponential_moving_average(losses, period)
14.
15.   // Relative strength
16.   rs ← avg_gain / avg_loss
17.
18.   // RSI formula
19.   rsi ← 100 - (100 / (1 + rs))

```

```

20.
21.     RETURN rsi
22.

```

Stochastic Oscillator:

Compares the current closing price to the recent high–low range over a fixed window and then smooths the result using moving averages. This shows where the current price sits relative to its recent range, giving the model contextual information about short-term price behaviour rather than absolute price values.

```

1. FUNCTION calculate_stochastic(high_prices, low_prices, close_prices, k_period, d_period,
smooth_k):
2.     // Find lowest low and highest high over k_period
3.     lowest_low ← []
4.     highest_high ← []
5.
6.     FOR i ← k_period TO length(close_prices) DO:
7.         window_low ← low_prices[i-k_period:i]
8.         window_high ← high_prices[i-k_period:i]
9.
10.        lowest_low[i] ← min(window_low)
11.        highest_high[i] ← max(window_high)
12.
13.        // Calculate raw %K
14.        raw_k ← []
15.        FOR i ← k_period TO length(close_prices) DO:
16.            range ← highest_high[i] - lowest_low[i]
17.            IF range = 0 THEN:
18.                raw_k[i] ← 0
19.            ELSE:
20.                raw_k[i] ← ((close_prices[i] - lowest_low[i]) / range) × 100
21.
22.        // Smooth %K
23.        percent_k ← simple_moving_average(raw_k, smooth_k)
24.
25.        // Calculate %D (signal line)
26.        percent_d ← simple_moving_average(percent_k, d_period)
27.        RETURN (percent_k, percent_d)

```

Average True Range:

Measures how much the price changes between time steps by considering both the daily range and gaps from the previous closing price, then averaging this over a fixed period. This provides a scale-independent measure of price variation, helping the model account for how volatile the market is rather than just the direction of movement.

```

1. FUNCTION calculate_atr(high_prices, low_prices, close_prices, period):
2.     // Calculate True Range for each day
3.     true_range ← []
4.
5.     FOR i ← 1 TO length(close_prices) - 1 DO:
6.         high_low ← high_prices[i] - low_prices[i]
7.         high_close ← |high_prices[i] - close_prices[i-1]|
8.         low_close ← |low_prices[i] - close_prices[i-1]|
9.
10.        true_range[i] ← max(high_low, high_close, low_close)
11.
12.        // Average True Range is the moving average of TR
13.        atr ← simple_moving_average(true_range, period)
14.
15.        RETURN atr
16.

```

Statistical Volatility:

This is a derived statistical feature rather than a traditional technical indicator. It calculates percentage price returns and then measures how much these returns vary over different time windows using standard deviation. This directly quantifies recent price variation at multiple timescales, giving the model explicit information about how stable or unstable price movements have been.

```
1. FUNCTION calculate_volatility(close_prices, windows):
2.   volatility_features ← {}
3.
4.   // Calculate returns
5.   returns ← []
6.   FOR i ← 1 TO length(close_prices) - 1 DO:
7.     returns[i] ← (close_prices[i] - close_prices[i-1]) / close_prices[i-1]
8.
9.   // Calculate volatility for each window
10.  FOR EACH window IN windows DO:
11.    vol ← []
12.    FOR i ← window TO length(returns) DO:
13.      window_returns ← returns[i-window:i]
14.      vol[i] ← standard_deviation(window_returns)
15.
16.    volatility_features[f"Vol_{window}d"] ← vol
17.
18.  // Volatility ratio (short-term to medium-term)
19.  short_window ← min(windows)
20.  medium_window ← median(windows)
21.
22.  volatility_features["Vol_Ratio"] ←
23.    volatility_features[f"Vol_{short_window}d"] /
24.    volatility_features[f"Vol_{medium_window}d"]
25.
26.  RETURN volatility_features
27.
```

Gap Features:

Calculates the difference between the opening price and previous closing prices, including a normalised version using a moving average. It was included because a research paper⁷ on LightGBM for assets forecasting showed that it can be useful for predicting short-term price movements, providing the model with information about sudden changes between time steps.

```
1. FUNCTION calculate_gap_features(open_prices, close_prices):
2.   // Previous close
3.   prev_close ← shift(close_prices, 1)
4.
5.   // Gap calculation (opening gap from previous close)
6.   gap ← (open_prices - prev_close) / prev_close
7.
8.   // EMA for normalization
9.   ema_14 ← exponential_moving_average(close_prices, 14)
10.
11.  // Gap normalized by EMA (highly ranked feature from research paper)8
12.  gap_ema_norm ← (open_prices - prev_close) / ema_14
13.
```

⁸ See references: (Bisdoulis, 2024)

```

14.    // 2-day gap
15.    prev_close_2d ← shift(close_prices, 2)
16.    gap_2d ← (open_prices - prev_close_2d) / ema_14
17.
18.    RETURN (gap, gap_ema_norm, gap_2d)
19.

```

EMA Ratios:

Calculates ratios between exponential moving averages of different lengths, as well as the ratio of the current price to the shortest EMA. This captures the relative position of recent prices to different trend scales, providing the model with information about momentum and short-term versus longer-term trends.

```

1. FUNCTION calculate_ema_ratios(close_prices, ema_lengths):
2.     ema_features ← {}
3.
4.     // Calculate all EMAs
5.     emas ← {}
6.     FOR EACH length IN ema_lengths DO:
7.         emas[length] ← exponential_moving_average(close_prices, length)
8.
9.     // EMA cross ratios (short EMA / long EMA)
10.    FOR i ← 0 TO length(ema_lengths) - 2 DO:
11.        short ← ema_lengths[i]
12.        long ← ema_lengths[i+1]
13.
14.        ema_features[f"EMA_Ratio_{short}_{long}"] ← emas[short] / emas[long]
15.
16.    // Price to EMA ratio
17.    reference_ema ← min(ema_lengths)
18.    ema_features["Price_EMA_Ratio"] ← close_prices / emas[reference_ema]
19.
20.    RETURN ema_features
21.

```

Price and Indicator Slopes with Divergences:

This feature set calculates the rate of change (slope) of the closing price, MACD, RSI, and volume over different windows, and then measures the divergence between the price slope and each of the other indicators. This captures short-term trends and discrepancies between price movements and technical indicators, giving the model additional context about momentum and potential reversals. It was also a highly ranked feature in the aforementioned research paper⁹.

```

1. FUNCTION calculate_slopes_and_divergences(close_prices, macd, rsi, volume, windows):
2.     slope_features ← {}
3.
4.     FOR EACH window IN windows DO:
5.         // Price slope (rate of change)
6.         price_slope ← (close_prices - shift(close_prices, window)) / window
7.         slope_features[f"Price_Slope_{window}d"] ← price_slope
8.
9.         // MACD slope
10.        macd_slope ← (macd - shift(macd, window)) / window
11.        slope_features[f"MACD_Slope_{window}d"] ← macd_slope
12.
13.        // MACD divergence (price slope vs MACD slope)
14.        slope_features[f"MACD_Slope_Div_{window}d"] ← price_slope - macd_slope

```

⁹ (Bisdoulis, 2024)

```

15.
16.      // RSI slope divergence
17.      rsi_slope ← (rsi - shift(rsi, window)) / window
18.      slope_features[f"RSI_Slope_Div_{window}d"] ← price_slope - rsi_slope
19.
20.      // Volume slope divergence
21.      vol_slope ← (volume - shift(volume, window)) / window
22.      slope_features[f"Vol_Slope_Div_{window}d"] ← price_slope - vol_slope
23.
24.      RETURN slope_features
25.

```

Psychological Line:

This feature calculates the percentage of days in a given period where the closing price increased compared to the previous day. This gives a simple measure of market sentiment, indicating whether recent price movements are mostly upwards or downwards, which can help the model identify short-term momentum trends.

```

1. FUNCTION calculate_psy(close_prices, periods):
2.   psy_features ← {}
3.
4.   // Calculate daily price changes
5.   price_changes ← []
6.   FOR i ← 1 TO length(close_prices) - 1 DO:
7.     price_changes[i] ← close_prices[i] - close_prices[i-1]
8.
9.   // Calculate PSY for each period
10.  FOR EACH period IN periods DO:
11.    psy ← []
12.
13.    FOR i ← period TO length(price_changes) DO:
14.      window ← price_changes[i-period:i]
15.
16.      // Count up days
17.      up_days ← count(window > 0)
18.
19.      // PSY is percentage of up days
20.      psy[i] ← (up_days / period) × 100
21.
22.      psy_features[f"PSY_{period}"] ← psy
23.
24.  RETURN psy_features
25.

```

Commodity Channel Index:

This feature calculates the deviation of the typical price (average of high, low, and close) from its moving average, scaled by the mean absolute deviation. This measures how far prices deviate from recent trends, providing the model with information about overbought or oversold conditions and short-term price extremes.

```

1. FUNCTION calculate_cci(high_prices, low_prices, close_prices, periods):
2.   cci_features ← {}
3.
4.   FOR EACH period IN periods DO:
5.     // Typical price
6.     typical_price ← (high_prices + low_prices + close_prices) / 3
7.

```

```

8.      // Simple moving average of typical price
9.      sma_tp ← simple_moving_average(typical_price, period)
10.
11.     // Mean absolute deviation
12.     mad ← []
13.     FOR i ← period TO length(typical_price) DO:
14.         window ← typical_price[i-period:i]
15.         mean_val ← mean(window)
16.
17.         // Calculate absolute deviations
18.         deviations ← [|x - mean_val| FOR x IN window]
19.         mad[i] ← mean(deviations)
20.
21.     // CCI formula (0.015 is Lambert's constant)
22.     cci ← (typical_price - sma_tp) / (0.015 × mad)
23.
24.     cci_features[f"CCI_{period}"] ← cci
25.
26.     RETURN cci_features
27.

```

Chande Momentum Oscillator:

This feature calculates the difference between the sum of gains and the sum of losses over a given period, scaled to a value between -100 and 100. This quantifies short-term momentum, indicating whether recent price changes are mostly upwards or downwards, which helps the model identify trend strength and potential reversals.

```

1. FUNCTION calculate_cmo(close_prices, periods):
2.     cmo_features ← {}
3.
4.     // Calculate price changes
5.     deltas ← []
6.     FOR i ← 1 TO length(close_prices) - 1 DO:
7.         deltas[i] ← close_prices[i] - close_prices[i-1]
8.
9.     FOR EACH period IN periods DO:
10.        cmo ← []
11.
12.        FOR i ← period TO length(deltas) DO:
13.            window ← deltas[i-period:i]
14.
15.            // Separate gains and losses
16.            gains ← [max(0, d) FOR d IN window]
17.            losses ← [max(0, -d) FOR d IN window]
18.
19.            // Sum of gains and losses
20.            sum_gains ← sum(gains)
21.            sum_losses ← sum(losses)
22.
23.            // CMO formula
24.            IF sum_gains + sum_losses = 0 THEN:
25.                cmo[i] ← 0
26.            ELSE:
27.                cmo[i] ← 100 × (sum_gains - sum_losses) / (sum_gains + sum_losses)
28.
29.        cmo_features[f"CMO_{period}"] ← cmo
30.
31.    RETURN cmo_features
32.

```

Parameter Override:

Allows a default configuration to have parameters overridden by a config passed by a user.

```

1. FUNCTION initialize_indicator_calculator(params):
2.     // Default parameters for all indicators
3.     default_params ← {
4.         "bb_period": 20,
5.         "bb_std": 2,
6.         "rsi_period": 14,
7.         "macd_fast": 12,
8.         "macd_slow": 26,
9.         "macd_signal": 9,
10.        "ema_periods": [10, 20, 50, 200],
11.        "atr_period": 14,
12.        "stoch_period": 14
13.    }
14.
15.    // Override with user-provided parameters
16.    FOR EACH key, value IN params DO:
17.        IF key IN default_params THEN:
18.            default_params[key] ← value
19.
20.    calculator.params ← default_params
21.    RETURN calculator

```

Exponential Moving Average:

This feature calculates a weighted average of past values where more recent data points have greater influence. This smooths out short-term fluctuations while still responding to recent price changes, providing the model with a measure of the underlying trend.

```

1. FUNCTION exponential_moving_average(values, period):
2.     // Calculate smoothing factor
3.     alpha ← 2 / (period + 1)
4.
5.     ema ← []
6.     ema[0] ← values[0] // First value is seed
7.
8.     // Calculate EMA iteratively
9.     FOR i ← 1 TO length(values) - 1 DO:
10.        ema[i] ← alpha × values[i] + (1 - alpha) × ema[i-1]
11.
12.     RETURN ema
13.

```

Objective 5

Averaging Sentiment and Decaying Missing Values:

Gets a single score for each day of sentiment, and decays previous sentiment when there is none. This reflects the idea that sentiment does not instantly vanish, allowing the model to maintain a realistic representation of ongoing market sentiment even on days with missing input.

```

1. FUNCTION apply_exponential_decay(sentiment_series, decay_factor):
2.     filled ← []
3.     last_value ← 0.0 // Neutral starting point
4.
5.     FOR EACH date, value IN sentiment_series DO:
6.         IF value is not NaN THEN:
7.             last_value ← value
8.         ELSE:
9.             // Decay previous sentiment
10.            last_value ← last_value × decay_factor
11.
12.     filled.append(last_value)

```

```

13.
14.     RETURN filled
15.

```

Target Label Generation:

Generates a variety of targets for LightGBM to predict.

```

1. FUNCTION compute_targets(dataset, config):
2.     trading_idx ← indices where dataset["is_trading_day"] == TRUE
3.     close_prices ← dataset["Close"] restricted to trading_idx
4.
5.
6.     // Simple percentage change targets (percent)
7.
8.     FOR EACH h IN config.pct_change_targets DO:
9.         future_close ← close_prices.shift(-h)
10.        target ← (future_close / close_prices - 1) × 100
11.        dataset["Future_Pct_Change_" + h + "d"] at trading_idx ← target
12.
13.
14.     // Simple return targets (for baseline)
15.
16.     FOR EACH h IN config.return_targets DO:
17.         future_close ← close_prices.shift(-h)
18.         target ← future_close / close_prices - 1
19.         dataset["Future_Return_" + h + "d"] at trading_idx ← target
20.
21.
22.     // 3. Volatility-adjusted return targets
23.
24.     returns ← close_prices.pct_change()
25.     volatility ← rolling_std(returns, window=20, min_periods=10)
26.     volatility_safe ← replace zeros in volatility with NaN
27.
28.     FOR EACH h IN config.vol_adjusted_targets DO:
29.         raw_return ← close_prices.shift(-h) / close_prices - 1
30.         target ← raw_return / volatility_safe
31.         dataset["Target_VolAdjusted_" + h + "d"] at trading_idx ← target
32.
33.
34.     // EMA-relative positioning targets
35.
36.     FOR EACH h IN config.ema_targets DO:
37.         ema ← exponential_moving_average(close_prices, span=h)
38.         future_close ← close_prices.shift(-h)
39.         target ← future_close / ema - 1
40.         dataset["Target_EMA_Position_" + h + "d"] at trading_idx ← target
41.
42.
43.     // Normalised return targets (time-scaled risk adjustment)
44.
45.     returns ← close_prices.pct_change()
46.     volatility ← rolling_std(returns, window=20, min_periods=10)
47.     volatility_safe ← replace zeros in volatility with NaN
48.
49.     FOR EACH h IN config.normalised_return_targets DO:
50.         raw_return ← close_prices.shift(-h) / close_prices - 1
51.         target ← raw_return / (volatility_safe × sqrt(h))
52.         dataset["Target_Normalised_Return_" + h + "d"] at trading_idx ← target
53.
54.     RETURN dataset
55.

```

Combining Features into a Unified Dataset:

Merge features into one environment for machine learning.

```
1. FUNCTION generate_dataset(ohlc_data, sentiment_data, indicators):
2.     // Create date range covering all data
3.     full_index <- date_range(start_date, end_date)
4.
5.     // Initialize dataset
6.     dataset <- DataFrame(index=full_index)
7.
8.     // Join OHLC data
9.     dataset <- dataset.join(ohlc_data)
10.
11.    // Mark trading days
12.    dataset["is_trading_day"] <- NOT is_null(dataset["Close"])
13.
14.    // Add sentiment (reindex and forward-fill)
15.    dataset["Sentiment"] <- sentiment_data.reindex(full_index).ffill().fillna(0.0)
16.
17.    // Forward-fill OHLC for continuity
18.    dataset[["Open", "High", "Low", "Close"]] <- forward_fill(dataset[["Open", "High", "Low",
19.    "Close"]])
20.
21.    // Add technical indicators
22.    dataset <- dataset.join(indicators)
23.
24.    // Compute features and targets
25.    dataset <- compute_features_and_targets(dataset)
26.
27.    // Filter to trading days only
28.    dataset <- dataset[dataset["is_trading_day"] = true]
29.
30.    // Remove rows with missing features
31.    dataset <- dataset.dropna(subset=feature_columns)
32.
33.    RETURN dataset
```

Objective 6

Quantile Regression Training:

Fits a LightGBM model for each quantile on the training data.

```
1. FUNCTION train_quantile_models(X_train, y_train, quantiles, lgbm_params):
2.     quantile_models <- {}
3.
4.     FOR EACH q IN quantiles DO:
5.         // Configure for quantile regression
6.         params <- copy(lgbm_params)
7.         params["objective"] <- "quantile"
8.         params["alpha"] <- q
9.
10.        // Train model
11.        model <- LGBMRegressor(**params)
12.        model.fit(X_train, y_train)
13.
14.        quantile_models[q] <- model
15.
16.    RETURN quantile_models
17.
18.
19. FUNCTION predict_quantiles(quantile_models, X):
20.     predictions <- {}
21.
```

```

22.     FOR EACH q, model IN quantile_models DO:
23.         predictions[q] ← model.predict(X)
24.
25.     RETURN predictions

```

Conformal Prediction Calibration:

Adjusts model predictions to provide reliable uncertainty estimates via a calibration set, in this case a small cut from the training data. For each quantile, it calculates the typical difference between predicted and actual values and then applies this as a correction to future predictions.

A conformal correction is saved, which can later be applied to future predictions.

```

1. FUNCTION calibrate_conformal(quantile_models, X_cal, y_cal, quantiles):
2.     corrections ← {}
3.
4.     FOR EACH q IN quantiles DO:
5.         // Get predictions on calibration set
6.         y_pred ← quantile_models[q].predict(X_cal)
7.
8.         // Calculate residuals (actual - predicted)
9.         residuals ← y_cal - y_pred
10.
11.        // Find q-th quantile of residuals
12.        // This gives the correction needed for proper coverage
13.        correction ← quantile(residuals, q)
14.
15.        corrections[q] ← correction
16.
17.    RETURN corrections
18.
19.
20. FUNCTION predict_with_conformal(quantile_models, conformal_corrections, X):
21.     predictions ← {}
22.
23.     FOR EACH q IN quantiles DO:
24.         base_prediction ← quantile_models[q].predict(X)
25.
26.         // Apply conformal correction
27.         predictions[q] ← base_prediction + conformal_corrections[q]
28.
29.     RETURN predictions
30.

```

Walk-Forward Training and Prediction, with Cascade Support for Pretrained Models:

This algorithm is found within the dataset generator rather than LightGBM, allowing for several models to be trained together so that the DQN has predictive meta-features when training.

```

1. FUNCTION add_lgbm_predictions(dataset, config, optional_pretrained_models):
2.
3.
4.     // Case 1: Use pre-trained models
5.
6.     IF optional_pretrained_models is provided THEN:
7.
8.         models_by_horizon ← load all models
9.
10.        // models will have cascade order stored with them
11.        cascade_order ← models_by_horizon.cascade_order
12.
13.        FOR EACH horizon IN horizons ordered by cascade_order DO:
14.            forecaster ← model for horizon

```

```

15.         feature_cols ← model.feature_columns
16.         quantiles ← model.quantiles
17.
18.         valid_rows ← rows where all feature_cols are non-null
19.         IF no valid_rows:
20.             CONTINUE
21.
22.         predictions ← forecaster.predict(dataset[valid_rows, feature_cols])
23.
24.         FOR EACH q IN quantiles DO:
25.             col_name ← "LGBM_" + horizon + "d_q" + (q × 100)
26.             dataset[col_name][valid_rows] ← predictions[q]
27.
28.     RETURN dataset
29.
30.
31.
32.     // Case 2: Train models from scratch
33.
34.     FOR EACH horizon IN config.horizons DO:
35.
36.         // select target column based on config
37.         target_col ← resolve_target_column(config.target_type, horizon)
38.         IF target_col not in dataset:
39.             CONTINUE
40.
41.         // feature set excludes all target columns
42.         feature_cols ← all columns NOT starting with "Target" or "Future"
43.
44.         mark rows with valid target as has_target
45.         IF count(has_target) < config.train_min:
46.             CONTINUE
47.
48.         initialize LGBM prediction columns for this horizon
49.
50.         current_idx ← 0
51.         days_since_train ← config.retrain
52.
53.         WHILE current_idx < length(dataset) DO:
54.
55.
56.             // Retrain model if needed
57.
58.             IF days_since_train ≥ config.retrain OR model not yet trained THEN:
59.
60.                 train_end ← current_idx
61.                 train_start ← max(0, train_end - config.train_max)
62.
63.                 training_slice ← dataset[train_start : train_end]
64.                 training_data ← rows in training_slice where has_target == TRUE
65.
66.                 IF size(training_data) < config.train_min:
67.                     advance current_idx by retrain window
68.                     CONTINUE
69.
70.                 X_train ← training_data[feature_cols]
71.                 y_train ← training_data[target_col]
72.
73.                 train LGBM forecaster
74.                 optionally apply conformal calibration
75.
76.                 days_since_train ← 0
77.
78.
79.             // Predict next batch
80.             batch_end ← min(current_idx + config.retrain, length(dataset))
81.             X_pred ← dataset[current_idx : batch_end][feature_cols]
82.
83.             predictions ← forecaster.predict(X_pred)

```

```

84.
85.         IF prediction_mode == quantile:
86.             FOR EACH q IN quantiles DO:
87.                 dataset["LGBM_" + horizon + "d_q" + (q × 100)]
88.                     [current_idx : batch_end] ← predictions[q]
89.         ELSE:
90.             dataset["LGBM_" + horizon + "d"]
91.                 [current_idx : batch_end] ← predictions
92.
93.         days_since_train += (batch_end - current_idx)
94.         current_idx ← batch_end
95.
96.         remove temporary target-validity markers
97.
98.     drop any LGBM columns that contain only NaNs
99.     RETURN dataset

```

Objective 7

Extra Features for Observation Space:

Other than the dataset features, more observations are generated, such as LightGBM's current accuracy (hit rate) as well as the days remaining so it does not blow up its capital buying a dip that will never recover by the end of the episode. It also has an observation of its own positions so it has an idea of how much risk it is taking.

```

1. FUNCTION get_state(current_step, lookback_window):
2.     // Extract historical features
3.     start ← current_step - lookback_window
4.     end ← current_step
5.
6.     state ← normalized_features[start:end]
7.
8.     // Add LGBM prediction features
9.     lgbm_features ← extract_lgbm_features(current_step)
10.
11.    // Add episode hit rate
12.    hit_rate ← calculate_lgbm_hit_rate()
13.    hit_rate_expanded ← repeat(hit_rate, lookback_window)
14.
15.    // Add days remaining (for time awareness)
16.    days_remaining ← (episode_length - current_step) / episode_length
17.    days_expanded ← repeat(days_remaining, lookback_window)
18.
19.    // Add position information
20.    position_info ← [
21.        has_positions,
22.        num_positions / max_positions,
23.        position_value / portfolio_value,
24.        unrealized_pnl / initial_balance,
25.        (current_price - avg_entry_price) / avg_entry_price
26.    ]
27.    position_expanded ← repeat(position_info, lookback_window)
28.
29.    // Concatenate all features
30.    state ← concatenate([
31.        state,
32.        lgbm_features,
33.        hit_rate_expanded,
34.        days_expanded,
35.        position_expanded
36.    ], axis=1)
37.
38.    RETURN state
39.

```

```

40.
41. FUNCTION extract_lgbm_features(current_step):
42.     // Separate short-term and long-term horizons
43.     short_term_preds ← []
44.     long_term_preds ← []
45.
46.     FOR EACH horizon IN horizons DO:
47.         pred ← lgbm_predictions[horizon][current_step]
48.
49.         IF horizon ≤ split_threshold THEN:
50.             short_term_preds.append(pred)
51.         ELSE:
52.             long_term_preds.append(pred)
53.
54.     // Aggregate features
55.     features ← []
56.
57.     IF short_term_preds not empty THEN:
58.         features.append(mean(short_term_preds)) // Average prediction
59.         features.append(std(short_term_preds)) // Uncertainty
60.         features.append(mean(|short_term_preds|)) // Strength
61.         features.append(mean(sign(short_term_preds))) // Consensus
62.
63.     IF long_term_preds not empty THEN:
64.         features.append(mean(long_term_preds))
65.         features.append(std(long_term_preds))
66.         features.append(mean(|long_term_preds|))
67.         features.append(mean(sign(long_term_preds)))
68.
69.     RETURN features
70.

```

DQN Training Environment Step Function:

Defines how the environment changes with each action taken by the DQN.

```

1. FUNCTION step(action):
2.     current_price ← get_current_price()
3.     reward ← 0
4.
5.     // ACTION 0: Close all positions
6.     IF action = 0 THEN:
7.         IF has_positions() THEN:
8.             capital_returned, total_pnl ← close_all_positions(current_price)
9.
10.            transaction_cost ← capital_returned × transaction_cost_rate
11.            balance ← balance + capital_returned - transaction_cost
12.
13.            realized_pnl ← total_pnl - transaction_cost
14.            cumulative_realized_pnl ← cumulative_realized_pnl + realized_pnl
15.
16.            // Reward based on profitability
17.            IF realized_pnl > 0 THEN:
18.                reward ← reward_profit_scale × (realized_pnl / initial_balance)
19.            ELSE:
20.                loss_pct ← |realized_pnl / initial_balance| × 100
21.                reward ← -punish_loss_scale × loss_pct2
22.        ELSE:
23.            // Apply indecision penalty
24.            indecision_steps ← indecision_steps + 1
25.            penalty_progress ← min(indecision_steps / ramp_steps, 1.0)
26.            indecision_penalty ← max_penalty × penalty_progress
27.            reward ← reward - indecision_penalty
28.
29.    // ACTION 1: Hold
30.    ELSE IF action = 1 THEN:
31.        IF has_positions() THEN:

```

```

32.         // Step-based PnL reward
33.         current_pnl ← get_total_pnl(current_price)
34.         pnl_change ← current_pnl - prev_unrealized_pnl
35.
36.         IF pnl_change > 0 THEN:
37.             reward ← reward + reward_rising_hold × (pnl_change / initial_balance)
38.         ELSE:
39.             reward ← reward + punish_falling_hold × (pnl_change / initial_balance)
40.
41.         prev_unrealized_pnl ← current_pnl
42.     ELSE:
43.         // Indecision penalty
44.         indecision_steps ← indecision_steps + 1
45.
46.     // ACTIONS 2-4: Pyramid positions
47.     ELSE IF action ≥ 2 THEN:
48.         pyramid_size ← pyramid_sizes[action - 2]
49.         capital_to_allocate ← balance × pyramid_size
50.
51.         IF capital_to_allocate > 10 AND num_positions < max_positions THEN:
52.             transaction_cost ← capital_to_allocate × transaction_cost_rate
53.             total_cost ← capital_to_allocate + transaction_cost
54.
55.             IF total_cost ≤ balance THEN:
56.                 add_position(current_price, capital_to_allocate)
57.                 balance ← balance - total_cost
58.
59.                 reward ← reward - opening_penalty
60.                 indecision_steps ← 0
61.
62.         current_step ← current_step + 1
63.
64.         // Update metrics
65.         update_lgbm_hit_rate()
66.
67.         position_value ← get_total_value(current_price)
68.         current_balance ← balance + position_value
69.         balance_history.append(current_balance)
70.
71.         realized_balance ← initial_balance + cumulative_realized_pnl
72.         realized_balance_history.append(realized_balance)
73.
74.         // Check for bankruptcy
75.         IF current_balance ≤ 0 THEN:
76.             done ← true
77.             reward ← -100
78.             RETURN (null, reward, done, info)
79.
80.         // Check for episode completion
81.         IF current_step ≥ episode_length THEN:
82.             done ← true
83.
84.         // Close remaining positions
85.         IF has_positions() THEN:
86.             capital_returned, total_pnl ← close_all_positions(current_price)
87.             balance ← balance + capital_returned - transaction_costs
88.             cumulative_realized_pnl ← cumulative_realized_pnl + (total_pnl -
transaction_costs)
89.
90.         // Calculate Sharpe ratio bonus
91.         realized_returns ← diff(realized_balance_history) / realized_balance_history[:-1]
92.         IF length(realized_returns) > 1 AND std(realized_returns) > 0 THEN:
93.             sharpe_ratio ← (mean(realized_returns) / std(realized_returns)) × √252
94.             sharpe_clipped ← clip(sharpe_ratio, sharpe_min, sharpe_max)
95.
96.             reward ← reward + reward_sharpe_bonus × sharpe_clipped
97.
98.         // Clip reward for stability
99.         reward ← clip(reward, reward_clip_min, reward_clip_max)

```

```

100.
101.     next_state ← get_state() IF NOT done ELSE null
102.
103.     RETURN (next_state, reward, done, info)
104.

```

Sharpe Ratio Calculation:

This algorithm measures the risk-adjusted performance of a trading strategy by comparing the average return to the variation in returns over time. The calculation scales daily returns to a yearly measure, providing a standardised way to evaluate how much return is achieved per unit of risk. It is a useful metric for evaluating a trained DQN Agent.

```

1. FUNCTION calculate_sharpe_ratio(balance_history:
2.     // Calculate returns
3.     returns ← []
4.     FOR i ← 1 TO length(balance_history) - 1 DO:
5.         r ← (balance_history[i] - balance_history[i-1]) / balance_history[i-1]
6.         returns.append(r)
7.
8.     IF length(returns) < 2 OR std(returns) = 0 THEN:
9.         RETURN 0
10.
11.    // Sharpe = (avg return / std return) × √252 (periods over a year of trading daily)
12.    avg_return ← mean(returns)
13.    std_return ← std(returns)
14.
15.    sharpe ← (avg_return / std_return) × sqrt(252)
16.
17.    RETURN sharpe
18.

```

Max Drawdown Calculation:

Calculates the largest drop from a peak in the agent's capital over time. This provides a measure of downside risk, showing how much the agent's capital can decrease during a losing period, which complements return-based metrics like the Sharpe ratio.

```

1. FUNCTION calculate_max_drawdown(balance_history):
2.     peak ← balance_history[0]
3.     max_dd ← 0
4.
5.     FOR EACH balance IN balance_history DO:
6.         IF balance > peak THEN:
7.             peak ← balance
8.
9.         drawdown ← (peak - balance) / peak
10.
11.        IF drawdown > max_dd THEN:
12.            max_dd ← drawdown
13.
14.    RETURN max_dd × 100 // As percentage
15.

```

Buy & Hold Baseline Comparison:

Calculates the performance of a simple buy-and-hold strategy as a baseline to evaluate whether the DQN agent's active trading adds value. It simulates purchasing the asset at the episode start and holding until the end, providing a comparison point for the agent's realized returns.

DQN Training Step:

How the agent learns and updates its network based on experiences.

```
1. FUNCTION train_step(agent):
2.   IF length(replay_buffer) < batch_size THEN:
3.     RETURN null
4.
5.   // Sample random batch from experience replay
6.   states, actions, rewards, next_states, dones ← replay_buffer.sample(batch_size)
7.
8.   // Compute current Q-values
9.   current_q_values ← policy_network.forward(states)
10.  current_q ← gather(current_q_values, actions)
11.
12.  // Compute target Q-values using target network
13.  WITH no_gradient_tracking DO:
14.    next_q_values ← target_network.forward(next_states)
15.    max_next_q ← max(next_q_values, axis=1)
16.
17.    // Bellman equation:  $Q(s,a) = r + \gamma \times \max Q(s',a')$ 
18.    target_q ← rewards + (1 - dones) × gamma × max_next_q
19.
20.  // Compute loss
21.  loss ← mean_squared_error(current_q, target_q)
22.
23.  // Backpropagation
24.  optimizer.zero_grad()
25.  loss.backward()
26.  clip_gradient_norm(policy_network.parameters, max_norm=1.0)
27.  optimizer.step()
28.
29.  RETURN loss
30.
31.
32. FUNCTION soft_update_target_network(policy_net, target_net, tau):
33.   // Soft update:  $\theta_{\text{target}} \leftarrow \tau \times \theta_{\text{policy}} + (1-\tau) \times \theta_{\text{target}}$ 
34.   FOR policy_param, target_param IN zip(policy_net.params, target_net.params) DO:
35.     target_param ← tau × policy_param + (1 - tau) × target_param
36.
37.
38. FUNCTION select_action(state, epsilon, num_actions):
39.   // Epsilon-greedy exploration
40.   IF random() < epsilon THEN:
41.     RETURN random_integer(0, num_actions - 1)
42.   ELSE:
43.     WITH no_gradient_tracking DO:
44.       q_values ← policy_network.forward(state)
45.       RETURN argmax(q_values)
46.
```

Convolutional Neural Network:

Processes input with 1D convolutions to extract temporal features, aggregates them, and maps them to Q-values for actions.

```
1. FUNCTION forward(state):
2.   // state shape: (batch, timesteps, features)
3.
4.   // Transpose for Conv1D (batch, features, timesteps)
5.   x ← transpose(state, dimensions=[0, 2, 1])
6.
7.   // Convolutional layers with layer normalization
```

```

8.    x ← conv1d(x, filters=64, kernel_size=3, padding=1)
9.    x ← layer_norm(x)
10.   x ← relu(x)
11.
12.   x ← conv1d(x, filters=128, kernel_size=3, padding=1)
13.   x ← layer_norm(x)
14.   x ← relu(x)
15.
16.   x ← conv1d(x, filters=64, kernel_size=3, padding=1)
17.   x ← layer_norm(x)
18.   x ← relu(x)
19.
20.   // Global average pooling
21.   x ← adaptive_avg_pool(x, output_size=1)
22.   x ← squeeze(x, dimension=-1)
23.
24.   // Fully connected layers
25.   x ← linear(x, output_size=256)
26.   x ← relu(x)
27.
28.   x ← linear(x, output_size=128)
29.   x ← relu(x)
30.
31.   // Output layer (Q-values for each action)
32.   q_values ← linear(x, output_size=num_actions)
33.
34.   RETURN q_values
35.

```

Circular Queue Replay Buffer:

Stores past experiences in a fixed-size circular buffer and allows random sampling of batches to train the agent.

```

1. FUNCTION initialize_replay_buffer(capacity, state_shape, device):
2.   buffer.capacity ← capacity
3.   buffer.position ← 0
4.   buffer.size ← 0
5.
6.   // Pre-allocate GPU memory
7.   lookback, num_features ← state_shape
8.
9.   buffer.states ← zeros((capacity, lookback, num_features), device=device)
10.  buffer.next_states ← zeros((capacity, lookback, num_features), device=device)
11.  buffer.actions ← zeros(capacity, dtype=long, device=device)
12.  buffer.rewards ← zeros(capacity, dtype=float, device=device)
13.  buffer.dones ← zeros(capacity, dtype=float, device=device)
14.
15.
16. FUNCTION push(state, action, reward, next_state, done):
17.   pos ← buffer.position
18.
19.   buffer.states[pos] ← state
20.   buffer.actions[pos] ← action
21.   buffer.rewards[pos] ← reward
22.   buffer.dones[pos] ← done
23.
24.   IF next_state is not null THEN:
25.     buffer.next_states[pos] ← next_state
26.   ELSE:
27.     buffer.next_states[pos] ← 0
28.
29.   // Circular increment
30.   buffer.position ← (buffer.position + 1) MOD buffer.capacity

```

```

31.     buffer.size ← min(buffer.size + 1, buffer.capacity)
32.
33.
34. FUNCTION sample(batch_size):
35.     // Random sampling without replacement
36.     indices ← random_integers(0, buffer.size - 1, size=batch_size)
37.
38.     RETURN (
39.         buffer.states[indices],
40.         buffer.actions[indices],
41.         buffer.rewards[indices],
42.         buffer.next_states[indices],
43.         buffer.dones[indices]
44.     )
45.

```

Trade Performance Metrics:

Calculations for generic metrics that are useful for researchers to see.

```

1. FUNCTION track_trade_statistics(trades):
2.     winners ← [t FOR t IN trades IF t.pnl > 0]
3.     losers ← [t FOR t IN trades IF t.pnl ≤ 0]
4.
5.     total_trades ← length(trades)
6.     num_winners ← length(winners)
7.     num_losers ← length(losers)
8.
9.     win_rate ← num_winners / total_trades
10.
11.     avg_win ← mean([t.pnl FOR t IN winners]) IF winners ELSE 0
12.     avg_loss ← mean([t.pnl FOR t IN losers]) IF losers ELSE 0
13.
14.     total_profit ← sum([t.pnl FOR t IN winners])
15.     total_loss ← |sum([t.pnl FOR t IN losers])|
16.
17.     profit_factor ← total_profit / total_loss IF total_loss > 0 ELSE ∞
18.
19.     expectancy ← mean([t.pnl FOR t IN trades])
20.
21.     RETURN {
22.         "total_trades": total_trades,
23.         "win_rate": win_rate,
24.         "avg_win": avg_win,
25.         "avg_loss": avg_loss,
26.         "profit_factor": profit_factor,
27.         "expectancy": expectancy
28.     }
29.

```

Position Management:

Tracks positions held by agent.

```

1. FUNCTION manage_positions():
2.     // Position tracking structure
3.     positions ← [] // List of individual positions
4.
5.     FUNCTION add_position(entry_price, capital):
6.         position ← {
7.             "entry_price": entry_price,

```

```

8.         "allocation": capital,
9.         "shares": capital / entry_price,
10.        "entry_date": current_date
11.    }
12.    positions.append(position)
13.
14.    FUNCTION close_all_positions(current_price):
15.        total_capital_returned ← 0
16.        total_pnl ← 0
17.
18.        FOR EACH position IN positions DO:
19.            shares ← position.shares
20.            pnl ← (current_price - position.entry_price) × shares
21.            capital_returned ← position.allocation + pnl
22.
23.            total_capital_returned ← total_capital_returned + capital_returned
24.            total_pnl ← total_pnl + pnl
25.
26.        positions ← [] // Clear all positions
27.
28.        RETURN (total_capital_returned, total_pnl)
29.
30.    FUNCTION get_total_value(current_price):
31.        total_value ← 0
32.        FOR EACH position IN positions DO:
33.            total_value ← total_value + (position.shares × current_price)
34.        RETURN total_value
35.
36.    FUNCTION get_total_pnl(current_price):
37.        total_pnl ← 0
38.        FOR EACH position IN positions DO:
39.            pnl ← (current_price - position.entry_price) × position.shares
40.            total_pnl ← total_pnl + pnl
41.        RETURN total_pnl
42.

```

File Structures

Since much of this system relies on machine learning, extensive datasets are required to train new models. The first of these models is *LightGBM*, which will use a dataset similar to the one below to fit data.

Date	Open	High	Low	Close	Sent_Score	ATR	RSI	Stochastic_Osc	Bolling_Low	Bolling_Med	Bolling_High	MACD	Movement
2023-01-01	100.5	102.0	99.0	101.0	-0.25	1.2	40.0	75	98.5	100.0	101.5	0.5	0.49%
2023-01-02	101.0	102.5	100.0	101.5	0.10	1.1	42.5	78	99.0	101.0	102.0	0.6	0.50%
2023-01-03	101.5	103.0	100.5	102.0	-0.12	1.3	45.0	80	99.5	101.5	102.5	0.7	0.49%
2023-01-04	102.0	103.5	101.0	102.5	0.05	1.0	50.0	85	100.0	102.0	103.0	0.8	0.49%

The next dataset is to train the DQN model. To ensure the DQN has enough information to make optimal decisions, many columns are reused. There will also be additional columns for the LightGBM predictions with upper and lower quantile bounds for each horizon. The movement column will *not* be reused, as realistically if this model were to trade live, it would not be able to tell how much the market will move within a day and would have to solely rely on LightGBM for this information. The model will not be able to access the entire dataset. One row will be revealed to the model after every timestep, to prevent the model from using real future prices to make its decisions. It will look similar to the one below:

Date	Open	High	Low	Close	Sent_Score	ATR	RSI	Stochastic_Osc	Bolling_Low	Bolling_Med	Bolling_High	MACD	LGBM_7d_q10	LGBM_7d_q50	LGBM_7d_q90
2023-03-01	103.0	104.5	102.0	103.5	0.08	1.0	52.0	82	101.0	103.0	105.0	0.75	-0.5%	0.2%	0.9%
2023-03-02	103.5	105.0	102.5	104.0	-0.02	1.1	54.5	85	101.5	103.5	105.5	0.80	-0.4%	0.1%	1.0%
2023-03-03	104.0	106.0	103.0	105.0	0.15	1.2	56.0	88	102.0	104.0	106.0	0.85	-0.3%	0.3%	1.2%
2023-03-04	105.0	106.5	104.0	105.5	0.25	1.3	58.0	89	102.5	104.5	106.5	0.90	-0.2%	0.5%	1.3%

LightGBM sometimes finds reverse correlations, and therefore it is necessary to reverse predictions to make them accurate. A csv file will be stored containing which horizons must be flipped

Horizon	Flip
7	False
14	True
30	True

When a LightGBM model is saved, a model will be stored for each horizon involved in a .pkl (pickle) file. Although the configuration may remain the same, the target variable differs for each horizon, meaning multiple models are required. Pickling is a process done via the pickle library in python, used to convert objects into a bytestream, preserving the object state in a format that can easily be saved. This makes it appropriate for saving LightGBM models. When a DQN is

saved, it will store four files: the policy network, the target network, and the optimiser as .pt (PyTorch) files, and metadata as a .json file to allow other modules to use it correctly.

Networking & APIs

To ensure that training data reflects real market conditions, the system will retrieve data directly from online sources using HTTP requests. The two data types being acquired are historical price data and financial news metadata, which will be required at later processing stages.

Historical OHLC price data will be obtained using the *yfinance* library, which provides access to Yahoo Finance data through python. The library sends HTTP requests to Yahoo Finance servers and returns structured responses that can be converted into Pandas DataFrames. This allows large volumes of historical price data to be collected automatically over a specified time period, removing the need for manually downloading OHLC, and ensuring consistency across datasets.

For financial news data, the system will directly access a web API endpoint that returns article metadata in JSON format. Requests will be sent using the requests library.

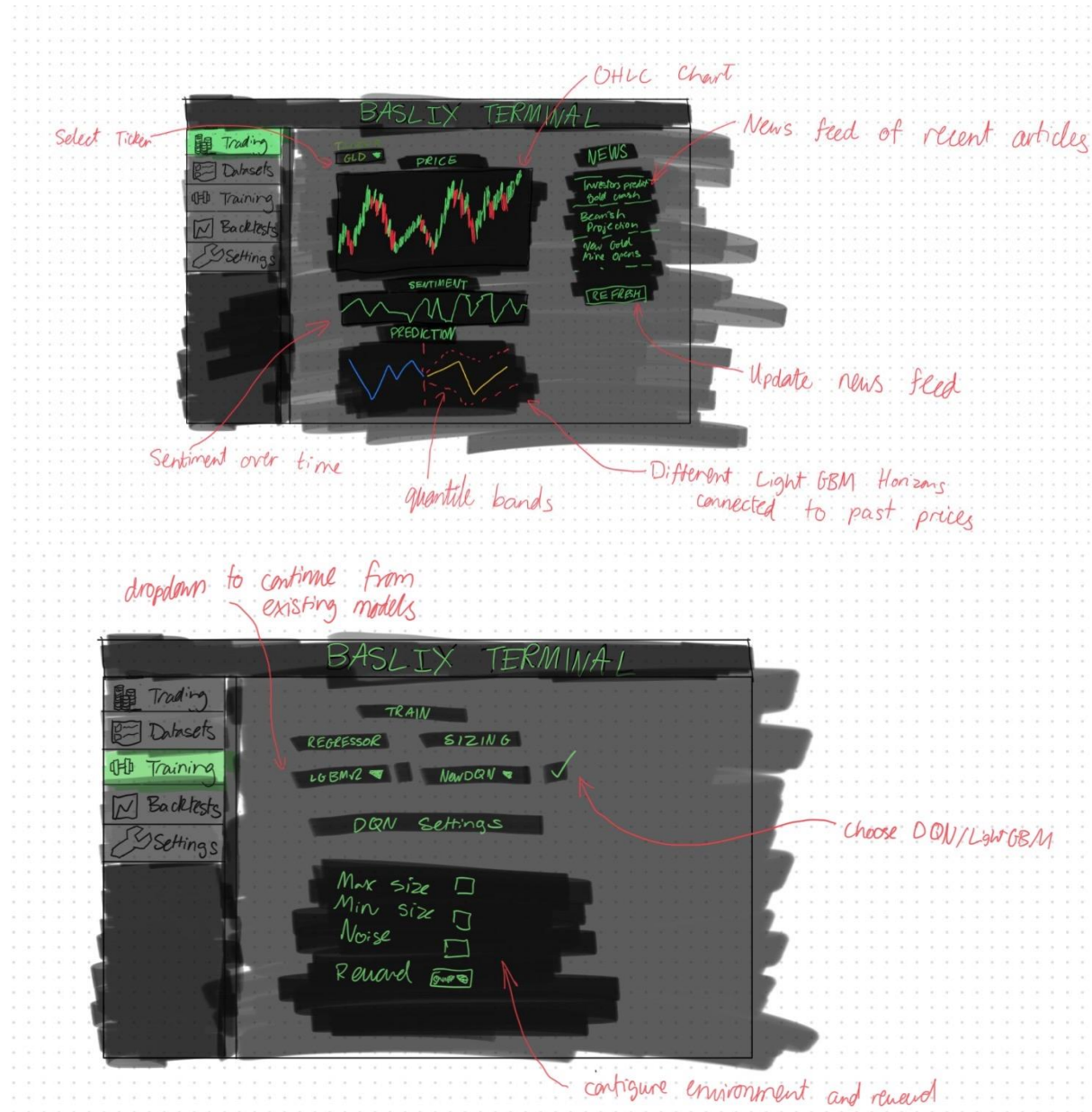
The API will return responses as nested JSON objects containing article information which includes publication date, title, associated tickers, and sentiment labels. These responses will be parsed programmatically by navigating the JSON structure and extracting only the required fields. Articles will be filtered based on relevant ticker tags and a defined date range to ensure that only applicable data is retained.

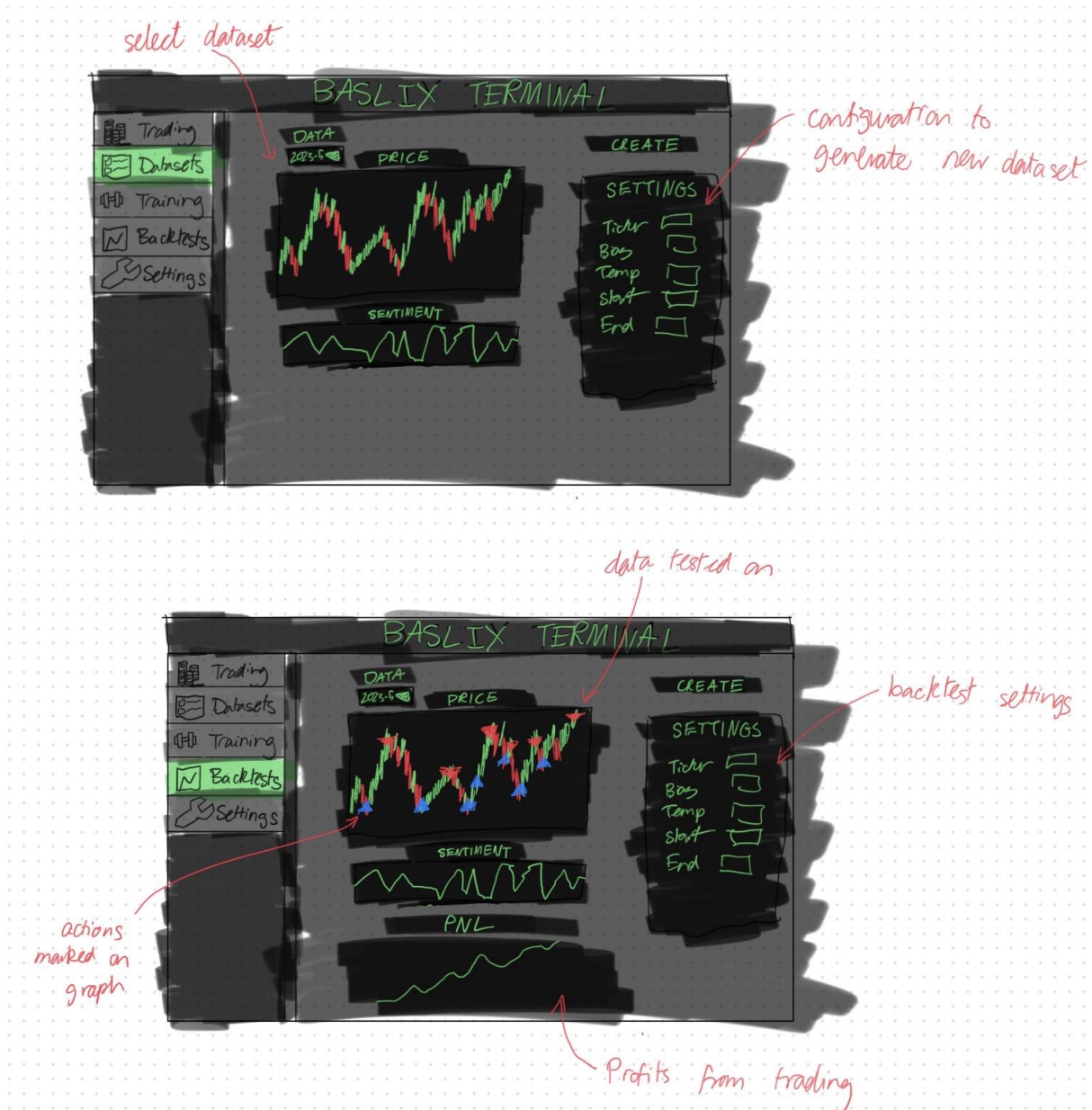
As the API returns results in pages, pagination will be handled by repeatedly requesting successive pages until no further results are returned. To reduce the likelihood of request blocking and minimise server load, a randomised delay will be introduced between requests. HTTP status codes and malformed JSON responses will be checked to ensure that errors are handled safely without crashing the system.

Metadata required for training can either be stored locally or in memory. Such metadata includes article date, title, and sentiment label. This data can be passed on later data processing and machine learning stages. The networking component is designed to be modular so that alternative or licensed data sources could be substituted in future without changes to the rest of the system.

Human Computer Interface

It will be useful to have a clear GUI to make research metrics easier to view. The *Streamlit* python library is used for GUI components. The design will be based off the sketches below:





A black and green theme is used, like that of an old terminal, to fit with the *BASLIX Terminal* name. There will be four tabs for all the required parts of the research environment: trading, training, dataset management, and backtesting.

The user should be able to modify configurations for relevant modules being called and be able to run them in separate processes to prevent the UI from freezing up. There should also be a stop button to kill processes.

Hardware Selection

A computer with a moderately powerful CPU and GPU is preferable when running this program for reasonable wait times. In this specific case, an NVIDIA RTX 4060 with 8GB of vRAM, and an AMD Ryzen 7 7435HS CPU will be used.

Library Usage

This system uses libraries exclusively for low-level infrastructure. The logic required to apply them to the financial domain, as well as the algorithmic designs will be custom built. For example, the requests library handles HTTP protocol compliance and TLS encryption (which are well-established standards and outside the scope of this research system), but all scraping logic, pagination, rate limiting, and data extraction will be designed specifically for this system. Similarly, PyTorch provides tensor operations and automatic differentiation (standard practice for building neural networks in python), but the entire DQN architecture (multi-layer CNN, replay buffer, epsilon-greedy exploration, soft target updates, reward function, training environment) will be an original implementation. This pattern extends to the rest of the codebase. Libraries should handle atomic operations (array arithmetic, HTTP requests, JSON parsing) that would otherwise require thousands of lines to reimplement from first principles, while all financial logic, statistical modelling, feature engineering, and machine learning pipelines will be manually constructed for this system. This separation ensures the project demonstrates genuine programming competency rather than library assembly, while maintaining a reasonable scope.

PyTorch

- Handles tensor operations and gradient computation
- Is necessary due to its use of low-level CUDA code, which is out of scope for this project and unnecessary to meet the objectives

LightGBM

- Efficiently trains gradient boosted decision trees
- Is an industry standard for forecasting in machine learning contexts

Transformers (HuggingFace)

- Loads a pre-trained FinRoBERTa model and runs inference for label probabilities only
- Does *not* output continuous polarity scores
- Would require a GPU cluster to train an alternative model (for likely worse performance anyway)

Pandas

- Stores data in DataFrames, and allows for basic operations to be conducted on them (filtering and grouping)
- Is a commonly used library to work with tables, and is a *required dependency* for *yfinance*

NumPy

- Allows for fast array operations to be conducted (mean, standard deviation)
- Is helpful for fast operations that are needed as part of calculating statistics

Yfinance

- Downloads OHLC price data from Yahoo Finance
- Is a research standard, and is *required* to gather price data

Requests

- Makes HTTP requests
- Would otherwise require reimplementing low-level networking protocols, which is outside the scope of this project

Matplotlib

- Is not used as part of the solution
- Can be found in the code for backend testing purposes
- Does not serve any purpose other than this, and can effectively be ignored

Tqdm

- Renders a progress bar for consoles
- Used primarily for backend testing purposes
- Can also be ignored

Multiprocessing

- Spawns worker processes
- Is used to parallelise sentiment analysis architecture, and for running processing components without stopping UI functionality

Pickle

- Serialises python objects to disk
- Is used to save LightGBM models

Datetime

- Represents dates and times in a format that can have arithmetic applied (such as adding days to a date)

Pathlib (OS)

- File system operations
- Is needed to access different directories (due to the number of modules used)

Json

- Parses/writes json
- Would otherwise require complex parsing and tokenisation, which again, does not align with the objectives and is out of this project's scope

Streamlit

- Provides a lightweight web interface for interacting with the system
- Is used to aesthetically display model outputs, configuration options, and results
- Does *not* perform data processing, modelling, or decision-making

Technical Solution

Technical Overview

This section will cover the codebase for *BASLIX Terminal* in detail. Wherever an **objective** is met, a **red** annotation/text box will be used. Demonstrated skills will be marked with **blue** annotations/text boxes. The code itself is commented to show technical understanding within the programming language. All scripts are in python. The code will be provided in different sections, according to their placement within the system's files.

Programming Technique Used	Page Numbers
Complex OOP:	
<i>Classes</i>	83-94, 127-138, 124-125, 126-127
<i>Composition</i>	83, 129, 124, 126
<i>Methods</i>	84, 129, 124, 126
<i>Attributes</i>	83, 127, 124, 126
Complex financial model:	
<i>DatasetGenerator</i>	83-94
<i>TradingEnvironment</i>	127-138
List Operations:	
<i>Append</i>	126
<i>Sort</i>	99
User-defined Algorithms:	
<i>scale_score()</i>	108-108
<i>update_expanding_normalisation()</i>	131
<i>calculate_historical_lgbm_confidence()</i>	129-131
<i>add_lgbm_predictions()</i>	89-93
Neural Network:	
<i>Conv1dDQN</i>	121-122
<i>Training Loop</i>	117-121
Dictionaries:	
<i>Indicator Configuration</i>	101
<i>Progress Tracker</i>	120
Read/Write Files:	
<i>Save dataset</i>	94
User Defined Data Structures:	
<i>Position</i>	126
<i>ReplayBuffer</i>	122-124
Parameterised Web Service API:	
<i>Web scraper</i>	96-100
Multi-dimensional Arrays:	
<i>Transition States</i>	123-123
<i>Raw Features</i>	128
Multiprocessing:	
<i>Sentiment Processing</i>	83-85
<i>UI Workers</i>	155
Defensive Programming:	
<i>Divide by Zero Protection</i>	86
<i>Index Bound Validation</i>	213

Training Resources

dataset_generator.py

This module orchestrates dataset generation and manages the fetching and processing modules to gather relevant data and process it into a unified dataset for training machine learning models and RL agents. In some places, there are references to a research paper, which can be found under references for [\(Bisdoulis, 2024\)](#).

Objective 1

All scripts seen in the technical solution contribute to completing objective 1

```
1. import pandas as pd
2. import numpy as np
3. from datetime import datetime, timedelta
4. from pathlib import Path
5. import pickle
6. from tqdm import tqdm
7.
8. from multiprocessing import Pool
9.
10. import matplotlib.pyplot as plt
11. import sys
12. import os
13. import copy
14.
15. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
16. if project_root not in sys.path:
17.     sys.path.insert(0, project_root)
18.
19. from training_resources.fetching.price_fetcher import fetch_prices
20. from training_resources.fetching.scrafer.scrafer import ArticleScrafer
21. from training_resources.processing.indicator_calculator.indicator_calculator import
IndicatorCalculator
22. from training_resources.processing.sentiment_analysis import SentimentAnalyser
23. from prediction_tools.light_gbm import LGBMForecaster
24.
25. DEFAULT_REGRESSION_CONFIG = {
26.     'lgbm': {
27.         'prediction_mode': 'quantile',
28.         'quantiles': [0.1, 0.5, 0.9],
29.         'conformal': False,
30.         'lgbm_params': {
31.             'n_estimators': 250,
32.             'learning_rate': 0.008,
33.             'max_depth': 12,
34.             'num_leaves': 84,
35.             'min_child_samples': 30,
36.             'subsample': 0.85,
37.             'colsample_bytree': 0.7,
38.             'reg_alpha': 0.0005,
39.             'reg_lambda': 2.0,
40.             'max_bin': 125,
41.             'boosting_type': 'gbdt',
42.             'random_state': 42,
43.             'n_jobs': -1,
44.             'verbose': -1
45.         },
46.     },
47.     'target': 'pct',
48.     'horizons': [365, 90, 30, 14, 7],
49.     'train_min': 500,
50.     'train_max': 1000,
51.     'retrain': 100,
52.     'conformal_split': 0.15,
53.     'conformal_retrain': False
54. }
```

Objective 5

This script in its entirety is designed to meet objective 5. Specific steps will be labelled

Similar imports will be seen throughout project, setting the project root to allow for easy importing of other modules

```

56. #worker functions for parallel sentiment processing
57. analysis_worker = None
58.
59. def create_sentiment_worker(sentiment_params):
60.     global analysis_worker
61.     from training_resources.processing.sentiment_analysis import SentimentAnalyser
62.     analysis_worker = SentimentAnalyser(**sentiment_params)
63.
64. def compute_single_sentiment(args):
65.     global analysis_worker
66.     title, true_sentiment, weight = args
67.     #base sentiment
68.     model_score = analysis_worker.analyse_sentiment(title)
69.
70.     #add weighting if it does exist
71.     if true_sentiment != "none":
72.         return weight * true_sentiment + (1 - weight) * model_score
73.
74.     return model_score
75.
76. class DatasetGenerator:
77.     def __init__(self, generator_config, scraper_config, sentiment_calculations=None,
calculator_params=None, adjust_prices=False):
78.         #defining attributes for dataset generation
79.         self.ticker = generator_config['ticker']
80.         self.start_date = generator_config['start_date']
81.         self.end_date = generator_config.get('end_date') or datetime.now()
82.         self.max_workers = generator_config.get('max_workers', 8)
83.         self.pct_change_features = generator_config.get('pct_change_features', [1, 3,
7])
84.         self.pct_change_targets = generator_config.get('pct_change_targets', [])
85.         self.ema_targets = generator_config.get('ema_targets', [])
86.         self.return_targets = generator_config.get('return_targets', [])
87.         self.vol_adjusted_targets = generator_config.get('vol_targets', [])
88.         self.normalised_return_targets =
generator_config.get('normalised_return_targets', [])
89.
90.         self.adjust_prices = adjust_prices
91.         self.scraper_config = scraper_config
92.
93.         sentiment_calculations = sentiment_calculations or {}
94.         self.sentiment_params = sentiment_calculations.get('sentiment_params', {})
95.         self.true_sentiment_weight =
sentiment_calculations.get('true_sentiment_weight', 0.7)
96.         self.sentiment_rolling_window = sentiment_calculations.get('rolling_window', 3)
97.         self.sentiment_decay = sentiment_calculations.get('decay', 0.7)
98.         self.sentiment_lags = sentiment_calculations.get('lags', [])
99.
100.        self.sentiment_analyser = SentimentAnalyser(**self.sentiment_params)
101.        self.indicator_calculator = IndicatorCalculator(params=calculator_params)
102.
103.        #x1.5 warmup to account for non trading days
104.        indicator_max_lookback = int(self.indicator_calculator.max_warmup()*1.5)
105.        sentiment_max_lookback = int(max(self.sentiment_lags + [0])*1.5 +
self.sentiment_rolling_window)
106.        pct_change_lookback = int(max(self.pct_change_features + [0])*1.5)
107.        ema_lookback = int(max(self.ema_targets + [0])*1.5)
108.
109.        necessary_warmup = max(indicator_max_lookback, sentiment_max_lookback,
pct_change_lookback, ema_lookback) + 5
110.        warmup_override = generator_config.get('warmup_days')
111.
112.        if warmup_override is not None:
113.            self.warmup_days = max(warmup_override, necessary_warmup)
114.        else:
115.            self.warmup_days = necessary_warmup
116.
117.        print(f"DEBUG WARMUP: necessary_warmup={necessary_warmup},
pct_change_lookback={pct_change_lookback}, final warmup_days={self.warmup_days}")

```

Initialises a sentiment analysis model once per worker process to avoid repeated model loading and reduce overhead

DatasetGenerator coordinates multiple dependent components using configuration inputs for initialisation, isolating preprocessing complexity from the rest of the system

```

118.         scraper_config = copy.deepcopy(self.scraper_config)
119.         self.scraper = ArticleScraper(
120.             cookies_path=scraper_config.get('cookies_path', ""),
121.             target_tag=scraper_config.get('target_tag', '80'),
122.             page_size=scraper_config.get('page_size', 50),
123.             category=scraper_config.get('category', ""),
124.             min_delay=scraper_config.get('min_delay', 1),
125.             max_delay=scraper_config.get('max_delay', 5)
126.         )
127.
128.         self.ohlc_data = None
129.         self.articles = []
130.         self.daily_sentiment = None
131.         self.full_dataset = None
132.
133.         #added to make UI easier
134.         self.progress = {
135.             "current": 0,
136.             "total": 0,
137.             "state": "idle"
138.         }
139.
140.         #yfinance call
141.         def fetch_ohlc(self):
142.             fetch_start = self.start_date - timedelta(days=self.warmup_days)
143.             self.ohlc_data = fetch_prices(
144.                 ticker=self.ticker,
145.                 start_date=fetch_start,
146.                 end_date=self.end_date,
147.                 adjust=self.adjust_prices
148.             )
149.             self.ohlc_data.index = pd.to_datetime(self.ohlc_data.index)
150.
151.         #article scraper
152.         def fetch_articles(self):
153.             fetch_start = self.start_date - timedelta(days=self.warmup_days)
154.             self.articles = self.scraper.fetch_articles(start=fetch_start, end=self.end_date)
155.
156.         #all days sentiment
157.         def compute_daily_sentiment(self):
158.             print("Computing daily sentiment...")
159.
160.             articles_by_date = {}
161.             for date_str, title, true_sentiment in self.articles:
162.                 date = pd.to_datetime(date_str).date()
163.                 articles_by_date.setdefault(date, []).append((title, true_sentiment))
164.
165.             all_tasks = []
166.             task_dates = []
167.             for date, articles in articles_by_date.items():
168.                 for title, ts in articles:
169.                     all_tasks.append((title, ts, self.true_sentiment_weight))
170.                     task_dates.append(date)
171.
172.             #seems to be a sweet spot for speed from testing
173.             chunksize = 8
174.
175.             self.progress["state"], self.progress["current"], self.progress["total"] =
176.             "Computing sentiment...", 0, len(all_tasks)
177.             scores = []
178.
179.             #parallel processing because this is terribly slow
180.             print(f"Processing {len(all_tasks)} articles across {self.max_workers} workers...")
181.             with Pool(
182.                 processes=self.max_workers,
183.                 initializer=create_sentiment_worker,
184.                 initargs=(self.sentiment_params,)
185.             ) as pool:

```

Uses multiprocessing to parallelise sentiment analysis across multiple worker processes, significantly reducing dataset generation time for large article sets

```

186.         for score in tqdm(
187.             pool.imap(compute_single_sentiment, all_tasks, chunksize=chunksize),
188.             total=len(all_tasks),
189.             desc="Sentiment",
190.             unit="article"
191.         ):
192.             scores.append(score)
193.             self.progress["current"] += 1
194.
195.         self.progress["state"] = "idle"
196.         #create a dictionary of scores for each date
197.         score_by_date = {}
198.
199.         for i in range(len(task_dates)):
200.             date = task_dates[i]
201.             score = scores[i]
202.
203.             if date not in score_by_date:
204.                 score_by_date[date] = []
205.
206.             score_by_date[date].append(score)
207.
208.         all_dates = pd.date_range(self.start_date - timedelta(days=self.warmup_days),
209. self.end_date)
210.         sentiment_series = pd.Series(index=all_dates, dtype=float)
211.         for date in all_dates:
212.             day_scores = score_by_date.get(date.date(), [])
213.             sentiment_series[date] = np.mean(day_scores) if day_scores else np.nan
214.
215.         #in case the user inputs dumb stuff from the UI
216.         if hasattr(self, "sentiment_rolling_window") and self.sentiment_rolling_window > 1:
217.             sentiment_series = sentiment_series.rolling(window=self.sentiment_rolling_window, min_periods=1).mean()
218.
219.         self.daily_sentiment = self.apply_decay(sentiment_series)
220.
221.         def apply_decay(self, sentiment_series):
222.             filled = []
223.             last_val = 0.0 #neutrality...
224.
225.             #check if sentiment exists, otherwise decay last val
226.             for date, val in sentiment_series.items():
227.                 if not pd.isna(val):
228.                     last_val = val
229.                 else:
230.                     last_val *= self.sentiment_decay
231.                 filled.append(last_val)
232.
233.             return pd.Series(filled, index=sentiment_series.index)
234.
235.         #call indicator calculator
236.         def compute_indicators(self, df):
237.             print("Calculating technical indicators...")
238.
239.             self.indicator_calculator.load_dataframe(df)
240.             self.ohlcv_data = self.indicator_calculator.add_all_indicators()
241.
242.         #lagged/rolling sentiment and pct change for features. pct and ema for target
243.         def compute_features_and_targets(self, dataset, debug=False):
244.             trading_idx = dataset.index[dataset['is_trading_day']]
245.             closes_trading = dataset['Close'].loc[trading_idx]
246.
247.             #historical pct_change features
248.             for h in self.pct_change_features:
249.                 col_name = f'Pct_Change_{h}d'
250.                 s = closes_trading.pct_change(periods=h).shift(1) * 100.0
251.                 dataset.loc[trading_idx, col_name] = s
252.

```

Objective 5.1

Meets the objective of the dataset to include each the mean sentiment for each day as a feature

Sentiment exponentially decays away when no articles are published, modelling how in the real-world sentiment doesn't simply "disappear"

```

253.
254.
255.     #historical sentiment features
256.     for lag in self.sentiment_lags:
257.         s = dataset['Sentiment'].shift(lag)
258.         dataset.loc[trading_idx, f'Sentiment_lag_{lag}'] = s.loc[trading_idx]
259.
260.     #rolling sentiment
261.     rolling = dataset['Sentiment'].rolling(
262.         window=self.sentiment_rolling_window, min_periods=1
263.     ).mean()
264.     dataset.loc[trading_idx, 'Sentiment_Rolling'] = rolling.loc[trading_idx]
265.
266.     #target calculations
267.     #added quite a few to experiment with for accuracy
268.
269.     # 1. Simple Pct Change Targets
270.     if self.pct_change_targets:
271.         for h in self.pct_change_targets:
272.             col_name = f'Future_Pct_Change_{h}d'
273.             s = (closes_trading.shift(-h) / closes_trading - 1.0) * 100.0
274.             dataset.loc[trading_idx, col_name] = s
275.
276.     # 2. Simple Return Targets (decimal, not percentage - baseline comparison)
277.     if self.return_targets:
278.         for h in self.return_targets:
279.             col_name = f'Future_Return_{h}d'
280.             s = closes_trading.shift(-h) / closes_trading - 1.0
281.             dataset.loc[trading_idx, col_name] = s
282.
283.     # 3. Volatility-Aware Targets (risk-adjusted returns)
284.     if self.vol_adjusted_targets:
285.         trading_days_df = dataset.loc[trading_idx, ['Close']].copy()
286.
287.         #calculate returns and rolling volatility
288.         returns = trading_days_df['Close'].pct_change()
289.         vol = returns.rolling(window=20, min_periods=10).std()
290.
291.         for h in self.vol_adjusted_targets:
292.             #raw return
293.             raw_return = trading_days_df['Close'].shift(-h) / trading_days_df['Close']
294.             - 1.0
295.
296.             #risk-adjusted target (return / volatility)
297.             #replace zero or near zero volatility with nan to avoid division issues
298.             vol_safe = vol.replace(0, np.nan)
299.             target = raw_return / vol_safe
300.
301.             col_name = f'Target_VolAdjusted_{h}d'
302.             dataset.loc[trading_idx, col_name] = target.values
303.
304.             if debug:
305.                 print(f"\n[DEBUG] Vol-Adjusted Target {h}d calculation:")
306.                 print(f"    Raw return mean: {raw_return.mean():.4f}")
307.                 print(f"    Volatility mean: {vol.mean():.4f}")
308.                 print(f"    Vol-adjusted mean: {target.mean():.4f}")
309.                 print(f"    Vol-adjusted std: {target.std():.4f}")
310.                 print(f"    Non-null count: {target.notna().sum()}")
311.
312.     # 4. EMA-Relative Positioning Targets (mean reversion/continuation)
313.     if self.ema_targets:
314.         trading_days_df = dataset.loc[trading_idx, ['Close']].copy()
315.
316.         if debug:
317.             print(f"[DEBUG] Trading days in sequence: {len(trading_days_df)}")
318.
319.         for h in self.ema_targets:
320.             #calculate EMA
321.             ema = trading_days_df['Close'].ewm(span=h, adjust=False).mean()

```

Objective 5.3

Multiple target labels are generated for supervised learning

Divide by zero protection by replacing zero volatility with NaN during feature and target calculations

```

321.
322.         #future close relative to current ema
323.         future_close = trading_days_df['Close'].shift(-h)
324.         target = future_close / ema - 1.0
325.
326.         col_name = f'Target_EMA_Position_{h}d'
327.         dataset.loc[trading_idx, col_name] = target.values
328.
329.         if debug:
330.             print(f"\n[DEBUG] EMA({h}) Position Target {h}d calculation:")
331.             print(f"    EMA mean: {ema.mean():.4f}")
332.             print(f"    Target mean: {target.mean():.4f}")
333.             print(f"    Target std: {target.std():.4f}")
334.             print(f"    Non-null count: {target.notna().sum()}")
335.
336.     # 5. Normalised Return Targets (forward return / recent volatility)
337.     if self.normalised_return_targets:
338.         trading_days_df = dataset.loc[trading_idx, ['Close']].copy()
339.
340.         #compute daily returns and rolling volatility (causal)
341.         returns = trading_days_df['Close'].pct_change()
342.         vol = returns.rolling(window=20, min_periods=10).std()
343.
344.         vol_safe = vol.replace(0, np.nan)
345.
346.         for h in self.normalised_return_targets:
347.             #forward raw return
348.             raw_return = trading_days_df['Close'].shift(-h) / trading_days_df['Close']
349.             - 1.0
350.
351.             #normalized return
352.             normalized_return = raw_return / (vol_safe * np.sqrt(h))
353.
354.             col_name = f'Target_Normalised_Return_{h}d'
355.             dataset.loc[trading_idx, col_name] = normalized_return.values
356.
357.             if debug:
358.                 print(f"\n[DEBUG] Normalised Return Target {h}d calculation:")
359.                 print(f"    Raw return mean: {raw_return.mean():.4f}")
360.                 print(f"    Volatility mean: {vol.mean():.4f}")
361.                 print(f"    Normalised mean: {normalized_return.mean():.4f}")
362.                 print(f"    Normalised std: {normalized_return.std():.4f}")
363.                 print(f"    Non-null count: {normalized_return.notna().sum()}")
364.
365.     #only drop rows that lack PAST features
366.     #this is to ensure that it is possible to pull up to the present day
367.     max_lag = max(self.pct_change_features + [0])
368.
369.     if max_lag > 0:
370.         dataset = dataset.iloc[max_lag:]
371.
372.     if debug:
373.         print(f"\n[DEBUG] Dataset shape after feature computation: {dataset.shape}")
374.         print(f"[DEBUG] Trading days: {dataset['is_trading_day'].sum()}")
375.
376.     #show all target types
377.     target_cols = []
378.     if self.pct_change_targets:
379.         target_cols += [f'Future_Pct_Change_{h}d' for h in self.pct_change_targets]
380.     if self.return_targets:
381.         target_cols += [f'Future_Return_{h}d' for h in self.return_targets]
382.     if self.vol_adjusted_targets:
383.         target_cols += [f'Target_VolAdjusted_{h}d' for h in
384. self.vol_adjusted_targets]
385.     if self.ema_targets:
386.         target_cols += [f'Target_EMA_Position_{h}d' for h in self.ema_targets]
387.
388.     if target_cols:

```

```

388.         print(f"[DEBUG] Last 15 rows of targets:")
389.         existing_targets = [c for c in target_cols if c in
390.             if existing_targets:
391.                 print(dataset.loc[trading_idx, existing_targets])
392.
393.         return dataset
394.
395.     def generate_dataset(self):
396.
397.         #pull data if it doesnt already exist for some reason (testing different sentiment
with different tickers)
398.         if self.ohlcv_data is None:
399.             self.fetch_ohlcv()
400.         if self.daily_sentiment is None:
401.             if self.articles == []:
402.                 self.fetch_articles()
403.             self.compute_daily_sentiment()
404.         self.compute_indicators(df=self.ohlcv_data)
405.
406.         fetch_start = self.start_date - timedelta(days=self.warmup_days)
407.         full_idx = pd.date_range(fetch_start, self.end_date)
408.
409.         dataset = pd.DataFrame(index=full_idx)
410.         dataset = dataset.join(self.ohlcv_data)
411.         dataset['is_trading_day'] = ~dataset['Close'].isna()
412.
413.         #sentiment processing
414.         dataset['Sentiment'] = self.daily_sentiment.reindex(full_idx).ffill().fillna(0.0)
415.
416.         #forward-fill OHLC for continuity
417.         dataset[['Open', 'High', 'Low', 'Close']] =
dataset[['Open', 'High', 'Low', 'Close']].ffill()
418.         vol_cols = [col for col in dataset.columns if col.startswith("Vol_")]
419.         if vol_cols:
420.             dataset[vol_cols] = dataset[vol_cols].ffill()
421.
422.         #compute all features and targets
423.         dataset = self.compute_features_and_targets(dataset)
424.
425.         #list of features
426.         feature_cols = []
427.
428.         #pct change features
429.         feature_cols.extend([f'Pct_Change_{h}d' for h in self.pct_change_features])
430.
431.         #sentiment features
432.         feature_cols.extend([f'Sentiment_lag_{lag}' for lag in self.sentiment_lags])
433.         if self.sentiment_rolling_window:
434.             feature_cols.append('Sentiment_Rolling')
435.
436.         required_indicators = self.indicator_calculator.get_indicator_columns()
437.
438.         #slice off warmup period
439.         dataset = dataset[dataset.index >= self.start_date]
440.         dataset = dataset[dataset.index >= self.start_date]
441.
442.         #keep only trading days
443.         dataset = dataset[dataset['is_trading_day']].copy()
444.
445.         #only drop rows missing FEATURES (not targets!)
446.         required_cols = feature_cols + required_indicators
447.
448.         #filter to only columns that actually exist
449.         required_cols = [c for c in required_cols if c in dataset.columns]
450.
451.         for col in required_cols:
452.             nan_count = dataset[col].isna().sum()
453.             if nan_count > 0:
454.                 first_valid = dataset[col].first_valid_index()

```

Objective 5.2

This function combines all data into a single dataset

The final dataset generated has complex technical features, sentiment reflecting how people feel about the market, and prices, creating a complex model of financial markets

```

455.         print(f" {col}: {nan_count} NaNs, first valid = {first_valid}")
456.
457.     if required_cols:
458.         dataset = dataset.dropna(subset=required_cols)
459.
460.     #dont need this flag anymore
461.     if "is_trading_day" in dataset.columns:
462.         dataset = dataset.drop(columns=["is_trading_day"])
463.
464.     #store full dataset (includes rows without targets)
465.     self.full_dataset = dataset.copy()
466.
467.
468.     print(f"\n[DATASET INFO]")
469.     print(f" Total rows (with features): {len(dataset)}")
470.     print(f" Date range: {dataset.index.min().date()} to
471. {dataset.index.max().date()}")
472.
473.     if self.ema_targets:
474.         print(f"\n[EMA TARGETS]")
475.         for h in self.ema_targets:
476.             col = f'Target_EMA_Position_{h}d'
477.             valid = dataset[col].notna().sum()
478.             print(f" {h}d: {valid} valid targets")
479.
480.     if len(dataset) == 0:
481.         print("Dataset is empty after filtering!")
482.
483.     return dataset
484.
485. def add_lgbm_predictions(self, config=None, model=None):
486.
487.     #only replace parameters that need replacing, and fill in if needed
488.     final_config = copy.deepcopy(DEFAULT_REGRESSION_CONFIG)
489.     if config is not None:
490.         for key in config:
491.             if key in DEFAULT_REGRESSION_CONFIG:
492.                 final_config[key] = config[key]
493.             else:
494.                 print(f"{key} is not a valid parameter for the regressor")
495.
496.     df = self.full_dataset.copy()
497.
498.     if model is not None:
499.         print("\n" + "="*60)
500.         print("LOADING PRE-TRAINED MODELS")
501.         print("="*60)
502.
503.         #model should be a path to directory containing .pkl files
504.         model_dir = Path(model)
505.         if not model_dir.exists():
506.             print(f"ERROR: Model directory not found: {model_dir}")
507.             self.full_dataset = df
508.         else:
509.             model_files = list(model_dir.glob("*.pkl"))
510.             if not model_files:
511.                 print(f"ERROR: No .pkl files found in {model_dir}")
512.                 self.full_dataset = df
513.             else:
514.                 print(f"Found {len(model_files)} model files")
515.
516.                 #load models and organize by horizon
517.                 models_by_horizon = {}
518.                 cascade_order = None
519.
520.                 for model_file in model_files:
521.                     print(f>Loading: {model_file.name}")
522.                     with open(model_file, 'rb') as f:

```

This algorithm uses a walk-forward expanding window based on parameters to add predictions without causing data leakage

```

523.         model_data = pickle.load(f)
524.
525.         horizon = model_data['horizon']
526.         forecaster = model_data['forecaster']
527.         feature_cols = model_data['feature_cols']
528.         target_col = model_data['target_col']
529.         use_cascading = model_data.get('use_cascading', False)
530.
531.         #get cascade order from first model
532.         if cascade_order is None:
533.             cascade_order = model_data.get('cascade_order', 'ascending')
534.
535.         models_by_horizon[horizon] = {
536.             'forecaster': forecaster,
537.             'feature_cols': feature_cols,
538.             'target_col': target_col,
539.             'quantiles': forecaster.config['quantiles']
540.         }
541.
542.         print(f" Horizon: {horizon}d")
543.         print(f" Quantiles: {forecaster.config['quantiles']}")
544.         print(f" Features: {len(feature_cols)}")
545.
546.     print(f"\nCascading: {use_cascading}")
547.     print(f"Cascade order: {cascade_order}")
548.
549.     if cascade_order == 'ascending':
550.         sorted_horizons = sorted(models_by_horizon.keys())
551.         print("Processing order: shortest to longest horizons")
552.     else:
553.         sorted_horizons = sorted(models_by_horizon.keys(), reverse=True)
554.         print("Processing order: longest to shortest horizons")
555.
556.
557.     for horizon in sorted_horizons:
558.         print(f"\n{'='*60}")
559.         print(f"GENERATING PREDICTIONS: {horizon}d Horizon")
560.         print(f"{'='*60}")
561.
562.         model_info = models_by_horizon[horizon]
563.         forecaster = model_info['forecaster']
564.         feature_cols = model_info['feature_cols']
565.         quantiles = model_info['quantiles']
566.
567.         missing_features = [col for col in feature_cols if col not in
df.columns]
568.
569.         if missing_features:
570.             print(f"ERROR: Missing features: {missing_features[:5]}...")
571.             print(f"Skipping {horizon}d horizon")
572.             continue
573.
574.         #valid rows that have all required features
575.         valid_rows = df[feature_cols].notna().all(axis=1)
576.         X = df.loc[valid_rows, feature_cols]
577.
578.         print(f"Valid samples: {len(X)}")
579.
580.         if len(X) == 0:
581.             print(f"WARNING: No valid samples for {horizon}d horizon")
582.             continue
583.
584.         # Generate predictions
585.         print("Generating predictions...")
586.         predictions = forecaster.predict(X)
587.
588.         # Add prediction columns to dataframe
589.         for q in quantiles:
590.             col_name = f'LGBM_{horizon}d_q{int(q*100)}'

```

```

591.                 # Initialize column if it doesn't exist
592.                 if col_name not in df.columns:
593.                     df[col_name] = np.nan
594.
595.                 # Fill predictions where we have valid features
596.                 df.loc[valid_rows, col_name] = predictions[q]
597.
598.                 print(f" Added: {col_name} ({valid_rows.sum()} predictions)")
599.
600.                 print(f"✓ Completed {horizon}d horizon")
601.
602.                 # Update full dataset
603.                 self.full_dataset = df
604.                 print("\n" + "="*60)
605.                 print("PREDICTION GENERATION COMPLETE")
606.                 print("="*60)
607.
608. #
609. # Train models from scratch
610. else:
611.     min_train = final_config['train_min']
612.     max_train = final_config['train_max']
613.     retrain = final_config['retrain']
614.     conformal_split = final_config['conformal_split']
615.
616.     using_quantiles = True
617.
618.     for horizon in final_config['horizons']:
619.
620.         #grab feature columns (recalculated each horizon to include previous LGBM
621.         #predictions for cascading predictions)
622.         feature_cols = []
623.         for col in df.columns:
624.             if col.startswith("Target") or col.startswith("Future"):
625.                 continue
626.             else:
627.                 feature_cols.append(col)
628.
629.         if final_config['target'] == 'ema':
630.             target_col = f"Target_EMA_Position_{horizon}d"
631.         elif final_config['target'] == 'pct':
632.             target_col = f"Future_Pct_Change_{horizon}d"
633.         elif final_config['target'] == 'vol':
634.             target_col = f"Target_VolAdjusted_{horizon}d"
635.         elif final_config['target'] == 'returns':
636.             target_col = f"Future_Return_{horizon}d"
637.         elif final_config['target'] == 'norm':
638.             target_col = f"Target_Normalised_Return_{horizon}d"
639.         else:
640.             raise ValueError(f"Unrecognised target: {final_config['target']}")
641.
642.         if target_col not in df.columns:
643.             print(f"Target column {target_col} not found, skipping horizon
644. {horizon}d")
645.             continue
646.
647.         print(f"\n{'='*60}")
648.         print(f"Training model for {horizon}d horizon")
649.         print(f"{'='*60}")
650.
651.         df['_has_target'] = df[target_col].notna()
652.
653.         if df['_has_target'].sum() < min_train:
654.             print(f"Insufficient data: {df['_has_target'].sum()} rows are present")
655.             print(f"{min_train} rows are required")
656.             df = df.drop(columns=['_has_target'])
657.             continue
658.
659.         #initialize prediction columns

```

```

658.         first_time = True
659.
660.
661.         current_idx = 0
662.         days_since_train = retrain
663.
664.         self.progress["state"], self.progress["current"], self.progress["total"] =
665.         f"Generating {horizon}d Predictions...", 0, len(df)
666.
667.         print(f"Starting predictions...")
668.         pbar = tqdm(total=len(df), desc=f"Predicting {horizon}d", unit="day")
669.
670.         while current_idx < len(df):
671.
672.             forecaster = LGBMForecaster(config=final_config['lgbm'])
673.
674.             if first_time:
675.                 conformal = forecaster.config['conformal']
676.
677.                 if conformal:
678.                     print("Using conformal predictions")
679.
680.                     if forecaster.prediction_mode == 'quantile':
681.                         for q in forecaster.config['quantiles']:
682.                             df[f'LGBM_{horizon}d_q{int(q*100)}'] = np.nan
683.                     else:
684.                         df[f'LGBM_{horizon}d'] = np.nan
685.
686.                     for key, entry in forecaster.config.items():
687.                         print(f"{key}: {entry}")
688.
689.                     first_time = False
690.
691.             if days_since_train >= retrain or forecaster.model is None:
692.
693.                 train_end_idx = current_idx
694.                 train_start_idx = max(0, train_end_idx - max_train)
695.
696.                 #get training data from df, filter by target validity
697.                 train_slice = df.iloc[train_start_idx:train_end_idx]
698.                 train_data = train_slice[train_slice['_has_target']].copy()
699.
700.                 if len(train_data) < min_train:
701.                     current_idx += retrain
702.                     days_since_train = retrain
703.                     pbar.update(min(retrain, len(df) - current_idx + retrain))
704.                     continue
705.
706.                 X_train = train_data[feature_cols]
707.                 y_train = train_data[target_col]
708.
709.                 #train model
710.                 if conformal and forecaster.prediction_mode == 'quantile':
711.                     forecaster.fit(X_train, y_train, recalibrate=True,
conformal_split=conformal_split)
712.                     if final_config['conformal_retrain']:
713.                         forecaster.fit(X_train, y_train, recalibrate=False)
714.                 else:
715.                     forecaster.fit(X_train, y_train, recalibrate=False)
716.
717.                 days_since_train = 0
718.
719.                 #predict batch (for SPEED)
720.                 batch_end_idx = min(current_idx + retrain, len(df))
721.                 batch_size = batch_end_idx - current_idx
722.
723.                 X_pred = df.iloc[current_idx:batch_end_idx][feature_cols]
724.                 pred = forecaster.predict(X_pred)

```

The model is only retrained after a configurable number of new observations, which simulates how the model might be deployed in a live trading environment. Also reduces computational cost

```

725.
726.         #store predictions
727.         if forecaster.prediction_mode == 'quantile':
728.             for q in forecaster.config['quantiles']:
729.                 col_name = f'LGBM_{horizon}d_q{int(q*100)}'
730.                 df.iloc[current_idx:batch_end_idx,
df.columns.get_loc(col_name)] = pred[q]
731.             else:
732.                 col_name = f'LGBM_{horizon}d'
733.                 df.iloc[current_idx:batch_end_idx, df.columns.get_loc(col_name)] =
pred
734.
735.                 current_idx = batch_end_idx
736.                 days_since_train += batch_size
737.
738.                 self.progress["current"] = current_idx
739.                 pbar.update(batch_size)
740.
741.         pbar.close()
742.         self.progress["state"] = "idle"
743.
744.         #cleanup
745.         df = df.drop(columns=['_has_target'])
746.
747.         #for debugging
748.         if forecaster.prediction_mode == 'quantile':
749.             pred_count = df[f'LGBM_{horizon}d_q50'].notna().sum()
750.         else:
751.             using_quantiles = False
752.             pred_count = df[f'LGBM_{horizon}d'].notna().sum()
753.
754.         print(f"Generated {pred_count} predictions for {horizon}d horizon")
755.
756.
757.
758.         price_col = 'Close' if 'Close' in df.columns else 'close' if 'close' in
df.columns else None
759.
760.         if price_col is None:
761.             print("Warning: No 'Close' price column found.")
762.         else:
763.             for horizon in final_config['horizons']:
764.                 if using_quantiles:
765.                     pred_col = f'LGBM_{horizon}d_q50'
766.                 else:
767.                     pred_col = f'LGBM_{horizon}d'
768.
769.                 if pred_col not in df.columns:
770.                     continue
771.
772.         #cleanup at the end
773.         print("\nCleaning up dataset...")
774.
775.
776.         prediction_cols = [col for col in df.columns if col.startswith('LGBM_')]
777.         empty_pred_cols = [col for col in prediction_cols if df[col].isna().all()]
778.
779.
780.         #more debug stuff
781.         if empty_pred_cols:
782.             df = df.drop(columns=empty_pred_cols)
783.             print(f"Dropped {len(empty_pred_cols)} empty prediction columns")
784.
785.         print(f"Final dataset shape: {df.shape}")
786.
787.         self.full_dataset = df
788.
789.
790.         #this was for backend testing

```

```

791.     def plot_price_and_sentiment(self, title=None, sentiment_lags=True, show=True):
792.
793.         #happened a few times when full dataset didnt save for some reason
794.         if self.full_dataset is None:
795.             raise ValueError("Trading dataset not generated yet. Call generate_dataset()
first.")
796.
797.         dataset = self.full_dataset
798.         fig, (ax1, ax2) = plt.subplots(
799.             2, 1, figsize=(14,10), sharex=True, gridspec_kw={'height_ratios':[2,1]}
800.         )
801.
802.         #price and bollinger
803.         ax1.plot(dataset.index, dataset['Close'], label='Close Price', color='blue')
804.         ax1.plot(dataset.index, dataset['BB_Mid'], label='BB Mid', color='orange',
linestyle='--')
805.         ax1.plot(dataset.index, dataset['BB_Upper'], label='BB Upper', color='green',
linestyle='--')
806.         ax1.plot(dataset.index, dataset['BB_Lower'], label='BB Lower', color='red',
linestyle='--')
807.         ax1.fill_between(dataset.index, dataset['BB_Lower'], dataset['BB_Upper'],
color='grey', alpha=0.1)
808.         ax1.set_ylabel("Price")
809.         ax1.set_title(title or "Price with Bollinger Bands & Sentiment")
810.         ax1.legend()
811.         ax1.grid(True)
812.
813.         #sentiment
814.         ax2.plot(dataset.index, dataset['Sentiment'], label='Sentiment', color='blue')
815.         if sentiment_lags:
816.             lag_cols = [col for col in dataset.columns if col.startswith("Sentiment_lag")]
817.             for col in lag_cols:
818.                 ax2.plot(dataset.index, dataset[col], linestyle='--', alpha=0.7, label=col)
819.         ax2.set_ylabel("Sentiment")
820.         ax2.set_xlabel("Date")
821.         ax2.legend()
822.         ax2.grid(True)
823.
824.         plt.tight_layout()
825.         if show:
826.             plt.show()
827.         else:
828.             return fig
829.
830.         #save dataset
831.         def save_dataset(self, path):
832.             if self.full_dataset is not None:
833.                 df_to_save = self.full_dataset.copy()
834.                 df_to_save.insert(0, "Date", df_to_save.index)
835.                 df_to_save.to_csv(path, index=False)
836.                 print(f"Dataset saved to {path}")
837.                 print(f"Total rows: {len(df_to_save)}")
838.                 print(f"Date range: {df_to_save['Date'].min().date()} to
{df_to_save['Date'].max().date()}")
839.             else:
840.                 raise ValueError("Dataset not generated yet. Call generate_dataset() first.")
841.
842.
843.         #testing
844.         if __name__ == "__main__":
845.
846.             generator_config = {
847.                 'ticker': 'BTC-USD',
848.                 'start_date': datetime(2017,1,1),
849.                 'end_date': datetime(2026, 1, 16),
850.                 'max_workers': 8,
851.                 'pct_change_features': [3, 7, 14, 30, 90, 365],
852.                 'pct_change_targets': [3,7, 14, 30, 90, 365],
853.                 'return_targets': [3,7, 14, 30, 90, 365],

```

Generated dataset is saved to a .csv file at the specified path

```

854.         'vol_targets': [3,7, 14, 30, 90, 365],
855.         'ema_targets': [3,7, 14, 30, 90, 365],
856.         'normalised_return_targets': [3,7,9,14, 30, 90, 365]
857.     }
858.
859.     scraper_config = {
860.         'cookies_path': "training_resources\\fetching\\scraper\\cookies.json",
861.         'target_tag': '581328',
862.         'page_size': 100,
863.         'category': "market-outlook::cryptocurrency",
864.         'min_delay': 0,
865.         'max_delay': 1,
866.     }
867.
868.     sentiment_calculations = {
869.         'sentiment_params': {
870.             'bias_prompt': "Bitcoin market outlook: ",
871.             'temperature': 1.6,
872.             'neutrality_scaler': 7.7,
873.             'testing': False
874.         },
875.         'true_sentiment_weight': 0.7,
876.         'rolling_window': 3,
877.         'decay': 0.7,
878.         'lags': [3, 7, 14, 28]
879.     }
880.
881.     generator = DatasetGenerator(
882.         generator_config=generator_config,
883.         scraper_config=scraper_config,
884.         sentiment_calculations=sentiment_calculations,
885.         calculator_params=None
886.     )
887.
888.     dataset = generator.generate_dataset()
889.
890.     #experimented according to the paper for LightGBM assets forecasting (Bisdoulis, 2024),
891.     #see References
892.     lgbm_config = {
893.         'lgbm': {
894.             'prediction_mode': 'quantile',
895.             'quantiles': [0.1, 0.5, 0.9],
896.             'conformal': False,
897.             'lgbm_params': {
898.                 'n_estimators': 250, #paper used 500-2200
899.                 'learning_rate': 0.008, #lower LR for better convergence
900.                 'max_depth': 12, #paper searched -1 to 30
901.                 'num_leaves': 84, #paper searched 10-80
902.                 'min_child_samples': 30, #increase for stability
903.                 'subsample': 0.85,
904.                 'colsample_bytree': 0.7,
905.                 'reg_alpha': 0.0005, #L1 regularization
906.                 'reg_lambda': 2.0, #L2 regularization (paper used 1e-6 to 10)
907.                 'max_bin': 125, #paper searched 125-750
908.                 'boosting_type': 'gbdt',
909.                 'random_state': 42,
910.                 'n_jobs': -1,
911.                 'verbose': -1
912.             },
913.             'target': 'pct',
914.             'horizons': [365, 90, 30, 14, 7], #in this order for cascading predictions
915.             'train_min': 500,
916.             'train_max': 1000,
917.             'retrain': 100,
918.             'conformal_split': 0.15,
919.             'conformal_retrain': False
920.         }
921.     }

```

```

922.     generator.add_lgbm_predictions(config=lgbm_config)
923.
924.     #save
925.     path = r"training_resources\datasets\gen_test.csv"
926.     generator.save_dataset(path)
927.     print("\nDataset generated and saved.")
928.
929.     #plot
930.     generator.plot_price_and_sentiment(title="BTC Price and Sentiment")
931.

```

Fetching

scraper.py

This module pulls articles from [REDACTED] to provide the rest of the pipeline with news articles and dates to analyse. A setting can easily be added for any ticker present on the website, provided the tag used within the API is known as well as the category it will be found in. Several YouTube tutorials were referred to when building the web scraper¹⁰ as well as tutorials for locating API endpoints¹¹.

```

1. import requests
2. import json
3. import time
4. import random
5. import csv
6. import os
7. from datetime import datetime, timedelta
8. from tqdm import tqdm
9. import pandas as pd
10.
11. class ArticleScraper:
12.
13.     def __init__(self, cookies_path, target_tag, page_size, category, min_delay,
max_delay):
14.
15.         self.cookies_path = cookies_path
16.
17.         #accept single tag or list of tags
18.         if isinstance(target_tag, str):
19.             self.target_tags = [target_tag]
20.         else:
21.             self.target_tags = target_tag
22.         self.page_size = page_size
23.         self.category = category
24.         self.min_delay = min_delay
25.         self.max_delay = max_delay
26.         self.session = None
27.
28.         #added for ui progress bars, still keeping tqdm to avoid a massive rewrite
29.         self.progress = {
30.             "current": 0,
31.             "total": 0,
32.             "state": "idle"
33.         }
34.
35.
36.

```

Web service API is constructed via a parameterised URL

Objective 2.1

This function dynamically constructs request URLs

¹⁰ <https://www.youtube.com/watch?v=kJyOp12PgP4&t=2507s>,

Web Scraping APIs with Python for Data Science - Beginner to Advanced - Part 1

¹¹ <https://www.youtube.com/watch?v=kPe3wtA9aPM>,

Web Scraping: discovering hidden website APIs using Chrome Dev Tools

```

37.     #theres not much of a downside to pulling more fields, so ive included a few irrelevant
ones i might use later.
38.     def build_url(self, start_timestamp, end_timestamp, page, category=None):
39.
40.         #the default category is for gold and precious metals, but you can change it if
necessary
41.         if category is None:
42.             category = self.category
43.
44.         return (
45.             "https://[REDACTED]?"
46.
47.         )
48.
49.     def load_cookies(self):
50.
51.         if not os.path.exists(self.cookies_path):
52.             raise FileNotFoundError(f"Could not find cookie file. Currently checking:
'{self.cookies_path}'")
53.
54.         with open(self.cookies_path, "r", encoding="utf-8") as file:
55.             cookies = json.load(file)
56.
57.             cookie_string = ""
58.
59.             #standard cookie formatting
60.             for i in range(0, len(cookies)):
61.                 cookie_string += cookies[i]['name'] + "=" + cookies[i]['value']
62.                 if i < len(cookies) - 1:
63.                     cookie_string += "; "
64.
65.             return cookie_string
66.
67.     def relevant_sentiments(self, included):
68.
69.         sentiments = {}
70.
71.         sentiment_map = {
72.             "bullish": 1.0,
73.             "bearish": -1.0,
74.             "neutral": 0
75.         }
76.
77.         for item in included:
78.             if item.get("type") == "sentiment":
79.                 attributes = item.get("attributes", {})
80.                 article_id = str(attributes.get("articleId"))
81.                 true_sentiment = attributes.get("type")
82.
83.                 tag_id = item.get("relationships", {}).get("tag", {}).get("data",
{}).get("id", "")
84.
85.                 if tag_id in self.target_tags:#
86.                     sentiments[article_id] = sentiment_map.get(true_sentiment, "none")
87.
88.         for article_id in sentiments:
89.             if not sentiments[article_id]:
90.                 sentiments[article_id] = "none"
91.
92.         return sentiments
93.
94.     def has_relevant_tag(self, article):
95.         metadata = article.get("relationships", {})
96.         for section in ["primaryTickers", "secondaryTickers"]:
97.             tickers = metadata.get(section, {}).get("data", [])
98.             for ticker in tickers:
99.                 ticker_id = ticker.get("id")
100.                 if ticker_id in self.target_tags:
101.                     return True

```

Objective 2.3

Checks for the target tag as
a primary or secondary
ticker on the article

```

109.         return False
110.
111.
112.     def get_page(self, url):
113.         try:
114.             #using requests library to access url
115.             response = self.session.get(url, timeout=15)
116.         except requests.RequestException as error:
117.             print(f"Request failed: {error}")
118.             return None
119.
120.         #typical response if bot detection kicks in
121.         if response.status_code == 403:
122.             print("Error 403: You got blocked :(")
123.             print("try switching cookies or manually accessing the site, if it wont work
you might be IP blocked")
124.             return None
125.         elif response.status_code == 404:
126.             print("attempted to access a page that does not exist")
127.             return None
128.         elif response.status_code != 200: #200 means status is okay
129.             print(f"some other error occurred: {response.status_code}")
130.             return None
131.
132.         try:
133.             return response.json()
134.         except:
135.             print("something went wrong with the json")
136.             return None
137.
138.     def fetch_articles(self, start, end, output_file=None,):
139.         min_delay = self.min_delay
140.         max_delay = self.max_delay
141.
142.         since = int(time.mktime(start.timetuple()))
143.         until = int(time.mktime((end + timedelta(days=1)).timetuple()))
144.
145.         page = 1
146.         relevant_articles = []
147.
148.         cookie_string = self.load_cookies()
149.         self.session = requests.Session()
150.         self.session.headers.update({
151.             "User-Agent": (
152.                 "Mozilla/5.0 (Windows NT 10.0; Win64; x64) "
153.                 "AppleWebKit/537.36 (KHTML, like Gecko) "
154.                 "Chrome/115.0.0.0 Safari/537.36"
155.             ),
156.             "Accept": "application/json",
157.             "Referer": "https://[REDACTED].com/",
158.             "Origin": "https://[REDACTED].com",
159.             "Cookie": cookie_string
160.         })
161.
162.         print("Starting article fetch...")
163.
164.
165.
166.         #initial dummy total for progress bar
167.         #tqdm was initially used for backend testing. i kept it because it doesnt hurt to
have at all
168.         #the progress dict is what is actually used for displaying GUI later
169.         pbar = tqdm(total=1, desc="Fetching pages", unit="page")
170.
171.         self.progress["state"] = "Fetching pages..."
172.         #for dynamic estimation
173.         all_page_days = []
174.
175.         while True:

```

Objective 2.2

Outputs an appropriate message to console if a HTTP error is hit, and appropriately stops the rest of the scraping process

Fetches articles via the web API and constructed URL, handling pages and random delays to respect rate limits

```

176.         url = self.build_url(since, until, page)
177.         data = self.get_page(url)
178.
179.         if not data:
180.             break
181.
182.         articles = data.get("data", [])
183.         if not articles:
184.             break
185.
186.         sentiments = self.relevant_sentiments(data.get("included", []))
187.
188.         new_articles_this_page = 0
189.         page_dates = []
190.
191.         for i in range(len(articles)):
192.             article = articles[i]
193.             if self.has_relevant_tag(article):
194.                 attrs = article.get("attributes", {})
195.                 article_id = article.get("id")
196.                 title = attrs.get("title", "").strip()
197.                 publish_date_str = attrs.get("publishOn")
198.                 publish_date = pd.to_datetime(publish_date_str).date()
199.                 sentiment = sentiments.get(article_id, "none")
200.                 relevant_articles.append((publish_date_str, title, sentiment))
201.                 new_articles_this_page += 1
202.                 page_dates.append(publish_date)
203.
204.         #update dynamic days per page estimate
205.         #figures out the date range you cover with each page pull, and therefore how
many pages will be needed
206.         if page_dates:
207.             earliest = min(page_dates)
208.             latest = max(page_dates)
209.             days_covered = (latest - earliest).days + 1
210.             all_page_days.append(days_covered)
211.             avg_days_per_page = sum(all_page_days) / len(all_page_days)
212.         else:
213.             avg_days_per_page = 1 # fallback
214.
215.         #recalculate estimated total pages
216.         total_days = (end.date() - start.date()).days + 1
217.         estimated_total_pages = max(1, int(total_days / max(1, avg_days_per_page)))
218.         pbar.total = max(page, estimated_total_pages)
219.
220.         self.progress["total"] = max(page, estimated_total_pages)
221.
222.         pbar.refresh()
223.
224.         #advance page and progress bar
225.         self.progress["current"] = page
226.         page += 1
227.         pbar.update(1)
228.
229.         #delay between requests
230.         delay = random.uniform(min_delay, max_delay)
231.         time.sleep(delay)
232.
233.         pbar.close()
234.         self.progress["state"] = "idle"
235.         # Sort articles by date using built-in sort
236.         relevant_articles.sort(key=lambda x: x[0])
237.
238.         if output_file is not None:
239.             with open(output_file, "w", newline="", encoding="utf-8") as f:
240.                 writer = csv.writer(f)
241.                 writer.writerow(["Date", "Title", "Sentiment"])
242.                 for a in relevant_articles:
243.                     writer.writerow(a)

```

Objective 2.4

Waits a random time
between the min and max
set delay

Sorts the articles list by
index 0, which in this
case is the date

```

244.         print(f"\nSuccess! Saved {len(relevant_articles)} relevant articles to
{output_file}")
245.     else:
246.         return relevant_articles
247.

248. #for backend testing, this will run if the script is run without being called by another
249. if __name__ == "__main__":
250.     start_date = datetime(2025, 1, 1)
251.     end_date = datetime(2025, 6, 1)
252.
253.     scraper = ArticleScraper(
254.         cookies_path= "training_resources\\fetching\\scraper\\cookies.json",
255.         target_tag="80",
256.         page_size=50,
257.         category="market-outlook::gold-and-precious-metals",
258.         min_delay=0,
259.         max_delay=1
260.     )
261.
262.     scraper.fetch_articles(start_date, end_date,
"training_resources\\fetching\\scraper\\articles.csv")
263.

```

price_fetcher.py

This script calls the *yfinance* API and returns a DataFrame formatted the way the rest of the pipeline expects it.

```

1. import yfinance as yf
2. from datetime import datetime
3. import os
4. import pandas as pd
5.
6. def fetch_prices(ticker, start_date, end_date="today", interval="1d", adjust=False):
7.
8.     if end_date == "today":
9.         end_date = datetime.now().strftime('%Y-%m-%d')
10.
11.     print("Pulling OHLC...")
12.     data = yf.download(ticker, start=start_date, end=end_date, interval=interval,
auto_adjust=adjust)
13.     print("OHLC Pull Complete.")
14.
15.     open_col = data[("Open", ticker)]
16.     high_col = data[("High", ticker)]
17.     low_col = data[("Low", ticker)]
18.     close_col = data[("Close", ticker)]
19.
20.     if adjust:
21.         adj_close_col = close_col
22.     else:
23.         adj_close_col = data[("Adj Close", ticker)]
24.
25.     clean_data = pd.DataFrame({
26.         "Open": open_col,
27.         "High": high_col,
28.         "Low": low_col,
29.         "Close": close_col,
30.         "Adj Close": adj_close_col
31.     }, index=data.index)
32.
33.     return clean_data
34.

```

Objective 2.5

Fetches OHLC data from
yfinance

```

35. if __name__ == "__main__":
36.     csv_path = "training_resources\\processing\\indicator_calculator\\ohlc_data.csv"
37.
38.     try:
39.         data = fetch_prices("GLD", "2016-01-01", "today")
40.         print(data)
41.
42.         save_csv = input("\nOutput CSV? (y/n): ").lower()
43.         if save_csv == "y":
44.             data.to_csv(csv_path)
45.             print(f"CSV saved at: {os.path.abspath(csv_path)}")
46.
47.     except Exception as e:
48.         print(f"Error: {e}")
49.
50.

```

Processing

indicator_calculator.py

Calculates indicators based on the DataFrame passed in. You can customise the parameters used for the calculations in question. For specific formulae, refer to the research section on technical indicators. To enforce DataFrame loading, a wrapper is used. Implementation was added following methods shown in *GeeksForGeeks*¹². All technical indicators implemented are standard definitions taken from financial analysis literature, implemented independently in Python.

```

1. import pandas as pd
2. import numpy as np
3. import copy
4.
5. #default parameters
6. DEFAULT_PARAMS = {
7.     "rsi_length": [14],
8.     "macd": {"fast": 12, "slow": 26, "signal": 9},
9.     "bollinger": {"length": 20, "std": 2},
10.    "stochastic_osc": {"k": 14, "d": 3, "smooth_k": 3},
11.    "atr_length": [14],
12.    "volatility_windows": [5, 10, 20, 30],
13.    "ema_lengths": [10, 14, 20, 50],
14.    "slope_windows": [5, 10, 14, 20],
15.    "psy_length": [12],
16.    "cci_length": [20],
17.    "cmo_length": [14],
18.    "gap_features": True
19. }
20.
21. #decorator to enforce all other functions have df loaded before running
22. def requires_df(func):
23.     #even if all funcs currently just use self, i dont want it to break if i add anything
24.     else
25.         def wrapper(self, *args, **kwargs):
26.             if self.df is None:
27.                 raise ValueError(f"A dataset is required to run {func.__name__}, and has not
28.                 been provided")
29.             return func(self, *args, **kwargs)
30.         return wrapper
31.
32. class IndicatorCalculator:
33.

```

This dictionary contains nested dictionaries to define necessary parameters for indicator calculations

Objective 4

This script as a whole is designed process OHLC data into technical features

Objective 4.1

Parameters for calculations are defined, and can be overwritten later

¹² <https://www.geeksforgeeks.org/python/decorators-in-python/>

```

32.     def __init__(self, params=None):
33.
34.         self.df = None
35.         self.price_col = None
36.         #avoid changing the defaults
37.         self.params = copy.deepcopy(DEFAULT_PARAMS)
38.
39.         #validate parameters
40.         if params == None:
41.             print("Using default parameters to calculate indicators")
42.         else:
43.             for p in params:
44.                 if p in DEFAULT_PARAMS:
45.                     self.params[p] = params[p]
46.                 else:
47.                     print(f"{p} is not a valid parameter, using defaults")
48.
49.             for p in DEFAULT_PARAMS:
50.                 if p not in params:
51.                     del self.params[p]
52.                     print(f"Removed default feature: {p}")
53.
54.
55.         #initially this was part of instanting the class, but as the df wont exist all the
time, its helpful to be able to instance without one
56.         def load_dataframe(self, df):
57.             self.df = df
58.             self.price_col = "Adj Close" if "Adj Close" in self.df.columns else "Close"
59.
60.         #to calculate warmup
61.         def max_warmup(self):
62.             lookback = []
63.
64.             for key, value in self.params.items():
65.                 if isinstance(value, list) and len(value) > 0 and isinstance(value[0],
int):
66.                     lookback.append(max(value))
67.
68.             if 'macd' in self.params:
69.                 lookback.append(self.params['macd']['slow'])
70.             if 'stochastic_osc' in self.params:
71.                 lookback.append(self.params['stochastic_osc']['k'])
72.             if 'bollinger' in self.params:
73.                 lookback.append(self.params['bollinger']['length'])
74.
75.             return max(lookback) if lookback else 0
76.
77.         def get_indicator_columns(self):
78.             cols = []
79.
80.             if "rsi_length" in self.params:
81.                 for length in self.params["rsi_length"]:
82.                     cols.append(f"RSI_{length}")
83.
84.             if "macd" in self.params:
85.                 cols.extend(["MACD", "MACD_Signal", "MACD_Hist"])
86.
87.             if "bollinger" in self.params:
88.                 cols.extend(["BB_Mid", "BB_Upper", "BB_Lower"])
89.
90.             if "stochastic_osc" in self.params:
91.                 cols.extend(["%K", "%D"])
92.
93.             if "atr_length" in self.params:
94.                 for length in self.params["atr_length"]:
95.                     cols.append(f"ATR_{length}")
96.
97.             if "volatility_windows" in self.params:
98.                 for w in self.params["volatility_windows"]:

```

```

99.         cols.append(f"Vol_{w}d")
100.
101.         windows = self.params["volatility_windows"]
102.         if len(windows) >= 2:
103.             shortest = min(windows)
104.             medium = min(windows, key=lambda w: abs(w - np.mean(windows)))
105.             cols.append(f"Vol_Ratio_{shortest}_{medium}")
106.
107.         if "gap_features" in self.params and self.params["gap_features"]:
108.             cols.append("Gap")
109.             cols.extend(["Gap_EMA_Norm", "Gap_2d"])
110.
111.         if "psy_length" in self.params:
112.             for length in self.params["psy_length"]:
113.                 cols.append(f"PSY_{length}")
114.
115.         if "cci_length" in self.params:
116.             for length in self.params["cci_length"]:
117.                 cols.append(f"CCI_{length}")
118.
119.         if "cmo_length" in self.params:
120.             for length in self.params["cmo_length"]:
121.                 cols.append(f"CMO_{length}")
122.
123.         if "ema_lengths" in self.params:
124.             cols.extend(["Price_EMA14_Ratio", "ATR_EMA_Ratio"])
125.
126.             ema_lengths = self.params["ema_lengths"]
127.             if len(ema_lengths) >= 2:
128.                 for i in range(len(ema_lengths)-1):
129.                     cols.append(f"EMA_Ratio_{ema_lengths[i]}_{ema_lengths[i+1]}")
130.
131.         if "slope_windows" in self.params:
132.             for window in self.params["slope_windows"]:
133.                 cols.append(f"Price_Slope_{window}d")
134.                 cols.append(f"MACD_Slope_{window}d")
135.                 cols.append(f"RSI_Slope_Div_{window}d")
136.                 cols.append(f"MACD_Slope_Div_{window}d")
137.                 cols.append(f"Vol_Slope_Div_{window}d")
138.
139.         return cols
140.
141.         #all indicators are standard calculations
142.
143.         @requires_df
144.         def add_rsi(self):
145.             for length in self.params["rsi_length"]:
146.                 delta = self.df[self.price_col].diff()
147.                 gain = delta.clip(lower=0)
148.                 loss = -delta.clip(upper=0)
149.
150.                 avg_gain = gain.ewm(alpha=1/length, adjust=False).mean()
151.                 avg_loss = loss.ewm(alpha=1/length, adjust=False).mean()
152.
153.                 rs = avg_gain / avg_loss
154.                 self.df[f"RSI_{length}"] = 100 - (100 / (1 + rs))
155.             return self.df
156.
157.         @requires_df
158.         def add_macd(self):
159.             fast = self.params["macd"]["fast"]
160.             slow = self.params["macd"]["slow"]
161.             signal = self.params["macd"]["signal"]
162.
163.             #use exponential weighted average for macd. might work better
164.
165.             ema_fast = self.df[self.price_col].ewm(span=fast, adjust=False).mean()
166.             ema_slow = self.df[self.price_col].ewm(span=slow, adjust=False).mean()
167.

```

Objective 4.2

Below are all technical features that use the defined parameters

```

168.         self.df["MACD"] = ema_fast - ema_slow
169.         self.df["MACD_Signal"] = self.df["MACD"].ewm(span=signal, adjust=False).mean()
170.         self.df["MACD_Hist"] = self.df["MACD"] - self.df["MACD_Signal"]
171.         return self.df
172.
173.     @requires_df
174.     def add_bollinger_bands(self):
175.         length = self.params["bollinger"]["length"]
176.         sdev_multiplier = self.params["bollinger"]["std"]
177.
178.         moving_average = self.df[self.price_col].rolling(length).mean()
179.         standard_dev = self.df[self.price_col].rolling(length).std()
180.
181.         self.df["BB_Mid"] = moving_average
182.         self.df["BB_Upper"] = moving_average + sdev_multiplier * standard_dev
183.         self.df["BB_Lower"] = moving_average - sdev_multiplier * standard_dev
184.         return self.df
185.
186.     @requires_df
187.     def add_stochastic(self):
188.         k_length = self.params["stochastic_osc"]["k"]
189.         d_length = self.params["stochastic_osc"]["d"]
190.         smooth_k = self.params["stochastic_osc"]["smooth_k"]
191.
192.         low_min = self.df["Low"].rolling(k_length).min()
193.         high_max = self.df["High"].rolling(k_length).max()
194.
195.         k = ((self.df[self.price_col] - low_min) / (high_max - low_min)) * 100
196.         self.df["%K"] = k.rolling(smooth_k).mean()
197.         self.df["%D"] = self.df["%K"].rolling(d_length).mean()
198.         return self.df
199.
200.     @requires_df
201.     def add_atr(self):
202.         for length in self.params["atr_length"]:
203.             high_low = self.df["High"] - self.df["Low"]
204.             high_close = (self.df["High"] - self.df[self.price_col].shift()).abs()
205.             low_close = (self.df["Low"] - self.df[self.price_col].shift()).abs()
206.
207.             tr = pd.concat([high_low, high_close, low_close], axis=1).max(axis=1)
208.             self.df[f"ATR_{length}"] = tr.rolling(length).mean()
209.         return self.df
210.
211.     @requires_df
212.     def add_volatility(self):
213.         windows = self.params["volatility_windows"]
214.         ret = self.df[self.price_col].pct_change()
215.
216.         for w in windows:
217.             self.df[f"Vol_{w}d"] = ret.rolling(w).std()
218.
219.         short_ratio = min(windows)
220.         #looking for the closest window to the average length
221.         medium_ratio = min(windows, key=lambda w: abs(w-np.mean(windows)))
222.
223.         # short to medium ratio
224.         self.df[f"Vol_Ratio_{short_ratio}_{medium_ratio}"] = self.df[f"Vol_{short_ratio}d"]
225.         / self.df[f"Vol_{medium_ratio}d"]
226.         return self.df
227.
228.     @requires_df
229.     def add_gap(self):
230.         if ("Open" not in self.df.columns) or (not self.params["gap_features"]):
231.             return self.df
232.
233.         prev_close = self.df[self.price_col].shift(1)
234.         self.df["Gap"] = (self.df["Open"] - prev_close) / prev_close
235.
236.         #paper's most important feature was EMA normalized gap

```

```

236.         #this was ranked #1 in feature importance across all their models
237.         ema_14 = self.df[self.price_col].ewm(span=14, adjust=False).mean()
238.         self.df["Gap_EMA_Norm"] = (self.df["Open"] - prev_close) / ema_14
239.
240.         #2-day gap (paper also found this useful)
241.         self.df["Gap_2d"] = (self.df["Open"] - self.df[self.price_col].shift(2)) / ema_14
242.
243.         return self.df
244.
245.     @requires_df
246.     def add_ema_ratios(self):
247.         if 'ema_lengths' not in self.params or not self.params['ema_lengths']:
248.             return self.df
249.
250.         ema_lengths = self.params["ema_lengths"]
251.
252.         emas = {}
253.         for length in ema_lengths:
254.             emas[length] = self.df[self.price_col].ewm(span=length, adjust=False).mean()
255.
256.         #short long ema ratio
257.         for i in range(len(ema_lengths)-1):
258.             short = ema_lengths[i]
259.             long = ema_lengths[i+1]
260.             self.df[f"EMA_Ratio_{short}_{long}"] = emas[short] / emas[long]
261.
262.         #price ema ratio
263.         if len(emas) > 0:
264.             reference_ema = min(ema_lengths) #shortest ema for ref
265.             self.df[f"Price_EMA{reference_ema}_Ratio"] = self.df[self.price_col] /
emas[reference_ema]
266.
267.         #atr ema ratio
268.         if 'atr_length' in self.params and len(self.params['atr_length']) > 0:
269.             atr_col = f"ATR_{self.params['atr_length']}[0]"
270.             if atr_col in self.df.columns:
271.                 self.df[f"ATR_EMA{reference_ema}_Ratio"] = self.df[atr_col] /
emas[reference_ema]
272.
273.         return self.df
274.
275.     @requires_df
276.     def add_slope_divergences(self):
277.         #this ranked high in paper importance
278.
279.         for window in self.params["slope_windows"]:
280.             #price slope (rate of change over window)
281.             price_slope = self.df[self.price_col].diff(window) / window
282.             self.df[f"Price_Slope_{window}d"] = price_slope
283.
284.             #MACD slope divergence
285.             if "MACD" in self.df.columns:
286.                 macd_slope = self.df["MACD"].diff(window) / window
287.                 self.df[f"MACD_Slope_{window}d"] = macd_slope
288.                 #divergence
289.                 self.df[f"MACD_Slope_Div_{window}d"] = price_slope - macd_slope
290.
291.             #RSI slope divergence
292.             if "RSI_14" in self.df.columns:
293.                 rsi_slope = self.df["RSI_14"].diff(window) / window
294.                 self.df[f"RSI_Slope_Div_{window}d"] = price_slope - rsi_slope
295.
296.             #volume slope divergence
297.             if "Volume" in self.df.columns:
298.                 vol_slope = self.df["Volume"].diff(window) / window
299.                 self.df[f"Vol_Slope_Div_{window}d"] = price_slope - vol_slope
300.
301.         return self.df
302.

```

```

303.     @requires_df
304.     def add_psy(self):
305.         #psychological line is the percentage of up days over a period
306.         # Paper ranked this highly for feature importance
307.         for length in self.params["psy_length"]:
308.             up_days = (self.df[self.price_col].diff() > 0).rolling(length).sum()
309.             self.df[f"PSY_{length}"] = (up_days / length) * 100
310.         return self.df
311.
312.     @requires_df
313.     def add_cci(self):
314.         for length in self.params["cci_length"]:
315.             #measures deviation from average price
316.             #particularly useful for commodities like gold
317.
318.             typical_price = (self.df["High"] + self.df["Low"] + self.df[self.price_col]) /
319.             3
320.             sma_tp = typical_price.rolling(length).mean()
321.
322.             #mean absolute deviation
323.             mad = typical_price.rolling(length).apply(lambda x: np.abs(x -
324.             x.mean()).mean())
325.             self.df[f"CCI_{length}"] = (typical_price - sma_tp) / (0.015 * mad)
326.         return self.df
327.
328.     @requires_df
329.     def add_cmo(self):
330.         for length in self.params["cmo_length"]:
331.             #momentum indicator similar to RSI
332.             #paper found this useful for capturing momentum shifts
333.
334.             delta = self.df[self.price_col].diff()
335.             gain = delta.clip(lower=0).rolling(length).sum()
336.             loss = -delta.clip(upper=0).rolling(length).sum()
337.
338.             self.df[f"CMO_{length}"] = 100 * (gain - loss) / (gain + loss)
339.         return self.df
340.
341.     @requires_df
342.     def add_all_indicators(self):
343.         #a dict of functions (just figured out i can do this, time to have some fun :p)
344.         methods = {
345.             'rsi_length': self.add_rsi,
346.             'macd': self.add_macd,
347.             'bollinger': self.add_bollinger_bands,
348.             'stochastic_osc': self.add_stochastic,
349.             'atr_length': self.add_atr,
350.             'volatility_windows': self.add_volatility,
351.             'gap_features': self.add_gap,
352.             'ema_lengths': self.add_ema_ratios,
353.             'slope_windows': self.add_slope_divergences,
354.             'psy_length': self.add_psy,
355.             'cci_length': self.add_cci,
356.             'cmo_length': self.add_cmo,
357.         }
358.         for key, method in methods.items():
359.             if key in self.params:
360.                 method()
361.
362.         return self.df
363.
364.
365. #testing
366. if __name__ == "__main__":
367.     csv_file = "training_resources\\processing\\indicator_calculator\\ohlcv_data.csv"
368.     df = pd.read_csv(csv_file, index_col=0, parse_dates=True)
369.

```

```

370.     numeric_cols = ["Open", "High", "Low", "Close", "Adj Close"]
371.     for col in numeric_cols:
372.         if col in df.columns:
373.             df[col] = df[col].astype(float)
374.
375.     calc = IndicatorCalculator()
376.     calc.load_dataframe(df)
377.     df_with_indicators = calc.add_all_indicators()
378.
379.     print(df_with_indicators)
380.
381.     df_with_indicators.to_csv(
382.         "training_resources\\processing\\indicator_calculator\\ohlcv_data_with_indicators.csv",
383.         index=True
384.     )
385.     print("Indicators added and saved to 'ohlcv_data_with_indicators.csv'")
386.

```

sentiment analysis.py

This module integrates a pretrained financial sentiment transformer model (FinRoBERTa) to classify news headlines into negative, neutral, and positive sentiment. Rather than using discrete labels, the model's output probabilities are extracted and converted into a continuous sentiment score suitable for use as a numerical feature. Temperature scaling, contextual prompting, and a custom nonlinear scaling function are applied to reduce overconfidence and preserve strong directional sentiment. All calibration parameters are configurable to support controlled experimentation.

```

1. from transformers import AutoModelForSequenceClassification, AutoTokenizer
2. import torch
3. import torch.nn.functional as F
4. import math
5.
6. class SentimentAnalyser:
7.
8.     #I changed this to a class so that if I change the ticker, I can make presets for
different tickers
9.     #For the sake of the NEA, I will use the gold market as a default
10.    #Also, be careful with the bias prompt, subtle wording changes can lead to different
results
11.    #It is a lot easier to tell how results will change if you tweak temperature and
neutrality
12.    #Temperature brings mid values closer to 0, while neutrality scales everything down
based on how neutral it is
13.    #Default values were empirically derived by testing on articles and seeing how
appropriate the outputs were.
14.    def __init__(self, bias_prompt="Gold market outlook: ", temperature=1.6,
neutrality_scaler=7.7, testing=False):
15.
16.        self.tokenizer = AutoTokenizer.from_pretrained("soleimanian/financial-roberta-large-
sentiment")
17.        self.model =
AutoModelForSequenceClassification.from_pretrained("soleimanian/financial-roberta-large-
sentiment")
18.        self.bias_prompt = bias_prompt
19.        self.temperature = temperature
20.        self.neutrality_scaler = neutrality_scaler
21.        self.testing = testing
22.
23.    @staticmethod
24.
25.    #i know its only used one time but this looks CLEAN

```

Objective 3

The script as a whole is designed to meet this objective

Objective 3.6

Key parameters are exposed in this class

Objective 3.1

Loads a pretrained FinRoBERTa model

```

26. def sign(x):
27.     if x > 0:
28.         return 1
29.     elif x < 0:
30.         return -1
31.     else:
32.         return 0
33.
34. def scale_score(self, score):
35.     #initial implementation: scaled_score = score * (1 - neutral * neut
36.     #i found a working polynomial function by messing around on desmos
37.     #it works such that scores closer to 1 are scaled down harder, but still is possible
38.     to get a score of 1
39.     #at the same time, the gradient levels off around 0.75, and is relatively straight
40.     till 0.25. after this point it curves down
41.     #i also designed it such that the y intercept is 0, so that truly neutral scores are
42.     still possible.
43.
44.     score_sign = self.sign(score)
45.     score = abs(score)
46.
47.     #all of the (score-0.5) mentions are just to shift the graph to the right
48.     scaled_score = score_sign * ((score - 0.5) - self.neutrality_scaler*(score-0.5)**3 +
49.     4*self.neutrality_scaler*(score-0.5)**5 + 0.5)
50.
51.     #letting neutrality_scaler be a constant makes the function look a lot simpler
52.     #for values above 0: f(x) = x - kx^3 + 4kx^5 + 0.5
53.     #the graph is transformed such that the new function g(x) = f(x-0.5)
54.     #then i just used the sign so it works for negative values as well
55.
56.     return scaled_score
57.
58. def analyse_sentiment(self, text):
59.     inputs = self.tokenizer(self.bias_prompt + text, return_tensors="pt",
60.     truncation=True, padding=True, max_length=512)
61.
62.     with torch.no_grad():
63.         outputs = self.model(**inputs)
64.         logits = outputs.logits
65.
66.         # reduce overconfidence
67.         probs = F.softmax(logits / self.temperature, dim=1)
68.
69.         negative = probs[0][0].item()
70.         neutral = probs[0][1].item()
71.         positive = probs[0][2].item()
72.
73.         score = self.scale_score(positive - negative)
74.
75.         if self.testing:
76.             return score, negative, neutral, positive
77.         else:
78.             return score
79.
80. if __name__ == "__main__":
81.     print("TESTING SENTIMENT ANALYSIS")
82.
83.     #parameters to mess with :D
84.     analyser = SentimentAnalyser(bias_prompt="Gold market outlook: ",
85.     temperature=1.6,
86.     neutrality_scaler=7.7,
87.     testing=True)
88.
89.     while True:
90.         headline = input("Enter a headline: ")
91.
92.         if headline.lower() == "quit":

```

Objective 3.5

This
normal
fun

A complex polynomial function is used to scale outputs, empirically designed to bound outputs into a stable range for downstream decision-making

Objective 3.3

Bias prompt is appended to text

Objective 3.4

Logits are scaled down via the temperature parameter

Objective 3.2

(positive - negative) probabilities creates a continuous score

```

90.         break
91.
92.         score, negative, neutral, positive = analyser.analyse_sentiment(headline)
93.         print(f"Score: {score:.4f}, Negative: {negative:.4f}, Neutral: {neutral:.4f},
Positive: {positive:.4f}")
94.

```

Prediction Tools

light_gbm.py

This module allows for a LightGBM model to be quickly instanced and trained based on a passed configuration. It also allows for conformal calibration. A helper function for feature importances is included as this will be helpful for determining overfit conditions later and could help in ablation studies.

```

1. import pandas as pd
2. import numpy as np
3. import lightgbm as lgb
4. from datetime import datetime, timedelta
5. import warnings
6. import copy
7. warnings.filterwarnings('ignore')
8.
9. #these were set in place before i did extensive testing, so the defaults here arent great
but do work as a fallback
10.
11. DEFAULT_PARAMS = {
12.     'n_estimators': 250,
13.     'learning_rate': 0.008,
14.     'max_depth': 12,
15.     'num_leaves': 84,
16.     'min_child_samples': 30,
17.     'subsample': 0.85,
18.     'colsample_bytree': 0.7,
19.     'reg_alpha': 0.0005,
20.     'reg_lambda': 2.0,
21.     'max_bin': 125,
22.     'boosting_type': 'gbdt',
23.     'random_state': 42,
24.     'n_jobs': -1,
25.     'verbose': -1
26. }
27.
28.
29. DEFAULT_CONFIG = {
30.     'prediction_mode': 'quantile',
31.     'quantiles': [0.2,0.5,0.8],
32.     'lgbm_params': DEFAULT_PARAMS,
33.     'conformal': True
34. }
35.
36. class LGBMForecaster:
37.     def __init__(self, config):
38.
39.         self.model = None
40.         self.quantile_models = {}
41.
42.         self.config = copy.deepcopy(DEFAULT_CONFIG)
43.         #ensure all necessary config options exist, and use defaults otherwise (this code
snippet is reused from Indicator Calculator)

```

Objective 6

This script provides a reusable class for a LightGBM quantile forecaster

Objective 6.1

LightGBM parameters are predefined and can be configured via the config dictionary

```

44.     if config is None:
45.         print("Using default parameters to calculate indicators")
46.     else:
47.         for c in config:
48.             if c in DEFAULT_CONFIG:
49.                 self.config[c] = config[c]
50.             else:
51.                 print(f"{c} is not a valid parameter, using defaults")
52.
53.     self.prediction_mode = self.config['prediction_mode']
54.     self.conformal_calibration = None
55.
56.     def create_model(self, quantile=None):
57.         params = self.config['lgbm_params'].copy()
58.
59.         if self.prediction_mode == 'quantile' and quantile is not None:
60.             params['objective'] = 'quantile'
61.             params['alpha'] = quantile
62.
63.         return lgb.LGBMRegressor(**params)
64.
65.     def calibrate_conformal(self, X_cal, y_cal):
66.         corrections = {}
67.
68.         for q in self.config['quantiles']:
69.             y_pred = self.quantile_models[q].predict(X_cal)
70.
71.             #residuals are actuals - predicted
72.             residuals = y_cal - y_pred
73.
74.             #get the qth quantile of residuals
75.             #works out the value to give correct coverage when added to preds
76.
77.             correction = np.quantile(residuals, q)
78.
79.             corrections[q] = correction
80.
81.         return corrections
82.
83.     #training, but fit is the name of the function being used so i thought it was more
appropriate of a name.
84.     #point prediction still exists from early experiments, but its mostly redundant
85.     def fit(self, X, y, recalibrate=False, conformal_split=0.2):
86.
87.         if self.prediction_mode == 'point':
88.             self.model = self.create_model()
89.             self.model.fit(X, y)
90.
91.         elif self.prediction_mode == 'quantile':
92.
93.             X_train, y_train = X, y
94.
95.             if recalibrate and self.config['conformal']:
96.                 calib_days = int(len(X)*conformal_split)
97.                 X_cal, X_train = X[:calib_days], X[calib_days:]
98.                 y_cal, y_train = y[:calib_days], y[calib_days:]
99.
100.            for q in self.config['quantiles']:
101.                model = self.create_model(quantile=q)
102.                model.fit(X_train, y_train)
103.                self.quantile_models[q] = model
104.
105.            if recalibrate and self.config['conformal']:
106.                self.conformal_calibration = self.calibrate_conformal(X_cal, y_cal)
107.        else:
108.            raise ValueError(f"no compatibility currently exists for the selected
prediction: {self.prediction_mode}")
109.
110.    def predict(self, X, raw=False):

```

Objective 6.2

Conformal predictions can be added. This function provides the necessary logic

```

111.
112.     if self.prediction_mode == 'point':
113.         return self.model.predict(X)
114.
115.     elif self.prediction_mode == 'quantile':
116.         preds = {}
117.
118.         for q, model in self.quantile_models.items():
119.             preds[q] = model.predict(X)
120.
121.         #apply conformal if it exists
122.         if not raw and self.conformal_calibration is not None:
123.             for q in self.config['quantiles']:
124.                 if q in self.conformal_calibration:
125.                     preds[q] += self.conformal_calibration[q]
126.
127.         return preds
128.     else:
129.         raise ValueError(f"Unknown prediction_mode: {self.prediction_mode}")
130.
131.
132.     def get_feature_importance(self, features):
133.
134.         if self.prediction_mode == 'point':
135.             importance_df = pd.DataFrame({
136.                 'feature': features,
137.                 'importance': self.model.feature_importances_
138.             })
139.             return importance_df.sort_values('importance',
ascending=False).reset_index(drop=True)
140.
141.         elif self.prediction_mode == 'quantile':
142.             importance_df = pd.DataFrame({'feature': features})
143.
144.             for q, model in self.quantile_models.items():
145.                 importance_df[f'importance_q{int(q*100)}'] = model.feature_importances_
146.
147.             importance_cols = [c for c in importance_df.columns if 'importance_' in c]
148.             importance_df['importance_avg'] = importance_df[importance_cols].mean(axis=1)
149.             importance_df = importance_df.sort_values('importance_avg',
ascending=False).reset_index(drop=True)
150.
151.             return importance_df
152.
153.

```

Objective 6.3

Target is predicted for all requested quantiles

DQN Modules

trainer.py

Objective 7

The scripts in this section are designed to meet objective seven

This script manages the training and configuration of the DQN, including the actions, rewards, and observations. The PyTorch documentation ([PyTorch, 2025](#)) was consulted to understand DQN implementation details, particularly for defining the replay buffer, policy and target networks, and training steps. See references for details.

```
1. import os
2. import sys
3. import numpy as np
4. import pandas as pd
5. import torch
6. import matplotlib.pyplot as plt
7. from tqdm import tqdm
8. from typing import List
9. from datetime import datetime
10. import json
11. import copy
12.
13. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
14. if project_root not in sys.path:
15.     sys.path.insert(0, project_root)
16.
17. from prediction_tools.DQN.agent import DQNAgent
18. from prediction_tools.DQN.trading_env import TradingEnvironment
19. from prediction_tools.DQN.testing import test_agent
20.
21. def discover_features(df, config):
22.     exclude_prefixes = ['LGBM_', 'Target_', 'Future_']
23.     exclude_exact = ['Date', 'is_trading_day']
24.
25.     all_cols = df.columns.tolist()
26.     features = {}
27.
28.     for col in all_cols:
29.         if col in exclude_exact:
30.             continue
31.
32.         #check if column should be excluded
33.         exclude = False
34.         for p in exclude_prefixes:
35.             if p in col:
36.                 exclude = True
37.                 break
38.
39.         if exclude:
40.             continue
41.
42.
43.         #default to enabled
44.         features[col] = config.get('enabled_features', {}).get(col, True)
```

```

45.
46.     return features
47.
48. #default configuration
49. DEFAULT_CONFIG = {
50.     #model params
51.     'model_name': 'DQN_GLD_3', #base name for saved models
52.     'load_model': 'DQN_GLD_3',
53.     'model_save_path': r'prediction_tools\saved_models\DQN',
54.
55.     #data settings
56.     'csv_path': r'training_resources\datasets\dqn_all_gld.csv',
57.     'plot_path': 'test_images',
58.
59.     'horizons': [7, 14, 30, 90, 365], #multiple horizons to observe
60.     'horizon_split_threshold': 30, #days - horizons ≤ this are short-term, > this are
long-term
61.     'lgbm_upper': 0.9,
62.     'lgbm_lower': 0.1,
63.
64.     'train_split': 0.8,
65.     'test_from_start': False,
66.
67.     #episode settings
68.     'lookback_window': 30,
69.     'episode_length_days': 252,
70.     'initial_balance': 10000.0,
71.     'transaction_costs': 0.001,
72.     'max_positions': 10,
73.
74.     'enabled_features': {}, #is populated later
75.
76.     #training parameters
77.     'episodes': 1000,
78.     'batch_size': 128,
79.     'learning_rate': 0.0002,
80.     'gamma': 0.985,
81.     'epsilon_start': 1.0,
82.     'epsilon_end': 0.01,
83.     'epsilon_decay': 0.985,
84.     'target_update_freq': 50,
85.     'tau': 0.01,
86.     'memory_size': 30000,
87.
88.     # regularization
89.     'noise_std': 0.03,
90.
91.     #what each integer dqn action corresponds to (for human readability)
92.     'action_types': ['close', 'hold', 'pyramid_0.1', 'pyramid_0.25', 'pyramid_0.5'],
93.     'pyramid_sizes': [0.1, 0.25, 0.5],
94.
95.     #reward parameters
96.     'reward_profit_scale': 5.0,
97.     'punish_loss_scale': 4.2,
98.     'loss_penalty_nonlinear': True,
99.     'reward_rising_hold': 2.0,
100.    'punish_falling_hold': 2.0,
101.    'reward_sharpe_bonus': 0.5,
102.    'sharpe_clip_min': -1.0,
103.    'sharpe_clip_max': 1.0,
104.
105.
106.    #drawdown penalties
107.    'drawdown_penalty_enabled': False,
108.    'drawdown_hold_threshold': 0.08,
109.    'drawdown_hold_penalty_scale': 0.5,
110.    'drawdown_hold_penalty_nonlinear': False,
111.
112.    #progressive indecision penalty (replaces cowardly penalty)

```

Objective 7.1

Extensive customisation options are available for all aspects of the DQN

```

113.     'indecision_penalty_enabled': True,
114.     'indecision_penalty_start': 0.0, # Starts at zero
115.     'indecision_penalty_max': 0.01, # Maximum penalty per step
116.     'indecision_penalty_ramp_steps': 60, # Steps to reach max penalty
117.     'indecision_penalty_decay_rate': 0.95, # How fast it decays after taking action
118.     'indecision_use_lgbm_confidence': True,
119.     'indecision_confidence_threshold': 0.75,
120.     'confidence_horizons': [7, 365], #which horizons to use: 'all', 'short', 'long', or
list like [7, 30]
121.     'lgbm_confidence_window': 10,
122.
123.     'opening_penalty': 0.001,
124.
125.     #reward frequency
126.     'reward_frequency': 'step',
127.     'step_reward_scale': 0.08,
128.
129.     #reward clipping
130.     'reward_clip': True,
131.     'reward_clip_min': -5.0,
132.     'reward_clip_max': 5.0,
133.
134.     #network architecture
135.     'conv_filters': [64, 128, 64],
136.     'fc_layers': [256, 128],
137.     'kernel_size': 3,
138.
139.     #optimisation settings
140.     'train_steps_per_episode': 4,
141.
142.     #device
143.     'device': 'cuda' if torch.cuda.is_available() else 'cpu'
144. }
145.
146. class DQNTrainer:
147.     def __init__(self, config=None):
148.
149.         #if none use an empty dict so default can override properly
150.         config = config or {}
151.
152.         override_config = copy.deepcopy(DEFAULT_CONFIG)
153.
154.         for c in config:
155.             if c in override_config:
156.                 override_config[c] = config[c]
157.             else:
158.                 print(f"Invalid parameter {c} in config override")
159.
160.         self.config = override_config
161.
162.         self.save_dir = self.config.get('model_save_path',
r'prediction_tools\saved_models\DQN')
163.         self.model_name = self.config.get('model_name', 'DQN')
164.         self.load_name = self.config.get('load_model', 'DQN')
165.
166.         self.policy_path = os.path.join(self.save_dir, f"{self.model_name}_policy.pt")
167.         self.policy_load_path = os.path.join(self.save_dir, f"{self.load_name}_policy.pt")
168.
169.         self.target_path = os.path.join(self.save_dir, f"{self.model_name}_target.pt")
170.         self.target_load_path = os.path.join(self.save_dir, f"{self.load_name}_target.pt")
171.
172.         self.optimiser_path = os.path.join(self.save_dir,
f"{self.model_name}_optimiser.pt")
173.         self.optimiser_load_path = os.path.join(self.save_dir,
f"{self.load_name}_optimiser.pt")
174.
175.         #progress tracking dict
176.         #absolute lifesaver for the UI, before adding this dict i had to use regex to read
tqdm bars

```

```

177.         #that was such a waste of time and not even very accurate. this was a much easier
solution...
178.         self.progress = {
179.             "current": 0,
180.             "total": 0,
181.             "state": "idle",
182.             "metadata": {},
183.             "history": {
184.                 'rewards': [],
185.                 'returns': [],
186.                 'sharpes': [],
187.                 'losses': []
188.             }
189.         }
190.
191.     def load_metadata(self):
192.         metadata_path = os.path.join(self.save_dir, f"{self.load_name}_metadata.json")
193.
194.         if os.path.exists(metadata_path):
195.             try:
196.                 with open(metadata_path, 'r') as f:
197.                     metadata = json.load(f)
198.
199.                     if 'enabled_features' in metadata and metadata['enabled_features']:
200.                         print(f"Found enabled_features in metadata:
{len(metadata['enabled_features'])} features")
201.                         return metadata['enabled_features']
202.                     else:
203.                         print("No enabled_features found in metadata. Using all features.")
204.                         return None
205.             except Exception as e:
206.                 print(f"Warning: Could not load metadata: {e}")
207.                 return None
208.         else:
209.             print("No metadata file found. Using all features.")
210.             return None
211.
212.     def save_model(self, agent, test_stats):
213.         os.makedirs(self.save_dir, exist_ok=True)
214.
215.         torch.save(agent.policy_net.state_dict(), self.policy_path)
216.         torch.save(agent.target_net.state_dict(), self.target_path)
217.         torch.save(agent.optimiser.state_dict(), self.optimiser_path)
218.
219.         #save config and metadata
220.         metadata = {
221.             'config': self.config,
222.             'epsilon': agent.epsilon,
223.             'test_stats': test_stats,
224.             'timestamp': datetime.now().strftime('%Y%m%d_%H%M%S'),
225.             'enabled_features': self.config.get('enabled_features', {}),
226.             'feature_list': [col for col, enabled in self.config.get('enabled_features',
{}).items() if enabled],
227.             'num_features': len([col for col, enabled in
self.config.get('enabled_features', {}).items() if enabled])
228.         }
229.         metadata_path = os.path.join(self.save_dir, f"{self.model_name}_metadata.json")
230.
231.
232.         metadata['config'] = self.config
233.
234.         with open(metadata_path, 'w') as f:
235.             json.dump(metadata, f, indent=4)
236.
237.         print(f"\nModel saved to {self.save_dir}")
238.
239.     def load_model(self, agent):
240.

```

```

241.         if os.path.exists(self.policy_path) and os.path.exists(self.target_path) and
os.path.exists(self.optimiser_path):
242.             #load weights
243.             agent.policy_net.load_state_dict(torch.load(self.policy_load_path,
map_location=self.config['device']))
244.             agent.target_net.load_state_dict(torch.load(self.target_load_path,
map_location=self.config['device']))
245.             agent.optimiser.load_state_dict(torch.load(self.optimiser_load_path,
map_location=self.config['device']))
246.
247.             print(f"Loaded existing model from {self.save_dir}")
248.         else:
249.             print("No saved model found. Starting training from scratch.")
250.
251.     def plot_training_progress(self, episode_returns, episode_sharpes, losses,
bankruptcies):
252.
253.
254.         #learning curves
255.         fig, axes = plt.subplots(2, 2, figsize=(15, 10))
256.
257.         episodes = np.arange(1, len(episode_returns) + 1)
258.
259.         #episode Returns
260.         ax1 = axes[0, 0]
261.         returns_pct = np.array(episode_returns) * 100
262.         ax1.plot(episodes, returns_pct, alpha=0.3, color='blue', label='Episode Return')
263.
264.         window = 20
265.         if len(returns_pct) >= window:
266.             rolling_avg = pd.Series(returns_pct).rolling(window=window,
min_periods=1).mean()
267.             ax1.plot(episodes, rolling_avg, color='red', linewidth=2, label=f'{window}-
Episode Average')
268.
269.             ax1.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
270.             ax1.set_xlabel('Episode', fontsize=11)
271.             ax1.set_ylabel('Return (%)', fontsize=11)
272.             ax1.set_title('Training Returns Over Time', fontsize=12, fontweight='bold')
273.             ax1.legend()
274.             ax1.grid(True, alpha=0.3)
275.
276.         #sharpe ratios
277.         ax2 = axes[0, 1]
278.         sharpes = np.array(episode_sharpes)
279.         ax2.plot(episodes, sharpes, alpha=0.3, color='green', label='Episode Sharpe')
280.
281.         if len(sharpes) >= window:
282.             rolling_sharpe = pd.Series(sharpes).rolling(window=window,
min_periods=1).mean()
283.             ax2.plot(episodes, rolling_sharpe, color='darkgreen', linewidth=2,
label=f'{window}-Episode Average')
284.
285.             ax2.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
286.             ax2.axhline(y=1, color='orange', linestyle='--', alpha=0.5, label='Sharpe = 1.0')
287.             ax2.set_xlabel('Episode', fontsize=11)
288.             ax2.set_ylabel('Sharpe Ratio', fontsize=11)
289.             ax2.set_title('Sharpe Ratio Over Time', fontsize=12, fontweight='bold')
290.             ax2.legend()
291.             ax2.grid(True, alpha=0.3)
292.
293.         #training Loss
294.         ax3 = axes[1, 0]
295.         if losses:
296.             loss_episodes = np.arange(1, len(losses) + 1)
297.             ax3.plot(loss_episodes, losses, alpha=0.4, color='red', label='Training Loss')
298.
299.             if len(losses) >= window:

```

```

300.         rolling_loss = pd.Series(losses).rolling(window=window,
min_periods=1).mean()
301.         ax3.plot(loss_episodes, rolling_loss, color='darkred', linewidth=2,
label=f'{window}-Episode Average')
302.
303.         ax3.set_xlabel('Episode', fontsize=11)
304.         ax3.set_ylabel('Loss', fontsize=11)
305.         ax3.set_title('Training Loss Convergence', fontsize=12, fontweight='bold')
306.         ax3.legend()
307.         ax3.grid(True, alpha=0.3)
308.         ax3.set_yscale('log')
309.
310.         #summary statistics
311.         ax4 = axes[1, 1]
312.         ax4.axis('off')
313.
314.         final_return = episode_returns[-1] * 100
315.         avg_return = np.mean(episode_returns[-50:]) * 100 if len(episode_returns) >= 50
else np.mean(episode_returns) * 100
316.         best_return = max(episode_returns) * 100
317.         worst_return = min(episode_returns) * 100
318.
319.         final_sharpe = episode_sharpes[-1]
320.         avg_sharpe = np.mean(episode_sharpes[-50:]) if len(episode_sharpes) >= 50 else
np.mean(episode_sharpes)
321.
322.         bankruptcy_rate = bankruptcies / len(episode_returns) * 100
323.
324.         summary_text = f"""
325.         TRAINING STATS
326.         {'='*35}
327.         Total Episodes:      {len(episode_returns)}
328.
329.         RETURNS
330.         Final Episode:      {final_return:+.2f}%
331.         Best Episode:       {best_return:+.2f}%
332.         Worst Episode:      {worst_return:+.2f}%
333.         Avg (Last 50):      {avg_return:+.2f}%
334.
335.         SHARPE RATIOS
336.         Final Episode:      {final_sharpe:.2f}
337.         Avg (Last 50):      {avg_sharpe:.2f}
338.
339.         Bankruptcies:       {bankruptcies} ({bankruptcy_rate:.1f}%)
340.
341.         LEARNING
342.         Final Epsilon:      {self.config.get('epsilon_end', 0.01):.3f}
343.         Final Loss:         {losses[-1]:.4f}
344.         """
345.
346.         ax4.text(0.1, 0.95, summary_text, transform=ax4.transAxes,
                 fontsize=10, verticalalignment='top', fontfamily='monospace',
347.                 bbox=dict(boxstyle='round', facecolor='lightblue', alpha=0.3))
348.
349.
350.         plt.suptitle('Training Progress', fontsize=14, fontweight='bold')
351.         plt.tight_layout()
352.
353.
354.         os.makedirs(self.config['plot_path'], exist_ok=True)
355.         save_path = os.path.join(self.config['plot_path'], 'training_progress.png')
356.         plt.savefig(save_path, dpi=150, bbox_inches='tight')
357.         print(f"\nTraining plot saved to {save_path}")
358.         plt.close()
359.
360.     def train_dqn(self, use_pretrained=False):
361.
362.         config = self.config
363.         print(f"Using device: {self.config['device']}")
364.         if config['device'] == 'cuda':

```

Custom training loop
managing episodes,
agent memory, network
updates, rewards, and
interaction with the
trading environment

```

365.         print(f"GPU: {torch.cuda.get_device_name(0)}")
366.
367.
368.
369.         print(f"Loading data from: {config['csv_path']}")
370.
371.         df = pd.read_csv(config['csv_path'])
372.
373.         target_cols = [col for col in df.columns if col.startswith("Target") or
374. col.startswith("Future")]
375.         df = df.drop(columns=target_cols)
376.
377.         horizons = config['horizons'] if isinstance(config.get('horizons'), list) else
378. [config.get('horizon', 7)]
379.
380.         #checking what lgbm columns are available
381.
382.         for horizon in horizons:
383.             lgbm_col = f"LGBM_{horizon}d_q50"
384.             if lgbm_col not in df.columns:
385.                 raise ValueError(f"Missing LGBM column: {lgbm_col}")
386.
387.             valid_count = df[lgbm_col].notna().sum()
388.             print(f" {horizon}d horizon: {valid_count:,} / {len(df):,} valid predictions
389. ({valid_count/len(df)*100:.1f}%")
390.
391.             #filter to rows where ALL horizons have predictions (so all episodes are fair in
392. training)
393.             valid_rows = pd.Series([True] * len(df))
394.             for horizon in horizons:
395.                 lgbm_col = f"LGBM_{horizon}d_q50"
396.                 valid_rows &= df[lgbm_col].notna()
397.
398.             rows_before = len(df)
399.             df = df[valid_rows].reset_index(drop=True)
400.             rows_after = len(df)
401.
402.             print(f"\nFiltered rows without LGBM Predictions:")
403.             print(f" Rows before: {rows_before:,}")
404.             print(f" Rows after: {rows_after:,}")
405.             print(f" Removed: {rows_before - rows_after:,} ({(rows_before -
406. rows_after)/rows_before*100:.1f}%")
407.
408.
409.
410.             split_idx = int(len(df) * config['train_split'])
411.
412.             if config.get('test_from_start', False):
413.                 test_data = df.iloc[:split_idx].reset_index(drop=True)
414.                 train_data = df.iloc[split_idx:].reset_index(drop=True)
415.                 print(f"Split: TEST FROM START")
416.             else:
417.                 train_data = df.iloc[:split_idx].reset_index(drop=True)
418.                 test_data = df.iloc[split_idx:].reset_index(drop=True)
419.                 print(f"Split: TEST FROM END")
420.
421.             print(f"Train: {len(train_data)} rows, Test: {len(test_data)} rows\n")
422.
423.             train_env = TradingEnvironment(train_data, config, config['device'],
424. is_training=True)
425.
426.             state = train_env.reset()
427.             state_shape = state.shape
428.             num_actions = len(config['action_types'])
429.
430.             print(f"State shape: {state_shape}, Actions: {num_actions}")
431.
432.             lookback_window, num_features = state_shape
433.             agent = DQNAgent(num_features, lookback_window, num_actions, config)

```

```

428.
429.     episode_rewards = []
430.     episode_returns = []
431.     episode_sharpes = []
432.     losses = []
433.     bankruptcies = 0
434.
435.
436.     if use_pretrained:
437.         print("\nLoading metadata from pretrained model...")
438.         pretrained_features = self.load_metadata()
439.
440.         if pretrained_features is not None:
441.             #override config with pretrained model's features
442.             self.config['enabled_features'] = pretrained_features
443.             enabled_feature_cols = [col for col, enabled in pretrained_features.items()
if enabled]
444.             disabled_count = len([v for v in pretrained_features.values() if not v])
445.             print(f"Using pretrained model's features: {len(enabled_feature_cols)}
enabled, {disabled_count} disabled")
446.         else:
447.             #default use all
448.             print("\nDiscovering features...")
449.             discovered_features = discover_features(df, self.config)
450.             self.config['enabled_features'] = discovered_features
451.             enabled_feature_cols = [col for col, enabled in discovered_features.items()
if enabled]
452.             disabled_count = len([v for v in discovered_features.values() if not v])
453.             print(f"Features: {len(enabled_feature_cols)} enabled, {disabled_count}
disabled")
454.         else:
455.             #fresh training
456.             print("\nDiscovering features...")
457.             discovered_features = discover_features(df, self.config)
458.             self.config['enabled_features'] = discovered_features
459.             enabled_feature_cols = [col for col, enabled in discovered_features.items() if
enabled]
460.             disabled_count = len([v for v in discovered_features.values() if not v])
461.             print(f"Features: {len(enabled_feature_cols)} enabled, {disabled_count}
disabled")
462.
463.
464.         print("\nStarting training...")
465.
466.         pbar = tqdm(range(config['episodes']), desc="Training", ncols=150)
467.         self.progress["state"], self.progress["current"], self.progress["total"] =
"Training...", 0, config['episodes']
468.
469.         self.progress["metadata"] = {
470.             'Reward': '0',
471.             'Avg(10)': '0',
472.             'Return%': '0',
473.             'Avg%(10)': '0',
474.             'Sharpe': '0',
475.             'Eps': '0',
476.             'Loss': '0',
477.             'Bankrupt': '0'
478.         }
479.
480.         #main training logic
481.         for episode in pbar:
482.             state = train_env.reset()
483.             episode_reward = 0
484.             episode_loss = []
485.
486.             step_count = 0
487.
488.             #logic for each episode
489.             while not train_env.done:

```

Agent selects actions and steps through the environment, receiving rewards and next states

```

490.         action = agent.select_action(state, training=True)
491.         next_state, reward, done, info = train_env.step(action)
492.
493.         agent.memory.push(state, action, reward, next_state, done)
494.         episode_reward += reward
495.
496.         #avoid updating weights every single step
497.         if step_count % config['train_steps_per_episode'] == 0:
498.             for _ in range(config['train_steps_per_episode']):
499.                 loss = agent.train_step()
500.                 if loss is not None:
501.                     episode_loss.append(loss)
502.                     agent.soft_update_target_network()
503.
504.         state = next_state
505.         step_count += 1
506.
507.         #bankruptcy protection
508.         if info.get('bankruptcy', False):
509.             bankruptcies += 1
510.             break
511.
512.         episode_rewards.append(episode_reward)
513.         episode_returns.append(info['total_return'])
514.         episode_sharpes.append(info['sharpe_ratio'])
515.         if episode_loss:
516.             losses.append(np.mean(episode_loss))
517.
518.         agent.decay_epsilon()
519.
520.         if episode >= 10:
521.             avg_reward_10 = np.mean(episode_rewards[-10:])
522.             avg_return_10 = np.mean(episode_returns[-10:])
523.         else:
524.             avg_reward_10 = np.mean(episode_rewards)
525.             avg_return_10 = np.mean(episode_returns)
526.
527.         avg_loss = np.mean(episode_loss) if episode_loss else 0
528.
529.         pbar.set_postfix({
530.             'Reward': f'{episode_reward:.2f}',
531.             'Avg(10)': f'{avg_reward_10:.2f}',
532.             'Return%': f'{info["total_return"]*100:.1f}',
533.             'Avg%(10)': f'{avg_return_10*100:.1f}',
534.             'Sharpe': f'{info["sharpe_ratio"]:.2f}',
535.             'Eps': f'{agent.epsilon:.3f}',
536.             'Loss': f'{avg_loss:.4f}',
537.             'Bankrupt': bankruptcies
538.         })
539.
540.         self.progress["metadata"] = {
541.             'Reward': f'{episode_reward:.2f}',
542.             'Avg(10)': f'{avg_reward_10:.2f}',
543.             'Return%': f'{info["total_return"]*100:.1f}',
544.             'Avg%(10)': f'{avg_return_10*100:.1f}',
545.             'Sharpe': f'{info["sharpe_ratio"]:.2f}',
546.             'Eps': f'{agent.epsilon:.3f}',
547.             'Loss': f'{avg_loss:.4f}',
548.             'Bankrupt': bankruptcies
549.         }
550.
551.         self.progress["history"]["rewards"].append(avg_reward_10)
552.         self.progress["history"]["returns"].append(avg_return_10*100)
553.         self.progress["history"]["sharpes"].append(info["sharpe_ratio"])
554.         self.progress["history"]["losses"].append(avg_loss)
555.
556.
557.         self.progress["current"] = episode
558.

```

Stores transitions in
ReplayBuffer and
periodically updates policy
and target networks

Progress dictionary
tracks metrics in real
time to later display
on UI elements while
training

```

559.         pbar.close()
560.         self.progress["state"] = "idle"
561.
562.         print(f"\nTraining completed! Bankruptcies: {bankruptcies}/{config['episodes']}")
563.
564.         self.plot_training_progress(episode_returns, episode_sharpes, losses, bankruptcies)
565.
566.         train_norm_stats = (train_env.initial_feature_means,
train_env.initial_feature_stds)
567.
568.         print("\nTesting on OOS data...")
569.         test_stats, test_balances, test_actions = test_agent(agent, test_data, config,
train_norm_stats, plot=True)
570.
571.         #save model after testing
572.         self.save_model(agent, test_stats)
573.
574.         return agent, episode_rewards, episode_returns, test_stats
575.
576. #backend testing
577. if __name__ == "__main__":
578.
579.     override_config = {
580.     }
581.
582.     trainer = DQNTrainer(override_config)
583.
584.     use_pretrained = input("Load existing model? (y/n)").lower()
585.
586.     if use_pretrained == "y":
587.         agent, rewards, returns, test_stats = trainer.train_dqn(use_pretrained=True)
588.     else:
589.         agent, rewards, returns, test_stats = trainer.train_dqn()
590.
591.     print("\n" + "="*60)
592.     print("TRAINING COMPLETE!")
593.     print("="*60)
594.     print(f"Best episode return: {max(returns)*100:.2f}%")
595.     print(f"Final test win rate: {test_stats['win_rate']*100:.1f}%")
596.     print(f"Test profit factor: {test_stats['profit_factor']:.2f}")
597.     print("="*60)
598.

```

agent.py

This is the neural network's actual structure and defines how data moves through the network and how the network is optimised. Implementation is adapted from YouTube tutorials¹³, modified to use a one-dimensional CNN (convolutional neural network).

```

1. import numpy as np
2. import torch
3. import torch.nn as nn
4. import torch.optim as optim
5. import random
6. from typing import Tuple, Dict
7.
8. #class describing the structure of the neural network overall, and how data flows through
the layers
9. class Conv1dDQN(nn.Module):

```

Defines a custom Conv1D Deep Q-Network architecture and explicitly controls the forward data flow for reinforcement learning

¹³ <https://www.youtube.com/watch?v=qKePPepISiA>,
Convolutional Neural Network (CNN) in Deep Q-Learning (DQN) Explained

```

10.     def __init__(self, num_features, num_actions, config):
11.         #base pytorch module init
12.         super(Conv1dDQN, self).__init__()
13.
14.
15.         # when kernels slide across a vector, there will inevitably be places at the ends
with missing values
16.         # padding adds zeros to keep the output length roughly the same as input
17.         #rest of params are relativley self-explanatory, check CONFIG (trainer.py)
18.
19.         #defining all layers
20.         self.conv1 = nn.Conv1d(num_features, config['conv_filters'][0],
kernel_size=config['kernel_size'], padding=1)
21.         self.ln1 = nn.LayerNorm([config['conv_filters'][0], config['lookback_window']])
22.
23.         self.conv2 = nn.Conv1d(config['conv_filters'][0], config['conv_filters'][1],
kernel_size=config['kernel_size'], padding=1)
24.         self.ln2 = nn.LayerNorm([config['conv_filters'][1], config['lookback_window']])
25.
26.
27.         self.conv3 = nn.Conv1d(config['conv_filters'][1], config['conv_filters'][2],
kernel_size=config['kernel_size'], padding=1)
28.         self.ln3 = nn.LayerNorm([config['conv_filters'][2], config['lookback_window']])
29.
30.         #defined as reuseable functions for experimentation
31.         #ReLU sets negative values to zero so that nonlinear relationships can be learnt
32.         self.relu = nn.ReLU(inplace=True)
33.         #pooling compresses the time dimension to a single value per feature
34.         self.pool = nn.AdaptiveAvgPool1d(1)
35.
36.         #fully connected layers
37.         self.fc1 = nn.Linear(config['conv_filters'][2], config['fc_layers'][0])
38.         self.fc2 = nn.Linear(config['fc_layers'][0], config['fc_layers'][1])
39.         self.out = nn.Linear(config['fc_layers'][1], num_actions)
40.
41.
42.         #this defines how data flows through the neural net
43.         def forward(self, x):
44.             #changing input vector into what pytorch expects (original was just to make working
with dataframes easier)
45.             #expected vector: batch, channels (AKA features), timesteps
46.             x = x.transpose(1, 2)
47.
48.             #step by step process of passing through layers
49.             x = self.conv1(x)
50.             x = self.ln1(x)
51.             x = self.relu(x)
52.
53.             x = self.conv2(x)
54.             x = self.ln2(x)
55.             x = self.relu(x)
56.
57.             x = self.conv3(x)
58.             x = self.ln3(x)
59.             x = self.relu(x)
60.
61.             x = self.pool(x).squeeze(-1)
62.
63.             #output layer has one neuron per action
64.             #actions are decided upon based on Q values ("strength" of each neuron in a sense)
65.             x = self.relu(self.fc1(x))
66.             x = self.relu(self.fc2(x))
67.             x = self.out(x)
68.
69.             return x
70.
71. #replay buffer (dqn's memory)
72. class ReplayBuffer:
73.     def __init__(self, capacity, num_features, lookback_window, device):

```

1D convolution layers are used to extract temporal patterns from time-series market features so that the agent can learn trends and market structures

Fully connected layers map extracted features to Q-values, with one output neuron per possible action. Allows for decision making based on expected reward.

Forward pass reshapes input tensors to expected formats, applies normalisation and the ReLU activation, pools the time dimension, and produces Q-Value estimates

Objective 7.4

This class creates a custom ReplayBuffer, complete with operations for a circular queue with random samples for past transitions

```

74.         #number of steps that can be saved at a time
75.         self.capacity = capacity
76.         self.device = torch.device(device)
77.         self.position = 0
78.         self.size = 0
79.
80.         self.num_features = num_features
81.         self.lookback_window = lookback_window
82.
83.         #allocating GPU blocks once so that less time is wasted with GPU transfers.
training is horribly slow so im trying to boost efficiency here.
84.         #what torch.zeros() actually does is fill up part of the GPU memory with zeros to
store these matrices.
85.         #storing floats expected by neural net as 32 bit floats, it is precise enough and
64 doubles memory use for little gain
86.
87.         #these allocated blocks are later used like circular queues, when
taken up in order, and after all is full, oldest is overwritten
88.         self.pre_transition_states = torch.zeros(
89.             (capacity, lookback_window, num_features),
90.             dtype=torch.float32,
91.             device=self.device
92.         )
93.
94.         self.post_transition_states = torch.zeros(
95.             (capacity, lookback_window, num_features),
96.             dtype=torch.float32,
97.             device=self.device
98.         )
99.
100.        #long dtype is used so torch.gather() works when matching q-values to actions
101.        self.actions = torch.zeros(capacity, dtype=torch.long, device=self.device)
102.        self.rewards = torch.zeros(capacity, dtype=torch.float32, device=self.device)
103.        #it's only a flag, but pytorch expects floats when math is involved
104.        self.terminal_steps = torch.zeros(capacity, dtype=torch.float32,
device=self.device)
105.
106.
107.        #push a new transition into the replay buffer (circular queues!)
108.        def push(self, prev_state, action, reward, next_state, done):
109.            #keep track of queue position
110.            pos = self.position
111.
112.            #make sure the state being observed is on the right device
113.            self.pre_transition_states[pos] = prev_state.to(self.device)
114.
115.            self.actions[pos] = action
116.            self.rewards[pos] = reward
117.            self.terminal_steps[pos] = float(done)
118.
119.            #store the next state after an action, and leave a placeholder if it's terminal
120.            if next_state is not None:
121.                self.post_transition_states[pos] = next_state.to(self.device)
122.            else:
123.                self.post_transition_states[pos] = 0
124.
125.            #move ahead in the queue and wrap around if capacity is exceeded (circular queue)
126.            self.position = (self.position + 1) % self.capacity
127.            self.size = min(self.size + 1, self.capacity)
128.
129.
130.        #sample a random set of transitions to learn from
131.        def sample(self, batch_size: int):
132.            #torch expects a tuple for size, so no, the comma is NOT a typo
133.            selected_samples = torch.randint(0, self.size, (batch_size,), device=self.device)
134.
135.            return (
136.                self.pre_transition_states[selected_samples],
137.                self.actions[selected_samples],

```

User-defined data structure implementing a replay buffer with a fixed capacity for storing transitions in a queue

Pre-allocates 3D tensors on the GPU for states, actions, rewards, and terminal flags to maximise training efficiency and avoid repeated memory allocation

Adds a new transition to the buffer using circular queue logic, overwriting oldest entries when full

Randomly samples a batch of transitions for training, providing stochastic updates for the neural network

```

138.         self.rewards[selected_samples],
139.         self.post_transition_states[selected_samples],
140.         self.terminal_steps[selected_samples]
141.     )
142.
143.     def __len__(self):
144.         return self.size
145.
146.
147. #actual agent
148. class DQNAgent:
149.     def __init__(self, num_features, lookback_window, num_actions, config):
150.         self.config = config
151.         self.device = torch.device(config['device'])
152.
153.         #policy network is what predicts q-values
154.         #the biggest q value is chosen as what would bring the highest long term reward by
155.         #the agent later (if not randomly acting)
156.         self.policy_net = Conv1dDQN(num_features, num_actions, config).to(self.device)
157.
158.         #slower updating network stabilises training by providing a more consistent
159.         #estimation of future rewards
160.         self.target_net = Conv1dDQN(num_features, num_actions, config).to(self.device)
161.
162.         self.target_net.load_state_dict(self.policy_net.state_dict())
163.         self.target_net.eval()
164.
165.         #optimiser computes gradients from loss (predicted vs actual q-values) and updates
166.         #policy to reduce loss
167.         #AdamW takes into account how consistent gradients are to determine if the current
168.         #gradient is just noise
169.         #it does this with running averages of gradient magnitudes to smooth out updates
170.         self.optimiser = optim.AdamW(
171.             self.policy_net.parameters(),
172.             lr=config['learning_rate'],
173.             fused=True if config['device'] == 'cuda' else False
174.         )
175.
176.         #store of past transitions to "learn from experience"
177.         #randomised to reduce correlation with more recent transitions (see ReplayBuffer
178.         #class)
179.         self.memory = ReplayBuffer(
180.             config['memory_size'],
181.             num_features,
182.             lookback_window,
183.             self.device
184.         )
185.
186.         # "exploration", AKA do random stuff till something works. This chance of random
187.         #actions decay with time
188.         self.epsilon = config['epsilon_start']
189.         self.num_actions = num_actions
190.
191.     def select_action(self, state_tensor, training = True):
192.
193.         if training and random.random() < self.epsilon:
194.             action = random.randint(0, self.num_actions - 1)
195.         else:
196.             #not tracking gradients to save computation (it's not needed rn)
197.             with torch.no_grad():
198.                 #compute q-values
199.                 q_values = self.policy_net(state_tensor.unsqueeze(0))
200.                 #choose the best!
201.                 action = q_values.argmax().item()
202.
203.         return action
204.
205.     def train_step(self):
206.         #cant learn if theres nothing to learn from -_-

```

DQNAgent handles reinforcement learning logic by coordinating policy and target neural networks with the Conv1dDQN class, alongside a ReplayBuffer, to manage the agent's past transitions.

Replay Buffer
Instance

```

201.         if len(self.memory) < self.config['batch_size']:
202.             return None
203.
204.         pre_transition_states, actions, rewards, post_transition_states, terminal_steps =
self.memory.sample(self.config['batch_size'])
205.
206.         #based on actions taken on the past, what does the new policy think the q-values
would be?
207.         current_q = self.policy_net(pre_transition_states).gather(1,
actions.unsqueeze(1)).squeeze(1)
208.
209.         with torch.no_grad():
210.             #estimate the maximum future Q-value using the target network
211.             next_q = self.target_net(post_transition_states).max(1)[0]
212.
213.             #the bellman equation, in all its glory :D
214.             #now the target takes into account both the immediate reward gain AND future
gain
215.             target_q = rewards + (1 - terminal_steps) * self.config['gamma'] * next_q
216.
217.             #how far off were predicted q-values from optimal?
218.             loss = nn.MSELoss()(current_q, target_q)
219.
220.             #clear stored gradients (they accumulate so it would get mixed in with new
calculations)
221.             self.optimiser.zero_grad(set_to_none=True)
222.
223.             #work out how much loss is affected by tweaking weights
224.             loss.backward()
225.             #clip large gradients to stabilise training, important in noisy data like finance
226.             torch.nn.utils.clip_grad_norm_(self.policy_net.parameters(), 1.0)
227.             #change weights according to gradients that show direction that reduces loss
228.             self.optimiser.step()
229.
230.             #for logging
231.             return loss.item()
232.
233.         def soft_update_target_network(self):
234.             #tau is how strongly target follows policy
235.             tau = self.config['tau']
236.
237.             #match target net params up to policy net params
238.             for target_param, policy_param in zip(self.target_net.parameters(),
self.policy_net.parameters()):
239.                 target_param.data.copy_(tau * policy_param.data + (1 - tau) *
target_param.data)
240.
241.             #reduce exploration, increase exploitation!
242.             def decay_epsilon(self):
243.                 self.epsilon = max(self.config['epsilon_end'], self.epsilon *
self.config['epsilon_decay'])
244.

```

trading env.py

This is the environment which the agent can act in and defines the rewards that the agent gets from its actions. Agent interaction functions such as `reset()`, `step()` and `get_state()` are inspired by OpenAI gym, but the implementation is entirely custom to handle trading-specific rewards and features.

```

1. import numpy as np
2. import pandas as pd
3. import torch
4. from typing import List, Tuple, Dict, Optional

```

Position class acts as a data structure to represent a trading position

```

5.
6. #individual position tracker
7. class Position:
8.     def __init__(self, entry_price, allocation):
9.         self.entry_price = entry_price
10.        self.allocation = allocation
11.
12.        #this assumes fractional shares can be owned
13.        self.shares = allocation / entry_price
14.
15.    def get_pnl(self, current_price):
16.        return (current_price - self.entry_price) * self.shares
17.
18.    def get_value(self, current_price):
19.        return self.shares * current_price
20.
21. #tracks multiple positions
22. class PositionManager:
23.     def __init__(self):
24.         #list datatype allows for a custom number of positions to be kept
25.         self.positions: List[Position] = []
26.
27.         #append to the list to allow for pyramiding
28.     def add_position(self, entry_price, capital):
29.         self.positions.append(Position(entry_price, capital))
30.
31.
32.         #this is the only way for the DQN to close positions
33.         #action space is kept simple for learning
34.     def close_all_positions(self, current_price):
35.         total_capital_returned = 0
36.         total_pnl = 0
37.
38.         for pos in self.positions:
39.             pnl = pos.get_pnl(current_price)
40.             capital_returned = pos.allocation + pnl
41.             total_capital_returned += capital_returned
42.             total_pnl += pnl
43.
44.         self.positions = []
45.
46.         #total_pnl has useage in reward shaping, capital is for actual calculations
47.         return total_capital_returned, total_pnl
48.
49.     #helper functions for reward shaping and pnl calculations
50.     def get_total_exposure(self):
51.         exposure = 0
52.         for pos in self.positions:
53.             exposure += pos.allocation
54.
55.         return exposure
56.
57.     def get_total_value(self, current_price):
58.         value = 0
59.         for pos in self.positions:
60.             value += pos.get_value(current_price)
61.
62.         return value
63.
64.     def get_total_pnl(self, current_price):
65.         pnl = 0
66.         for pos in self.positions:
67.             pnl += pos.get_pnl(current_price)
68.
69.         return pnl
70.
71.     def get_average_entry_price(self):
72.         avg = 0
73.

```

Initialises position attributes and calculates number of shares assuming fractional ownership

PositionManager handles the agent's trading portfolio state through a collection of Position objects

Appends a position class to the list

The trading environment is a complex financial model, creating an environment for a DQN agent to interact with the features generated by the dataset generator, as well as meta-features such as the hit rate of a LightGBM model and an observation window of held positions, allowing the agent to see how they change over time

TradingEnvironment composes a PositionManager object to handle portfolio management, separating environment state, action

```

74.         if self.positions:
75.             total_shares, total_value = 0, 0
76.
77.             for pos in self.positions:
78.                 total_shares += pos.shares
79.                 total_value += pos.entry_price * pos.shares
80.
81.             if total_shares > 0:
82.                 avg = total_value / total_shares
83.
84.             return avg
85.
86.         def has_positions(self):
87.             return (len(self.positions) > 0)
88.
89.         def get_position_count(self):
90.             return len(self.positions)
91.
92. #simulated environment for dqn to act in
93. class TradingEnvironment:
94.     #train_normalisation_stats is only needed for testing
95.     def __init__(self, trading_df, config, device, is_training,
train_normalisation_stats = None, auto_reset=True):
96.         self.trading_df = trading_df
97.         self.config = config
98.         self.device = device
99.         self.is_training = is_training
100.
101.         #separate feature columns
102.         base_cols = ['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close']
103.
104.         lgbm_cols = [col for col in trading_df.columns if col.startswith('LGBM_')]
105.         feature_cols = [col for col in trading_df.columns
106.                         if col not in base_cols and col not in lgbm_cols]
107.
108.         # Filter features based on enabled_features config
109.         enabled_features = config.get('enabled_features', {})
110.         if enabled_features:
111.             feature_cols = [col for col in feature_cols
112.                             if enabled_features.get(col, True)]
113.             print(f"TradingEnvironment: Using {len(feature_cols)} enabled features")
114.
115.         self.horizons = config['horizons']
116.
117.         #separate short-term and long-term horizons
118.         #default is 30
119.         split_threshold = config.get('horizon_split_threshold', 30)
120.
121.         self.short_term_horizons = []
122.         self.long_term_horizons = []
123.
124.         for h in self.horizons:
125.             if h <= split_threshold:
126.                 self.short_term_horizons.append(h)
127.             else:
128.                 self.long_term_horizons.append(h)
129.
130.         #prepare lgbm predictions split by time scale
131.         lgbm_features_list = []
132.         self.lgbm_predictions = {}
133.
134.         #processing horizons
135.         short_term_preds = []
136.         short_term_magnitudes = []
137.         short_term_signs = []
138.
139.         for horizon in self.short_term_horizons:
140.             lgbm_col = f"LGBM_{horizon}d_q50"
141.             if lgbm_col not in trading_df.columns:

```

Objective 7.2

This class defines a training environment for the agent to act in

```

142.         raise ValueError(f"Missing LGBM column: {lgbm_col}")
143.
144.         lgbm_pred = trading_df[lgbm_col].values
145.         self.lgbm_predictions[horizon] = lgbm_pred
146.
147.         #explicitly extracting what i want the agent to be looking for
148.         short_term_preds.append(lgbm_pred)
149.         short_term_magnitudes.append(np.abs(lgbm_pred))
150.         short_term_signs.append(np.sign(lgbm_pred))
151.
152.     #same thing for long term
153.
154.     long_term_preds = []
155.     long_term_magnitudes = []
156.     long_term_signs = []
157.
158.     for horizon in self.long_term_horizons:
159.         lgbm_col = f"LGBM_{horizon}d_q50"
160.         if lgbm_col not in trading_df.columns:
161.             raise ValueError(f"Missing LGBM column: {lgbm_col}")
162.
163.         lgbm_pred = trading_df[lgbm_col].values
164.         self.lgbm_predictions[horizon] = lgbm_pred
165.
166.         long_term_preds.append(lgbm_pred)
167.         long_term_magnitudes.append(np.abs(lgbm_pred))
168.         long_term_signs.append(np.sign(lgbm_pred))
169.
170.     #averaging the predictions so its less noisy for the model
171.     if short_term_preds:
172.         st_pred_mean = np.mean(short_term_preds, axis=0)
173.
174.         #capture variation between models, helps with uncertainty
175.         st_pred_std = np.std(short_term_preds, axis=0)
176.
177.         #strength
178.         st_mag_mean = np.mean(short_term_magnitudes, axis=0)
179.
180.         #"agreement" between models, again helpful for uncertainty
181.         st_sign_consensus = np.mean(short_term_signs, axis=0)
182.
183.         lgbm_features_list.extend([
184.             st_pred_mean,
185.             st_pred_std,
186.             st_mag_mean,
187.             st_sign_consensus
188.         ])
189.
190.
191.     if long_term_preds:
192.         lt_pred_mean = np.mean(long_term_preds, axis=0)
193.         lt_pred_std = np.std(long_term_preds, axis=0)
194.         lt_mag_mean = np.mean(long_term_magnitudes, axis=0)
195.         lt_sign_consensus = np.mean(long_term_signs, axis=0)
196.
197.         lgbm_features_list.extend([
198.             lt_pred_mean,
199.             lt_pred_std,
200.             lt_mag_mean,
201.             lt_sign_consensus
202.         ])
203.
204.     #matrix for dqn to observe
205.     #rows are timesteps and columns are lgbm features
206.     lgbm_features = np.column_stack(lgbm_features_list)
207.
208.     #combine all features
209.     feature_data = trading_df[feature_cols].values
210.     self.raw_features = np.concatenate([feature_data, lgbm_features], axis=1)

```

This 2D array contains the raw features before further normalisation steps

```

211.
212.     #normalisation with full dataset for training
213.     #assumes somewhat similar market conditions, and is faster than having to compute
for each episode
214.     if is_training:
215.         self.initial_feature_means = np.nanmean(self.raw_features, axis=0)
216.         self.initial_feature_stds = np.nanstd(self.raw_features, axis=0)
217.         self.initial_feature_stds[self.initial_feature_stds == 0] = 1
218.
219.         normalised = (self.raw_features - self.initial_feature_means) /
self.initial_feature_stds
220.         normalised = np.nan_to_num(normalised, 0)
221.
222.         self.feature_means = self.initial_feature_means
223.         self.feature_stds = self.initial_feature_stds
224.         #for testing, use the statistics from training to normalise new data (again assumes
similar environment)
225.     else:
226.         if train_normalisation_stats is None:
227.             raise ValueError("test environment requires train_normalisation_stats")
228.
229.         self.initial_feature_means, self.initial_feature_stds =
train_normalisation_stats
230.
231.         self.feature_means = self.initial_feature_means.copy()
232.         self.feature_stds = self.initial_feature_stds.copy()
233.
234.         normalised = np.zeros_like(self.raw_features)
235.
236.         #store normalised features on GPU if available
237.         if device == 'cuda':
238.             self.normalised_features = torch.from_numpy(normalised).float().to(device)
239.         else:
240.             self.normalised_features = torch.from_numpy(normalised).float()
241.
242.         self.prices = trading_df['Close'].values
243.
244.         self.lgbm_confidence_history =
self.calculate_historical_lgbm_confidence(trading_df)
245.
246.         if auto_reset:
247.             self.reset()
248.         else:
249.             # For live trading, initialize without reset (will be set manually)
250.             self.start_idx = 0
251.             self.current_step = 0
252.             self.balance = float(self.config['initial_balance'])
253.             self.initial_balance = float(self.config['initial_balance'])
254.             self.peak_balance = float(self.config['initial_balance'])
255.             self.position_manager = PositionManager()
256.             self.done = False
257.             self.balance_history = [self.balance]
258.             self.realized_balance_history = [self.balance]
259.             self.cumulative_realized_pnl = 0.0
260.             self.prev_action = 1
261.             self.action_history = []
262.             self.prev_unrealized_pnl = 0.0
263.             self.lgbm_hit_history = []
264.             self.episode_hit_rate = 0.0
265.             self.indecision_steps = 0
266.             self.indecision_penalty_current = 0.0
267.
268.         #only looks at data up till current timestep to avoid leakage
269.         def calculate_historical_lgbm_confidence(self, df) :
270.             confidence_setting = self.config.get('confidence_horizons', 'all')
271.             #choose horizons to use to determine confidence based on quantile width
272.             if confidence_setting == 'all':
273.                 confidence_horizons = self.horizons
274.             elif confidence_setting == 'short':

```

Position Manager is instances to manage several position objects and track trades from the DQN Agent

This algorithm either uses quantile widths or hit rates to determine how much LightGBM should be trusted

```

275.         confidence_horizons = self.short_term_horizons
276.     elif confidence_setting == 'long':
277.         confidence_horizons = self.long_term_horizons
278.
279.         #allows for a custom list to be used for confidence in the config
280.     elif isinstance(confidence_setting, list):
281.         confidence_horizons = [h for h in confidence_setting if h in self.horizons]
282.         if not confidence_horizons:
283.             raise ValueError(f"None of the specified confidence_horizons
284. {confidence_setting} are in horizons {self.horizons}")
285.         else:
286.             raise ValueError(f"Invalid confidence_horizons setting: {confidence_setting}")
287.
288.         #confidence levels for each horizon, averaged later depending on settings
289.         confidence_by_horizon = []
290.
291.         low_q_setting = self.config.get('lgbm_lower', 0.1)
292.         high_q_setting = self.config.get('lgbm_upper', 0.9)
293.
294.         for horizon in confidence_horizons:
295.             low_q_col = f"LGBM_{horizon}d_q{int(low_q_setting*100)}"
296.             high_q_col = f"LGBM_{horizon}d_q{int(high_q_setting*100)}"
297.
298.             if low_q_col in df.columns and high_q_col in df.columns:
299.                 lower_q = df[low_q_col].values
300.                 upper_q = df[high_q_col].values
301.
302.                 interval_width = np.abs(upper_q - lower_q)
303.
304.                 window = self.config.get('lgbm_confidence_window', 20)
305.                 #pre-allocating zeros for performance in a fixed length array
306.                 confidence = np.zeros(len(df))
307.
308.                 #quantile width can vary depending on the specifics of the market
309.                 #rather than hardcoding levels, it compares to historical widths to see if
310.                 #its more confident than usual
311.                 for i in range(len(df)):
312.                     #neutral confidence with no history (0.5)
313.                     if i == 0:
314.                         confidence[i] = 0.5
315.                         continue
316.                     elif i < window:
317.                         past_widths = interval_width[:i]
318.                         median_width = np.median(past_widths)
319.                     else:
320.                         past_widths = interval_width[i-window:i]
321.                         median_width = np.median(past_widths)
322.
323.                     current_width = interval_width[i]
324.
325.                     median_width = max(median_width, 0.001)
326.                     current_width = max(current_width, 0.001)
327.
328.                     conf_score = median_width / current_width
329.                     confidence[i] = np.clip(conf_score, 0, 2) / 2.0
330.
331.                 confidence_by_horizon.append(confidence)
332.
333.             #if quantile widths are not available, use hit rate for confidence
334.             else:
335.                 window = self.config.get('lgbm_confidence_window', 20)
336.                 confidence = np.zeros(len(df))
337.
338.                 lgbm_pred = self.lgbm_predictions[horizon]
339.
340.                 for i in range(horizon, len(df)):
341.                     if i < window + horizon:
342.                         lookback = i - horizon
343.                     else:

```

Checks if LightGBM is "more confident than usual" using median quantile widths

```

342.         lookback = window
343.
344.         start_idx = max(0, i - lookback - horizon)
345.         end_idx = i - horizon
346.
347.         if end_idx > start_idx:
348.             past_preds = lgbm_pred[start_idx:end_idx]
349.             future_prices = self.prices[start_idx + horizon:end_idx + horizon]
350.             past_prices = self.prices[start_idx:end_idx]
351.
352.             actual_returns = (future_prices - past_prices) / past_prices
353.             pred_signs = np.sign(past_preds)
354.             actual_signs = np.sign(actual_returns)
355.
356.             hit_rate = np.mean((pred_signs * actual_signs) > 0)
357.             confidence[i] = hit_rate
358.
359.         confidence_by_horizon.append(confidence)
360.
361.     if confidence_by_horizon:
362.         avg_confidence = np.mean(confidence_by_horizon, axis=0)
363.         return avg_confidence
364.     else:
365.         raise RuntimeError("No horizons were processed for confidence calculation")
366.
367.
368.     #for useage in testing only
369.     #expanding window prevents leakage when normalising data for tests
370.     def update_expanding_normalisation(self, current_global_idx):
371.         if self.is_training:
372.             return
373.
374.         if current_global_idx > 0:
375.             available_features = self.raw_features[:current_global_idx]
376.
377.             self.feature_means = np.nanmean(available_features, axis=0)
378.             self.feature_stds = np.nanstd(available_features, axis=0)
379.             self.feature_stds[self.feature_stds == 0] = 1
380.
381.             if current_global_idx < len(self.raw_features):
382.                 normalised = (self.raw_features[current_global_idx] - self.feature_means) /
self.feature_stds
383.                 normalised = np.nan_to_num(normalised, 0)
384.
385.                 if self.device == 'cuda':
386.                     self.normalised_features[current_global_idx] =
torch.from_numpy(normalised).float().to(self.device)
387.                 else:
388.                     self.normalised_features[current_global_idx] =
torch.from_numpy(normalised).float()
389.
390.         #starting a new episode
391.         def reset(self):
392.             #make sure you dont start an episode where there is no data ahead
393.             max_start = len(self.trading_df) - self.config['episode_length_days'] -
self.config['lookback_window']
394.
395.             self.start_idx = np.random.randint(self.config['lookback_window'], max_start)
396.
397.             balance_setting = float(self.config['initial_balance'])
398.
399.             self.current_step = 0
400.             self.balance = balance_setting
401.             self.initial_balance = balance_setting
402.             self.peak_balance = balance_setting
403.             self.position_manager = PositionManager()
404.             self.done = False
405.
406.             self.balance_history = [balance_setting] #total balance (cash + unrealized)

```

This algorithm normalises features with only data before the current index, to ensure no leakage during tests

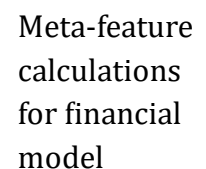
Composition is used here to embed a PositionManager within the TradingEnvironment, delegating portfolio responsibilities to a dedicated object

```

407.     self.realized_balance_history = [balance_setting] #only realized gains/losses
408.     self.cumulative_realized_pnl = 0.0 #track total realized PnL
409.
410.     self.prev_action = 1
411.     self.action_history = []
412.     self.prev_unrealized_pnl = 0.0
413.
414.     self.lgbm_hit_history = []
415.     self.episode_hit_rate = 0.0
416.
417.     self.indecision_steps = 0
418.     self.indecision_penalty_current = 0.0
419.
420.     return self.get_state()
421.
422. #dqn observation
423. def get_state(self):
424.
425.     #walk forward expanding norm if testing (prevent leakage)
426.     if not self.is_training:
427.         current_global_idx = self.start_idx + self.current_step
428.         self.update_expanding_normalisation(current_global_idx)
429.
430.     start = self.start_idx + self.current_step - self.config['lookback_window']
431.     end = self.start_idx + self.current_step
432.
433.     state = self.normalised_features[start:end].clone()
434.
435.     #adding noise to reduce overfit
436.     if self.is_training and self.config.get('noise_std', 0) > 0:
437.         noise = torch.randn_like(state) * self.config['noise_std']
438.         state = state + noise
439.
440.     hit_rate_tensor = torch.tensor(
441.         self.episode_hit_rate,
442.         dtype=torch.float32,
443.         device=state.device
444.     ).reshape(1, 1)
445.
446.     hit_rate_expanded = hit_rate_tensor.expand(state.shape[0], 1)
447.
448.     #days remaining might prove useful, as the model blows up near the end of episodes
449.     #due to its predicted trends not coming to pass
450.     days_remaining = self.config['episode_length_days'] - self.current_step
451.     days_remaining_normalised = days_remaining / self.config['episode_length_days']
452.
453.     days_remaining_tensor = torch.tensor(
454.         days_remaining_normalised,
455.         dtype=torch.float32,
456.         device=state.device
457.     ).reshape(1, 1)
458.
459.     days_remaining_expanded = days_remaining_tensor.expand(state.shape[0], 1)
460.
461.     current_price = self.get_current_price()
462.
463.     #has positions (0 or 1)
464.     has_positions = float(self.position_manager.has_positions())
465.
466.     #number of positions normalised
467.     num_positions = self.position_manager.get_position_count()
468.     max_positions = self.config.get('max_positions', 10)
469.     num_positions_norm = num_positions / max_positions
470.
471.     # Position exposure (% of total portfolio value)
472.     position_value = self.position_manager.get_total_value(current_price)
473.     total_portfolio = self.balance + position_value
474.     exposure_pct = position_value / total_portfolio if total_portfolio > 0 else 0.0

```

Meta-feature
calculations
for financial
model



```

475.         #unrealized P&L (% of initial balance)
476.         unrealized_pnl = self.position_manager.get_total_pnl(current_price)
477.         unrealized_pnl_pct = unrealized_pnl / self.initial_balance
478.
479.         #average entry price vs current price (% difference)
480.         avg_entry = self.position_manager.get_average_entry_price()
481.         if avg_entry > 0:
482.             price_diff_pct = (current_price - avg_entry) / avg_entry
483.         else:
484.             price_diff_pct = 0.0
485.
486.         # Create tensors for position features
487.         position_features = torch.tensor([
488.             has_positions,
489.             num_positions_norm,
490.             exposure_pct,
491.             unrealized_pnl_pct,
492.             price_diff_pct
493.         ], dtype=torch.float32, device=state.device).reshape(1, 5)
494.
495.         # Expand to match lookback window
496.         position_features_expanded = position_features.expand(state.shape[0], 5)
497.
498.         # Concatenate all features including position info
499.         state = torch.cat([
500.             state,
501.             hit_rate_expanded,
502.             days_remaining_expanded,
503.             position_features_expanded
504.         ], dim=1)
505.
506.         return state
507.
508.     def get_current_price(self):
509.         idx = self.start_idx + self.current_step
510.         return self.prices[idx]
511.
512.     #checks the precalculated confidences up to the current step
513.     def get_current_lgbm_confidence(self):
514.         idx = self.start_idx + self.current_step
515.         return self.lgbm_confidence_history[idx]
516.
517.     #for dynamic hit rates in an episode
518.     def update_lgbm_hit_rate(self):
519.         hit_rates_this_step = []
520.
521.         for horizon in self.horizons:
522.             if self.current_step < horizon:
523.                 continue
524.
525.             eval_step = self.current_step - horizon
526.             pred_idx = self.start_idx + eval_step
527.             current_idx = self.start_idx + self.current_step
528.
529.             lgbm_pred = self.lgbm_predictions[horizon][pred_idx]
530.             pred_sign = np.sign(lgbm_pred)
531.
532.             price_then = self.prices[pred_idx]
533.             price_now = self.prices[current_idx]
534.             actual_return = price_now / price_then - 1.0
535.             actual_sign = np.sign(actual_return)
536.
537.             was_correct = (pred_sign * actual_sign) > 0
538.             hit_rates_this_step.append(1.0 if was_correct else 0.0)
539.
540.         if hit_rates_this_step:
541.             self.lgbm_hit_history.append(np.mean(hit_rates_this_step))
542.             self.episode_hit_rate = np.mean(self.lgbm_hit_history)
543.

```

```

544.     def step(self, action):
545.         current_price = self.get_current_price()
546.         reward = 0
547.
548.         self.action_history.append(action)
549.         reward_freq = self.config['reward_frequency']
550.
551.         # universal reinforcements
552.         if self.position_manager.has_positions():
553.             position_value = self.position_manager.get_total_value(current_price)
554.
555.             #DRAWDOWN PENALTY
556.             if self.config.get('drawdown_penalty_enabled', False):
557.                 hypothetical_balance = self.balance + position_value
558.                 drawdown = (self.peak_balance - hypothetical_balance) / self.peak_balance
559.
560.                 threshold = self.config.get('drawdown_hold_threshold', 0.08)
561.                 if drawdown > threshold:
562.                     excess_drawdown = drawdown - threshold
563.                     penalty = excess_drawdown * self.config['drawdown_hold_penalty_scale']
564.
565.                     if self.config.get('drawdown_hold_penalty_nonlinear', False):
566.                         excess_drawdown_pct = excess_drawdown * 100
567.                         penalty = self.config['drawdown_hold_penalty_scale'] *
568.                         (excess_drawdown_pct ** 2) / 100
569.
570.                     reward -= penalty
571.
572.             #STEP-BASED PNL
573.             if reward_freq == 'step':
574.                 current_unrealized_pnl = self.position_manager.get_total_pnl(current_price)
575.                 pnl_change = current_unrealized_pnl - self.prev_unrealized_pnl
576.                 step_scale = self.config.get('step_reward_scale', 0.1)
577.
578.                 if abs(pnl_change) > 0.001:
579.                     if pnl_change > 0:
580.                         reward += self.config['reward_rising_hold'] * step_scale *
581.                         (pnl_change / self.initial_balance)
582.                     else:
583.                         reward += self.config['punish_falling_hold'] * step_scale *
584.                         (pnl_change / self.initial_balance)
585.
586.             # Update for next step
587.             self.prev_unrealized_pnl = current_unrealized_pnl
588.
589.             #ACTIONS
590.
591.             # Action 0: Close all positions
592.             if action == 0:
593.                 if self.position_manager.has_positions():
594.                     capital_returned, total_pnl =
595.                     self.position_manager.close_all_positions(current_price)
596.
597.                     transaction_cost = capital_returned * self.config.get('transaction_costs',
598.                     0.001)
599.
600.                     capital_after_costs = capital_returned - transaction_cost
601.
602.                     self.balance += capital_after_costs
603.
604.                     #track realized PnL
605.                     realized_pnl = total_pnl - transaction_cost
606.                     self.cumulative_realized_pnl += realized_pnl
607.
608.                     # PnL-based rewards
609.                     if reward_freq in ['close', 'step']:
610.                         if realized_pnl > 0:
611.                             reward += self.config['reward_profit_scale'] * (realized_pnl /
612.                             self.initial_balance)
613.                         else:

```

Agent can interact with financial model in realistic ways, including transaction costs

```

607.                loss_pct = abs(realized_pnl / self.initial_balance) * 100
608.
609.                if self.config.get('loss_penalty_nonlinear', True):
610.                    reward -= self.config['punish_loss_scale'] * (loss_pct ** 2) /
100
611.                else:
612.                    reward += self.config['punish_loss_scale'] *
(realized_pnl/self.initial_balance)
613.
614.                self.prev_unrealized_pnl = 0.0
615.                self.indecision_steps = 0
616.                self.indecision_penalty_current = 0.0
617.            else:
618.                #no positions, apply indecision penalty
619.                if self.config.get('indecision_penalty_enabled', False):
620.                    apply_penalty = True
621.
622.                if self.config.get('indecision_use_lgbm_confidence', False):
623.                    lgbm_confidence = self.get_current_lgbm_confidence()
624.                    confidence_threshold =
self.config.get('indecision_confidence_threshold', 0.6)
625.                    apply_penalty = (lgbm_confidence >= confidence_threshold)
626.
627.                if apply_penalty:
628.                    self.indecision_steps += 1
629.                    max_penalty = self.config.get('indecision_penalty_max', 0.05)
630.                    ramp_steps = self.config.get('indecision_penalty_ramp_steps', 20)
631.                    progress = min(self.indecision_steps / ramp_steps, 1.0)
632.                    self.indecision_penalty_current = max_penalty * progress
633.                    reward -= self.indecision_penalty_current
634.                else:
635.                    self.indecision_steps += 1
636.                    self.indecision_penalty_current = 0.0
637.            else:
638.                self.indecision_steps += 1
639.
640.        # Action 1: Hold
641.        elif action == 1:
642.            if self.position_manager.has_positions():
643.                #reset indecision (holding positions is decisive)
644.                self.indecision_steps = 0
645.                self.indecision_penalty_current = 0.0
646.            else:
647.                #indecision penalty again
648.                if self.config.get('indecision_penalty_enabled', False):
649.                    apply_penalty = True
650.
651.                if self.config.get('indecision_use_lgbm_confidence', False):
652.                    lgbm_confidence = self.get_current_lgbm_confidence()
653.                    confidence_threshold =
self.config.get('indecision_confidence_threshold', 0.6)
654.                    apply_penalty = (lgbm_confidence >= confidence_threshold)
655.
656.                if apply_penalty:
657.                    self.indecision_steps += 1
658.                    max_penalty = self.config.get('indecision_penalty_max', 0.05)
659.                    ramp_steps = self.config.get('indecision_penalty_ramp_steps', 20)
660.                    progress = min(self.indecision_steps / ramp_steps, 1.0)
661.                    self.indecision_penalty_current = max_penalty * progress
662.                    reward -= self.indecision_penalty_current
663.                else:
664.                    self.indecision_steps += 1
665.                    self.indecision_penalty_current = 0.0
666.            else:
667.                self.indecision_steps += 1
668.
669.                self.prev_unrealized_pnl = 0.0
670.
671.        # Actions 2, 3, 4: Pyramid

```

```

672.         else:
673.             max_positions = self.config.get('max_positions', 10)
674.
675.             if self.position_manager.get_position_count() >= max_positions:
676.                 pass
677.             else:
678.                 pyramid_idx = action - 2
679.                 pyramid_size = self.config['pyramid_sizes'][pyramid_idx]
680.                 capital_to_allocate = self.balance * pyramid_size
681.
682.                 if capital_to_allocate > 10:
683.
684.                     transaction_cost = capital_to_allocate *
self.config.get('transaction_costs', 0.001)
685.                     total_cost = capital_to_allocate + transaction_cost
686.
687.                     if total_cost <= self.balance:
688.                         self.position_manager.add_position(current_price,
capital_to_allocate)
689.                         self.balance -= total_cost
690.
691.                         reward -= self.config.get('opening_penalty', 0.0)
692.
693.                         # Update prev_unrealized_pnl for next step's tracking
694.                         if reward_freq == 'step':
695.                             self.prev_unrealized_pnl =
self.position_manager.get_total_pnl(current_price)
696.
697.                             self.indecision_steps = 0
698.                             self.indecision_penalty_current = 0.0
699.
700.
701.                 self.prev_action = action
702.                 self.current_step += 1
703.
704.                 self.update_lgbm_hit_rate()
705.
706.                 position_value = self.position_manager.get_total_value(current_price)
707.                 current_balance = self.balance + position_value
708.                 self.balance_history.append(current_balance)
709.
710.                 if current_balance > self.peak_balance:
711.                     self.peak_balance = current_balance
712.
713.                 realized_balance = self.initial_balance + self.cumulative_realized_pnl
714.                 self.realized_balance_history.append(realized_balance)
715.
716.                 #bankruptcy check
717.                 portfolio_value = self.balance + position_value
718.
719.                 #it honestly takes skill to go fully bankrupt when trading this way...
720.                 if portfolio_value <= 0:
721.                     self.done = True
722.                     reward = -100.0
723.
724.                     if self.position_manager.has_positions():
725.                         capital_returned, total_pnl =
self.position_manager.close_all_positions(current_price)
726.                         self.balance += capital_returned
727.                         self.cumulative_realized_pnl += total_pnl
728.
729.                     next_state = None
730.
731.                 return next_state, reward, self.done, {
732.                     'balance': max(0, portfolio_value),
733.                     'realized_balance': self.initial_balance + self.cumulative_realized_pnl,
734.                     'total_return': (portfolio_value - self.initial_balance) /
self.initial_balance,
735.                     'realized_return': self.cumulative_realized_pnl / self.initial_balance,

```

```

736.         'sharpe_ratio': 0.0,
737.         'cash': self.balance,
738.         'position_value': position_value,
739.         'num_positions': 0,
740.         'avg_entry_price': 0,
741.         'lgbm_hit_rate': self.episode_hit_rate,
742.         'lgbm_confidence': self.get_current_lgbm_confidence(),
743.         'bankruptcy': True
744.     }
745.
746.     #check if episode complete
747.     if self.current_step >= self.config['episode_length_days']:
748.
749.         self.done = True
750.
751.         if self.position_manager.has_positions():
752.             capital_returned, total_pnl =
753. self.position_manager.close_all_positions(current_price)
754.
755.             transaction_cost = capital_returned * self.config.get('transaction_costs',
756. 0.001)
757.             capital_after_costs = capital_returned - transaction_cost
758.
759.             self.balance += capital_after_costs
760.
761.             #track realized PnL from final close
762.             realized_pnl = total_pnl - transaction_cost
763.             self.cumulative_realized_pnl += realized_pnl
764.
765.             if reward_freq in ['close', 'step']:
766.                 if realized_pnl > 0:
767.                     reward += self.config['reward_profit_scale'] * (realized_pnl /
768. self.initial_balance)
769.                 else:
770.                     loss_pct = abs(realized_pnl / self.initial_balance) * 100
771.
772.                     if self.config.get('loss_penalty_nonlinear', True):
773.                         reward -= self.config['punish_loss_scale'] * (loss_pct ** 2)
774.                     else:
775.                         reward += self.config['punish_loss_scale'] * (realized_pnl /
776. self.initial_balance)
777.
778.             #calculate Sharpe using REALIZED returns only
779.             realized_returns = np.diff(self.realized_balance_history) /
780. self.realized_balance_history[:-1]
781.
782.             if len(realized_returns) > 1 and np.std(realized_returns) > 0:
783.                 #sharpe is avg return/avg volatility, and annualised with 252 trading days
784.                 a year
785.                 sharpe_ratio = np.mean(realized_returns) / np.std(realized_returns) *
786. np.sqrt(252)
787.
788.                 #clip so it doesnt get weighted too much
789.                 sharpe_clipped = np.clip(
790.                     sharpe_ratio,
791.                     self.config.get('sharpe_clip_min', -1.0),
792.                     self.config.get('sharpe_clip_max', 1.0)
793.                 )
794.
795.                 final_return = self.cumulative_realized_pnl / self.initial_balance
796.
797.                 #no positive reward if its losing overall (can happen if just a few trades
798.                 #were REALLY bad)
799.                 if final_return < 0:
800.                     sharpe_clipped = min(sharpe_clipped, 0.0)
801.
802.                 reward += self.config['reward_sharpe_bonus'] * sharpe_clipped

```

Objective 7.3

The rest of the script from here calculates trading metrics to display while training

```

797.         else:
798.             sharpe_ratio = 0.0
799.
800.         #clip reward for stability
801.         if self.config.get('reward_clip', True):
802.             reward = np.clip(
803.                 reward,
804.                 self.config.get('reward_clip_min', -2.0),
805.                 self.config.get('reward_clip_max', 2.0)
806.             )
807.
808.         next_state = None if self.done else self.get_state()
809.         total_return = (current_balance - self.initial_balance) / self.initial_balance
810.         realized_return = self.cumulative_realized_pnl / self.initial_balance
811.
812.         realized_returns = np.diff(self.realized_balance_history) /
self.realized_balance_history[:-1]
813.         sharpe_ratio = 0
814.         if len(realized_returns) > 1 and np.std(realized_returns) > 0:
815.             sharpe_ratio = np.mean(realized_returns) / np.std(realized_returns) *
np.sqrt(252)
816.
817.         return next_state, reward, self.done, {
818.             'balance': current_balance,
819.             'realized_balance': self.initial_balance + self.cumulative_realized_pnl,
820.             'total_return': total_return,
821.             'realized_return': realized_return,
822.             'sharpe_ratio': sharpe_ratio,
823.             'cash': self.balance,
824.             'position_value': position_value,
825.             'num_positions': self.position_manager.get_position_count(),
826.             'avg_entry_price': self.position_manager.get_average_entry_price(),
827.             'lgbm_hit_rate': self.episode_hit_rate,
828.             'lgbm_confidence': self.get_current_lgbm_confidence(),
829.             'bankruptcy': False
830.         }
831.

```

testing.py

This script tests an agent on a specific dataset (usually an out of sample slice from a bigger dataset) to calculate standard trading metrics for the model including Sharpe Ratio, returns, and how well it did as oppose to a basic Buy and Hold strategy (to show if it really learnt an edge or has a “dumb” strategy).

Objective 7.3

This entire script is used to calculate post-training metrics. Some are calculated by the trading environment, and displayed here, and some are directly calculated here.

```

1. import os
2. import sys
3. import numpy as np
4. import pandas as pd
5. import matplotlib.pyplot as plt
6. from typing import List, Dict, Tuple
7.
8. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
9. if project_root not in sys.path:
10.     sys.path.insert(0, project_root)
11.
12. from prediction_tools.DQN.agent import DQNAgent
13. from prediction_tools.DQN.trading_env import TradingEnvironment
14.
15. #tracks each individual trade to compute statistics
16. class TradeTracker:
17.     def __init__(self, initial_balance=None, initial_price=None):

```

```

18.         #list of dicts containing entry price, exit price, capital, pnl, pnl pct, winner
19.         (if it was a winning trade or not)
20.         #for completed trades only
21.         self.trades = []
22.         #tracking for currently open positions
23.         self.current_entries = []
24.
25.         # For buy-and-hold comparison
26.         self.initial_balance = initial_balance
27.         self.initial_price = initial_price
28.         self.final_price = None
29.
30.     def open_position(self, entry_price, capital):
31.         self.current_entries.append({
32.             'entry_price': entry_price,
33.             'capital': capital,
34.             'shares': capital / entry_price
35.         })
36.
37.     #records all positions closed
38.     def close_all_positions(self, exit_price):
39.         for entry in self.current_entries:
40.             pnl = (exit_price - entry['entry_price']) * entry['shares']
41.             pnl_pct = (exit_price - entry['entry_price']) / entry['entry_price']
42.
43.             self.trades.append({
44.                 'entry_price': entry['entry_price'],
45.                 'exit_price': exit_price,
46.                 'capital': entry['capital'],
47.                 'pnl': pnl,
48.                 'pnl_pct': pnl_pct,
49.                 'winner': pnl > 0
50.             })
51.
52.         self.current_entries = []
53.
54.     def get_statistics(self):
55.         if not self.trades:
56.             raise RuntimeError("No trades recorded.")
57.
58.         #tracking stats for each winning and losing trade
59.         winners = [t for t in self.trades if t['winner']]
60.         losers = [t for t in self.trades if not t['winner']]
61.
62.         total_trades = len(self.trades)
63.         num_winners = len(winners)
64.         num_losers = len(losers)
65.
66.         win_rate = num_winners / total_trades
67.
68.         #in the unlikely event there are zero winners/losers, avoid a div by zero error
69.         avg_win = np.mean([t['pnl'] for t in winners]) if winners else 0
70.         avg_loss = np.mean([t['pnl'] for t in losers]) if losers else 0
71.
72.         avg_win_pct = np.mean([t['pnl_pct'] for t in winners]) if winners else 0
73.         avg_loss_pct = np.mean([t['pnl_pct'] for t in losers]) if losers else 0
74.
75.         total_profit = sum(t['pnl'] for t in winners)
76.         total_loss = abs(sum(t['pnl'] for t in losers))
77.
78.         #if there were no losers at all, the ratio is technically infinite
79.         #its not a helpful stat in this case, but inf is used to avoid a type error
80.         profit_factor = total_profit / total_loss if total_loss > 0 else float('inf')
81.
82.         #expected outcome of each trade
83.         expectancy = np.mean([t['pnl'] for t in self.trades])
84.
85.         stats = {

```

```

86.         'total_trades': total_trades,
87.         'winners': num_winners,
88.         'losers': num_losers,
89.         'win_rate': win_rate,
90.         'avg_win': avg_win,
91.         'avg_loss': avg_loss,
92.         'avg_win_pct': avg_win_pct,
93.         'avg_loss_pct': avg_loss_pct,
94.         'profit_factor': profit_factor,
95.         'expectancy': expectancy
96.     }
97.
98.     # Add buy-and-hold comparison if we have the required data
99.     if self.initial_balance and self.initial_price and self.final_price:
100.         bh_shares = self.initial_balance / self.initial_price
101.         bh_final_value = bh_shares * self.final_price
102.         bh_return = (bh_final_value - self.initial_balance) / self.initial_balance
103.         bh_pnl = bh_final_value - self.initial_balance
104.
105.         stats['buy_hold_return'] = bh_return
106.         stats['buy_hold_pnl'] = bh_pnl
107.         stats['buy_hold_final_value'] = bh_final_value
108.
109.     return stats
110.
111.
112. #for backend testing, again this isnt used for UI
113. def plot_results(balances, realized_balances, actions, config, trade_stats, title_prefix =
"Test"):
114.
115.     balance_setting = float(config['initial_balance'])
116.
117.     fig = plt.figure(figsize=(16, 10))
118.     #3x2 gridspec for subplots
119.     gs = fig.add_gridspec(3, 2, hspace=0.3, wspace=0.3)
120.
121.     #plot both realized and unrealized capital curves
122.     ax1 = fig.add_subplot(gs[0, :])
123.     days = np.arange(len(balances))
124.
125.     #plot unrealized (total portfolio value)
126.     ax1.plot(days, balances, linewidth=2, color="#2A73AF", alpha=0.5, label='Total Value
(Unrealized)', linestyle='--')
127.
128.     #plot realized (actual cash performance)
129.     ax1.plot(days, realized_balances, linewidth=2.5, color="#0EFF0E", label='Realized PnL')
130.
131.     ax1.axhline(y=balance_setting, color='gray', linestyle='--', alpha=0.5, label='Initial
Capital')
132.
133.     #colour fills for realized gain and loss
134.     ax1.fill_between(days, balance_setting, realized_balances,
where=(np.array(realized_balances) >= balance_setting),
135.                     alpha=0.3, color='green', label='Realized Profit')
136.     ax1.fill_between(days, balance_setting, realized_balances,
where=(np.array(realized_balances) < balance_setting),
137.                     alpha=0.3, color='red', label='Realized Loss')
138.
139.     #grab last element for final balance
140.     final_balance = balances[-1]
141.     final_realized = realized_balances[-1]
142.     total_return = (final_balance - balance_setting) / balance_setting * 100
143.     realized_return = (final_realized - balance_setting) / balance_setting * 100
144.
145.     ax1.set_xlabel('Date', fontsize=11)
146.     ax1.set_ylabel('Portfolio Value ($)', fontsize=11)
147.     ax1.set_title(rf'{title_prefix} Capital Curve - Final: \${final_balance:,.2f}
({total_return:+.1f}%) | Realized: \${final_realized:,.2f} ({realized_return:+.1f}%)',
148.                  fontsize=13, fontweight='bold')

```

```

149.     ax1.grid(True, alpha=0.3)
150.     ax1.legend(loc='upper left')
151.
152.     realized_peak = np.maximum.accumulate(realized_balances)
153.     realized_drawdown = (np.array(realized_balances) - realized_peak) / realized_peak * 100
154.     max_dd = np.min(realized_drawdown)
155.     max_dd_idx = np.argmin(realized_drawdown)
156.
157.     ax1.annotate(f'Max Realized DD: {max_dd:.1f}%',
158.                 xy=(max_dd_idx, realized_balances[max_dd_idx]),
159.                 xytext=(max_dd_idx + 20, realized_balances[max_dd_idx] - 500),
160.                 arrowprops=dict(arrowstyle='->', color='red', lw=1.5),
161.                 fontsize=10, color='red', fontweight='bold')
162.
163.     ax2 = fig.add_subplot(gs[1, :])
164.     ax2.fill_between(days, 0, realized_drawdown, color='red', alpha=0.4)
165.     ax2.plot(days, realized_drawdown, color='darkred', linewidth=1.5)
166.     ax2.set_xlabel('Trading Day', fontsize=11)
167.     ax2.set_ylabel('Realized Drawdown (%)', fontsize=11)
168.     ax2.set_title(f'Realized Drawdown Over Time - Max: {max_dd:.2f}%', fontsize=13,
fontweight='bold')
169.     ax2.grid(True, alpha=0.3)
170.     ax2.axhline(y=-20, color='orange', linestyle='--', alpha=0.5, label='20% DD')
171.     ax2.legend()
172.
173.     #action distribution
174.     ax3 = fig.add_subplot(gs[2, 0])
175.     action_counts = np.bincount(actions, minlength=len(config['action_types']))
176.     action_pcts = action_counts / len(actions) * 100
177.
178.     colors = ["#E32A31", "#4284AF", "#23A999", "#E2C553", "#EAA149"]
179.     bars = ax3.bar(config['action_types'], action_pcts, color=colors, alpha=0.8,
edgecolor='black')
180.
181.     for bar, count, pct in zip(bars, action_counts, action_pcts):
182.         height = bar.get_height()
183.         ax3.text(bar.get_x() + bar.get_width()/2., height,
184.                 f'{count}\n({pct:.1f}%',
185.                 ha='center', va='bottom', fontsize=9, fontweight='bold')
186.
187.     ax3.set_ylabel('Percentage (%)', fontsize=11)
188.     ax3.set_title('Action Distribution', fontsize=13, fontweight='bold')
189.     ax3.set_ylim(0, max(action_pcts) * 1.2)
190.     plt.setp(ax3.xaxis.get_majorticklabels(), rotation=45, ha='right')
191.     ax3.grid(True, alpha=0.3, axis='y')
192.
193.     #trade statistics
194.     ax4 = fig.add_subplot(gs[2, 1])
195.     ax4.axis('off')
196.
197.     stats_text = f"""
198.     TRADE STATISTICS
199.     {'='*35}
200.     Total Trades:      {trade_stats['total_trades']}
201.     Winners:           {trade_stats['winners']} ({trade_stats['win_rate']*100:.1f}%)
202.     Losers:            {trade_stats['losers']} ((1-trade_stats['win_rate'])*100:.1f}%)
203.
204.     Avg Win:           ${trade_stats['avg_win']:.2f}
({trade_stats['avg_win_pct']*100:+.2f}%)
205.     Avg Loss:          ${trade_stats['avg_loss']:.2f}
({trade_stats['avg_loss_pct']*100:.2f}%)
206.
207.     Profit Factor:     {trade_stats['profit_factor']:.2f}
208.     Expectancy:        ${trade_stats['expectancy']:.2f}
209.
210.     PORTFOLIO METRICS
211.     {'='*35}
212.     Final Balance:     ${final_balance:,.2f}
213.     Total Return:      {total_return:+.2f}%

```

```

214.     Realized Return:      {realized_return:+.2f}%
215.     Max Drawdown:        {max_dd:.2f}%
216.     """
217.
218.     ax4.text(0.1, 0.95, stats_text, transform=ax4.transAxes,
219.             fontsize=10, verticalalignment='top', fontfamily='monospace',
220.             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.3))
221.
222.     plt.suptitle(f'{title_prefix} Performance Analysis', fontsize=16, fontweight='bold',
223. y=0.995)
224.
225.     return fig
226.
227. #test agent using expanding windows to avoid leakage
228. def test_agent(agent, test_data, config, train_norm_stats, plot = True,
229. start_date_override=None):
230.     env = TradingEnvironment(
231.         test_data,
232.         config,
233.         config['device'],
234.         is_training=False,
235.         train_normalisation_stats=train_norm_stats
236.     )
237.     state = env.reset()
238.
239.     #override start_idx if date provided
240.     if start_date_override is not None:
241.         test_data['Date'] = pd.to_datetime(test_data['Date'])
242.         try:
243.             #find the index for the requested start date
244.             idx = test_data[test_data['Date'] ==
245. pd.to_datetime(start_date_override)].index[0]
246.
247.             #ensure there's enough room for lookback window and episode length
248.             min_idx = config['lookback_window']
249.             max_idx = len(test_data) - config['episode_length_days'] - 1
250.
251.             if min_idx <= idx <= max_idx:
252.                 env.start_idx = idx
253.                 env.current_step = 0
254.                 state = env.get_state()
255.             else:
256.                 print(f"Warning: Date {start_date_override} out of valid range
257. [{test_data.iloc[min_idx]['Date']} to {test_data.iloc[max_idx]['Date']}. Using random start.")
258.             except (IndexError, KeyError):
259.                 print(f"Warning: Date {start_date_override} not found in data. Using random
260. start.")
261.
262.         start_date = test_data.iloc[env.start_idx]['Date']
263.         end_idx = min(env.start_idx + config['episode_length_days'], len(test_data) - 1)
264.         end_date = test_data.iloc[end_idx]['Date']
265.
266.         print(f"Testing agent on random episode 005")
267.         print(f"Start Date: {start_date}")
268.         print(f"End Date: {end_date}")
269.
270.         total_reward = 0
271.         actions_taken = []
272.         balances = []
273.         realized_balances = []
274.
275.         initial_price = test_data.iloc[env.start_idx]['Close']
276.         trade_tracker = TradeTracker(
277.             initial_balance=float(config['initial_balance']),
278.             initial_price=initial_price
279.         )
280.         while not env.done:

```

```

278.         action = agent.select_action(state, training=False)
279.
280.         positions_before = env.position_manager.get_position_count()
281.
282.         next_state, reward, done, info = env.step(action)
283.
284.         positions_after = env.position_manager.get_position_count()
285.
286.         if action >= 2 and positions_after > positions_before:
287.             new_position = env.position_manager.positions[-1]
288.             trade_tracker.open_position(new_position.entry_price, new_position.allocation)
289.         elif action == 0 and positions_before > 0:
290.             current_price = test_data.iloc[env.start_idx + env.current_step - 1]['Close']
291.             trade_tracker.close_all_positions(current_price)
292.
293.         total_reward += reward
294.         actions_taken.append(action)
295.         balances.append(info['balance'])
296.         realized_balances.append(info['realized_balance'])
297.
298.         state = next_state
299.
300.         if info.get('bankruptcy', False):
301.             print("\nAgent went BANKRUPT during testing!")
302.             break
303.
304.         final_price = test_data.iloc[env.start_idx + env.current_step - 1]['Close']
305.         if trade_tracker.current_entries:
306.             trade_tracker.close_all_positions(final_price)
307.         trade_tracker.final_price = final_price
308.
309.         trade_stats = trade_tracker.get_statistics()
310.
311.         #print results
312.         print(f"\n{'='*60}")
313.         print("TEST RESULTS")
314.         print(f"{'='*60}")
315.         print(f"Total Reward:           {total_reward:.2f}")
316.         print(f"Final Balance:           ${info['balance']:, .2f}")
317.         print(f"Realized Balance:        ${info['realized_balance']:, .2f}")
318.         print(f"Total Return:           {info['total_return']*100:+.2f}%")
319.         print(f"Realized Return:        {info['realized_return']*100:+.2f}%")
320.         print(f"Sharpe Ratio:           {info['sharpe_ratio']:.2f}")
321.         print(f"Bankruptcy:             {info.get('bankruptcy', False)}")
322.
323.         print(f"\n{'='*60}")
324.         print("TRADE STATISTICS")
325.         print(f"{'='*60}")
326.         print(f"Total Trades:           {trade_stats['total_trades']}")
327.         print(f"Winners:                 {trade_stats['winners']}")
328.         print(f"Losers:                 {trade_stats['losers']} ({(1-
329.         trade_stats['win_rate'])*100:.1f}%")
330.         #these edge cases happened with small sample sizes and leaky models, so they are red
331.         #flags
332.         avg_win = trade_stats['avg_win']
333.         avg_win_str = f"${avg_win:.2f}" if not np.isinf(avg_win) else "N/A (no losses)"
334.         avg_win_pct_str = f"{trade_stats['avg_win_pct']*100:+.2f}%" if not
335.         np.isinf(trade_stats['avg_win_pct']) else "N/A"
336.         avg_loss = trade_stats['avg_loss']
337.         avg_loss_str = f"${avg_loss:.2f}" if not np.isinf(avg_loss) else "N/A (no losses)"
338.         avg_loss_pct_str = f"{trade_stats['avg_loss_pct']*100:.2f}%" if not
339.         np.isinf(trade_stats['avg_loss_pct']) else "N/A"
340.         profit_factor = trade_stats['profit_factor']
341.         pf_str = f"{profit_factor:.2f}" if not np.isinf(profit_factor) else "∞ (no losses)"

```

```

342.     expectancy = trade_stats['expectancy']
343.     exp_str = f"${expectancy:.2f}" if not np.isinf(expectancy) else "N/A"
344.
345.     print(f"Avg Win:           {avg_win_str} ({avg_win_pct_str})")
346.     print(f"Avg Loss:          {avg_loss_str} ({avg_loss_pct_str})")
347.     print(f"Profit Factor:      {pf_str}")
348.     print(f"Expectancy:         {exp_str}")
349.
350.     if 'buy_hold_return' in trade_stats:
351.         print(f"\n{'='*60}")
352.         print(f"BUY & HOLD COMPARISON")
353.         print(f"{'='*60}")
354.         print(f"B&H Final Value:      ${trade_stats['buy_hold_final_value']:.2f}")
355.         print(f"B&H Return:          {trade_stats['buy_hold_return']*100:+.2f}%")
356.         print(f"B&H PnL:             ${trade_stats['buy_hold_pnl']:.2f}")
357.         print(f"Strategy Outperformance: {(info['realized_return'] -
trade_stats['buy_hold_return'])*100:+.2f}%")
358.
359.         print(f"\n{'='*60}")
360.         print(f"ACTION DISTRIBUTION")
361.         print(f"{'='*60}")
362.         action_counts = np.bincount(actions_taken, minlength=len(config['action_types']))
363.         for i, (action_type, count) in enumerate(zip(config['action_types'], action_counts)):
364.             print(f"  {action_type:20s}: {count:4d} ({count/len(actions_taken)*100:5.1f}%")
365.
366.         if plot:
367.
368.             fig = plot_results(balances, realized_balances, actions_taken, config, trade_stats,
title_prefix="Test")
369.             os.makedirs(config['plot_path'], exist_ok=True)
370.             save_path = os.path.join(config['plot_path'], 'test_results.png')
371.             plt.savefig(save_path, dpi=150, bbox_inches='tight')
372.             print(f"\n Plot saved to {save_path}")
373.             plt.close()
374.
375.     return trade_stats, balances, actions_taken

```

User Interface

Due to time constraints, parts of the UI code prioritise clarity and reliability over refactoring and abstraction, which has resulted in some repeated sections. The focus was on ensuring the application functioned correctly and remained close to the original design sketches. Producing an exact replica of the sketches was not essential to meeting the project objectives and would have required significantly more development time. For this reason, emojis and Unicode symbols are used for icons and info screens rather than designed images. Error handling has a focus on places where user input can cause failure.

app.py

This is the main script run for the research environment and handles the displaying of all other pages. It in itself is quite simple, just a container that applies a CSS theme to the pages and lists them on a side tab. Streamlit is used for all rendering of UI components.

```

1. import streamlit as st
2. from components import dataset_manager, backtesting, training, trading
3.
4. st.set_page_config(
5.     page_title="BASLIX TERMINAL",

```

Objective 8

This script is the orchestrator for subobjectives under objective eight

```

6.     layout="wide",
7.     initial_sidebar_state="expanded"
8. )
9.
10. def apply_theme():
11.     st.markdown(
12.         """
13.         <style>
14.         .stApp {
15.             background-color: #000000;
16.         }
17.
18.         h1, h2, h3, h4, h5, h6, p, label {
19.             color: #00FF41;
20.         }
21.
22.         .block-container {
23.             padding-top: 1rem;
24.         }
25.
26.         div[data-testid="stSidebarNav"] {
27.             display: none;
28.         }
29.         </style>
30.         """,
31.         unsafe_allow_html=True
32.     )
33.
34. def render_header():
35.     st.markdown(
36.         "<h1 style='text-align:center; font-family:monospace;'>BASLIX TERMINAL</h1>",
37.         unsafe_allow_html=True
38.     )
39.
40. PAGES = {
41.     "📊 Trading": trading,
42.     "📁 Datasets": dataset_manager,
43.     "🔄 Backtesting": backtesting,
44.     "🎓 Training": training,
45. }
46.
47. apply_theme()
48. render_header()
49.
50. with st.sidebar:
51.     st.markdown("### Navigation")
52.     selection = st.radio(
53.         "Navigation",
54.         list(PAGES.keys()),
55.         label_visibility="collapsed"
56.     )
57.
58. page = PAGES.get(selection)
59. if page:
60.     page.render()
61. else:
62.     st.error("Page not found")
63.

```

trading.py

This screen displays the most recent market data, and what the recommended DQN actions are. This can be used for research purposes and evaluate the accuracy of models in real time. It also includes LightGBM forecasts for the same purpose.

News headlines and sentiment scores are included to allow a researcher to compare the model outputs to what their own intuition says about the sentiment.

```
1. import streamlit as st
2. from pathlib import Path
3. import pandas as pd
4. import plotly.graph_objects as go
5. import sys
6. import os
7. from datetime import datetime, timedelta
8. import multiprocessing
9. import threading
10. import queue
11. import time
12. import copy
13. import numpy as np
14. import torch
15. import json
16.
17. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
18. if project_root not in sys.path:
19.     sys.path.insert(0, project_root)
20.
21. from training_resources.dataset_generator import DatasetGenerator
22. from training_resources.fetching.scraperscraper import ArticleScraper
23. from training_resources.processing.sentiment_analysis import SentimentAnalyser
24.
25. DATASET_DIR = Path("application/datasets/trading")
26. LGBM_MODEL_DIR = Path("prediction_tools/saved_models/LGBM")
27. LGBM_CORRELATIONS_DIR = Path("prediction_tools/saved_models/LGBM/correlations")
28.
29. #ticker configs
30. TICKER_CONFIGS = {
31.     "GLD": {
32.         "scraper_config": {
33.             "cookies_path": "training_resources\\fetching\\scraper\\cookies.json",
34.             "target_tag": "80",
35.             "page_size": 80,
36.             "category": "market-outlook::gold-and-precious-metals",
37.             "min_delay": 0,
38.             "max_delay": 1
39.         },
40.         "sentiment_params": {
41.             "bias_prompt": "Gold market outlook: ",
42.             "temperature": 1.6,
43.             "neutrality_scaler": 7.7,
44.         },
45.         "sentiment_lags": [3, 7, 14, 28]
46.     },
47.     "BTC-USD": {
48.         "scraper_config": {
49.             "cookies_path": "training_resources\\fetching\\scraper\\cookies.json",
50.             "target_tag": "581328",
51.             "page_size": 80,
52.             "category": "market-outlook::cryptocurrency",
53.             "min_delay": 0,
54.             "max_delay": 1
55.         },
56.         "sentiment_params": {
57.             "bias_prompt": "Bitcoin market outlook: ",
58.             "temperature": 1.8,
59.             "neutrality_scaler": 7.5,
60.         },
61.         "sentiment_lags": [3, 7, 14, 28]
62.     }
63. }
64.
65. DEFAULT_GENERATOR_CONFIG = {
```

```

66.     'start_date': datetime(2017, 1, 1),
67.     'max_workers': 8,
68.     'pct_change_features': [3, 7, 14, 30, 90, 365],
69.     'pct_change_targets': [3, 7, 14, 30, 90, 365],
70.     'return_targets': [3, 7, 14, 30, 90, 365],
71.     'vol_targets': [3, 7, 14, 30, 90, 365],
72.     'ema_targets': [3, 7, 14, 30, 90, 365],
73.     'normalised_return_targets': [3, 7, 9, 14, 30, 90, 365]
74. }
75.
76. DEFAULT_SENTIMENT_BASE = {
77.     'true_sentiment_weight': 0.7,
78.     'rolling_window': 3,
79.     'decay': 0.7,
80. }
81.
82. #class to gather console output from processing modules
83. class StreamToQueue:
84.     def __init__(self, queue):
85.         self.queue = queue
86.
87.     def write(self, msg):
88.         if msg.strip():
89.             self.queue.put(msg)
90.
91.     def flush(self):
92.         pass
93.
94. #using the progress dictionaries mentioned in other scripts
95. def progress_monitor(generator_instance, shared_dict, stop_event):
96.     while not stop_event.is_set():
97.         try:
98.             if hasattr(generator_instance, 'scraper') and
generator_instance.scraper.progress['state'] != 'idle':
99.                 shared_dict['current'] =
generator_instance.scraper.progress.get('current', 0)
100.                 shared_dict['total'] = generator_instance.scraper.progress.get('total', 0)
101.                 shared_dict['state'] = generator_instance.scraper.progress.get('state',
'Scraping...')
102.             elif hasattr(generator_instance, 'progress'):
103.                 shared_dict['current'] = generator_instance.progress.get('current', 0)
104.                 shared_dict['total'] = generator_instance.progress.get('total', 0)
105.                 shared_dict['state'] = generator_instance.progress.get('state',
'Processing...')
106.                 time.sleep(0.5)
107.             except Exception:
108.                 break
109.
110.
111. #below are worker functions for use in multiprocessing
112. #if this was done in the same process, it would freeze the UI, which is very annoying.
113.
114. def news_fetcher_worker(scraper_config, shared_dict, log_queue):
115.     sys.stdout = StreamToQueue(log_queue)
116.     sys.stderr = StreamToQueue(log_queue)
117.
118.     print("--- News Fetching Started ---")
119.
120.     stop_event = threading.Event()
121.
122.     try:
123.         #calculate date range (last month)
124.         end_date = datetime.now()
125.         start_date = end_date - timedelta(days=30)
126.
127.         print(f"Fetching articles from {start_date.date()} to {end_date.date()}")
128.
129.         scraper = ArticleScraper(
130.             cookies_path=scraper_config['cookies_path'],

```

```

131.         target_tag=scrapper_config['target_tag'],
132.         page_size=scrapper_config['page_size'],
133.         category=scrapper_config['category'],
134.         min_delay=scrapper_config['min_delay'],
135.         max_delay=scrapper_config['max_delay']
136.     )
137.
138.     monitor_thread = threading.Thread(
139.         target=progress_monitor,
140.         args=(scraper, shared_dict, stop_event),
141.         daemon=True
142.     )
143.     monitor_thread.start()
144.
145.     articles = scraper.fetch_articles(start_date, end_date, output_file=None)
146.
147.     print(f"Fetched {len(articles)} articles")
148.
149.     #save to shared dict
150.     shared_dict['articles'] = articles
151.
152. except Exception as e:
153.     print(f"ERROR: {str(e)}")
154.     import traceback
155.     traceback.print_exc(file=sys.stdout)
156. finally:
157.     stop_event.set()
158.     print("--- News Fetching Finished ---")
159.
160. def lgbm_prediction_worker(dataset_path, model_dir, flip_config, shared_dict, log_queue,
output_path):
161.     sys.stdout = StreamToQueue(log_queue)
162.     sys.stderr = StreamToQueue(log_queue)
163.
164.     print("--- LGBM Prediction Generation Started ---")
165.
166.     try:
167.         #load dataset
168.         print(f"Loading dataset: {dataset_path}")
169.         df = pd.read_csv(dataset_path)
170.
171.         #remove existing LGBM columns
172.         lgbm_cols = [col for col in df.columns if col.startswith('LGBM')]
173.         if lgbm_cols:
174.             print(f"Removing {len(lgbm_cols)} existing LGBM columns...")
175.             df = df.drop(columns=lgbm_cols)
176.
177.         #save cleaned dataset temporarily
178.         df.to_csv(dataset_path, index=False)
179.
180.         #load flip configuration
181.         flip_settings = {}
182.         if flip_config and Path(flip_config).exists():
183.             flip_df = pd.read_csv(flip_config)
184.             flip_settings = dict(zip(flip_df['horizon'], flip_df['flip']))
185.             print(f"Loaded flip configuration: {flip_settings}")
186.
187.         #initialize generator with dataset
188.         print("Initializing DatasetGenerator...")
189.         #create minimal config just to load the dataset
190.         gen_config = {
191.             'ticker': 'TEMP',
192.             'start_date': datetime(2017, 1, 1),
193.             'max_workers': 8,
194.             'pct_change_features': [3, 7, 14, 30, 90, 365],
195.             'pct_change_targets': [3, 7, 14, 30, 90, 365],
196.             'return_targets': [3, 7, 14, 30, 90, 365],
197.             'vol_targets': [3, 7, 14, 30, 90, 365],
198.             'ema_targets': [3, 7, 14, 30, 90, 365],

```

```

199.         'normalised_return_targets': [3, 7, 9, 14, 30, 90, 365]
200.     }
201.
202.     scraper_config = {
203.         'cookies_path': "",
204.         'target_tag': '80',
205.         'page_size': 50,
206.         'category': "",
207.         'min_delay': 1,
208.         'max_delay': 5
209.     }
210.
211.     generator = DatasetGenerator(
212.         generator_config=gen_config,
213.         scraper_config=scraper_config,
214.         sentiment_calculations=None,
215.         calculator_params=None
216.     )
217.
218.     #load the dataset into generator
219.     generator.full_dataset = df.set_index('Date') if 'Date' in df.columns else df
220.
221.     stop_event = threading.Event()
222.     monitor_thread = threading.Thread(
223.         target=progress_monitor,
224.         args=(generator, shared_dict, stop_event),
225.         daemon=True
226.     )
227.     monitor_thread.start()
228.
229.     #add LGBM predictions (using quantiles from the model files)
230.     print(f"Adding LGBM predictions from models in: {model_dir}")
231.
232.     generator.add_lgbm_predictions(config=None, model=model_dir)
233.
234.     #apply flipping if configured
235.     if flip_settings:
236.         print("\nApplying flip configuration...")
237.         for horizon, should_flip in flip_settings.items():
238.             if should_flip:
239.                 print(f" Flipping predictions for {horizon}d horizon")
240.                 #find all quantile columns for this horizon
241.                 horizon_cols = [col for col in generator.full_dataset.columns if
col.startswith(f'LGBM_{horizon}d_q')]
242.
243.                 if horizon_cols:
244.                     #extract quantile values and sort them
245.                     quantile_map = {}
246.                     for col in horizon_cols:
247.                         q_str = col.split('_q')[-1]
248.                         try:
249.                             q_val = int(q_str)
250.                             quantile_map[q_val] = generator.full_dataset[col].copy()
251.                         except:
252.                             continue
253.
254.                     #sort quantiles to pair them (lowest with highest)
255.                     sorted_qs = sorted(quantile_map.keys())
256.
257.                     #create temporary storage for swapped data
258.                     temp_data = {}
259.
260.                     #swap quantiles by pairing lowest with highest, invert values
261.                     for i in range(len(sorted_qs)):
262.                         old_q = sorted_qs[i]
263.                         new_q = sorted_qs[-(i+1)] # Pair from opposite end
264.
265.                         # Invert the values and assign to opposite quantile
266.                         temp_data[new_q] = -quantile_map[old_q]

```

```

267.
268.                 # Apply the swapped data
269.                 for q_val, data in temp_data.items():
270.                     col_name = f'LGBM_{horizon}d_q{q_val}'
271.                     generator.full_dataset[col_name] = data
272.
273.                     print(f"    Swapped quantile labels and inverted values for
{horizon}d")
274.                     print(f"    Quantiles swapped: {sorted_qs}")
275.
276.                 #save updated dataset
277.                 print(f"Saving updated dataset to: {output_path}")
278.                 df_to_save = generator.full_dataset.copy()
279.                 df_to_save.insert(0, "Date", df_to_save.index)
280.                 df_to_save.to_csv(output_path, index=False)
281.
282.                 print("LGBM predictions added to dataset")
283.
284.             except Exception as e:
285.                 print(f"ERROR: {str(e)}")
286.                 import traceback
287.                 traceback.print_exc(file=sys.stdout)
288.             finally:
289.                 stop_event.set()
290.                 print("--- LGBM Prediction Generation Finished ---")
291.
292. def dataset_worker(ticker, gen_config, scraper_config, sentiment_config, shared_dict,
log_queue, dataset_dir):
293.     sys.stdout = StreamToQueue(log_queue)
294.     sys.stderr = StreamToQueue(log_queue)
295.
296.     print("--- Dataset Generation Started ---")
297.     print(f"Generator Config: {gen_config}")
298.
299.     try:
300.         print("Initializing DatasetGenerator...")
301.         generator = DatasetGenerator(
302.             generator_config=gen_config,
303.             scraper_config=scraper_config,
304.             sentiment_calculations=sentiment_config,
305.             calculator_params=None
306.         )
307.
308.         stop_event = threading.Event()
309.         monitor_thread = threading.Thread(
310.             target=progress_monitor,
311.             args=(generator, shared_dict, stop_event),
312.             daemon=True
313.         )
314.         monitor_thread.start()
315.
316.         print("Generating dataset...")
317.         dataset = generator.generate_dataset()
318.
319.         #determine filename
320.         end = gen_config['end_date'].strftime('%Y%m%d')
321.         filename = f"{ticker}_latest_{end}.csv"
322.
323.         save_dir = Path(str(dataset_dir))
324.         save_dir.mkdir(parents=True, exist_ok=True)
325.
326.         save_path = save_dir / filename
327.
328.         print(f"Saving dataset to: {save_path}")
329.         generator.save_dataset(save_path)
330.         print(f"SUCCESS: Dataset saved as {filename}")
331.
332.         #cleanup old datasets for this ticker
333.         print("\nCleaning up old datasets...")

```

```

334.     all_ticker_datasets = list(save_dir.glob(f"{ticker}_latest_*.csv"))
335.     if len(all_ticker_datasets) > 1:
336.         #sort by modification time, keep newest
337.         all_ticker_datasets.sort(key=lambda x: x.stat().st_mtime, reverse=True)
338.         datasets_to_delete = all_ticker_datasets[1:] # All except the newest
339.
340.         for old_dataset in datasets_to_delete:
341.             try:
342.                 old_dataset.unlink()
343.                 print(f"Deleted old dataset: {old_dataset.name}")
344.             except Exception as e:
345.                 print(f"Failed to delete {old_dataset.name}: {e}")
346.         else:
347.             print("No old datasets to clean up")
348.
349.     except Exception as e:
350.         print(f"ERROR: {str(e)}")
351.         import traceback
352.         traceback.print_exc(file=sys.stdout)
353.     finally:
354.         stop_event.set()
355.         print("--- Dataset Generation Finished ---")
356.
357. def update_dataset_worker(ticker, existing_path, gen_config, scraper_config,
sentiment_config, shared_dict, log_queue, dataset_dir):
358.     sys.stdout = StreamToQueue(log_queue)
359.     sys.stderr = StreamToQueue(log_queue)
360.
361.     print("--- Dataset Update Started ---")
362.     print(f"Generator Config: {gen_config}")
363.
364.     try:
365.         print(f"Loading existing dataset: {existing_path}")
366.         existing_df = pd.read_csv(existing_path)
367.         existing_df['Date'] = pd.to_datetime(existing_df['Date'])
368.
369.         #get the last date in existing dataset
370.         last_date = existing_df['Date'].max()
371.         yesterday = datetime.now() - timedelta(days=1)
372.
373.         print(f"Last date in existing dataset: {last_date.date()}")
374.         print(f"Target date (yesterday): {yesterday.date()}")
375.
376.         #check if already up to date
377.         if last_date.date() >= yesterday.date():
378.             print("Dataset is already up to date!")
379.             return
380.
381.         #set new date range for update
382.         new_start = last_date + timedelta(days=1)
383.         new_end = yesterday
384.
385.         print(f"Fetching data from {new_start.date()} to {new_end.date()}")
386.
387.         #update generator config
388.         update_config = gen_config.copy()
389.         update_config['start_date'] = new_start
390.         update_config['end_date'] = new_end
391.
392.         print("Initializing DatasetGenerator for update...")
393.         generator = DatasetGenerator(
394.             generator_config=update_config,
395.             scraper_config=scraper_config,
396.             sentiment_calculations=sentiment_config,
397.             calculator_params=None
398.         )
399.
400.         stop_event = threading.Event()
401.         monitor_thread = threading.Thread(

```

```

402.         target=progress_monitor,
403.         args=(generator, shared_dict, stop_event),
404.         daemon=True
405.     )
406.     monitor_thread.start()
407.
408.     print("Generating new data...")
409.     new_dataset = generator.generate_dataset()
410.
411.     #merge with existing dataset
412.     print("Merging with existing dataset...")
413.
414.     #remove 'Date' column from existing if it exists as a regular column
415.     if 'Date' in existing_df.columns:
416.         existing_df = existing_df.set_index('Date')
417.
418.     #combine datasets
419.     combined_df = pd.concat([existing_df, new_dataset], axis=0)
420.     combined_df = combined_df[~combined_df.index.duplicated(keep='last')]
421.     combined_df = combined_df.sort_index()
422.
423.     #save updated dataset
424.     end = yesterday.strftime('%Y%m%d')
425.     filename = f"{ticker}_latest_{end}.csv"
426.
427.     save_dir = Path(str(dataset_dir))
428.     save_dir.mkdir(parents=True, exist_ok=True)
429.
430.     save_path = save_dir / filename
431.
432.     print(f"Saving updated dataset to: {save_path}")
433.
434.     #prepare for saving
435.     combined_df_to_save = combined_df.copy()
436.     combined_df_to_save.insert(0, "Date", combined_df_to_save.index)
437.     combined_df_to_save.to_csv(save_path, index=False)
438.
439.     print(f"SUCCESS: Dataset updated and saved as {filename}")
440.     print(f"Total rows: {len(combined_df_to_save)}")
441.     print(f"Date range: {combined_df_to_save['Date'].min().date()} to
{combined_df_to_save['Date'].max().date()}")
442.
443.     #cleanup old datasets for this ticker
444.     print("\nCleaning up old datasets...")
445.     all_ticker_datasets = list(save_dir.glob(f"{ticker}_latest_*.csv"))
446.     if len(all_ticker_datasets) > 1:
447.         #sort by modification time, keep newest
448.         all_ticker_datasets.sort(key=lambda x: x.stat().st_mtime, reverse=True)
449.         datasets_to_delete = all_ticker_datasets[1:] # All except the newest
450.
451.         for old_dataset in datasets_to_delete:
452.             try:
453.                 old_dataset.unlink()
454.                 print(f"Deleted old dataset: {old_dataset.name}")
455.             except Exception as e:
456.                 print(f"Failed to delete {old_dataset.name}: {e}")
457.         else:
458.             print("No old datasets to clean up")
459.
460.     except Exception as e:
461.         print(f"ERROR: {str(e)}")
462.         import traceback
463.         traceback.print_exc(file=sys.stdout)
464.     finally:
465.         stop_event.set()
466.         print("--- Dataset Update Finished ---")
467.
468.     #styling for console
469.     def render_console(log_lines, height=300):

```

```

470.     log_text = "\n".join(log_lines[-100:])
471.     st.markdown(
472.         f"""
473.         <div style="
474.             background-color:#0e1117;
475.             color:#e5e7eb;
476.             padding:10px;
477.             border-radius:5px;
478.             height:{height}px;
479.             overflow-y:scroll;
480.             font-family: monospace;
481.             white-space: pre-wrap;
482.         ">
483.             {log_text}
484.         </div>
485.         """,
486.         unsafe_allow_html=True
487.     )
488.
489. #utility functions
490.
491. def convert_to_datetime(date_value):
492.     if isinstance(date_value, datetime):
493.         return date_value
494.     else:
495.         #its a date object from the widget, convert to datetime
496.         return datetime.combine(date_value, datetime.min.time())
497.
498. def build_generator_config():
499.     gen_config = {}
500.
501.     for key, default_val in DEFAULT_GENERATOR_CONFIG.items():
502.         session_key = f"gen_{key}"
503.
504.         if key == 'start_date':
505.             #get from session state and convert to datetime
506.             date_value = st.session_state.get(session_key, default_val)
507.             gen_config[key] = convert_to_datetime(date_value)
508.         elif isinstance(default_val, int):
509.             gen_config[key] = st.session_state.get(session_key, default_val)
510.         elif isinstance(default_val, list):
511.             #parse list from comma-separated string
512.             list_str = st.session_state.get(session_key, "").join(str(x) for x in
default_val))
513.             gen_config[key] = [int(x.strip()) for x in list_str.split(",") if x.strip()]
514.         else:
515.             gen_config[key] = st.session_state.get(session_key, default_val)
516.
517.     return gen_config
518.
519. def get_available_lgbm_models():
520.     if not LGBM_MODEL_DIR.exists():
521.         return []
522.
523.     #group models by base name
524.     model_files = list(LGBM_MODEL_DIR.glob("*.pkl"))
525.     model_groups = {}
526.
527.     for file in model_files:
528.         name = file.stem
529.         #extract base name (everything before the last _XXd)
530.         if '_' in name and name.split('_')[-1].endswith('d'):
531.             base_name = '_'.join(name.split('_')[:-1])
532.         else:
533.             base_name = name
534.
535.         if base_name not in model_groups:
536.             model_groups[base_name] = []
537.         model_groups[base_name].append(file)

```

```

538.
539.     return sorted(model_groups.keys())
540.
541.
542. #main display
543. def render():
544.     st.markdown("### 📊 Trading Terminal")
545.     st.divider()
546.
547.     #initialize session state
548.     if 'run_state' not in st.session_state:
549.         st.session_state.run_state = "idle"
550.     if 'console_logs' not in st.session_state:
551.         st.session_state.console_logs = []
552.     if 'proc_ref' not in st.session_state:
553.         st.session_state.proc_ref = None
554.     if 'shared_progress' not in st.session_state:
555.         st.session_state.shared_progress = None
556.     if 'log_queue' not in st.session_state:
557.         st.session_state.log_queue = None
558.     if 'manager' not in st.session_state:
559.         st.session_state.manager = None
560.     if 'selected_ticker' not in st.session_state:
561.         st.session_state.selected_ticker = "BTC-USD"
562.
563.     #get datasets
564.     datasets = (
565.         [f.name for f in DATASET_DIR.glob("*.csv")]
566.         if DATASET_DIR.exists()
567.         else []
568.     )
569.
570.     left, right = st.columns([4, 1.7])
571.
572.     with left:
573.         with st.container():
574.             st.markdown("#### 📈 Recent Price Data")
575.
576.             selected_ticker = st.selectbox(
577.                 "Select Ticker",
578.                 ["BTC-USD", "GLD"],
579.                 key="ticker_select"
580.             )
581.             st.session_state.selected_ticker = selected_ticker
582.
583.             #find latest dataset for this ticker
584.             ticker_datasets = [d for d in datasets if d.startswith(selected_ticker)]
585.
586.             if ticker_datasets:
587.                 #sort by date in filename to get latest
588.                 latest_dataset = sorted(ticker_datasets, reverse=True)[0]
589.
590.                 try:
591.                     df = pd.read_csv(DATASET_DIR / latest_dataset)
592.                     df["Date"] = pd.to_datetime(df["Date"])
593.
594.                     #candlestick chart
595.                     fig = go.Figure(data=[go.Candlestick(
596.                         x=df["Date"],
597.                         open=df["Open"],
598.                         high=df["High"],
599.                         low=df["Low"],
600.                         close=df["Close"],
601.                         increasing_line_color="#00ff1a",
602.                         decreasing_line_color="#fb3333",
603.                     )])
604.                     fig.update_layout(
605.                         title=f"{selected_ticker} Price",
606.                         xaxis_title="Date",

```

Objective 8.4

Renders dashboard to show both DQN decisions and LightGBM predictions on current day data

```

607.         yaxis_title="Price",
608.         height=450,
609.         margin=dict(l=10, r=10, t=40, b=10),
610.         dragmode="zoom",
611.         plot_bgcolor="#0e1117",
612.         paper_bgcolor="#0e1117",
613.         font=dict(color="#e5e7eb"),
614.         xaxis_rangeslider_visible=False,
615.     )
616.     st.plotly_chart(fig, use_container_width=True)
617.
618.     st.caption(f"Latest dataset: {latest_dataset}")
619.
620.     except Exception as e:
621.         st.error(f"Failed to load dataset: {e}")
622. else:
623.     st.info(f"No datasets found for {selected_ticker}. Generate one to see
charts.")
624.
625.     #buttons
626.     col1, col2 = st.columns(2)
627.
628.     is_running = st.session_state.run_state == "running"
629.     is_stopping = st.session_state.run_state == "stopping"
630.
631.     with col1:
632.         if st.button("🔄 Regenerate Dataset", use_container_width=True,
disabled=is_running or is_stopping, key="btn_regen_dataset"):
633.             st.session_state.console_logs = []
634.             st.session_state.run_state = "running"
635.
636.             #build configs from session state (UI inputs)
637.             ticker = st.session_state.selected_ticker
638.             ticker_config = TICKER_CONFIGS[ticker]
639.
640.             #build generator config from session state
641.             gen_config = build_generator_config()
642.             gen_config['ticker'] = ticker
643.             gen_config['end_date'] = datetime.now() - timedelta(days=1)
644.
645.             scraper_config = ticker_config['scraper_config']
646.
647.             sentiment_config = copy.deepcopy(DEFAULT_SENTIMENT_BASE)
648.             sentiment_config['sentiment_params'] =
ticker_config['sentiment_params']
649.             sentiment_config['lags'] = ticker_config['sentiment_lags']
650.
651.             #create manager and process
652.             st.session_state.manager = multiprocessing.Manager()
653.             st.session_state.log_queue = st.session_state.manager.Queue()
654.             st.session_state.shared_progress = st.session_state.manager.dict()
655.             st.session_state.shared_progress['current'] = 0
656.             st.session_state.shared_progress['total'] = 100
657.             st.session_state.shared_progress['state'] = "Initializing..."
658.
659.             p = multiprocessing.Process(
660.                 target=dataset_worker,
661.                 args=(
662.                     ticker,
663.                     gen_config,
664.                     scraper_config,
665.                     sentiment_config,
666.                     st.session_state.shared_progress,
667.                     st.session_state.log_queue,
668.                     str(DATASET_DIR)
669.                 )
670.             )
671.             st.session_state.proc_ref = p
672.             p.start()

```

Spawns a separate process to run dataset generation independently of the Streamlit UI, preventing the interface from freezing during long-running computations

```

673.             st.rerun()
674.
675.         with col2:
676.             if st.button("⚡ Update Dataset", use_container_width=True,
disabled=is_running or is_stopping, key="btn_update_dataset"):
677.                 if not ticker_datasets:
678.                     st.warning(f"No existing dataset found for {selected_ticker}.
Please regenerate first.")
679.                 else:
680.                     #get latest dataset
681.                     latest_dataset = sorted(ticker_datasets, reverse=True)[0]
682.                     existing_path = DATASET_DIR / latest_dataset
683.
684.                     #load to check start date
685.                     try:
686.                         existing_df = pd.read_csv(existing_path)
687.                         existing_df['Date'] = pd.to_datetime(existing_df['Date'])
688.                         existing_start = existing_df['Date'].min()
689.
690.                         #build generator config from session state
691.                         gen_config = build_generator_config()
692.                         requested_start = gen_config['start_date']
693.
694.                         #check if existing dataset goes back far enough (and add 3 day
tolerance in case of weekends and the like)
695.                         date_diff = (existing_start.date() -
requested_start.date()).days
696.                         if date_diff > 3:
697.                             st.warning(f"Existing dataset starts at
{existing_start.date()}, but config requests {requested_start.date()}. Regenerating...")
698.
699.                             #regenerate
700.                             st.session_state.console_logs = []
701.                             st.session_state.run_state = "running"
702.
703.                             ticker = st.session_state.selected_ticker
704.                             ticker_config = TICKER_CONFIGS[ticker]
705.
706.                             gen_config['ticker'] = ticker
707.                             gen_config['end_date'] = datetime.now() -
timedelta(days=1)
708.
709.                             scraper_config = ticker_config['scraper_config']
710.
711.                             sentiment_config = copy.deepcopy(DEFAULT_SENTIMENT_BASE)
712.                             sentiment_config['sentiment_params'] =
ticker_config['sentiment_params']
713.                             sentiment_config['lags'] = ticker_config['sentiment_lags']
714.
715.                             st.session_state.manager = multiprocessing.Manager()
716.                             st.session_state.log_queue =
st.session_state.manager.Queue()
717.                             st.session_state.shared_progress =
st.session_state.manager.dict()
718.                             st.session_state.shared_progress['current'] = 0
719.                             st.session_state.shared_progress['total'] = 100
720.                             st.session_state.shared_progress['state'] =
"Initializing..."
721.
722.                             p = multiprocessing.Process(
723.                                 target=dataset_worker,
724.                                 args=(
725.                                     ticker,
726.                                     gen_config,
727.                                     scraper_config,
728.                                     sentiment_config,
729.                                     st.session_state.shared_progress,
730.                                     st.session_state.log_queue,
731.                                     str(DATASET_DIR)

```

Note: Worker function logic is defined near the top of the UI scripts, as long-running tasks are executed in separate processes to avoid blocking the main interface

```

732.         )
733.     )
734.     st.session_state.proc_ref = p
735.     p.start()
736.     st.rerun()
737. else:
738.     #proceed with update
739.     st.session_state.console_logs = []
740.     st.session_state.run_state = "running"
741.
742.     ticker = st.session_state.selected_ticker
743.     ticker_config = TICKER_CONFIGS[ticker]
744.
745.     gen_config['ticker'] = ticker
746.     gen_config['end_date'] = datetime.now() -
timedelta(days=1)
747.
748.     scraper_config = ticker_config['scraper_config']
749.
750.     sentiment_config = copy.deepcopy(DEFAULT_SENTIMENT_BASE)
751.     sentiment_config['sentiment_params'] =
ticker_config['sentiment_params']
752.     sentiment_config['lags'] = ticker_config['sentiment_lags']
753.
754.     st.session_state.manager = multiprocessing.Manager()
755.     st.session_state.log_queue =
st.session_state.manager.Queue()
756.     st.session_state.shared_progress =
st.session_state.manager.dict()
757.     st.session_state.shared_progress['current'] = 0
758.     st.session_state.shared_progress['total'] = 100
759.     st.session_state.shared_progress['state'] = "Updating..."
760.
761.     p = multiprocessing.Process(
762.         target=update_dataset_worker,
763.         args=(
764.             ticker,
765.             str(existing_path),
766.             gen_config,
767.             scraper_config,
768.             sentiment_config,
769.             st.session_state.shared_progress,
770.             st.session_state.log_queue,
771.             str(DATASET_DIR)
772.         )
773.     )
774.     st.session_state.proc_ref = p
775.     p.start()
776.     st.rerun()
777.
778. except Exception as e:
779.     st.error(f"Failed to check existing dataset: {e}")
780.
781. #LGBM Controls and Chart (below dataset buttons)
782. if ticker_datasets:
783.     latest_dataset = sorted(ticker_datasets, reverse=True)[0]
784.
785.     try:
786.         df = pd.read_csv(DATASET_DIR / latest_dataset)
787.         df["Date"] = pd.to_datetime(df["Date"])
788.
789.         #check if LGBM predictions exist
790.         lgbm_cols = [col for col in df.columns if col.startswith('LGBM_')]
791.
792.         #always show the model selector and regenerate button
793.         available_models = get_available_lgbm_models()
794.         if available_models:
795.             st.markdown("---")
796.

```

```

797.                 col1, col2 = st.columns([3, 1])
798.                 with col1:
799.                     selected_model = st.selectbox(
800.                         "LGBM Model",
801.                         available_models,
802.                         key="lgbm_model_select"
803.                     )
804.
805.                 with col2:
806.                     st.markdown("<div style='height: 28px;'></div>",
unsafe_allow_html=True)
807.                     if st.button("🔄 Regen LGBM", use_container_width=True,
disabled=st.session_state.run_state == "running"):
808.                         st.session_state.console_logs = []
809.                         st.session_state.run_state = "running"
810.
811.                         #check for flip configuration
812.                         flip_config_path = LGBM_CORRELATIONS_DIR /
f"{selected_model}_flip_config.csv"
813.                         flip_config = str(flip_config_path) if
flip_config_path.exists() else None
814.
815.                         #create manager and process
816.                         st.session_state.manager = multiprocessing.Manager()
817.                         st.session_state.log_queue =
st.session_state.manager.Queue()
818.                         st.session_state.shared_progress =
st.session_state.manager.dict()
819.                         st.session_state.shared_progress['current'] = 0
820.                         st.session_state.shared_progress['total'] = 100
821.                         st.session_state.shared_progress['state'] =
"Initializing..."
822.
823.                         p = multiprocessing.Process(
824.                             target=lgbm_prediction_worker,
825.                             args=(
826.                                 str(DATASET_DIR / latest_dataset),
827.                                 str(LGBM_MODEL_DIR),
828.                                 flip_config,
829.                                 st.session_state.shared_progress,
830.                                 st.session_state.log_queue,
831.                                 str(DATASET_DIR / latest_dataset)
832.                             )
833.                         )
834.                         st.session_state.proc_ref = p
835.                         p.start()
836.                         st.rerun()
837.
838.                     st.markdown("#### 📊 LightGBM Predictions")
839.
840.                     #show chart if LGBM predictions exist, otherwise show placeholder
841.                     if lgbm_cols:
842.                         #extract horizons and quantiles from columns
843.                         horizons = set()
844.                         quantiles_set = set()
845.                         for col in lgbm_cols:
846.                             #format: LGBM_{horizon}d_q{quantile}
847.                             parts = col.split('_')
848.                             if len(parts) >= 3 and parts[2].startswith('q'):
849.                                 horizon_str = parts[1].replace('d', '')
850.                                 quantile_str = parts[2].replace('q', '')
851.                                 try:
852.                                     horizon_val = int(horizon_str)
853.                                     quantile_val = int(quantile_str)
854.                                     # Only accept quantiles between 0 and 100
855.                                     if 0 <= quantile_val <= 100:
856.                                         horizons.add(horizon_val)
857.                                         quantiles_set.add(quantile_val / 100.0)
858.                                 except:

```

```

859.                                     pass
860.
861.     horizons = sorted(horizons)
862.     quantiles = sorted(quantiles_set)
863.
864.     if horizons and quantiles:
865.         #get last 6 months of data
866.         six_months_ago = df['Date'].max() - timedelta(days=180)
867.         recent_df = df[df['Date'] >= six_months_ago].copy()
868.
869.         #create figure
870.         fig = go.Figure()
871.
872.         #plot historical closes
873.         fig.add_trace(go.Scatter(
874.             x=recent_df['Date'],
875.             y=recent_df['Close'],
876.             mode='lines',
877.             name='Close Price',
878.             line=dict(color='#00ff41', width=2)
879.         ))
880.
881.         #plot bollinger bands for historical section
882.         if all(col in recent_df.columns for col in ['BB_Upper',
'BB_Mid', 'BB_Lower']):
883.             # Upper band
884.             fig.add_trace(go.Scatter(
885.                 x=recent_df['Date'],
886.                 y=recent_df['BB_Upper'],
887.                 mode='lines',
888.                 name='BB Upper',
889.                 line=dict(color='rgba(100, 100, 100, 0.3)', width=1,
dash='dash'),
890.                 showlegend=False
891.             ))
892.
893.             fig.add_trace(go.Scatter(
894.                 x=recent_df['Date'],
895.                 y=recent_df['BB_Lower'],
896.                 mode='lines',
897.                 name='BB Lower',
898.                 line=dict(color='rgba(100, 100, 100, 0.3)', width=1,
dash='dash'),
899.                 fill='tonexty',
900.                 fillcolor='rgba(100, 100, 100, 0.1)',
901.                 showlegend=False
902.             ))
903.
904.             fig.add_trace(go.Scatter(
905.                 x=recent_df['Date'],
906.                 y=recent_df['BB_Mid'],
907.                 mode='lines',
908.                 name='BB Mid',
909.                 line=dict(color='rgba(150, 150, 150, 0.5)', width=1,
dash='dot'),
910.                 showlegend=False
911.             ))
912.
913.         last_close = df['Close'].iloc[-1]
914.         last_date = df['Date'].iloc[-1]
915.
916.         #find min and max quantiles
917.         min_q = min(quantiles)
918.         max_q = max(quantiles)
919.         median_q = min(quantiles, key=lambda q: abs(q - 0.5))
920.
921.         #connecting horizons
922.         colors_quantiles = {
923.

```

```

924.         min_q: '#f87171', #red for lower quantile
925.         median_q: '#facc15', #yellow for median
926.         max_q: '#3b82f6' #blue for upper quantile
927.     }
928.
929.     for q in quantiles:
930.         q_dates = []
931.         q_prices = []
932.
933.         #for median, connect to current close price
934.         if q == median_q:
935.             q_dates.append(last_date)
936.             q_prices.append(last_close)
937.
938.         for horizon in horizons:
939.             q_col = f'LGBM_{horizon}d_q{int(q*100)}'
940.
941.             if q_col in df.columns:
942.                 #get the most recent prediction for this quantile
943.                 last_pred_pct = df[q_col].iloc[-1]
944.
945.                 if not pd.isna(last_pred_pct):
946.                     future_date = last_date +
timedelta(days=horizon)
947.                     future_price = last_close * (1 + last_pred_pct
/ 100)
948.
949.                     q_dates.append(future_date)
950.                     q_prices.append(future_price)
951.
952.                 #only plot if points exist
953.                 if len(q_dates) >= 2:
954.                     color = colors_quantiles.get(q, '#8b5cf6')
955.                     line_style = 'solid' if q == median_q else 'dash'
956.                     line_width = 2 if q == median_q else 1.5
957.
958.                     #plot quantile line
959.                     fig.add_trace(go.Scatter(
960.                         x=q_dates,
961.                         y=q_prices,
962.                         mode='lines+markers',
963.                         name=f'q{int(q*100)} Forecast',
964.                         line=dict(color=color, width=line_width,
dash=line_style),
965.                         marker=dict(size=8 if q == median_q else 6,
color=color, symbol='diamond' if q == median_q else 'circle')
966.                     ))
967.
968.             fig.update_layout(
969.                 title=f"{selected_ticker} with LGBM Predictions",
970.                 xaxis_title="Date",
971.                 yaxis_title="Price",
972.                 height=450,
973.                 margin=dict(l=10, r=10, t=40, b=10),
974.                 dragmode="zoom",
975.                 plot_bgcolor="#0e1117",
976.                 paper_bgcolor="#0e1117",
977.                 font=dict(color="#e5e7eb"),
978.                 xaxis_rangeslider_visible=False,
979.                 showlegend=True,
980.                 legend=dict(
981.                     yanchor="top",
982.                     y=0.99,
983.                     xanchor="left",
984.                     x=0.01
985.                 )
986.             )
987.
988.     st.plotly_chart(fig, use_container_width=True)

```

```

989.
990.             st.caption(f"Horizons: {' '.join([f'{h}d' for h in
horizons])) | Quantiles: {' '.join([f'q{int(q*100)}' for q in quantiles]))")
991.         else:
992.             #placeholder when no LGBM predictions exist
993.             st.markdown(
994.                 """
995.                 <div style="
996.                 background-color:#1a1a1a;
997.                 height:450px;
998.                 border-radius:5px;
999.                 display:flex;
1000.                 align-items:center;
1001.                 justify-content:center;
1002.                 color:#666;
1003.                 font-size:18px;
1004.                 ">
1005.                 No LGBM predictions available. Select a model and click
"Regen LGBM" to generate predictions.
1006.                 </div>
1007.                 """,
1008.                 unsafe_allow_html=True
1009.             )
1010.
1011.     except Exception as e:
1012.         #placeholder on error
1013.         st.markdown(
1014.             """
1015.             <div style="
1016.             background-color:#1a1a1a;
1017.             height:450px;
1018.             border-radius:5px;
1019.             display:flex;
1020.             align-items:center;
1021.             justify-content:center;
1022.             color:#666;
1023.             font-size:18px;
1024.             ">
1025.             Unable to load prediction data.
1026.             </div>
1027.             """,
1028.             unsafe_allow_html=True
1029.         )
1030.
1031.     except Exception as e:
1032.         last_date = df['Date'].iloc[-1]
1033.
1034.         #find min and max quantiles
1035.         min_q = min(quantiles)
1036.         max_q = max(quantiles)
1037.         median_q = min(quantiles, key=lambda q: abs(q - 0.5))
1038.
1039.         #plot quantile bands
1040.         for horizon in horizons:
1041.             min_col = f'LGBM_{horizon}d_q{int(min_q*100)}'
1042.             max_col = f'LGBM_{horizon}d_q{int(max_q*100)}'
1043.
1044.             if min_col in recent_df.columns and max_col in
recent_df.columns:
1045.                 future_dates = [recent_df['Date'].iloc[i] +
timedelta(days=horizon)
1046.                                 for i in range(len(recent_df))]
1047.
1048.                 #calculate predicted prices
1049.                 min_prices = recent_df['Close'] * (1 +
recent_df[min_col] / 100)
1050.                 max_prices = recent_df['Close'] * (1 +
recent_df[max_col] / 100)
1051.

```

```

1052.         #plot as a band
1053.         fig.add_trace(go.Scatter(
1054.             x=future_dates + future_dates[::-1],
1055.             y=list(max_prices) + list(min_prices)[::-1],
1056.             fill='toself',
1057.             fillcolor=f'rgba({50 + horizon % 200}, {100 +
horizon % 150}, {200 - horizon % 100}, 0.2)'),
1058.             line=dict(width=0),
1059.             name=f'{horizon}d Range',
1060.             showlegend=True,
1061.             hoverinfo='skip'
1062.         ))
1063.
1064.         #plot median prediction line connecting horizons
1065.         median_dates = [last_date]
1066.         median_prices = [last_close]
1067.
1068.         for horizon in horizons:
1069.             median_col = f'LGBM_{horizon}d_q{int(median_q*100)}'
1070.
1071.             if median_col in df.columns:
1072.                 #get the most recent prediction
1073.                 last_pred_pct = df[median_col].iloc[-1]
1074.
1075.                 if not pd.isna(last_pred_pct):
1076.                     future_date = last_date + timedelta(days=horizon)
1077.                     future_price = last_close * (1 + last_pred_pct /
1078.                     100)
1079.
1080.                     median_dates.append(future_date)
1081.                     median_prices.append(future_price)
1082.
1083.         #plot median line
1084.         fig.add_trace(go.Scatter(
1085.             x=median_dates,
1086.             y=median_prices,
1087.             mode='lines+markers',
1088.             name='Median Forecast',
1089.             line=dict(color='#facc15', width=2, dash='dash'),
1090.             marker=dict(size=8, color='#facc15')
1091.         ))
1092.
1093.         fig.update_layout(
1094.             title=f"{selected_ticker} with LGBM Predictions",
1095.             xaxis_title="Date",
1096.             yaxis_title="Price",
1097.             height=450,
1098.             margin=dict(l=10, r=10, t=40, b=10),
1099.             dragmode="zoom",
1100.             plot_bgcolor="#0e1117",
1101.             paper_bgcolor="#0e1117",
1102.             font=dict(color="#e5e7eb"),
1103.             xaxis_rangeslider_visible=False,
1104.             showlegend=True,
1105.             legend=dict(
1106.                 yanchor="top",
1107.                 y=0.99,
1108.                 xanchor="left",
1109.                 x=0.01
1110.             )
1111.         )
1112.
1113.         st.plotly_chart(fig, use_container_width=True)
1114.
1115.         st.caption(f"Horizons: {'', '.join([f'{h}d' for h in
horizons])} | Quantiles: {'', '.join([f'q{int(q*100)}' for q in quantiles])}")
1116.
1117.         except Exception as e:
1118.             print("failed to read dataset.")

```

```

1118.
1119.     with right:
1120.         with st.container():
1121.             st.markdown("####  DQN Agent")
1122.
1123.             #initialize DQN state
1124.             if 'dqn_action' not in st.session_state:
1125.                 st.session_state.dqn_action = None
1126.                 st.session_state.dqn_action_raw = None
1127.             if 'dqn_agent' not in st.session_state:
1128.                 st.session_state.dqn_agent = None
1129.                 st.session_state.dqn_ticker = None
1130.
1131.             #clear action if ticker changed
1132.             if st.session_state.get('dqn_ticker') != st.session_state.selected_ticker:
1133.                 st.session_state.dqn_action = None
1134.                 st.session_state.dqn_action_raw = None
1135.
1136.             #display current action
1137.             if st.session_state.dqn_action is None:
1138.                 st.markdown(
1139.                     """
1140.                     <div style="
1141.                         text-align: center;
1142.                         padding: 20px;
1143.                         border: 2px solid #444;
1144.                         border-radius: 10px;
1145.                         background-color: #1a1a1a;
1146.                     ">
1147.                         <div style="font-size: 40px; font-weight: bold; color:
#888;"></div>
1148.                         <div style="font-size: 14px; color: #666; margin-top: 5px;">No
action yet</div>
1149.                     </div>
1150.                     """,
1151.                     unsafe_allow_html=True
1152.                 )
1153.             else:
1154.                 #parse action to determine display
1155.                 action_name = st.session_state.dqn_action_raw
1156.
1157.                 if 'close' in action_name.lower():
1158.                     action_text = "CLOSE"
1159.                     action_color = "#ff3b3b"
1160.                     sub_text = "Close all positions"
1161.                 elif 'hold' in action_name.lower():
1162.                     action_text = "HOLD"
1163.                     action_color = "#ffa500"
1164.                     sub_text = "Maintain positions"
1165.                 elif 'pyramid' in action_name.lower():
1166.                     #extract pyramid size
1167.                     import re
1168.                     match = re.search(r'pyramid_([0-9.]+)', action_name.lower())
1169.                     if match:
1170.                         pyramid_pct = float(match.group(1)) * 100
1171.                         action_text = f"BUY {pyramid_pct:.0f}%"
1172.                     else:
1173.                         action_text = "BUY"
1174.                         pyramid_pct = 0
1175.                         action_color = "#00ff41"
1176.                         sub_text = f"Allocate {pyramid_pct:.0f}% of capital"
1177.                 else:
1178.                     action_text = action_name.upper()
1179.                     action_color = "#888"
1180.                     sub_text = ""
1181.
1182.                 st.markdown(
1183.                     f"""
1184.                     <div style="

```

```

1185.             text-align: center;
1186.             padding: 20px;
1187.             border: 2px solid {action_color};
1188.             border-radius: 10px;
1189.             background-color: #1a1a1a;
1190.             box-shadow: 0 0 20px {action_color}33;
1191.         ">
1192.         <div style="font-size: 48px; font-weight: bold; color:
{action_color};">{action_text}</div>
1193.         <div style="font-size: 14px; color: #aaa; margin-top:
5px;">{sub_text}</div>
1194.     </div>
1195.     """,
1196.     unsafe_allow_html=True
1197. )
1198.
1199.     #DQN model selection and action button
1200.     dqn_model_dir = Path("prediction_tools/saved_models/DQN")
1201.     dqn_models = []
1202.     if dqn_model_dir.exists():
1203.         #find unique model names (looking for *_policy.pt files)
1204.         policy_files = list(dqn_model_dir.glob("*_policy.pt"))
1205.         dqn_models = sorted(set([f.stem.replace('_policy', '') for f in
policy_files]))
1206.
1207.         if dqn_models:
1208.             col1, col2 = st.columns([2, 1])
1209.             with col1:
1210.                 selected_dqn = st.selectbox(
1211.                     "DQN Model",
1212.                     dqn_models,
1213.                     key="dqn_model_select"
1214.                 )
1215.
1216.             with col2:
1217.                 st.markdown("<div style='height: 28px;'></div>",
unsafe_allow_html=True)
1218.                 if st.button("Get Action", use_container_width=True,
key="btn_dqn_action", disabled=st.session_state.run_state == "running"):
1219.                     #load model and get action
1220.                     try:
1221.                         #import DQN modules
1222.                         sys.path.insert(0, str(Path(__file__).parent.parent))
1223.                         from prediction_tools.DQN.agent import DQNAgent
1224.                         from prediction_tools.DQN.trading_env import
TradingEnvironment
1225.
1226.
1227.                         #get ticker-specific datasets
1228.                         current_ticker = st.session_state.selected_ticker
1229.                         ticker_specific = [d for d in datasets if
d.startswith(current_ticker)]
1230.
1231.                         if ticker_specific:
1232.                             #old logic is more robust (before dataset cleanup)
1233.                             #in case the process is killed and some are remenant, this
will work regardless
1234.
1235.                             latest_dataset = sorted(ticker_specific, reverse=True)[0]
1236.                             dataset_path = DATASET_DIR / latest_dataset
1237.
1238.                             st.info(f"Loading dataset: {latest_dataset} for
{current_ticker}")
1239.
1240.                             df = pd.read_csv(dataset_path)
1241.                             df["Date"] = pd.to_datetime(df["Date"])
1242.
1243.                             #load model metadata
1244.                             metadata_path = dqn_model_dir /
f"{selected_dqn}_metadata.json"

```

```

1244.         if metadata_path.exists():
1245.             with open(metadata_path, 'r') as f:
1246.                 metadata = json.load(f)
1247.
1248.                 config = metadata['config']
1249.
1250.                 target_cols = [col for col in df.columns if
col.startswith("Target") or col.startswith("Future")]
1251.                 df = df.drop(columns=target_cols)
1252.
1253.                 valid_rows = pd.Series([True] * len(df))
1254.                 for horizon in config['horizons']:
1255.                     lgbm_col = f"LGBM_{horizon}d_q50"
1256.                     if lgbm_col in df.columns:
1257.                         valid_rows &= df[lgbm_col].notna()
1258.
1259.                 df = df[valid_rows].reset_index(drop=True)
1260.
1261.                 #initialize environment with most recent data
1262.                 env = TradingEnvironment(df, config, 'cpu',
is_training=False,
1263.                                         train_normalisation_stats=(
1264.                                             np.zeros(df.shape[1]-1),
1265.                                             np.ones(df.shape[1]-1)
1266.                                         ),
1267.                                         auto_reset=False
1268.                                         )
1269.
1270.                 #override the random episode start for live trading
1271.                 #we want to observe the MOST RECENT data
1272.                 lookback = config['lookback_window']
1273.
1274.                 #mid episode
1275.                 env.current_step = config['episode_length_days'] // 2
1276.
1277.                 env.start_idx = len(df) - 1 - env.current_step
1278.                 if env.start_idx < lookback:
1279.                     raise ValueError(f"Not enough historical data.
Need at least {lookback + env.current_step} rows, have {len(df)}")
1280.
1281.                 state = env.get_state()
1282.
1283.                 #initialize agent if needed
1284.                 if (st.session_state.dqn_agent is None or
st.session_state.get('dqn_model_name') !=
1285.                     selected_dqn or
1286.                     current_ticker):
1287.
1288.                     lookback_window, num_features = state.shape
1289.                     num_actions = len(config['action_types'])
1290.
1291.                     agent = DQNAgent(num_features, lookback_window,
num_actions, config)
1292.
1293.                     #load weights
1294.                     policy_path = dqn_model_dir /
f"{selected_dqn}_policy.pt"
1295.                     target_path = dqn_model_dir /
f"{selected_dqn}_target.pt"
1296.
1297.                     agent.policy_net.load_state_dict(torch.load(policy_path, map_location='cpu'))
1298.                     agent.target_net.load_state_dict(torch.load(target_path, map_location='cpu'))
1299.
1300.                     #ensure model is on CPU and in eval mode
1301.                     agent.policy_net.to('cpu')
1302.                     agent.target_net.to('cpu')

```

```

1303.                agent.policy_net.eval()
1304.                agent.target_net.eval()
1305.
1306.                st.session_state.dqn_agent = agent
1307.                st.session_state.dqn_model_name = selected_dqn
1308.                st.session_state.dqn_ticker = current_ticker
1309.
1310.                #get action
1311.                #make sure state is on CPU
1312.                state_cpu = state.cpu() if state.is_cuda else state
1313.                action =
st.session_state.dqn_agent.select_action(state_cpu, training=False)
1314.                action_name = config['action_types'][action]
1315.
1316.
1317.                #get LGBM predictions for verification
1318.                lgbm_info = []
1319.                for h in config['horizons'][:3]: #show first 3
horizons
1320.                    col = f"LGBM_{h}d_q50"
1321.                    if col in df.columns:
1322.                        pred = df[col].iloc[-1]
1323.                        lgbm_info.append(f"{h}d: {pred:+.2f}%")
1324.
1325.                st.session_state.dqn_action = action_name
1326.                st.session_state.dqn_action_raw = action_name
1327.
1328.                st.rerun()
1329.
1330.            else:
1331.                st.error(f"Metadata not found for {selected_dqn}")
1332.        else:
1333.            st.warning(f"No dataset available for {current_ticker}.
Generate one first.")
1334.
1335.        except Exception as e:
1336.            st.error(f"Error getting DQN action: {str(e)}")
1337.            import traceback
1338.            st.code(traceback.format_exc())
1339.    else:
1340.        st.info("No DQN models found in prediction_tools/saved_models/DQN")
1341.
1342.    st.markdown("---")
1343.
1344.    #news and sentiment section
1345.    st.markdown("#### 📰 News Sentiment")
1346.
1347.    #initialize news state
1348.    if 'news_articles' not in st.session_state:
1349.        st.session_state.news_articles = []
1350.    if 'sentiment_score' not in st.session_state:
1351.        st.session_state.sentiment_score = None
1352.    if 'news_ticker' not in st.session_state:
1353.        st.session_state.news_ticker = None
1354.
1355.    #clear news if ticker changed
1356.    if st.session_state.get('news_ticker') != st.session_state.selected_ticker:
1357.        st.session_state.news_articles = []
1358.        st.session_state.sentiment_score = None
1359.        st.session_state.news_ticker = st.session_state.selected_ticker
1360.
1361.    #display sentiment score
1362.    if st.session_state.sentiment_score is None:
1363.        st.markdown(
1364.            """
1365.            <div style="
1366.                text-align: center;
1367.                padding: 20px;
1368.                border: 2px solid #444;

```

```

1369.         border-radius: 10px;
1370.         background-color: #1a1a1a;
1371.     ">
1372.         <div style="font-size: 40px; font-weight: bold; color:
#888;"></div>
1373.         <div style="font-size: 14px; color: #666; margin-top: 5px;">No
sentiment analyzed</div>
1374.     </div>
1375.     """,
1376.     unsafe_allow_html=True
1377. )
1378. else:
1379.     score = st.session_state.sentiment_score
1380.     if score > 20:
1381.         sentiment_color = "#00ff41" # Green
1382.         sentiment_label = "BULLISH"
1383.     elif score < -20:
1384.         sentiment_color = "#ff3b3b" # Red
1385.         sentiment_label = "BEARISH"
1386.     else:
1387.         sentiment_color = "#ffa500" # Orange
1388.         sentiment_label = "NEUTRAL"
1389.
1390.     st.markdown(
1391.         f"""
1392.         <div style="
1393.             text-align: center;
1394.             padding: 20px;
1395.             border: 2px solid {sentiment_color};
1396.             border-radius: 10px;
1397.             background-color: #1a1a1a;
1398.             box-shadow: 0 0 20px {sentiment_color}33;
1399.         ">
1400.             <div style="font-size: 48px; font-weight: bold; color:
{sentiment_color};">{score:+d}</div>
1401.             <div style="font-size: 14px; color: #aaa; margin-top:
5px;">{sentiment_label}</div>
1402.         </div>
1403.         """,
1404.         unsafe_allow_html=True
1405.     )
1406.
1407.     #news fetching and sentiment buttons
1408.     col1, col2 = st.columns(2)
1409.     with col1:
1410.         if st.button("📰 Fetch News", use_container_width=True,
key="btn_fetch_news", disabled=st.session_state.run_state == "running"):
1411.             st.session_state.console_logs = []
1412.             st.session_state.run_state = "running"
1413.             st.session_state.news_articles = []
1414.
1415.             ticker = st.session_state.selected_ticker
1416.             ticker_config = TICKER_CONFIGS[ticker]
1417.             scraper_config = ticker_config['scraper_config']
1418.
1419.             st.session_state.manager = multiprocessing.Manager()
1420.             st.session_state.log_queue = st.session_state.manager.Queue()
1421.             st.session_state.shared_progress = st.session_state.manager.dict()
1422.             st.session_state.shared_progress['current'] = 0
1423.             st.session_state.shared_progress['total'] = 100
1424.             st.session_state.shared_progress['state'] = "Fetching..."
1425.             st.session_state.shared_progress['articles'] = []
1426.
1427.             p = multiprocessing.Process(
1428.                 target=news_fetcher_worker,
1429.                 args=(
1430.                     scraper_config,
1431.                     st.session_state.shared_progress,
1432.                     st.session_state.log_queue

```

```

1433.         )
1434.     )
1435.     st.session_state.proc_ref = p
1436.     p.start()
1437.     st.rerun()
1438.
1439.     with col2:
1440.         if st.button("🔍 Analyse", use_container_width=True,
1441. key="btn_gen_sentiment", disabled=len(st.session_state.news_articles) == 0):
1442.             #get selected articles
1443.             selected_articles = [
1444.                 article for i, article in
1445. enumerate(st.session_state.news_articles)
1446.                 if st.session_state.get(f'article_check_{i}', False)
1447.             ]
1448.             if selected_articles:
1449.                 with st.spinner("Analyzing sentiment..."):
1450.                     ticker = st.session_state.selected_ticker
1451.                     ticker_config = TICKER_CONFIGS[ticker]
1452.                     sentiment_params = ticker_config['sentiment_params']
1453.
1454.                     analyser = SentimentAnalyser(
1455.                         bias_prompt=sentiment_params['bias_prompt'],
1456.                         temperature=sentiment_params['temperature'],
1457.                         neutrality_scaler=sentiment_params['neutrality_scaler'],
1458.                         testing=False
1459.                     )
1460.
1461.                     scores = []
1462.                     for date_str, title, _ in selected_articles:
1463.                         score = analyser.analyse_sentiment(title)
1464.                         scores.append(score)
1465.
1466.                     avg_score = np.mean(scores)
1467.                     final_score = round(avg_score * 100)
1468.
1469.                     st.session_state.sentiment_score = final_score
1470.                     st.success(f"Sentiment: {final_score:+d}")
1471.                     st.rerun()
1472.             else:
1473.                 st.warning("Please select at least one article")
1474.
1475.         #articles container
1476.         if st.session_state.news_articles:
1477.             col1, col2 = st.columns([0.7, 0.3])
1478.             with col1:
1479.                 st.markdown("**Recent Articles** (Last 30 days)")
1480.             with col2:
1481.                 if st.button("Select All", key="btn_select_all",
1482. use_container_width=True):
1483.                     for i in range(len(st.session_state.news_articles)):
1484.                         st.session_state[f'article_check_{i}'] = True
1485.                         st.rerun()
1486.
1487.             #scrollable container
1488.             with st.container(height=300, border=True):
1489.                 for i, (date_str, title, sentiment) in
1490. enumerate(st.session_state.news_articles):
1491.                     date_obj = pd.to_datetime(date_str).date()
1492.
1493.                     col1, col2 = st.columns([0.08, 0.92])
1494.                     with col1:
1495.                         st.checkbox("", key=f"article_check_{i}",
label_visibility="collapsed")
1496.                     with col2:
1497.                         st.markdown(f"**{date_obj}** - {title[:100]}{'...' if
len(title) > 100 else ''}")
1498.

```

```

1496.             if i < len(st.session_state.news_articles) - 1:
1497.                 st.divider()
1498.
1499.         st.markdown("---")
1500.
1501.         st.markdown("#### ⚙️ Dataset Configuration")
1502.
1503.         #force update configs when ticker changes
1504.         ticker = st.session_state.selected_ticker
1505.         ticker_config = TICKER_CONFIGS[ticker]
1506.
1507.         #clear sentiment config session state when ticker changes
1508.         if 'last_ticker' not in st.session_state:
1509.             st.session_state.last_ticker = ticker
1510.         elif st.session_state.last_ticker != ticker:
1511.             #ticker changed, clear sentiment config keys
1512.             keys_to_clear = [k for k in st.session_state.keys() if
k.startswith('sent_')]
1513.             for k in keys_to_clear:
1514.                 del st.session_state[k]
1515.             st.session_state.last_ticker = ticker
1516.
1517.         with st.expander("📄 Generator Config", expanded=True):
1518.             for key, default_val in DEFAULT_GENERATOR_CONFIG.items():
1519.                 if key == 'start_date':
1520.                     st.date_input(
1521.                         key.replace('_', ' ').title(),
1522.                         value=default_val,
1523.                         key=f"gen_{key}"
1524.                     )
1525.                 elif isinstance(default_val, int):
1526.                     st.number_input(
1527.                         key.replace('_', ' ').title(),
1528.                         value=default_val,
1529.                         step=1,
1530.                         key=f"gen_{key}"
1531.                     )
1532.                 elif isinstance(default_val, list):
1533.                     list_str = ",".join(str(x) for x in default_val)
1534.                     st.text_input(
1535.                         key.replace('_', ' ').title(),
1536.                         value=list_str,
1537.                         key=f"gen_{key}"
1538.                     )
1539.
1540.         with st.expander("🌐 Scraper Config"):
1541.             scraper_config = {}
1542.             for key, default_val in ticker_config['scraper_config'].items():
1543.                 if isinstance(default_val, int):
1544.                     scraper_config[key] = st.number_input(
1545.                         key.replace('_', ' ').title(),
1546.                         value=default_val,
1547.                         step=1,
1548.                         key=f"scraper_{key}"
1549.                     )
1550.                 else:
1551.                     scraper_config[key] = st.text_input(
1552.                         key.replace('_', ' ').title(),
1553.                         value=str(default_val),
1554.                         key=f"scraper_{key}"
1555.                     )
1556.
1557.         with st.expander("😊 Sentiment Config"):
1558.             sentiment_config = {}
1559.
1560.             st.markdown("**Sentiment Params**")
1561.             for key, default_val in ticker_config['sentiment_params'].items():
1562.                 if isinstance(default_val, float):
1563.                     #get current value from session state, or use default

```

```

1564.         current_val = st.session_state.get(f"sent_param_{key}",
default_val)
1565.         sentiment_config[key] = st.number_input(
1566.             key.replace('_', ' ').title(),
1567.             value=current_val,
1568.             step=0.1,
1569.             key=f"sent_param_{key}"
1570.         )
1571.     else:
1572.         current_val = st.session_state.get(f"sent_param_{key}",
str(default_val))
1573.         sentiment_config[key] = st.text_input(
1574.             key.replace('_', ' ').title(),
1575.             value=current_val,
1576.             key=f"sent_param_{key}"
1577.         )
1578.
1579.     st.markdown("----")
1580.
1581.     for key, default_val in DEFAULT_SENTIMENT_BASE.items():
1582.         if isinstance(default_val, float):
1583.             current_val = st.session_state.get(f"sent_{key}", default_val)
1584.             sentiment_config[key] = st.slider(
1585.                 key.replace('_', ' ').title(),
1586.                 0.0,
1587.                 1.0,
1588.                 current_val,
1589.                 0.05,
1590.                 key=f"sent_{key}"
1591.             )
1592.         elif isinstance(default_val, int):
1593.             current_val = st.session_state.get(f"sent_{key}", default_val)
1594.             sentiment_config[key] = st.number_input(
1595.                 key.replace('_', ' ').title(),
1596.                 value=current_val,
1597.                 step=1,
1598.                 key=f"sent_{key}"
1599.             )
1600.
1601.         list_str = ",".join(str(x) for x in ticker_config['sentiment_lags'])
1602.         current_lags = st.session_state.get("sent_lags", list_str)
1603.         input_str = st.text_input(
1604.             "Sentiment Lags",
1605.             value=current_lags,
1606.             key="sent_lags"
1607.         )
1608.         sentiment_config['lags'] = [int(x.strip()) for x in input_str.split(",")
if x.strip()]
1609.
1610.     st.markdown("----")
1611.
1612.     if st.button("■ Stop", use_container_width=True, disabled=not is_running):
1613.         if st.session_state.proc_ref and st.session_state.proc_ref.is_alive():
1614.             st.session_state.run_state = "stopping"
1615.             st.warning("Killing process...")
1616.             st.session_state.proc_ref.terminate()
1617.             st.session_state.proc_ref.join()
1618.             st.session_state.run_state = "idle"
1619.             st.session_state.proc_ref = None
1620.             st.error("Process terminated by user.")
1621.             st.rerun()
1622.
1623.     st.divider()
1624.
1625.     prog_container = st.container()
1626.
1627.     st.markdown("### 📄 LIVE CONSOLE")
1628.
1629.     if st.session_state.run_state == "running":

```

```

1630.         if st.session_state.proc_ref and not st.session_state.proc_ref.is_alive():
1631.             # Check if we have fetched articles
1632.             if 'articles' in st.session_state.shared_progress:
1633.                 articles = st.session_state.shared_progress['articles']
1634.                 if articles:
1635.                     st.session_state.news_articles = articles
1636.                     st.success(f"Fetches {len(articles)} articles!")
1637.
1638.                 st.session_state.run_state = "idle"
1639.                 st.session_state.proc_ref = None
1640.                 st.success("Task completed.")
1641.                 st.rerun()
1642.
1643.         if st.session_state.log_queue:
1644.             try:
1645.                 while not st.session_state.log_queue.empty():
1646.                     msg = st.session_state.log_queue.get_nowait()
1647.                     st.session_state.console_logs.append(msg)
1648.             except queue.Empty:
1649.                 pass
1650.
1651.         current = 0
1652.         total = 100
1653.         state_text = "Idle"
1654.
1655.         if st.session_state.shared_progress:
1656.             current = st.session_state.shared_progress.get('current', 0)
1657.             total = st.session_state.shared_progress.get('total', 100)
1658.             state_text = st.session_state.shared_progress.get('state', 'Running...')
1659.
1660.         with prog_container:
1661.             safe_total = total if total > 0 else 1
1662.             pct = min(1.0, max(0.0, current / safe_total))
1663.             st.progress(pct, text=f"{state_text} ({current}/{total})")
1664.
1665.         render_console(st.session_state.console_logs, height=400)
1666.
1667.         time.sleep(0.5)
1668.         st.rerun()
1669.     else:
1670.         render_console(st.session_state.console_logs, height=400)
1671.

```

dataset_manager.py

This script manages datasets and allows for the creation and modification of existing datasets.

```

1. import streamlit as st
2. from pathlib import Path
3. import pandas as pd
4. import plotly.graph_objects as go
5. import re
6. import sys
7. import os
8. import time
9. import multiprocessing
10. import threading
11. import queue
12. from datetime import datetime

```

Objective 8.1

This script as a whole provides an interface to manage and customise datasets

```

13.
14. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
15. if project_root not in sys.path:
16.     sys.path.insert(0, project_root)
17.
18. from training_resources.dataset_generator import DatasetGenerator
19. from training_resources.processing.indicator_calculator.indicator_calculator import
IndicatorCalculator
20. from datetime import timedelta
21. from training_resources.fetching.price_fetcher import fetch_prices
22.
23. DATASET_DIR = Path("application/datasets")
24.
25. #defaults
26. DEFAULT_GENERATOR_CONFIG = {
27.     'ticker': 'BTC-USD',
28.     'start_date': datetime(2017,1,1),
29.     'end_date': datetime(2026, 1, 1),
30.     'max_workers': 8,
31.     'pct_change_features': [3, 7, 14, 30, 90, 365],
32.     'pct_change_targets': [3,7, 14, 30, 90, 365],
33.     'return_targets': [3,7, 14, 30, 90, 365],
34.     'vol_targets': [3,7, 14, 30, 90, 365],
35.     'ema_targets': [3,7, 14, 30, 90, 365],
36.     'normalised_return_targets': [3,7,9,14, 30, 90, 365]
37. }
38.
39. DEFAULT_SCRAPER_CONFIG = {
40.     'cookies_path': "training_resources\\fetching\\scraper\\cookies.json",
41.     'target_tag': '581328',
42.     'page_size': 100,
43.     'category': "market-outlook::cryptocurrency",
44.     'min_delay': 0,
45.     'max_delay': 1,
46. }
47.
48. DEFAULT_SENTIMENT_CONFIG = {
49.     'sentiment_params': {
50.         'bias_prompt': "Bitcoin market outlook: ",
51.         'temperature': 1.6,
52.         'neutrality_scaler': 7.7,
53.     },
54.     'true_sentiment_weight': 0.7,
55.     'rolling_window': 3,
56.     'decay': 0.7,
57.     'lags': [3, 7, 14, 28]
58. }
59.
60. DEFAULT_LGBM_CONFIG = {
61.     'lgbm': {
62.         'prediction_mode': 'quantile',
63.         'quantiles': [0.1, 0.5, 0.9],
64.         'conformal': False,
65.         'lgbm_params': {
66.             'n_estimators': 250,
67.             'learning_rate': 0.008,
68.             'max_depth': 12,
69.             'num_leaves':84,
70.             'min_child_samples': 30,
71.             'subsample': 0.85,
72.             'colsample_bytree': 0.7,
73.             'reg_alpha': 0.0005,
74.             'reg_lambda': 2.0,
75.             'max_bin': 125,
76.             'boosting_type': 'gbdt',
77.             'random_state': 42,
78.             'n_jobs': -1,
79.             'verbose': -1
80.         },

```

```

81.     },
82.     'target': 'pct',
83.     'horizons': [365, 90, 30, 14, 7],
84.     'train_min': 500,
85.     'train_max': 1000,
86.     'retrain': 100,
87.     'conformal_split': 0.15,
88.     'conformal_retrain': False
89. }
90.
91. DEFAULT_CALCULATOR_CONFIG = {
92.     'rsi_length': [14],
93.     'macd': {'fast': 12, 'slow': 26, 'signal': 9},
94.     'bollinger': {'length': 20, 'std': 2},
95.     'stochastic_osc': {'k': 14, 'd': 3, 'smooth_k': 3},
96.     'atr_length': [14],
97.     'volatility_windows': [5, 10, 20, 30],
98.     'ema_lengths': [10, 14, 20, 50],
99.     'slope_windows': [5, 10, 14, 20],
100.    'psy_length': [12],
101.    'cci_length': [20],
102.    'cmo_length': [14],
103.    'gap_features': True
104. }
105.
106. INDICATOR_PREFIXES = [
107.     'RSI_', 'MACD', 'BB_', '%K', '%D', 'ATR_', 'Vol_', 'Gap',
108.     'EMA_Ratio_', 'Price_EMA', 'ATR_EMA', 'Price_Slope_',
109.     'MACD_Slope', 'RSI_Slope_Div', 'Vol_Slope_Div',
110.     'PSY', 'CCI', 'CMO'
111. ]
112.
113. #write logs to the queue so it can be read later
114. class StreamToQueue:
115.     def __init__(self, queue):
116.         self.queue = queue
117.
118.     def write(self, msg):
119.         if msg.strip(): #avoid empty newlines
120.             self.queue.put(msg)
121.
122.     def flush(self):
123.         pass
124.
125. #for loading bars
126. def progress_monitor(generator_instance, shared_dict, stop_event):
127.
128.     while not stop_event.is_set():
129.         try:
130.
131.             #check scraper first (runs inside the generator)
132.             if hasattr(generator_instance, 'scraper') and
generator_instance.scraper.progress['state'] != 'idle':
133.                 shared_dict['current'] =
generator_instance.scraper.progress.get('current', 0)
134.                 shared_dict['total'] = generator_instance.scraper.progress.get('total', 0)
135.                 shared_dict['state'] = generator_instance.scraper.progress.get('state',
'Scraping...')
136.
137.             #check generator (sentiment/features)
138.             elif hasattr(generator_instance, 'progress'):
139.                 shared_dict['current'] = generator_instance.progress.get('current', 0)
140.                 shared_dict['total'] = generator_instance.progress.get('total', 0)
141.                 shared_dict['state'] = generator_instance.progress.get('state',
'Processing...')
142.
143.             time.sleep(0.5)
144.         except Exception:
145.             break

```

```

146.
147. #multiprocess workers as seen before
148.
149. def replace_indicators_worker(dataset_path, calculator_config, new_filename, ticker,
shared_dict, log_queue):
150.     sys.stdout = StreamToQueue(log_queue)
151.     sys.stderr = StreamToQueue(log_queue)
152.
153.     print("--- Indicator Replacement Started ---")
154.
155.     try:
156.         print(f"Loading dataset: {dataset_path}")
157.         df = pd.read_csv(dataset_path)
158.         df['Date'] = pd.to_datetime(df['Date'])
159.         df = df.set_index('Date')
160.
161.         original_start = df.index.min()
162.
163.         #remove old cols
164.         indicator_cols = [col for col in df.columns
165.                             if any(col.startswith(prefix) for prefix in INDICATOR_PREFIXES)]
166.
167.         if indicator_cols:
168.             print(f"Removing {len(indicator_cols)} existing indicator columns...")
169.             df = df.drop(columns=indicator_cols)
170.         else:
171.             print("No existing indicator columns found.")
172.
173.         temp_calc = IndicatorCalculator(params=calculator_config)
174.         warmup_days = int(temp_calc.max_warmup() * 1.5) + 5
175.
176.         print(f"Warmup needed: {warmup_days} days")
177.
178.
179.         warmup_start = original_start - timedelta(days=warmup_days)
180.
181.         print(f"Fetching warmup OHLC from {warmup_start.date()} to
{original_start.date()}...")
182.         print(f"Using ticker: {ticker}")
183.
184.         warmup_ohlc = fetch_prices(
185.             ticker=ticker,
186.             start_date=warmup_start,
187.             end_date=original_start,
188.             adjust=False
189.         )
190.         warmup_ohlc.index = pd.to_datetime(warmup_ohlc.index)
191.
192.         #prepend warmup period to dataset
193.         df_with_warmup = pd.concat([warmup_ohlc, df])
194.
195.         print("Calculating indicators...")
196.         calculator = IndicatorCalculator(params=calculator_config)
197.         calculator.load_dataframe(df_with_warmup)
198.         df_with_warmup = calculator.add_all_indicators()
199.
200.         print(f"Added {len(calculator.get_indicator_columns())} indicator columns")
201.
202.         print(f"Removing warmup period, keeping data from {original_start.date()}
onwards...")
203.         df_final = df_with_warmup[df_with_warmup.index >= original_start].copy()
204.
205.         #save path logic
206.         if new_filename and new_filename.strip():
207.             if not new_filename.lower().endswith('.csv'):
208.                 new_filename += '.csv'
209.             save_path = Path(dataset_path).parent / new_filename
210.             print(f"Saving to file: {new_filename}")
211.

```

```

212.         else:
213.             save_path = dataset_path
214.             print(f"Overwriting original file: {Path(dataset_path).name}")
215.
216.             print(f"Saving updated dataset to: {save_path}")
217.             df_to_save = df_final.copy()
218.             df_to_save.insert(0, "Date", df_to_save.index)
219.             df_to_save.to_csv(save_path, index=False)
220.
221.             if new_filename and new_filename.strip():
222.                 print(f"SUCCESS: New dataset created as {Path(save_path).name}")
223.             else:
224.                 print(f"SUCCESS: Dataset {Path(save_path).name} updated with new indicators!")
225.
226.         except Exception as e:
227.             print(f"ERROR: {str(e)}")
228.             import traceback
229.             traceback.print_exc(file=sys.stdout)
230.         finally:
231.             print("--- Indicator Regeneration Finished ---")
232.
233.
234. #overriding lgbm in a dataset
235. def replace_lgbm_worker(dataset_path, lgbm_config, new_filename, shared_dict, log_queue):
236.     #redirect stdout
237.     sys.stdout = StreamToQueue(log_queue)
238.     sys.stderr = StreamToQueue(log_queue)
239.
240.     print("--- LGBM Replacement Started ---")
241.
242.     try:
243.         print(f"Loading dataset: {dataset_path}")
244.         df = pd.read_csv(dataset_path)
245.
246.         #remove existing LGBM columns
247.         lgbm_cols = [col for col in df.columns if col.startswith('LGBM_')]
248.         if lgbm_cols:
249.             print(f"Removing {len(lgbm_cols)} existing LGBM columns...")
250.             df = df.drop(columns=lgbm_cols)
251.         else:
252.             print("No existing LGBM columns found.")
253.
254.         #determine save path
255.         if new_filename and new_filename.strip():
256.             # Save as new file
257.             if not new_filename.lower().endswith('.csv'):
258.                 new_filename += '.csv'
259.             save_path = Path(dataset_path).parent / new_filename
260.             print(f"Will save as new file: {new_filename}")
261.         else:
262.             #overwrite original
263.             save_path = dataset_path
264.             print(f"Will overwrite original file: {Path(dataset_path).name}")
265.
266.         #extract date range from dataset
267.         df['Date'] = pd.to_datetime(df['Date'])
268.         df = df.set_index('Date')
269.
270.         #initialize a minimal DatasetGenerator instance just to use add_lgbm_predictions
271.         print("Preparing to add LGBM predictions...")
272.         from training_resources.dataset_generator import DatasetGenerator
273.
274.         #create instance without calling init to avoid long loading times for dataset
generator
275.         generator = reuse_dataset_lgbm(df)
276.
277.         #start progress monitor
278.         stop_event = threading.Event()
279.         monitor_thread = threading.Thread(

```

```

280.         target=progress_monitor,
281.         args=(generator, shared_dict, stop_event),
282.         daemon=True
283.     )
284.     monitor_thread.start()
285.
286.     #add new LGBM predictions
287.     print("Adding new LGBM predictions...")
288.     DatasetGenerator.add_lgbm_predictions(generator, config=lgbm_config)
289.
290.     #save using DatasetGenerator's save_dataset method
291.     print(f"Saving updated dataset to: {save_path}")
292.     generator.save_dataset(save_path)
293.
294.     if new_filename and new_filename.strip():
295.         print(f"SUCCESS: New dataset created as {Path(save_path).name}")
296.     else:
297.         print(f"SUCCESS: Dataset {Path(save_path).name} updated with new LGBM
predictions!")
298.
299.     except Exception as e:
300.         print(f"ERROR: {str(e)}")
301.         import traceback
302.         traceback.print_exc(file=sys.stdout)
303.     finally:
304.         stop_event.set()
305.         print("---- LGBM Replacement Finished ----")
306.
307. def dataset_worker(gen_config, scraper_config, sentiment_config, calculator_config,
lgbm_config, use_lgbm, shared_dict, log_queue, dataset_dir):
308.
309.     #redirect stdout
310.     sys.stdout = StreamToQueue(log_queue)
311.     sys.stderr = StreamToQueue(log_queue)
312.
313.     print("---- Process Started ----")
314.
315.     try:
316.         #initialize generator
317.         print("Initializing DatasetGenerator...")
318.         generator = DatasetGenerator(
319.             generator_config=gen_config,
320.             scraper_config=scraper_config,
321.             sentiment_calculations=sentiment_config,
322.             calculator_params=calculator_config
323.         )
324.
325.         #start progress monitor thread
326.         stop_event = threading.Event()
327.         monitor_thread = threading.Thread(
328.             target=progress_monitor,
329.             args=(generator, shared_dict, stop_event),
330.             daemon=True
331.         )
332.         monitor_thread.start()
333.
334.         #run Logic
335.         print("Generating dataset...")
336.         dataset = generator.generate_dataset()
337.
338.         if use_lgbm:
339.             print("Adding LGBM predictions...")
340.             generator.add_lgbm_predictions(config=lgbm_config)
341.
342.         #save Logic
343.         custom_name = gen_config.get('dataset_name_input', '').strip()
344.
345.         if custom_name:
346.             if not custom_name.lower().endswith('.csv'):

```

```

347.         custom_name += '.csv'
348.         filename = custom_name
349.     else:
350.         #fallback to default naming
351.         ticker = gen_config['ticker']
352.         start = gen_config['start_date'].strftime('%Y%m%d')
353.         end = gen_config['end_date'].strftime('%Y%m%d')
354.         filename = f"{ticker}_{start}_{end}.csv"
355.
356.         #ensure directory is valid
357.         save_dir = Path(str(dataset_dir))
358.         save_dir.mkdir(parents=True, exist_ok=True)
359.
360.         save_path = save_dir / filename
361.
362.         print(f"Saving dataset to: {save_path}")
363.         generator.save_dataset(save_path)
364.         print(f"SUCCESS: Dataset saved as {filename}")
365.
366.     except Exception as e:
367.         print(f"ERROR: {str(e)}")
368.         import traceback
369.         traceback.print_exc(file=sys.stdout)
370.     finally:
371.         stop_event.set()
372.         print("--- Process Finished ---")
373.
374. #css for console
375. def render_console(log_lines, height=300):
376.     log_text = "\n".join(log_lines[-100:]) # keep last 100 lines
377.     st.markdown(
378.         f"""
379.         <div style="
380.             background-color:#0e1117;
381.             color:#e5e7eb;
382.             padding:10px;
383.             border-radius:5px;
384.             height:{height}px;
385.             overflow-y:scroll;
386.             font-family: monospace;
387.             white-space: pre-wrap;
388.         ">
389.             {log_text}
390.         </div>
391.         """,
392.         unsafe_allow_html=True
393.     )
394.
395. #instancing a lighter dataset generator that does not load up the sentiment model, making
it much faster
396. #only one function is really needed and i didnt want to rewrite logic
397. def reuse_dataset_lgbm(df):
398.     generator = object.__new__(DatasetGenerator)
399.     generator.full_dataset = df
400.     generator.progress = {'current': 0, 'total': 0, 'state': 'idle'}
401.     return generator
402.
403. def render():
404.     st.markdown("### 📁 Dataset Manager")
405.     st.divider()
406.
407.     #state init
408.     if 'run_state' not in st.session_state:
409.         st.session_state.run_state = "idle"
410.     if 'console_logs' not in st.session_state:
411.         st.session_state.console_logs = []
412.     if 'proc_ref' not in st.session_state:
413.         st.session_state.proc_ref = None
414.     if 'shared_progress' not in st.session_state:

```

```

415.     st.session_state.shared_progress = None
416.     if 'log_queue' not in st.session_state:
417.         st.session_state.log_queue = None
418.     if 'manager' not in st.session_state:
419.         st.session_state.manager = None
420.
421.     #get list of datasets
422.     datasets = (
423.         [f.name for f in DATASET_DIR.glob("*.csv")]
424.         if DATASET_DIR.exists()
425.         else []
426.     )
427.
428.     left, right = st.columns([4, 1.7])
429.
430.     with left:
431.         with st.container():
432.             st.markdown("#### 📊 Price Data")
433.             selected_dataset = st.selectbox(
434.                 "Select Dataset",
435.                 datasets,
436.                 disabled=not datasets
437.             )
438.
439.             if selected_dataset:
440.                 try:
441.                     df = pd.read_csv(DATASET_DIR / selected_dataset)
442.                     df["Date"] = pd.to_datetime(df["Date"])
443.
444.                     #candlestick chart
445.                     fig = go.Figure(data=[go.Candlestick(
446.                         x=df["Date"],
447.                         open=df["Open"],
448.                         high=df["High"],
449.                         low=df["Low"],
450.                         close=df["Close"],
451.                         increasing_line_color="#00ff41",
452.                         decreasing_line_color="#ff3b3b",
453.                     )])
454.                     fig.update_layout(
455.                         title="Price", xaxis_title="Date", yaxis_title="Price",
456.                         height=450, margin=dict(l=10, r=10, t=40, b=10),
457.                         dragmode="zoom", plot_bgcolor="#0e1117", paper_bgcolor="#0e1117",
458.                         font=dict(color="#e5e7eb"), xaxis_rangeslider_visible=False,
459.                     )
460.                     st.plotly_chart(fig, use_container_width=True)
461.
462.                     # LGBM Predictions
463.                     st.markdown("#### 📈 LightGBM Predictions")
464.                     lgbm_cols = [c for c in df.columns if c.startswith("LGBM_")]
465.
466.                     if lgbm_cols:
467.                         horizons = sorted([
468.                             int(re.search(r"LGBM_(\d+)d", c).group(1))
469.                             for c in lgbm_cols if re.search(r"LGBM_(\d+)d", c)
470.                         ])
471.                         selected_h = st.selectbox("Prediction Horizon", horizons,
472. key="lgbm_horizon")
473.
474.                         quantiles = sorted([
475.                             float(re.search(r"q(\d+)", c).group(1))
476.                             for c in lgbm_cols
477.                             if f"LGBM_{selected_h}d_q" in c and "q50" not in c
478.                         ])
479.                         quantiles = [50] + quantiles
480.
481.                         fig_pred = go.Figure()
482.                         #actual
target_col = f"Future_Pct_Change_{selected_h}d"

```

```

483.         if target_col in df.columns:
484.             fig_pred.add_trace(go.Scatter(
485.                 x=df["Date"], y=df[target_col],
486.                 mode="lines", name="Actual", line=dict(color="#00ff41",
width=2)
487.             ))
488.
489.         #predictions
490.         colors = ["#facc15", "#f87171", "#3b82f6", "#10b981", "#8b5cf6"]
491.         for i, q in enumerate(quantiles):
492.             col_name = f"LGBM_{selected_h}d_q{int(q)}"
493.             if col_name in df.columns:
494.                 fig_pred.add_trace(go.Scatter(
495.                     x=df["Date"], y=df[col_name],
496.                     mode="lines", name=f"q{int(q)}",
497.                     line=dict(color=colors[i % len(colors)], dash="dash")
498.                 ))
499.
500.         fig_pred.update_layout(
501.             title=f"LGBM Predictions ({selected_h}d Horizon)",
502.             height=300, margin=dict(l=10, r=10, t=30, b=10),
503.             plot_bgcolor="#0e1117", paper_bgcolor="#0e1117",
504.             font=dict(color="#e5e7eb", dragmode="zoom",
505.         )
506.         st.plotly_chart(fig_pred, use_container_width=True)
507.     else:
508.         st.info("No LGBM predictions available.")
509.     except Exception as e:
510.         st.error(f"Failed to load dataset: {e}")
511. else:
512.     st.info("No datasets available. Upload one to see charts.")
513.
514. with right:
515.     with st.container():
516.         st.markdown("#### 🧠 Generator")
517.
518.         #expanders for each dict
519.         #need to validate each datatype possible if i make changes to dicts
520.         with st.expander("📄 Generator Config", expanded=True):
521.             gen_config = {}
522.
523.             st.markdown("***Output Filename**")
524.             gen_config['dataset_name_input'] = st.text_input(
525.                 "Dataset Name",
526.                 placeholder="enter name (without .csv)",
527.                 label_visibility="collapsed"
528.             )
529.             st.markdown("---")
530.
531.             for key, default_val in DEFAULT_GENERATOR_CONFIG.items():
532.                 if isinstance(default_val, datetime):
533.                     gen_config[key] = st.date_input(key.replace('_', ' ').title(),
value=default_val, key=f"gen_{key}")
534.                 elif isinstance(default_val, bool):
535.                     gen_config[key] = st.checkbox(key.replace('_', ' ').title(),
value=default_val, key=f"gen_{key}")
536.                 elif isinstance(default_val, int):
537.                     gen_config[key] = st.number_input(key.replace('_', ' ').title(),
value=default_val, step=1, key=f"gen_{key}")
538.                 elif isinstance(default_val, float):
539.                     gen_config[key] = st.number_input(key.replace('_', ' ').title(),
value=default_val, step=0.1, key=f"gen_{key}")
540.                 elif isinstance(default_val, list):
541.                     list_str = ",".join(str(x) for x in default_val)
542.                     input_str = st.text_input(key.replace('_', ' ').title(),
value=list_str, key=f"gen_{key}")
543.                     if len(default_val) > 0 and isinstance(default_val[0], int):
544.                         gen_config[key] = [int(x.strip()) for x in input_str.split(",")
if x.strip()]

```

```

545.         elif len(default_val) > 0 and isinstance(default_val[0], float):
546.             gen_config[key] = [float(x.strip()) for x in
input_str.split(",") if x.strip()]
547.         else:
548.             gen_config[key] = input_str.split(",")
549.         else:
550.             gen_config[key] = st.text_input(key.replace('_', ' ').title(),
value=str(default_val), key=f"gen_{key}")
551.
552.         with st.expander("🌐 Scraper Config"):
553.             scraper_config = {}
554.             for key, default_val in DEFAULT_SCRAPER_CONFIG.items():
555.                 if isinstance(default_val, bool):
556.                     scraper_config[key] = st.checkbox(key.replace('_', ' ').title(),
value=default_val, key=f"scraper_{key}")
557.                 elif isinstance(default_val, int):
558.                     scraper_config[key] = st.number_input(key.replace('_', '
').title(), value=default_val, step=1, key=f"scraper_{key}")
559.                 elif isinstance(default_val, float):
560.                     scraper_config[key] = st.number_input(key.replace('_', '
').title(), value=default_val, step=0.1, key=f"scraper_{key}")
561.                 else:
562.                     scraper_config[key] = st.text_input(key.replace('_', ' ').title(),
value=str(default_val), key=f"scraper_{key}")
563.
564.         with st.expander("😊 Sentiment Config"):
565.             sentiment_config = {}
566.             for key, default_val in DEFAULT_SENTIMENT_CONFIG.items():
567.                 if isinstance(default_val, dict):
568.                     st.markdown(f"***{key.replace('_', ' ').title()}**")
569.                     sentiment_config[key] = {}
570.                     for sub_key, sub_val in default_val.items():
571.                         if isinstance(sub_val, bool):
572.                             sentiment_config[key][sub_key] =
st.checkbox(sub_key.replace('_', ' ').title(), value=sub_val, key=f"sent_{key}_{sub_key}")
573.                         elif isinstance(sub_val, float):
574.                             sentiment_config[key][sub_key] =
st.number_input(sub_key.replace('_', ' ').title(), value=sub_val, step=0.1,
key=f"sent_{key}_{sub_key}")
575.                         else:
576.                             sentiment_config[key][sub_key] =
st.text_input(sub_key.replace('_', ' ').title(), value=str(sub_val),
key=f"sent_{key}_{sub_key}")
577.                 elif isinstance(default_val, float):
578.                     sentiment_config[key] = st.slider(key.replace('_', ' ').title(),
0.0, 1.0, default_val, 0.05, key=f"sent_{key}")
579.                 elif isinstance(default_val, int):
580.                     sentiment_config[key] = st.number_input(key.replace('_', '
').title(), value=default_val, step=1, key=f"sent_{key}")
581.                 elif isinstance(default_val, list):
582.                     list_str = ",".join(str(x) for x in default_val)
583.                     input_str = st.text_input(key.replace('_', ' ').title(),
value=list_str, key=f"sent_{key}")
584.                     sentiment_config[key] = [int(x.strip()) for x in
input_str.split(",") if x.strip()]
585.
586.         with st.expander("📊 Indicator Calculator Config"):
587.             calculator_config = {}
588.
589.             for key, default_val in DEFAULT_CALCULATOR_CONFIG.items():
590.                 if isinstance(default_val, dict):
591.                     # Add checkbox to enable/disable this indicator group
592.                     enabled = st.checkbox(
593.                         f"Enable {key.replace('_', ' ').title()}",
594.                         value=True,
595.                         key=f"calc_{key}_enabled"
596.                     )
597.
598.                     if enabled:

```

```

599.                 # Nested expander for dict parameters
600.                 with st.expander(f"🔍 {key.replace('_', ' ').title()}
Parameters", expanded=False):
601.                     calculator_config[key] = {}
602.                     for sub_key, sub_val in default_val.items():
603.                         if isinstance(sub_val, int):
604.                             calculator_config[key][sub_key] = st.number_input(
605.                                 sub_key.replace('_', ' ').title(),
606.                                 value=sub_val,
607.                                 step=1,
608.                                 key=f"calc_{key}_{sub_key}"
609.                             )
610.                         elif isinstance(sub_val, float):
611.                             calculator_config[key][sub_key] = st.number_input(
612.                                 sub_key.replace('_', ' ').title(),
613.                                 value=sub_val,
614.                                 step=0.1,
615.                                 key=f"calc_{key}_{sub_key}"
616.                             )
617.                 # If not enabled, don't add it to calculator_config
618.
619.                 elif isinstance(default_val, list):
620.                     list_str = ",".join(str(x) for x in default_val)
621.                     input_str = st.text_input(
622.                         key.replace('_', ' ').title(),
623.                         value=list_str,
624.                         key=f"calc_{key}",
625.                         help="Comma-separated values. Clear to disable."
626.                     )
627.                     if len(default_val) > 0 and isinstance(default_val[0], int):
628.                         calculator_config[key] = [int(x.strip()) for x in
input_str.split(",") if x.strip()]
629.                     else:
630.                         calculator_config[key] = [float(x.strip()) for x in
input_str.split(",") if x.strip()]
631.                 elif isinstance(default_val, bool):
632.                     calculator_config[key] = st.checkbox(
633.                         key.replace('_', ' ').title(),
634.                         value=default_val,
635.                         key=f"calc_{key}"
636.                     )
637.
638.                 use_lgbm = st.checkbox("Add LGBM Predictions", value=True)
639.                 lgbm_config = {}
640.                 if use_lgbm:
641.                     with st.expander("🔍 LGBM Config"):
642.                         for key, default_val in DEFAULT_LGBM_CONFIG.items():
643.                             if isinstance(default_val, dict) and key == 'lgbm':
644.                                 lgbm_config[key] = {}
645.                                 for sub_key, sub_val in default_val.items():
646.                                     if isinstance(sub_val, dict):
647.                                         st.markdown(f"***{sub_key.replace('_', ' ').title()}**")
648.                                         lgbm_config[key][sub_key] = {}
649.                                         for param_key, param_val in sub_val.items():
650.                                             if isinstance(param_val, bool):
651.                                                 lgbm_config[key][sub_key][param_key] =
st.checkbox(param_key.replace('_', ' ').title(), value=param_val,
key=f"lgbm_{sub_key}_{param_key}")
652.                                             elif isinstance(param_val, int):
653.                                                 lgbm_config[key][sub_key][param_key] =
st.number_input(param_key.replace('_', ' ').title(), value=param_val, step=1,
key=f"lgbm_{sub_key}_{param_key}")
654.                                             elif isinstance(param_val, float):
655.                                                 if param_val < 0.01:
656.                                                     lgbm_config[key][sub_key][param_key] =
st.number_input(param_key.replace('_', ' ').title(), value=param_val, step=0.0001,
format="%.4f", key=f"lgbm_{sub_key}_{param_key}")
657.                                                 else:

```

```

658.                                     lgbm_config[key][sub_key][param_key] =
st.number_input(param_key.replace('_', ' ').title(), value=param_val, step=0.1,
key=f"lgbm_{sub_key}_{param_key}")
659.                                     else:
660.                                     lgbm_config[key][sub_key][param_key] =
st.text_input(param_key.replace('_', ' ').title(), value=str(param_val),
key=f"lgbm_{sub_key}_{param_key}")
661.                                     elif isinstance(sub_val, list):
662.                                     list_str = ",".join(str(x) for x in sub_val)
663.                                     input_str = st.text_input(sub_key.replace('_', '
').title(), value=list_str, key=f"lgbm_{sub_key}")
664.                                     if len(sub_val) > 0 and isinstance(sub_val[0], float):
665.                                     lgbm_config[key][sub_key] = [float(x.strip()) for x
in input_str.split(",") if x.strip()]
666.                                     else:
667.                                     lgbm_config[key][sub_key] = input_str.split(",")
668.                                     elif isinstance(sub_val, bool):
669.                                     lgbm_config[key][sub_key] =
st.checkbox(sub_key.replace('_', ' ').title(), value=sub_val, key=f"lgbm_{sub_key}")
670.                                     else:
671.                                     lgbm_config[key][sub_key] =
st.text_input(sub_key.replace('_', ' ').title(), value=str(sub_val), key=f"lgbm_{sub_key}")
672.                                     elif isinstance(default_val, list):
673.                                     list_str = ",".join(str(x) for x in default_val)
674.                                     input_str = st.text_input(key.replace('_', ' ').title(),
value=list_str, key=f"lgbm_{key}")
675.                                     if len(default_val) > 0 and isinstance(default_val[0], int):
676.                                     lgbm_config[key] = [int(x.strip()) for x in
input_str.split(",") if x.strip()]
677.                                     else:
678.                                     lgbm_config[key] = input_str.split(",")
679.                                     elif isinstance(default_val, float):
680.                                     lgbm_config[key] = st.slider(key.replace('_', ' ').title(),
0.0, 1.0, default_val, 0.05, key=f"lgbm_{key}")
681.                                     elif isinstance(default_val, bool):
682.                                     lgbm_config[key] = st.checkbox(key.replace('_', ' ').title(),
value=default_val, key=f"lgbm_{key}")
683.                                     elif isinstance(default_val, int):
684.                                     lgbm_config[key] = st.number_input(key.replace('_', '
').title(), value=default_val, step=1, key=f"lgbm_{key}")
685.                                     else:
686.                                     lgbm_config[key] = st.text_input(key.replace('_', ' ').title(),
value=str(default_val), key=f"lgbm_{key}")
687.
688.                                     #buttons
689.                                     st.markdown("----")
690.
691.                                     is_running = st.session_state.run_state == "running"
692.                                     is_stopping = st.session_state.run_state == "stopping"
693.
694.                                     if st.button("▶ Run Generator", use_container_width=True, disabled=is_running
or is_stopping):
695.                                     st.session_state.console_logs = []
696.                                     st.session_state.run_state = "running"
697.
698.                                     #pre-processing config
699.                                     gen_config['start_date'] = pd.to_datetime(gen_config['start_date'])
700.                                     gen_config['end_date'] = pd.to_datetime(gen_config['end_date'])
701.                                     if gen_config.get('warmup_days', 0) == 0:
702.                                     gen_config.pop('warmup_days', None)
703.
704.                                     #create manager and process
705.                                     st.session_state.manager = multiprocessing.Manager()
706.                                     st.session_state.log_queue = st.session_state.manager.Queue()
707.                                     st.session_state.shared_progress = st.session_state.manager.dict()
708.                                     st.session_state.shared_progress['current'] = 0
709.                                     st.session_state.shared_progress['total'] = 100
710.                                     st.session_state.shared_progress['state'] = "Initializing..."
711.

```

```

712.         p = multiprocessing.Process(
713.             target=dataset_worker,
714.             args=(
715.                 gen_config,
716.                 scraper_config,
717.                 sentiment_config,
718.                 calculator_config,
719.                 lgbm_config,
720.                 use_lgbm,
721.                 st.session_state.shared_progress,
722.                 st.session_state.log_queue,
723.                 str(DATASET_DIR)
724.             )
725.         )
726.         st.session_state.proc_ref = p
727.         p.start()
728.         st.rerun()
729.
730.         if st.button("🔄 Replace Indicators", use_container_width=True,
disabled=is_running or is_stopping or not selected_dataset):
731.             st.session_state.console_logs = []
732.             st.session_state.run_state = "running"
733.
734.             # Create manager and process
735.             st.session_state.manager = multiprocessing.Manager()
736.             st.session_state.log_queue = st.session_state.manager.Queue()
737.             st.session_state.shared_progress = st.session_state.manager.dict()
738.             st.session_state.shared_progress['current'] = 0
739.             st.session_state.shared_progress['total'] = 100
740.             st.session_state.shared_progress['state'] = "Removing old indicators..."
741.
742.             dataset_path = str(DATASET_DIR / selected_dataset)
743.             new_filename = gen_config.get('dataset_name_input', '').strip()
744.             ticker = gen_config.get('ticker', 'BTC-USD')
745.             p = multiprocessing.Process(
746.                 target=replace_indicators_worker,
747.                 args=(
748.                     dataset_path,
749.                     calculator_config,
750.                     new_filename,
751.                     ticker,
752.                     st.session_state.shared_progress,
753.                     st.session_state.log_queue
754.                 )
755.             )
756.             st.session_state.proc_ref = p
757.             p.start()
758.             st.rerun()
759.             #replace LGBM button
760.             if st.button("🔄 Replace LGBM Predictions", use_container_width=True,
disabled=is_running or is_stopping or not selected_dataset):
761.                 st.session_state.console_logs = []
762.                 st.session_state.run_state = "running"
763.
764.                 #create manager and process
765.                 st.session_state.manager = multiprocessing.Manager()
766.                 st.session_state.log_queue = st.session_state.manager.Queue()
767.                 st.session_state.shared_progress = st.session_state.manager.dict()
768.                 st.session_state.shared_progress['current'] = 0
769.                 st.session_state.shared_progress['total'] = 100
770.                 st.session_state.shared_progress['state'] = "Removing old predictions..."
771.
772.                 dataset_path = str(DATASET_DIR / selected_dataset)
773.                 new_filename = gen_config.get('dataset_name_input', '').strip()
774.
775.                 p = multiprocessing.Process(
776.                     target=replace_lgbm_worker,
777.                     args=(
778.                         dataset_path,

```

```

779.             lgbm_config,
780.             new_filename,
781.             st.session_state.shared_progress,
782.             st.session_state.log_queue
783.         )
784.     )
785.     st.session_state.proc_ref = p
786.     p.start()
787.     st.rerun()
788.
789.     if st.button("■ Stop", use_container_width=True, disabled=not is_running):
790.         if st.session_state.proc_ref and st.session_state.proc_ref.is_alive():
791.             st.session_state.run_state = "stopping"
792.             st.warning("Killing process...")
793.             st.session_state.proc_ref.terminate() # Hard kill
794.             st.session_state.proc_ref.join()
795.             st.session_state.run_state = "idle"
796.             st.session_state.proc_ref = None
797.             st.error("Process terminated by user.")
798.             st.rerun()
799.
800.     st.divider()
801.
802.     prog_container = st.container()
803.
804.     st.markdown("### 📄 LIVE CONSOLE")
805.
806.
807.     if st.session_state.run_state == "running":
808.
809.         #check if process is still alive
810.         if st.session_state.proc_ref and not st.session_state.proc_ref.is_alive():
811.             st.session_state.run_state = "idle"
812.             st.session_state.proc_ref = None
813.             st.success("Task completed.")
814.             st.rerun()
815.
816.         #read from queue
817.         if st.session_state.log_queue:
818.             try:
819.                 while not st.session_state.log_queue.empty():
820.                     msg = st.session_state.log_queue.get_nowait()
821.                     st.session_state.console_logs.append(msg)
822.             except queue.Empty:
823.                 pass
824.
825.         #read from progress dict
826.         current = 0
827.         total = 100
828.         state_text = "Idle"
829.
830.         if st.session_state.shared_progress:
831.             current = st.session_state.shared_progress.get('current', 0)
832.             total = st.session_state.shared_progress.get('total', 100)
833.             state_text = st.session_state.shared_progress.get('state', 'Running...')
834.
835.         #render progress
836.         with prog_container:
837.             safe_total = total if total > 0 else 1
838.             pct = min(1.0, max(0.0, current / safe_total))
839.             st.progress(pct, text=f"{state_text} ({current}/{total})")
840.
841.         #render console
842.         render_console(st.session_state.console_logs, height=400)
843.
844.         time.sleep(0.5)
845.         st.rerun()
846.     else:
847.         render_console(st.session_state.console_logs, height=400)

```

training.py

This manages the training of new DQN agents as well as LightGBM models.

Objective 8.3

This script as a whole provides tools for training LightGBM and DQN models

```

1. import streamlit as st
2. from pathlib import Path
3. import pandas as pd
4. import plotly.graph_objects as go
5. import numpy as np
6. import os
7. import sys
8. import multiprocessing
9. import threading
10. import queue
11. import time
12. import pickle
13. from datetime import datetime
14.
15.
16. #add project root to path
17. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
18. if project_root not in sys.path:
19.     sys.path.insert(0, project_root)
20.
21. from prediction_tools.DQN.trainer import DEFAULT_CONFIG as DQN_DEFAULT_CONFIG, DQNTrainer,
discover_features
22. from prediction_tools.light_gbm import LGBMForecaster, DEFAULT_PARAMS as
LGBM_DEFAULT_PARAMS
23.
24.
25. DATASET_DIR = Path("application/datasets")
26. DQN_MODEL_DIR = Path("prediction_tools/saved_models/DQN")
27. LGBM_MODEL_DIR = Path("prediction_tools/saved_models/LGBM")
28.
29.
30.
31. def get_available_dqn_models():
32.     if not DQN_MODEL_DIR.exists():
33.         return []
34.
35.     models = set()
36.     for file in DQN_MODEL_DIR.glob("*_metadata.json"):
37.         base_name = file.stem.replace("_metadata", "")
38.         models.add(base_name)
39.
40.     return sorted(list(models))
41.
42. #redirect output
43. class StreamToQueue:
44.     def __init__(self, queue):
45.         self.queue = queue
46.
47.     def write(self, msg):
48.         if msg.strip():
49.             self.queue.put(msg)
50.
51.     def flush(self):
52.         pass
53.
54.

```

```

55. #worker functions
56.
57. def lgbm_testing_worker(config, shared_dict, log_queue):
58.     sys.stdout = StreamToQueue(log_queue)
59.     sys.stderr = StreamToQueue(log_queue)
60.
61.     try:
62.         print("="*60)
63.         print("LGBM TESTING STARTED")
64.         print("="*60)
65.
66.         #extract config
67.         model_files = config['model_files']
68.         dataset_path = config['dataset_path']
69.         test_split = config['test_split']
70.         test_from_start = config.get('test_from_start', False)
71.         model_base_name = config.get('model_base_name', 'unknown')
72.
73.         #load flip configuration if it exists
74.         config_dir = Path("prediction_tools/saved_models/LGBM/correlations")
75.         config_file = config_dir / f"{model_base_name}_flip_config.csv"
76.         flip_settings = {}
77.         if config_file.exists():
78.             flip_df = pd.read_csv(config_file)
79.             flip_settings = dict(zip(flip_df['horizon'], flip_df['flip']))
80.             print(f"Loaded flip configuration: {flip_settings}")
81.
82.         #load dataset
83.         print(f"\nLoading dataset: {dataset_path}")
84.         df = pd.read_csv(dataset_path)
85.         df['Date'] = pd.to_datetime(df['Date'])
86.
87.         #split dataset
88.         split_idx = int(len(df) * test_split)
89.
90.         if test_from_start:
91.             test_df = df.iloc[:split_idx].copy()
92.             print(f"Testing on FIRST {test_split*100:.0f}% of data")
93.         else:
94.             split_idx = int(len(df) * (1 - test_split))
95.             test_df = df.iloc[split_idx:].copy()
96.             print(f"Testing on LAST {test_split*100:.0f}% of data (OOS)")
97.
98.         print(f"Test set: {len(test_df)} rows ({test_df['Date'].min().date()} to
{test_df['Date'].max().date()})")
99.
100.        #store results
101.        all_results = {}
102.
103.        shared_dict['total'] = len(model_files)
104.        shared_dict['current'] = 0
105.
106.        for i, model_file in enumerate(model_files):
107.            print(f"\n{'='*60}")
108.            print(f"TESTING MODEL: {Path(model_file).name} ({i+1}/{len(model_files)})")
109.            print(f"{'='*60}")
110.
111.            shared_dict['current'] = i
112.            shared_dict['state'] = f"Testing {Path(model_file).name}..."
113.
114.            #load model
115.            with open(model_file, 'rb') as f:
116.                model_data = pickle.load(f)
117.
118.            forecaster = model_data['forecaster']
119.            horizon = model_data['horizon']
120.            target_type = model_data['target_type']
121.            target_col = model_data['target_col']
122.            feature_cols = model_data['feature_cols']

```

```

123.         quantiles = forecaster.config['quantiles']
124.
125.         print(f"Horizon: {horizon}d")
126.         print(f"Target: {target_col}")
127.         print(f"Features: {len(feature_cols)}")
128.
129.         #prepare test data
130.         test_data = test_df[test_df[target_col].notna()].copy()
131.         X_test = test_data[feature_cols]
132.         y_test = test_data[target_col]
133.
134.         print(f"Test samples: {len(X_test)}")
135.
136.         if len(X_test) == 0:
137.             print("WARNING: No test data available")
138.             continue
139.
140.         #get actual returns for directional accuracy evaluation
141.         actual_returns = []
142.         test_dates_list = []
143.         for idx in test_data.index:
144.             try:
145.                 current_idx = test_df.index.get_loc(idx)
146.                 future_idx = min(current_idx + horizon, len(test_df) - 1)
147.
148.                 current_price = test_df.loc[test_df.index[current_idx], 'Close']
149.                 future_price = test_df.loc[test_df.index[future_idx], 'Close']
150.
151.                 actual_return = (future_price - current_price) / current_price * 100
152.                 actual_returns.append(actual_return)
153.                 test_dates_list.append(test_df.loc[test_df.index[current_idx],
'Date'])
154.             except:
155.                 actual_returns.append(np.nan)
156.                 test_dates_list.append(test_data.loc[idx, 'Date'])
157.
158.         actual_returns = np.array(actual_returns)
159.
160.         #predict
161.         print("Running predictions...")
162.         predictions = forecaster.predict(X_test)
163.
164.         #apply flipping if configured
165.         should_flip = flip_settings.get(horizon, False)
166.         if should_flip:
167.             print(f"Flipping predictions for {horizon}d horizon (reverse
correlation)")
168.             #flip the prediction values (multiply by -1)
169.             flipped_predictions = {}
170.             #also reverse the quantile mapping and round it
171.             for q in quantiles:
172.                 opposite_q = round(1.0 - q, 2)
173.                 flipped_predictions[opposite_q] = -predictions[q]
174.             predictions = flipped_predictions
175.             quantiles = sorted(flipped_predictions.keys())
176.
177.         #calculate metrics using ACTUAL RETURNS for directional accuracy
178.         q_med = min(quantiles, key=lambda q: abs(q - 0.5))
179.         median_preds = predictions[q_med]
180.
181.         pred_direction = median_preds > 0
182.         actual_direction = actual_returns > 0
183.
184.         valid_rows = ~np.isnan(actual_returns)
185.         directional_acc = (pred_direction[valid_rows] ==
actual_direction[valid_rows]).mean()
186.
187.         coverage = {}
188.         for q in quantiles:

```

```

189.         cov = (y_test.values <= predictions[q]).mean()
190.         coverage[q] = cov
191.
192.         #compare Future_Pct_Change for plotting
193.         future_pct_col = f'Future_Pct_Change_{horizon}d'
194.         if future_pct_col in test_df.columns:
195.             plot_actual = test_data[future_pct_col].values if future_pct_col in
test_data.columns else y_test.values
196.             print(f"Plotting: {future_pct_col} (actual future percentage changes)")
197.         else:
198.             plot_actual = y_test.values
199.             print(f"ERROR: {future_pct_col} not found in dataset, using {target_col}
for plotting")
200.
201.             print(f"\n{'\-'*60}")
202.             print(f"RESULTS - {horizon}d Horizon")
203.             print(f{'\-'*60})
204.             print(f"Directional Accuracy (vs actual returns): {directional_acc*100:.2f}%")
205.             print(f"\nQuantile Coverage (vs {target_type} target):")
206.             for q in quantiles:
207.                 ideal = q
208.                 actual = coverage[q]
209.                 diff = actual - ideal
210.                 status = "✓" if abs(diff) < 0.05 else "△"
211.                 print(f"   q{int(round(q*100)):02d}: {actual:.2%} (ideal {ideal:.0%})
{status} [{diff:+.2%}]")
212.
213.             #store results for visualization (plot Future_Pct_Change, not the target)
214.             all_results[horizon] = {
215.                 'dates': test_data['Date'].values,
216.                 'actual': plot_actual,
217.                 'predictions': predictions,
218.                 'directional_acc': directional_acc,
219.                 'coverage': coverage,
220.                 'quantiles': quantiles
221.             }
222.
223.             shared_dict['current'] = len(model_files)
224.             shared_dict['state'] = 'completed'
225.             shared_dict['results'] = dict(all_results)
226.
227.             print(f"\n{'\-'*60}")
228.             print("TESTING COMPLETE!")
229.             print(f{'\-'*60})
230.
231.     except Exception as e:
232.         print(f"ERROR: {str(e)}")
233.         import traceback
234.         traceback.print_exc()
235.         shared_dict['state'] = 'error'
236.
237.
238. def lgbm_training_worker(config, shared_dict, log_queue):
239.
240.
241.     sys.stdout = StreamToQueue(log_queue)
242.     sys.stderr = StreamToQueue(log_queue)
243.
244.     try:
245.         print("="*60)
246.         print("LGBM TRAINING STARTED")
247.         print("="*60)
248.
249.         #extract config
250.         dataset_path = config['dataset_path']
251.         horizons = config['horizons']
252.         target_type = config['target_type']
253.         train_split = config['train_split']
254.         quantiles = config['quantiles']

```

Objective 8.3

Function to train LightGBM models

```

255.     lgbm_params = config['lgbm_params']
256.     model_name = config['model_name']
257.     use_conformal = config.get('use_conformal', False)
258.     conformal_split = config.get('conformal_split', 0.15)
259.     conformal_retrain = config.get('conformal_retrain', False)
260.     use_cascading = config.get('use_cascading', False)
261.     cascade_order = config.get('cascade_order', 'ascending')
262.
263.     #load flip configuration if it exists
264.     config_dir = Path("prediction_tools/saved_models/LGBM/correlations")
265.     config_file = config_dir / f"{model_name}_flip_config.csv"
266.     flip_settings = {}
267.     if config_file.exists():
268.         flip_df = pd.read_csv(config_file)
269.         flip_settings = dict(zip(flip_df['horizon'], flip_df['flip']))
270.         print(f"Loaded flip configuration: {flip_settings}")
271.
272.     print(f"\nLoading dataset: {dataset_path}")
273.     df = pd.read_csv(dataset_path)
274.     df['Date'] = pd.to_datetime(df['Date'])
275.
276.     split_idx = int(len(df) * train_split)
277.     train_df = df.iloc[:split_idx].copy()
278.     test_df = df.iloc[split_idx:].copy()
279.
280.     print(f"Train set: {len(train_df)} rows ({train_df['Date'].min().date()} to
{train_df['Date'].max().date()}")
281.     print(f"Test set: {len(test_df)} rows ({test_df['Date'].min().date()} to
{test_df['Date'].max().date()}")
282.     print(f"Cascading predictions: {'ENABLED' if use_cascading else 'DISABLED'}")
283.
284.     #store results for each horizon
285.     all_results = {}
286.
287.     #tack which LGBM columns created in THIS run (for cascades)
288.     created_lgbm_columns = set()
289.
290.     #sort horizons based on cascade order
291.     if cascade_order == 'ascending':
292.         sorted_horizons = sorted(horizons)
293.         print("Training order: shortest to longest horizons")
294.     else: #descending
295.         sorted_horizons = sorted(horizons, reverse=True)
296.         print("Training order: longest to shortest horizons")
297.
298.     shared_dict['total'] = len(sorted_horizons)
299.     shared_dict['current'] = 0
300.
301.     for i, horizon in enumerate(sorted_horizons):
302.         print(f"\n{'='*60}")
303.         print(f"TRAINING HORIZON: {horizon}d ({i+1}/{len(sorted_horizons)})")
304.         print(f"{'='*60}")
305.
306.         shared_dict['current'] = i
307.         shared_dict['state'] = f"Training {horizon}d horizon..."
308.
309.         #determine target column
310.         if target_type == 'pct':
311.             target_col = f'Future_Pct_Change_{horizon}d'
312.         elif target_type == 'ema':
313.             target_col = f'Target_EMA_Position_{horizon}d'
314.         elif target_type == 'vol':
315.             target_col = f'Target_VolAdjusted_{horizon}d'
316.         elif target_type == 'returns':
317.             target_col = f'Future_Return_{horizon}d'
318.         elif target_type == 'norm':
319.             target_col = f'Target_Normalised_Return_{horizon}d'
320.         else:
321.             print(f"ERROR: Unknown target type: {target_type}")

```

```

322.         continue
323.
324.     if target_col not in df.columns:
325.         print(f"ERROR: Target column {target_col} not found in dataset")
326.         continue
327.
328.     #get feature columns
329.     if use_cascading:
330.         feature_cols = []
331.         for col in train_df.columns:
332.             if col.startswith('Target') or col.startswith('Future'):
333.                 continue
334.             if col in ['Date', 'is_trading_day']:
335.                 continue
336.             if col.startswith('LGBM_'):
337.                 #only include LGBM columns created in THIS training run
338.                 if col not in created_lgbm_columns:
339.                     continue
340.                 try:
341.                     col_horizon = int(col.split('_')[1].replace('d', ''))
342.                     #include based on cascade order
343.                     if cascade_order == 'ascending':
344.                         #only include horizons smaller than current
345.                         if col_horizon < horizon:
346.                             feature_cols.append(col)
347.                     else: #descending
348.                         #only include horizons larger than current
349.                         if col_horizon > horizon:
350.                             feature_cols.append(col)
351.                 except:
352.                     continue
353.             else:
354.                 feature_cols.append(col)
355.
356.         if cascade_order == 'ascending':
357.             print(f"Cascading mode (ascending): Including LGBM predictions from
horizons < {horizon}d")
358.         else:
359.             print(f"Cascading mode (descending): Including LGBM predictions from
horizons > {horizon}d")
360.
361.         lgbm_features = [c for c in feature_cols if c.startswith('LGBM_')]
362.         if lgbm_features:
363.             print(f" Using {len(lgbm_features)} LGBM prediction features")
364.     else:
365.         #exclude ALL LGBM columns
366.         feature_cols = [col for col in train_df.columns
367.                         if not col.startswith('Target')
368.                         and not col.startswith('Future')
369.                         and not col.startswith('LGBM_')
370.                         and col not in ['Date', 'is_trading_day']]
371.         print(f"Non-cascading mode: Excluding all LGBM predictions")
372.
373.     print(f"Features: {len(feature_cols)}")
374.     print(f"Target: {target_col}")
375.
376.     #prepare training data
377.     train_data = train_df[train_df[target_col].notna()].copy()
378.     X_train = train_data[feature_cols]
379.     y_train = train_data[target_col]
380.
381.     print(f"Training samples: {len(X_train)}")
382.
383.     if len(X_train) < 100:
384.         print(f"ERROR: Insufficient training data ({len(X_train)} samples)")
385.         continue
386.
387.     #initialize forecaster
388.     forecaster_config = {

```

```

389.         'prediction_mode': 'quantile',
390.         'quantiles': quantiles,
391.         'lgbm_params': lgbm_params,
392.         'conformal': use_conformal,
393.         'conformal_split': conformal_split,
394.         'conformal_retrain': conformal_retrain
395.     }
396.
397.     forecaster = LGBMForecaster(config=forecaster_config)
398.
399.     #train
400.     print("Training model...")
401.     if use_conformal:
402.         print(f"Using conformal prediction (split={conformal_split},
retrain={conformal_retrain})")
403.         forecaster.fit(X_train, y_train)
404.         print("✓ Training complete")
405.
406.     #generate predictions on FULL dataset for cascading
407.     if use_cascading:
408.         print("Generating predictions for cascading...")
409.         #get valid rows (those with all required features)
410.         valid_rows = df[feature_cols].notna().all(axis=1)
411.         X_full = df.loc[valid_rows, feature_cols]
412.
413.         #predict
414.         predictions_full = forecaster.predict(X_full)
415.
416.         #apply flipping if configured
417.         should_flip = flip_settings.get(horizon, False)
418.         if should_flip:
419.             print(f"Flipping predictions for {horizon}d horizon (reverse
correlation)")
420.             flipped_predictions = {}
421.
422.             quantile_mapping = {}
423.             for q in quantiles:
424.                 #round because floats act silly sometimes
425.                 opposite_q = round(1.0 - q, 2)
426.                 flipped_predictions[opposite_q] = -predictions_full[q]
427.                 quantile_mapping[opposite_q] = opposite_q
428.             predictions_full = flipped_predictions
429.             quantiles_sorted = sorted(flipped_predictions.keys())
430.         else:
431.             quantiles_sorted = quantiles
432.
433.         #create prediction arrays aligned with the full dataframe
434.         valid_indices = df[valid_rows].index
435.
436.         for q in quantiles_sorted:
437.             col_name = f'LGBM_{horizon}d_q{int(round(q*100))}'
438.
439.             if col_name not in df.columns:
440.                 df[col_name] = np.nan
441.             if col_name not in train_df.columns:
442.                 train_df[col_name] = np.nan
443.             if col_name not in test_df.columns:
444.                 test_df[col_name] = np.nan
445.
446.             pred_series = pd.Series(predictions_full[q], index=valid_indices)
447.
448.             df.loc[valid_indices, col_name] = pred_series
449.
450.             #split predictions between train and test based on split_idx
451.             train_mask = valid_indices < split_idx
452.             test_mask = valid_indices >= split_idx
453.
454.             train_indices = valid_indices[train_mask]
455.             if len(train_indices) > 0:

```

```

456.         train_df.loc[train_indices, col_name] =
pred_series.loc[train_indices]
457.
458.         test_indices = valid_indices[test_mask]
459.         if len(test_indices) > 0:
460.             test_df.loc[test_indices, col_name] =
pred_series.loc[test_indices]
461.
462.             created_lgbm_columns.add(col_name)
463.
464.             print(f"✓ Added LGBM_{horizon}d predictions to dataset for cascading")
465.
466.             #OOS test
467.             print("\nTesting on holdout set...")
468.             test_data = test_df[test_df[target_col].notna()].copy()
469.             X_test = test_data[feature_cols]
470.             y_test = test_data[target_col]
471.
472.             print(f"Test samples: {len(X_test)}")
473.
474.             if len(X_test) == 0:
475.                 print("WARNING: No test data available")
476.                 continue
477.
478.             #get actual returns for directional accuracy
479.             actual_returns = []
480.             for idx in test_data.index:
481.                 try:
482.                     current_idx = test_df.index.get_loc(idx)
483.                     future_idx = min(current_idx + horizon, len(test_df) - 1)
484.
485.                     current_price = test_df.loc[test_df.index[current_idx], 'Close']
486.                     future_price = test_df.loc[test_df.index[future_idx], 'Close']
487.
488.                     actual_return = (future_price - current_price) / current_price * 100
489.                     actual_returns.append(actual_return)
490.                 except:
491.                     actual_returns.append(np.nan)
492.
493.             actual_returns = np.array(actual_returns)
494.
495.             #predict on test set
496.             predictions = forecaster.predict(X_test)
497.
498.             #apply flipping if configured
499.             should_flip = flip_settings.get(horizon, False)
500.             if should_flip:
501.                 print(f"Flipping test predictions for {horizon}d horizon")
502.                 flipped_predictions = {}
503.                 for q in quantiles:
504.                     #round to avoid floating point issues
505.                     opposite_q = round(1.0 - q, 2)
506.                     flipped_predictions[opposite_q] = -predictions[q]
507.                 predictions = flipped_predictions
508.                 quantiles = sorted(flipped_predictions.keys())
509.
510.             #calculate metrics
511.             q_med = min(quantiles, key=lambda q: abs(q - 0.5))
512.             median_preds = predictions[q_med]
513.
514.             pred_direction = median_preds > 0
515.             actual_direction = actual_returns > 0
516.             valid_rows = ~np.isnan(actual_returns)
517.             directional_acc = (pred_direction[valid_rows] ==
actual_direction[valid_rows]).mean()
518.
519.             coverage = {}
520.             for q in quantiles:
521.                 cov = (y_test.values <= predictions[q]).mean()

```

```

522.         coverage[q] = cov
523.
524.         #get Future_Pct_Change for plotting
525.         future_pct_col = f'Future_Pct_Change_{horizon}d'
526.         if future_pct_col in test_df.columns:
527.             plot_actual = test_data[future_pct_col].values
528.         else:
529.             plot_actual = y_test.values
530.             print(f"WARNING: {future_pct_col} not found, using {target_col} for
plotting")
531.
532.         print(f"\n{'-'*60}")
533.         print(f"RESULTS - {horizon}d Horizon")
534.         print(f"\n{'-'*60}")
535.         print(f"Directional Accuracy (vs actual returns): {directional_acc*100:.2f}%")
536.         print(f"\nQuantile Coverage (vs {target_type} target):")
537.         for q in quantiles:
538.             ideal = q
539.             actual = coverage[q]
540.             diff = actual - ideal
541.             status = "✓" if abs(diff) < 0.05 else "⚠"
542.             print(f"    q{int(round(q*100)):02d}: {actual:.2%} (ideal {ideal:.0%})
{status} [{diff:+.2%}]")
543.
544.         #save model
545.         os.makedirs(LGBM_MODEL_DIR, exist_ok=True)
546.         model_filename = f"{model_name}_{horizon}d.pkl"
547.         model_path = LGBM_MODEL_DIR / model_filename
548.
549.         with open(model_path, 'wb') as f:
550.             pickle.dump({
551.                 'forecaster': forecaster,
552.                 'config': forecaster_config,
553.                 'horizon': horizon,
554.                 'target_type': target_type,
555.                 'feature_cols': feature_cols,
556.                 'target_col': target_col,
557.                 'timestamp': datetime.now().strftime('%Y%m%d_%H%M%S'),
558.                 'train_samples': len(X_train),
559.                 'test_accuracy': directional_acc,
560.                 'test_coverage': coverage,
561.                 'use_cascading': use_cascading,
562.                 'cascade_order': cascade_order
563.             }, f)
564.
565.         print(f"\n✓ Model saved: {model_filename}")
566.
567.         #store results for visualization
568.         all_results[horizon] = {
569.             'dates': test_data['Date'].values,
570.             'actual': plot_actual,
571.             'predictions': predictions,
572.             'directional_acc': directional_acc,
573.             'coverage': coverage,
574.             'quantiles': quantiles
575.         }
576.
577.         shared_dict['current'] = len(sorted_horizons)
578.         shared_dict['state'] = 'completed'
579.         shared_dict['results'] = dict(all_results)
580.
581.         print(f"\n{'\n'}{'-'*60}")
582.         print("TRAINING COMPLETE!")
583.         print(f"\n{'\n'}{'-'*60}")
584.         print(f"Models saved to: {LGBM_MODEL_DIR}")
585.
586.     except Exception as e:
587.         print(f"ERROR: {str(e)}")
588.         import traceback

```

Objective 8.5

Displays directional
accuracy for LightGBM

Objective 8.3

Function to train DQN
models

```

589.         traceback.print_exc()
590.         shared_dict['state'] = 'error'
591.
592.
593. def training_worker(config, shared_dict, log_queue, use_pretrained=False):
594.
595.     sys.stdout = StreamToQueue(log_queue)
596.     sys.stderr = StreamToQueue(log_queue)
597.
598.     try:
599.         print("Initializing trainer...")
600.         trainer = DQNTrainer(config)
601.
602.         #start progress monitor
603.         def monitor_progress():
604.             while True:
605.                 try:
606.                     shared_dict['current'] = trainer.progress.get('current', 0)
607.                     shared_dict['total'] = trainer.progress.get('total', 0)
608.                     shared_dict['state'] = trainer.progress.get('state', 'idle')
609.                     shared_dict['metadata'] = dict(trainer.progress.get('metadata', {}))
610.                     shared_dict['history'] = dict(trainer.progress.get('history', {}))
611.                     time.sleep(0.5)
612.                 except:
613.                     break
614.
615.         monitor_thread = threading.Thread(target=monitor_progress, daemon=True)
616.         monitor_thread.start()
617.
618.         agent, rewards, returns, test_stats =
trainer.train_dqn(use_pretrained=use_pretrained)
619.
620.         print("\n" + "="*60)
621.         print("TRAINING COMPLETE!")
622.         print("="*60)
623.         print(f"Best episode return: {max(returns)*100:.2f}%")
624.         print(f"Final test win rate: {test_stats['win_rate']*100:.1f}%")
625.         print(f"Test profit factor: {test_stats['profit_factor']:.2f}")
626.         print("="*60)
627.
628.         shared_dict['state'] = 'completed'
629.         shared_dict['test_stats'] = test_stats
630.
631.     except Exception as e:
632.         print(f"ERROR: {str(e)}")
633.         import traceback
634.         traceback.print_exc()
635.         shared_dict['state'] = 'error'
636.
637.
638. def render_console_style(content, height=300):
639.     st.markdown(
640.         f"""
641.         <div style="
642.             background-color:#0e1117;
643.             color:#00FF41;
644.             padding:15px;
645.             border-radius:5px;
646.             height:{height}px;
647.             overflow-y:scroll;
648.             font-family: monospace;
649.             white-space: pre-wrap;
650.             border: 1px solid #00FF41;
651.         ">
652.             {content}
653.         """,
654.         unsafe_allow_html=True
655.     )
656.

```

Objective 8.5

Performance metrics for
final trained DQN

```

657.
658.
659.
660. def render():
661.     st.markdown("### 📖 Training")
662.     st.divider()
663.
664.     #get list of datasets
665.     datasets = (
666.         [f.name for f in DATASET_DIR.glob("*.csv")]
667.         if DATASET_DIR.exists()
668.         else []
669.     )
670.
671.     #initialize session state
672.     if 'training_active' not in st.session_state:
673.         st.session_state.training_active = False
674.     if 'training_completed' not in st.session_state:
675.         st.session_state.training_completed = False
676.     if 'training_process' not in st.session_state:
677.         st.session_state.training_process = None
678.     if 'training_logs' not in st.session_state:
679.         st.session_state.training_logs = []
680.     if 'training_progress' not in st.session_state:
681.         st.session_state.training_progress = None
682.     if 'log_queue' not in st.session_state:
683.         st.session_state.log_queue = None
684.     if 'manager' not in st.session_state:
685.         st.session_state.manager = None
686.
687.     left, right = st.columns([4, 1.7])
688.
689.     with right:
690.         with st.container():
691.             st.markdown("#### ⚙️ Settings")
692.
693.             #training type selector
694.             training_type = st.selectbox(
695.                 "Training Type",
696.                 ["DQN", "LightGBM"],
697.                 disabled=False
698.             )
699.
700.             st.markdown("---")
701.
702.             # LGBM Settings
703.             if training_type == "LightGBM":
704.                 st.markdown("***Model Mode**")
705.                 lgbm_mode = st.radio(
706.                     "Mode",
707.                     ["Train New Model", "Test Existing Model"],
708.                     help="Train a new model or test an existing one",
709.                     key="lgbm_mode"
710.                 )
711.
712.                 st.markdown("---")
713.
714.                 if lgbm_mode == "Test Existing Model":
715.                     #model selector
716.                     if not LGBM_MODEL_DIR.exists():
717.                         st.warning("No LGBM models found. Train a model first.")
718.                         st.stop()
719.
720.                     model_files = list(LGBM_MODEL_DIR.glob("*.pkl"))
721.                     if not model_files:
722.                         st.warning("No LGBM models found. Train a model first.")
723.                         st.stop()
724.
725.                     #group by base name

```

```

726.         models_by_name = {}
727.     for file in model_files:
728.         #extract base name and horizon
729.         name = file.stem
730.         if '_' in name and name.split('_')[-1].endswith('d'):
731.             base_name = '_'.join(name.split('_')[:-1])
732.             horizon = name.split('_')[-1]
733.         else:
734.             base_name = name
735.             horizon = "unknown"
736.
737.         if base_name not in models_by_name:
738.             models_by_name[base_name] = []
739.         models_by_name[base_name].append((horizon, file))
740.
741.     st.markdown("**Model Selection**")
742.     selected_model_name = st.selectbox(
743.         "Select Model Group",
744.         sorted(models_by_name.keys()),
745.         key="lgbm_test_model",
746.         help="All horizons for this model will be tested"
747.     )
748.
749.     st.markdown("**Dataset**")
750.     selected_dataset = st.selectbox("Dataset", datasets,
key="lgbm_test_dataset")
751.
752.     if not selected_dataset:
753.         st.info("Select a dataset to test on.")
754.         st.stop()
755.
756.     st.markdown("**Test Configuration**")
757.
758.     test_split = st.slider(
759.         "Test Split",
760.         min_value=0.05,
761.         max_value=0.5,
762.         value=0.2,
763.         step=0.05,
764.         help="Proportion of data to use for testing"
765.     )
766.
767.     test_from_start = st.checkbox(
768.         "Test from start",
769.         value=False,
770.         help="If checked, tests on FIRST test_split% of data. Otherwise
tests on LAST test_split% (recommended for OOS testing)"
771.     )
772.
773.     if test_from_start:
774.         st.info(f"Will test on the **first {test_split*100:.0f}%** of the
dataset ({len(models_by_name[selected_model_name])} horizons)")
775.     else:
776.         st.info(f"Will test on the **last {test_split*100:.0f}%** of the
dataset ({len(models_by_name[selected_model_name])} horizons)")
777.
778.     st.markdown("---")
779.
780.     if st.button("▶ Run Test", width='stretch', key="lgbm_test_button"):
781.         #get all model files for this base name
782.         model_files_to_test = [str(path) for _, path in
models_by_name[selected_model_name]]
783.
784.         #store model name for flip configuration
785.         st.session_state.lgbm_model_name = selected_model_name
786.
787.         test_config = {
788.             'model_files': model_files_to_test,
789.             'dataset_path': str(DATASET_DIR / selected_dataset),

```

```

790.             'test_split': test_split,
791.             'test_from_start': test_from_start,
792.             'model_base_name': selected_model_name
793.         }
794.
795.         st.session_state.training_logs = []
796.         st.session_state.manager = multiprocessing.Manager()
797.         st.session_state.log_queue = st.session_state.manager.Queue()
798.         st.session_state.training_progress =
st.session_state.manager.dict()
799.         st.session_state.training_progress['current'] = 0
800.         st.session_state.training_progress['total'] =
len(model_files_to_test)
801.         st.session_state.training_progress['state'] = 'Testing...'
802.         st.session_state.training_progress['results'] = {}
803.
804.         p = multiprocessing.Process(
805.             target=lgbm_testing_worker,
806.             args=(test_config, st.session_state.training_progress,
st.session_state.log_queue)
807.         )
808.         st.session_state.training_process = p
809.         st.session_state.training_active = True
810.         st.session_state.training_type = 'lgbm'
811.         p.start()
812.         st.rerun()
813.
814.     else: #train New Model
815.         st.markdown("***Model Configuration**")
816.
817.         lgbm_model_name = st.text_input(
818.             "Model Name",
819.             value="LGBM_model",
820.             help="Base name for saved models (horizon will be appended)"
821.         )
822.
823.         st.markdown("***Dataset**")
824.         selected_dataset = st.selectbox("Dataset", datasets,
key="lgbm_dataset")
825.
826.         if not selected_dataset:
827.             st.info("Select a dataset to begin.")
828.             st.stop()
829.
830.         st.markdown("***Training Parameters**")
831.
832.         horizons_str = st.text_input(
833.             "Horizons (comma-separated)",
834.             value="7,14,30,90,365",
835.             help="Prediction horizons in days"
836.         )
837.         horizons = [int(x.strip()) for x in horizons_str.split(",") if
x.strip()]
838.
839.         target_type = st.selectbox(
840.             "Target Type",
841.             ["pct", "ema", "vol", "returns", "norm"],
842.             help="Type of target to predict"
843.         )
844.
845.         train_split = st.slider(
846.             "Train/Test Split",
847.             min_value=0.5,
848.             max_value=0.95,
849.             value=0.8,
850.             step=0.05,
851.             help="Proportion of data to use for training"
852.         )
853.

```

```

854.         quantiles_str = st.text_input(
855.             "Quantiles (comma-separated)",
856.             value="0.1,0.5,0.9",
857.             help="Quantiles to predict"
858.         )
859.         quantiles = [float(x.strip()) for x in quantiles_str.split(",") if
x.strip()]
860.
861.
862.         use_cascading = st.checkbox(
863.             "Enable Cascading Predictions",
864.             value=False,
865.             help="Allow models to use predictions from shorter horizons as
features (trains in ascending order)"
866.         )
867.
868.         st.info(
869.             "☒ Cascading trains models in ascending order (7d, 14d, 30d...)
so longer horizons can use shorter horizon predictions as features."
870.             if use_cascading else
871.             "☐ Non-cascading mode: Each model trains independently without
seeing other LGBM predictions."
872.         )
873.
874.         if use_cascading:
875.             cascade_order = st.radio(
876.                 "Cascade Order",
877.                 ["ascending", "descending"],
878.                 index=0,
879.                 help="Ascending: short>long (use smaller horizons).
Descending: long>short (use larger horizons)"
880.             )
881.
882.             if cascade_order == "ascending":
883.                 st.info("▲ Ascending: Trains 7d>14d>30d... Each model uses
predictions from shorter horizons.")
884.             else:
885.                 st.info("▼ Descending: Trains 365d>90d>30d... Each model uses
predictions from longer horizons.")
886.             else:
887.                 cascade_order = 'ascending' #default when not using cascading
888.
889.         st.markdown("***Conformal Prediction***")
890.
891.         use_conformal = st.checkbox(
892.             "Enable Conformal Predictions",
893.             value=False,
894.             help="Use conformal prediction for calibrated prediction
intervals"
895.         )
896.
897.         if use_conformal:
898.             conformal_split = st.slider(
899.                 "Conformal Split",
900.                 min_value=0.05,
901.                 max_value=0.3,
902.                 value=0.15,
903.                 step=0.05,
904.                 help="Proportion of training data to use for conformal
calibration"
905.             )
906.
907.             conformal_retrain = st.checkbox(
908.                 "Retrain on Full Data",
909.                 value=False,
910.                 help="After calibration, retrain model on full training set
(including conformal calibration data)"
911.             )
912.             else:

```

```

913.         conformal_split = 0.15
914.         conformal_retrain = False
915.
916.     with st.expander("🔗 LGBM Parameters", expanded=False):
917.         lgbm_params = {}
918.         for key, default_val in LGBM_DEFAULT_PARAMS.items():
919.             if key in ['random_state', 'n_jobs', 'verbose']:
920.                 lgbm_params[key] = default_val
921.                 continue
922.
923.             if isinstance(default_val, int):
924.                 lgbm_params[key] = st.number_input(
925.                     key.replace('_', ' ').title(),
926.                     value=default_val,
927.                     step=1,
928.                     key=f"lgbm_param_{key}"
929.                 )
930.             elif isinstance(default_val, float):
931.                 if default_val < 0.01:
932.                     lgbm_params[key] = st.number_input(
933.                         key.replace('_', ' ').title(),
934.                         value=default_val,
935.                         step=0.0001,
936.                         format="%.4f",
937.                         key=f"lgbm_param_{key}"
938.                     )
939.                 else:
940.                     lgbm_params[key] = st.number_input(
941.                         key.replace('_', ' ').title(),
942.                         value=default_val,
943.                         step=0.01,
944.                         key=f"lgbm_param_{key}"
945.                     )
946.
947.     st.markdown("----")
948.
949.     #start training button
950.     if st.button("▶ Start Training", width='stretch',
key="lgbm_start_training"):
951.         st.session_state.training_completed = False
952.         lgbm_config = {
953.             'mode': 'train',
954.             'dataset_path': str(DATASET_DIR / selected_dataset),
955.             'horizons': horizons,
956.             'target_type': target_type,
957.             'train_split': train_split,
958.             'quantiles': quantiles,
959.             'lgbm_params': lgbm_params,
960.             'model_name': lgbm_model_name,
961.             'use_conformal': use_conformal,
962.             'conformal_split': conformal_split,
963.             'conformal_retrain': conformal_retrain,
964.             'use_cascading': use_cascading,
965.             'cascade_order': cascade_order
966.         }
967.
968.         #store model name for flip configuration
969.         st.session_state.lgbm_model_name = lgbm_model_name
970.
971.         st.session_state.training_logs = []
972.         st.session_state.manager = multiprocessing.Manager()
973.         st.session_state.log_queue = st.session_state.manager.Queue()
974.         st.session_state.training_progress =
st.session_state.manager.dict()
975.         st.session_state.training_progress['current'] = 0
976.         st.session_state.training_progress['total'] = len(horizons)
977.         st.session_state.training_progress['state'] = 'Initializing...'
978.         st.session_state.training_progress['results'] = {}
979.

```

```

980.                 p = multiprocessing.Process(
981.                     target=lgbm_training_worker,
982.                     args=(lgbm_config, st.session_state.training_progress,
st.session_state.log_queue)
983.                 )
984.                 st.session_state.training_process = p
985.                 st.session_state.training_active = True
986.                 st.session_state.training_type = 'lgbm'
987.                 p.start()
988.                 st.rerun()
989.
990.         elif training_type == "DQN":
991.             model_mode = st.radio(
992.                 "Model Mode",
993.                 ["Train New Model", "Continue Training Existing"],
994.                 help="Train a fresh model or load weights from an existing one"
995.             )
996.
997.             use_pretrained = (model_mode == "Continue Training Existing")
998.
999.             if use_pretrained:
1000.                 available_models = get_available_dqn_models()
1001.                 if not available_models:
1002.                     st.warning("No existing models found. Will train fresh model.")
1003.                     use_pretrained = False
1004.                 else:
1005.                     selected_model = st.selectbox("Select Model to Continue",
available_models)
1006.
1007.             st.markdown("---")
1008.
1009.             #configuration
1010.             with st.expander("⚙️ Training Configuration", expanded=True):
1011.                 config = {}
1012.
1013.                 if use_pretrained:
1014.                     config['load_model'] = selected_model
1015.                     st.info(f"📁 Loading weights from: **{selected_model}**")
1016.
1017.                 config['model_name'] = st.text_input(
1018.                     "Model Name (Save As)",
1019.                     value=DQN_DEFAULT_CONFIG['model_name'],
1020.                     help="Name for saving the trained model"
1021.                 )
1022.
1023.                 if use_pretrained:
1024.                     st.success(f"💾 Will save trained model as:
**{config['model_name']}**")
1025.                 else:
1026.                     config['load_model'] = config['model_name']
1027.
1028.                 st.markdown("***Dataset**")
1029.                 selected_dataset = st.selectbox("Dataset", datasets,
key="dqn_dataset")
1030.
1031.                 if selected_dataset:
1032.                     config['csv_path'] = str(DATASET_DIR / selected_dataset)
1033.
1034.                 st.markdown("***Key Parameters**")
1035.                 config['episodes'] = st.number_input("Episodes",
value=DQN_DEFAULT_CONFIG['episodes'], step=100)
1036.                 config['learning_rate'] = st.number_input("Learning Rate",
value=DQN_DEFAULT_CONFIG['learning_rate'], step=0.0001, format="%.4f")
1037.                 config['batch_size'] = st.number_input("Batch Size",
value=DQN_DEFAULT_CONFIG['batch_size'], step=16)
1038.                 config['gamma'] = st.number_input("Gamma (Discount)",
value=DQN_DEFAULT_CONFIG['gamma'], step=0.01, format="%.3f")
1039.
1040.                 with st.expander("🔗 Advanced Configuration", expanded=False):
1041.                     for key, default_val in DQN_DEFAULT_CONFIG.items():

```

```

1041.             #skips
1042.             if key in ['model_name', 'csv_path', 'episodes', 'learning_rate',
1043.                         'batch_size', 'gamma']:
1044.                 continue
1045.
1046.             if key == 'load_model':
1047.                 continue
1048.
1049.             if 'path' in key.lower():
1050.                 config[key] = default_val
1051.                 continue
1052.
1053.             if key == 'device':
1054.                 config[key] = default_val
1055.                 continue
1056.
1057.             if key == 'confidence_horizons':
1058.                 if isinstance(default_val, list):
1059.                     list_str = ",".join(str(x) for x in default_val)
1060.                 else:
1061.                     list_str = str(default_val)
1062.
1063.                 input_str = st.text_input(
1064.                     "Confidence Horizons",
1065.                     value=list_str,
1066.                     key=f"train_{key}",
1067.                     help='Enter "all", "short", "long", or comma-separated
1068.                         integers (e.g. "7,365")'
1069.                 )
1070.
1071.                 #parse as string or list of ints
1072.                 input_str = input_str.strip()
1073.                 if input_str in ['all', 'short', 'long']:
1074.                     config[key] = input_str
1075.                 else:
1076.                     #try to parse as list of integers
1077.                     try:
1078.                         config[key] = [int(x.strip()) for x in
1079.                                         input_str.split(",") if x.strip()]
1080.                     except:
1081.                         st.warning(f"Invalid confidence_horizons format. Using
1082.                             default: {default_val}")
1083.                         config[key] = default_val
1084.                 continue
1085.
1086.             #render based on type
1087.             if isinstance(default_val, bool):
1088.                 config[key] = st.checkbox(key.replace('_', ' ').title(),
1089.                                         value=default_val, key=f"train_{key}")
1090.             elif isinstance(default_val, int):
1091.                 config[key] = st.number_input(key.replace('_', ' ').title(),
1092.                                         value=default_val, step=1, key=f"train_{key}")
1093.             elif isinstance(default_val, float):
1094.                 if default_val < 0.01:
1095.                     config[key] = st.number_input(key.replace('_', ' ').title(),
1096.                                         value=default_val, step=0.0001, format="%.4f", key=f"train_{key}")
1097.                 else:
1098.                     config[key] = st.number_input(key.replace('_', ' ').title(),
1099.                                         value=default_val, step=0.1, key=f"train_{key}")
1100.             elif isinstance(default_val, list):
1101.                 list_str = ",".join(str(x) for x in default_val)
1102.                 input_str = st.text_input(key.replace('_', ' ').title(),
1103.                                         value=list_str, key=f"train_{key}")
1104.                 if len(default_val) > 0 and isinstance(default_val[0], int):
1105.                     config[key] = [int(x.strip()) for x in
1106.                                     input_str.split(",") if x.strip()]
1107.                 elif len(default_val) > 0 and isinstance(default_val[0],
1108.                                                             float):

```

```

1098.                 config[key] = [float(x.strip()) for x in
input_str.split(",") if x.strip()]
1099.                 else:
1100.                 config[key] = [x.strip() for x in input_str.split(",") if
x.strip()]
1101.                 elif isinstance(default_val, str):
1102.                 config[key] = st.text_input(key.replace('_', ' ').title(),
value=default_val, key=f"train_{key}")
1103.
1104.                 with st.expander("🔍 Feature Selection", expanded=False):
1105.                 if selected_dataset:
1106.                 # Load dataset to discover features
1107.                 dataset_path = DATASET_DIR / selected_dataset
1108.                 df_features = pd.read_csv(dataset_path)
1109.                 available_features = sorted(discover_features(df_features,
{ }).keys())
1110.
1111.                 st.markdown(f"***{len(available_features)} features detected**")
1112.
1113.                 # Select/Deselect all buttons
1114.                 col1, col2 = st.columns(2)
1115.                 with col1:
1116.                 if st.button("✅ Enable All", key="enable_all_features"):
1117.                 for feat in available_features:
1118.                 st.session_state[f'feature_{feat}'] = True
1119.                 st.rerun()
1120.                 with col2:
1121.                 if st.button("❌ Disable All", key="disable_all_features"):
1122.                 for feat in available_features:
1123.                 st.session_state[f'feature_{feat}'] = False
1124.                 st.rerun()
1125.
1126.                 st.markdown("----")
1127.
1128.                 num_cols = 2
1129.                 cols = st.columns(num_cols)
1130.
1131.                 for idx, feature in enumerate(available_features):
1132.                 col_idx = idx % num_cols
1133.                 with cols[col_idx]:
1134.                 default_state = st.session_state.get(f'feature_{feature}',
True)
1135.
1136.                 st.checkbox(
1137.                 feature,
1138.                 value=default_state,
1139.                 key=f'feature_{feature}'
1140.                 )
1141.
1142.                 config['enabled_features'] = {}
1143.                 for feat in available_features:
1144.                 enabled_state = st.session_state.get(f'feature_{feat}', True)
1145.                 config['enabled_features'][feat] = enabled_state
1146.
1147.                 enabled_count = sum(config['enabled_features'].values())
1148.                 st.info(f"📊 {enabled_count}/{len(available_features)} features
enabled")
1149.                 else:
1150.                 st.info("Select a dataset to configure features")
1151.
1152.                 st.markdown("----")
1153.
1154.                 #control buttons
1155.                 if not st.session_state.training_active:
1156.                 if st.button("▶ Start Training", width='stretch',
key="dqn_start_training"):
1157.                 st.session_state.training_completed = False
1158.                 #don't add _use_pretrained to config, pass it separately
1159.                 st.session_state.training_use_pretrained = use_pretrained

```

```

1160.
1161.         st.session_state.training_logs = []
1162.         st.session_state.manager = multiprocessing.Manager()
1163.         st.session_state.log_queue = st.session_state.manager.Queue()
1164.         st.session_state.training_progress =
st.session_state.manager.dict()
1165.         st.session_state.training_progress['current'] = 0
1166.         st.session_state.training_progress['total'] = config['episodes']
1167.         st.session_state.training_progress['state'] = 'Initializing...'
1168.         st.session_state.training_progress['metadata'] = {}
1169.         st.session_state.training_progress['history'] = {}
1170.
1171.         p = multiprocessing.Process(
1172.             target=training_worker,
1173.             args=(config, st.session_state.training_progress,
st.session_state.log_queue, use_pretrained)
1174.         )
1175.         st.session_state.training_process = p
1176.         st.session_state.training_active = True
1177.         st.session_state.training_type = 'dqn'
1178.         p.start()
1179.         st.rerun()
1180.     else:
1181.         if st.button("■ Stop Training", width='stretch'):
1182.             if st.session_state.training_process and
st.session_state.training_process.is_alive():
1183.                 st.session_state.training_process.terminate()
1184.                 st.session_state.training_process.join()
1185.                 st.session_state.training_active = False
1186.                 st.session_state.training_process = None
1187.                 st.warning("Training stopped by user.")
1188.                 st.rerun()
1189.
1190.     with left:
1191.         #show persistent results if they exist
1192.         if not st.session_state.training_active and 'lgbm_results' in st.session_state and
st.session_state.get('training_type') == 'lgbm':
1193.             results = st.session_state['lgbm_results']
1194.             if results:
1195.                 st.info("📄 Viewing saved results. Start new training to update.")
1196.                 st.markdown("#### 📊 Test Results by Horizon")
1197.
1198.                 #select horizon to view
1199.                 horizons = sorted(results.keys())
1200.                 selected_horizon = st.selectbox("Select Horizon", horizons,
key="result_horizon_persistent")
1201.
1202.                 if selected_horizon in results:
1203.                     result = results[selected_horizon]
1204.
1205.                     #metrics
1206.                     st.markdown(f"***{selected_horizon}d Horizon Metrics***")
1207.                     col1, col2 = st.columns(2)
1208.                     with col1:
1209.                         st.metric("Directional Accuracy",
f"{result['directional_acc']*100:.2f}%")
1210.                     with col2:
1211.                         coverage = result['coverage']
1212.                         q_med = min(result['quantiles'], key=lambda q: abs(q - 0.5))
1213.                         st.metric("Median Coverage", f"{coverage[q_med]*100:.2f}%")
1214.
1215.                     st.markdown("#### ☑ Predictions vs Actual")
1216.                     fig = go.Figure()
1217.
1218.                     fig.add_trace(go.Scatter(
1219.                         x=result['dates'],
1220.                         y=result['actual'],
1221.                         mode='lines',
1222.                         name='Actual',

```

```

1223.         line=dict(color='#00ff41', width=2)
1224.     ))
1225.
1226.     colors = ["#f87171", "#facc15", "#3b82f6", "#10b981", "#8b5cf6"]
1227.     for i, q in enumerate(result['quantiles']):
1228.         fig.add_trace(go.Scatter(
1229.             x=result['dates'],
1230.             y=result['predictions'][q],
1231.             mode='lines',
1232.             name=f'q{int(q*100):02d}',
1233.             line=dict(color=colors[i % len(colors)], dash='dash')
1234.         ))
1235.
1236.     fig.update_layout(
1237.         title=f"LGBM Predictions vs Actual ({selected_horizon}d Horizon)",
1238.         xaxis_title="Date",
1239.         yaxis_title="Future % Change",
1240.         height=400,
1241.         margin=dict(l=10, r=10, t=40, b=10),
1242.         plot_bgcolor="#0e1117",
1243.         paper_bgcolor="#0e1117",
1244.         font=dict(color="#00FF41"),
1245.         dragmode="zoom",
1246.         xaxis_rangeslider_visible=False,
1247.         showlegend=True
1248.     )
1249.
1250.     st.plotly_chart(fig, use_container_width=True)
1251.
1252.     #quantile coverage table
1253.     st.markdown("***Quantile Coverage**")
1254.     coverage_data = []
1255.     for q in result['quantiles']:
1256.         ideal = q
1257.         actual = coverage[q]
1258.         diff = actual - ideal
1259.         status = "✓" if abs(diff) < 0.05 else "△"
1260.         coverage_data.append({
1261.             'Quantile': f"q{int(q*100):02d}",
1262.             'Actual': f"{actual:.2%}",
1263.             'Ideal': f"{ideal:.0%}",
1264.             'Diff': f"{diff:+.2%}",
1265.             'Status': status
1266.         })
1267.
1268.     st.dataframe(pd.DataFrame(coverage_data), use_container_width=True,
1269.                 hide_index=True)
1270.
1271.     #flip configuration section
1272.     st.markdown("---")
1273.     st.markdown("####  Correlation Configuration")
1274.     st.info("Some models may find reverse correlations for certain horizons. Mark horizons that should have their predictions flipped.")
1275.
1276.     #load existing configuration if it exists
1277.     config_dir = Path("prediction_tools/saved_models/LGBM/correlations")
1278.     config_dir.mkdir(parents=True, exist_ok=True)
1279.
1280.     model_base_name = st.session_state.get('lgbm_model_name', 'unknown')
1281.     config_file = config_dir / f"{model_base_name}_flip_config.csv"
1282.
1283.     #initialize flip settings
1284.     if 'flip_settings_persistent' not in st.session_state:
1285.         if config_file.exists():
1286.             flip_df = pd.read_csv(config_file)
1287.             st.session_state.flip_settings_persistent = dict(zip(flip_df['horizon'], flip_df['flip']))
1288.         else:
1289.             st.session_state.flip_settings_persistent = {}

```

```

1289.
1290.         #create checkboxes for each horizon
1291.         flip_cols = st.columns(min(len(results.keys()), 5))
1292.         for i, horizon in enumerate(sorted(results.keys())):
1293.             with flip_cols[i % 5]:
1294.                 current_flip =
st.session_state.flip_settings_persistent.get(horizon, False)
1295.                 st.session_state.flip_settings_persistent[horizon] =
st.checkbox(
1296.                     f"{horizon}d flip",
1297.                     value=current_flip,
1298.                     key=f"flip_{horizon}_persistent"
1299.                 )
1300.
1301.         #save and retest buttons
1302.         if st.button("💾 Save Flip Configuration",
key="save_flip_persistent"):
1303.             flip_data = {
1304.                 'horizon':
list(st.session_state.flip_settings_persistent.keys()),
1305.                 'flip':
list(st.session_state.flip_settings_persistent.values())
1306.             }
1307.             pd.DataFrame(flip_data).to_csv(config_file, index=False)
1308.             st.success(f"✓ Configuration saved to {config_file.name}")
1309.             st.info("🔄 Use 'Test Existing Model' mode to re-test with new
flip settings")
1310.
1311.         elif not st.session_state.training_active and not
st.session_state.training_completed:
1312.             st.info("Configure settings and click 'Start Training' to begin.")
1313.         else:
1314.             #check if process is still alive
1315.             if st.session_state.training_process and not
st.session_state.training_process.is_alive():
1316.                 state = st.session_state.training_progress.get('state', 'idle')
1317.                 if state == 'completed':
1318.                     st.success("✅ Training completed successfully!")
1319.
1320.                 st.session_state.training_completed = True
1321.                 training_type = st.session_state.get('training_type', 'dqn')
1322.
1323.             #store results in session state for persistence
1324.             if training_type == 'lgbm':
1325.                 st.session_state['lgbm_results'] =
st.session_state.training_progress.get('results', {})
1326.             elif training_type == 'dqn':
1327.                 st.session_state['dqn_test_stats'] =
st.session_state.training_progress.get('test_stats', None)
1328.
1329.             #DQN results
1330.             if training_type == 'dqn':
1331.                 test_stats = st.session_state.get('dqn_test_stats')
1332.                 if test_stats:
1333.                     st.markdown("#### 📊 Test Results")
1334.
1335.                     col1, col2, col3 = st.columns(3)
1336.                     with col1:
1337.                         st.metric("Win Rate",
f"{test_stats['win_rate']*100:.1f}%")
1338.                     with col2:
1339.                         pf_str = f"{test_stats['profit_factor']:.2f}" if not
np.isinf(test_stats['profit_factor']) else "∞"
1340.                         st.metric("Profit Factor", pf_str)
1341.                     with col3:
1342.                         st.metric("Total Trades", test_stats['total_trades'])
1343.
1344.             #LGBM results
1345.             elif training_type == 'lgbm':

```

```

1346.         results = st.session_state.get('lgbm_results', {})
1347.         if results:
1348.             st.markdown("#### 📊 Test Results by Horizon")
1349.
1350.             #select horizon to view
1351.             horizons = sorted(results.keys())
1352.             selected_horizon = st.selectbox("Select Horizon", horizons,
key="result_horizon")
1353.
1354.             if selected_horizon in results:
1355.                 result = results[selected_horizon]
1356.
1357.                 #metrics
1358.                 st.markdown(f"***{selected_horizon}d Horizon Metrics**")
1359.                 col1, col2 = st.columns(2)
1360.                 with col1:
1361.                     st.metric("Directional Accuracy",
f"{result['directional_acc']*100:.2f}%")
1362.                 with col2:
1363.                     coverage = result['coverage']
1364.                     q_med = min(result['quantiles'], key=lambda q: abs(q -
0.5))
1365.                     st.metric("Median Coverage",
f"{coverage[q_med]*100:.2f}%")
1366.
1367.                 st.markdown("#### 📈 Predictions vs Actual")
1368.                 fig = go.Figure()
1369.
1370.                 fig.add_trace(go.Scatter(
1371.                     x=result['dates'],
1372.                     y=result['actual'],
1373.                     mode='lines',
1374.                     name='Actual',
1375.                     line=dict(color='#00ff41', width=2)
1376.                 ))
1377.
1378.                 colors = ["#f87171", "#facc15", "#3b82f6", "#10b981",
"#8b5cf6"]
1379.
1380.                 for i, q in enumerate(result['quantiles']):
1381.                     fig.add_trace(go.Scatter(
1382.                         x=result['dates'],
1383.                         y=result['predictions'][q],
1384.                         mode='lines',
1385.                         name=f'q{int(q*100):02d}',
1386.                         line=dict(color=colors[i % len(colors)],
dash='dash')
1387.                     ))
1388.
1389.                 fig.update_layout(
1390.                     title=f"LGBM Predictions vs Actual
({selected_horizon}d Horizon)",
1391.                     xaxis_title="Date",
1392.                     yaxis_title="Future % Change",
1393.                     height=400,
1394.                     margin=dict(l=10, r=10, t=40, b=10),
1395.                     plot_bgcolor="#0e1117",
1396.                     paper_bgcolor="#0e1117",
1397.                     font=dict(color="#00FF41"),
1398.                     dragmode="zoom",
1399.                     xaxis_rangeslider_visible=False,
1400.                     showlegend=True
1401.                 )
1402.
1403.                 st.plotly_chart(fig, use_container_width=True)
1404.
1405.                 #quantile coverage table
1406.                 st.markdown("***Quantile Coverage**")
1407.                 coverage_data = []
1408.                 for q in result['quantiles']:

```

```

1408.         ideal = q
1409.         actual = coverage[q]
1410.         diff = actual - ideal
1411.         status = "✓" if abs(diff) < 0.05 else "△"
1412.         coverage_data.append({
1413.             'Quantile': f"q{int(q*100):02d}",
1414.             'Actual': f"{actual:.2%}",
1415.             'Ideal': f"{ideal:.0%}",
1416.             'Diff': f"{diff:+.2%}",
1417.             'Status': status
1418.         })
1419.
1420.         st.dataframe(pd.DataFrame(coverage_data),
1421. use_container_width=True, hide_index=True)
1422.
1423.         #flip configuration section (again...)
1424.         st.markdown("---")
1425.         st.markdown("####  Correlation Configuration")
1426.         st.info("Some models may find reverse correlations for
1427. certain horizons. Mark horizons that should have their predictions flipped.")
1428.
1429.         config_dir =
1430. Path("prediction_tools/saved_models/LGBM/correlations")
1431.         config_dir.mkdir(parents=True, exist_ok=True)
1432.
1433.         model_base_name = st.session_state.get('lgbm_model_name',
1434. 'unknown')
1435.         config_file = config_dir /
1436. f"{model_base_name}_flip_config.csv"
1437.
1438.         if 'flip_settings' not in st.session_state:
1439.             if config_file.exists():
1440.                 flip_df = pd.read_csv(config_file)
1441.                 st.session_state.flip_settings =
1442. dict(zip(flip_df['horizon'], flip_df['flip']))
1443.             else:
1444.                 st.session_state.flip_settings = {}
1445.
1446.         flip_cols = st.columns(min(len(results.keys()), 5))
1447.         for i, horizon in enumerate(sorted(results.keys())):
1448.             with flip_cols[i % 5]:
1449.                 current_flip =
1450. st.session_state.flip_settings.get(horizon, False)
1451.                 st.session_state.flip_settings[horizon] =
1452. st.checkbox(
1453.                     f"{horizon}d flip",
1454.                     value=current_flip,
1455.                     key=f"flip_{horizon}_immediate"
1456.                 )
1457.
1458.         if st.button(" Save Flip Configuration",
1459. key="save_flip_immediate"):
1460.             flip_data = {
1461.                 'horizon':
1462. list(st.session_state.flip_settings.keys()),
1463.                 'flip':
1464. list(st.session_state.flip_settings.values())
1465.             }
1466.             pd.DataFrame(flip_data).to_csv(config_file,
1467. index=False)
1468.
1469.             st.success(f"✓ Configuration saved to
1470. {config_file.name}")
1471.
1472.             st.session_state.training_active = False
1473.             st.session_state.training_process = None
1474.
1475.             if st.session_state.log_queue:
1476.                 try:
1477.                     while not st.session_state.log_queue.empty():

```

```

1464.             msg = st.session_state.log_queue.get_nowait()
1465.             st.session_state.training_logs.append(msg)
1466.         except queue.Empty:
1467.             pass
1468.
1469.         current = st.session_state.training_progress.get('current', 0)
1470.         total = st.session_state.training_progress.get('total', 1)
1471.         state_text = st.session_state.training_progress.get('state', 'Training...')
1472.         metadata = st.session_state.training_progress.get('metadata', {})
1473.         history = st.session_state.training_progress.get('history', {})
1474.
1475.         #metadata progress bar
1476.         progress_pct = min(1.0, max(0.0, current / total if total > 0 else 0))
1477.
1478.         training_type = st.session_state.get('training_type', 'dqn')
1479.
1480.         #build metadata string
1481.         if training_type == 'dqn':
1482.             meta_str = f"Episode {current}/{total}"
1483.             if metadata:
1484.                 meta_items = []
1485.                 for key, val in metadata.items():
1486.                     meta_items.append(f"{key}: {val}")
1487.                 meta_str += " | " + " | ".join(meta_items)
1488.             else: # lgbm
1489.                 meta_str = f"Horizon {current}/{total} | {state_text}"
1490.
1491.         st.progress(progress_pct, text=meta_str)
1492.
1493.         #live training graphs
1494.         if training_type == 'dqn' and history and len(history.get('returns', [])) > 0:
1495.             st.markdown("#### ☒ Training Progress (10-Episode Rolling Average)")
1496.
1497.             episodes = list(range(1, len(history['returns']) + 1))
1498.
1499.             #create subplots
1500.             fig = go.Figure()
1501.
1502.             #returns
1503.             fig.add_trace(go.Scatter(
1504.                 x=episodes,
1505.                 y=history['returns'],
1506.                 mode='lines',
1507.                 name='Returns %',
1508.                 line=dict(color='#00ff41', width=2)
1509.             ))
1510.
1511.             fig.update_layout(
1512.                 title="Episode Returns (10-Ep Avg)",
1513.                 xaxis_title="Episode",
1514.                 yaxis_title="Return %",
1515.                 height=250,
1516.                 margin=dict(l=10, r=10, t=40, b=10),
1517.                 plot_bgcolor="#0e1117",
1518.                 paper_bgcolor="#0e1117",
1519.                 font=dict(color="#00FF41"),
1520.                 showlegend=False
1521.             )
1522.
1523.             st.plotly_chart(fig, use_container_width=True)
1524.
1525.             col1, col2 = st.columns(2)
1526.
1527.             with col1:
1528.                 fig_sharpe = go.Figure()
1529.                 fig_sharpe.add_trace(go.Scatter(
1530.                     x=episodes,
1531.                     y=history['sharpes'],
1532.                     mode='lines',

```

```

1533.         line=dict(color='#3b82f6', width=2)
1534.     ))
1535.     fig_sharpe.update_layout(
1536.         title="Sharpe Ratio",
1537.         xaxis_title="Episode",
1538.         yaxis_title="Sharpe",
1539.         height=200,
1540.         margin=dict(l=10, r=10, t=30, b=10),
1541.         plot_bgcolor="#0e1117",
1542.         paper_bgcolor="#0e1117",
1543.         font=dict(color="#00FF41"),
1544.         showlegend=False
1545.     )
1546.     st.plotly_chart(fig_sharpe, use_container_width=True)
1547.
1548.     with col2:
1549.         fig_loss = go.Figure()
1550.         fig_loss.add_trace(go.Scatter(
1551.             x=episodes,
1552.             y=history['losses'],
1553.             mode='lines',
1554.             line=dict(color='#f87171', width=2)
1555.         ))
1556.         fig_loss.update_layout(
1557.             title="Training Loss",
1558.             xaxis_title="Episode",
1559.             yaxis_title="Loss",
1560.             height=200,
1561.             margin=dict(l=10, r=10, t=30, b=10),
1562.             plot_bgcolor="#0e1117",
1563.             paper_bgcolor="#0e1117",
1564.             font=dict(color="#00FF41"),
1565.             showlegend=False,
1566.             yaxis_type="log"
1567.         )
1568.         st.plotly_chart(fig_loss, use_container_width=True)
1569.
1570.         #console logs
1571.         st.markdown("#### 📖 Training Logs")
1572.         log_text = "\n".join(st.session_state.training_logs[-100:])
1573.         render_console_style(log_text, height=300)
1574.
1575.         #auto-refresh while training
1576.         if st.session_state.training_active:
1577.             time.sleep(1)
1578.             st.rerun()
1579.

```

backtesting.py

This script handles backtesting for LightGBM and the DQN agent. For LightGBM specifically, its tests where it has been added to training datasets.

Objective 8.2

This script as a whole is designed to provide a backtesting environment for LightGBM and DON

```

1. import streamlit as st
2. from pathlib import Path
3. import pandas as pd
4. import plotly.graph_objects as go
5. import numpy as np
6. import re
7. import os
8. import sys
9. import torch
10. import json
11. from datetime import timedelta
12.
13. project_root = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
14. if project_root not in sys.path:
15.     sys.path.insert(0, project_root)
16.
17. DATASET_DIR = Path("application/datasets")
18. DQN_MODEL_DIR = Path("prediction_tools/saved_models/DQN")
19.
20. def extract_quantile_columns(df, horizon):
21.     #using regular expressions to extract quantile columns
22.     pattern = re.compile(rf"LGBM_{horizon}d_q(\d{{2}})")
23.     quantiles = {}
24.
25.     for col in df.columns:
26.         m = pattern.match(col)
27.         if m:
28.             q = int(m.group(1)) / 100.0
29.             quantiles[q] = col
30.
31.     return quantiles
32.
33. def calculate_lgbm_metrics(df, horizon, date_range=None):
34.
35.     #filter by date range if specified
36.     if date_range:
37.         df = df[(df['Date'] >= date_range[0]) & (df['Date'] <= date_range[1])].copy()
38.
39.     #get quantile columns
40.     quantile_cols = extract_quantile_columns(df, horizon)
41.     if not quantile_cols:
42.         return None, None, None
43.
44.     quantiles_sorted = sorted(quantile_cols.keys())
45.     q_med = min(quantiles_sorted, key=lambda q: abs(q - 0.5))
46.
47.     #get the target column from dataset
48.     target_col = f'Future_Pct_Change_{horizon}d'
49.     if target_col not in df.columns:
50.         return None, None, None
51.
52.     future_return = df[target_col]
53.     valid_idx = future_return.notna()
54.     future_return = future_return[valid_idx]
55.     preds = {q: df[col][valid_idx].copy() for q, col in quantile_cols.items()}
56.
57.     if len(future_return) == 0:
58.         return None, None, None
59.
60.     #calculate metrics
61.     metrics = {}
62.
63.     coverage = {}
64.     for q in quantiles_sorted:
65.         cov = (future_return <= preds[q]).mean()
66.         coverage[q] = cov
67.
68.     median_pred = preds[q_med] > 0
69.     actual_up = future_return > 0

```

Objective 8.2

LightGBM backtest on dataset

```

70.     overall_accuracy = (median_pred == actual_up).mean()
71.
72.     if len(quantiles_sorted) >= 2:
73.         q_low = quantiles_sorted[0]
74.         q_high = quantiles_sorted[-1]
75.         spread = preds[q_high] - preds[q_low]
76.
77.         spread_bins = []
78.         try:
79.             bins = pd.qcut(spread, 5, labels=False, duplicates='drop')
80.             for b in range(5):
81.                 mask = bins == b
82.                 if mask.sum() > 0:
83.                     acc = (np.sign(preds[q_med][mask]) ==
np.sign(future_return[mask])).mean()
84.                     avg_width = spread[mask].mean()
85.                     spread_bins.append({'bin': b+1, 'accuracy': acc, 'width': avg_width,
'count': mask.sum()})
86.             except:
87.                 pass
88.         else:
89.             spread_bins = []
90.
91.         metrics['coverage'] = coverage
92.         metrics['overall_accuracy'] = overall_accuracy
93.         metrics['spread_bins'] = spread_bins
94.
95.         # Return data for plotting
96.         plot_data = {
97.             'dates': df['Date'][valid_idx].values,
98.             'actual': future_return.values,
99.             'predictions': {q: preds[q].values for q in quantiles_sorted}
100.        }
101.
102.         return metrics, plot_data, quantiles_sorted
103.
104. def get_available_dqn_models():
105.     if not DQN_MODEL_DIR.exists():
106.         return []
107.
108.     models = set()
109.     for file in DQN_MODEL_DIR.glob("*_metadata.json"):
110.         base_name = file.stem.replace("_metadata", "")
111.         models.add(base_name)
112.
113.     return sorted(list(models))
114.
115. def load_dqn_model(model_name):
116.     try:
117.         #load metadata
118.         metadata_path = DQN_MODEL_DIR / f"{model_name}_metadata.json"
119.         with open(metadata_path, 'r') as f:
120.             metadata = json.load(f)
121.
122.         config = metadata['config']
123.
124.         #load model weights
125.         policy_path = DQN_MODEL_DIR / f"{model_name}_policy.pt"
126.         target_path = DQN_MODEL_DIR / f"{model_name}_target.pt"
127.         optimiser_path = DQN_MODEL_DIR / f"{model_name}_optimiser.pt"
128.
129.         if not all([policy_path.exists(), target_path.exists(), optimiser_path.exists()]):
130.             return None, None, "Model files incomplete"
131.
132.         return config, metadata, None
133.
134.     except Exception as e:
135.         return None, None, str(e)
136.

```

Objective 8.2

DQN backtest on specific date range

```

137. def run_dqn_backtest(model_name, dataset_path, start_date=None, episode_length=None):
138.     try:
139.         #local import
140.         from prediction_tools.DQN.agent import DQNAgent
141.         from prediction_tools.DQN.trading_env import TradingEnvironment
142.
143.         #load model config
144.         config, metadata, error = load_dqn_model(model_name)
145.         if error:
146.             return None, None, None, None, error
147.
148.         #store original trained episode length before overriding
149.         original_episode_length = config.get('episode_length_days', 252)
150.
151.         #override episode length if provided
152.         if episode_length is not None:
153.             config = config.copy()
154.             config['episode_length_days'] = episode_length
155.             config['trained_episode_length'] = original_episode_length # Keep original for
normalization
156.         else:
157.             config['trained_episode_length'] = original_episode_length
158.
159.         df = pd.read_csv(dataset_path)
160.         df['Date'] = pd.to_datetime(df['Date'])
161.
162.         target_cols = [col for col in df.columns if col.startswith("Target") or
col.startswith("Future")]
163.         df = df.drop(columns=target_cols)
164.
165.         #filter to only features the model was trained on
166.         required_features = metadata.get('feature_list', [])
167.
168.         if required_features:
169.             print(f"Model requires {len(required_features)} specific features")
170.
171.             #check if dataset has all required features
172.             missing_features = [f for f in required_features if f not in df.columns]
173.             if missing_features:
174.                 return None, None, None, None, None, f"Dataset missing required features:
{missing_features[:5]}..."
175.
176.             base_cols = ['Date', 'Open', 'High', 'Low', 'Close', 'Adj Close']
177.             lgbm_cols = [col for col in df.columns if col.startswith('LGBM')]
178.             keep_cols = base_cols + lgbm_cols + required_features
179.
180.             keep_cols = list(dict.fromkeys(keep_cols))
181.
182.             test_data = df[keep_cols].copy()
183.             print(f"Using {len(keep_cols)} columns for backtesting
({len(required_features)} features)")
184.         else:
185.             print("Model metadata missing feature list, using all features")
186.             test_data = df
187.
188.             train_env = TradingEnvironment(test_data, config, config['device'],
is_training=True)
189.             train_norm_stats = (train_env.initial_feature_means,
train_env.initial_feature_stds)
190.
191.             state = train_env.reset()
192.             lookback_window, num_features = state.shape
193.             num_actions = len(config['action_types'])
194.
195.             agent = DQNAgent(num_features, lookback_window, num_actions, config)
196.
197.             #load weights
198.             policy_path = DQN_MODEL_DIR / f"{model_name}_policy.pt"
199.             target_path = DQN_MODEL_DIR / f"{model_name}_target.pt"

```

```

200.         optimiser_path = DQN_MODEL_DIR / f"{model_name}_optimiser.pt"
201.
202.         agent.policy_net.load_state_dict(torch.load(policy_path,
map_location=config['device']))
203.         agent.target_net.load_state_dict(torch.load(target_path,
map_location=config['device']))
204.         agent.optimiser.load_state_dict(torch.load(optimiser_path,
map_location=config['device']))
205.
206.         #run test with position tracking
207.         position_events, trade_stats, balances, realized_balances, actions_taken =
run_test_with_tracking(
208.             agent, test_data, config, train_norm_stats, start_date
209.         )
210.
211.         return position_events, trade_stats, balances, realized_balances, actions_taken,
None
212.
213.     except Exception as e:
214.         import traceback
215.         return None, None, None, None, None, f"Error: {str(e)}\n{traceback.format_exc()}"
216.
217. def run_test_with_tracking(agent, test_data, config, train_norm_stats,
start_date_override=None):
218.     from prediction_tools.DQN.trading_env import TradingEnvironment, PositionManager
219.
220.     env = TradingEnvironment(
221.         test_data,
222.         config,
223.         config['device'],
224.         is_training=False,
225.         train_normalisation_stats=train_norm_stats
226.     )
227.
228.     if start_date_override is not None:
229.         try:
230.             idx = test_data[test_data['Date'] ==
pd.to_datetime(start_date_override)].index[0]
231.             min_idx = config['lookback_window']
232.             max_idx = len(test_data) - config['episode_length_days'] - 1
233.
234.             if min_idx <= idx <= max_idx:
235.                 #manually set start_idx before calling reset's random selection
236.                 env.start_idx = idx
237.
238.                 #now do the rest of reset manually
239.                 env.current_step = 0
240.                 env.balance = float(config['initial_balance'])
241.                 env.initial_balance = float(config['initial_balance'])
242.                 env.position_manager = PositionManager()
243.                 env.done = False
244.                 env.balance_history = [env.balance]
245.                 env.prev_action = 1
246.                 env.action_history = []
247.                 env.prev_unrealized_pnl = 0.0
248.                 env.lgbm_hit_history = []
249.                 env.episode_hit_rate = 0.0
250.                 env.indecision_steps = 0
251.                 env.indecision_penalty_current = 0.0
252.
253.                 state = env.get_state()
254.             else:
255.                 #date out of valid range, use random
256.                 state = env.reset()
257.         except:
258.             #date not found or error
259.             state = env.reset()
260.     else:
261.         #no date override

```

Ensures index
is valid before
setting state
manually

```

262.         state = env.reset()
263.
264.         #track position events
265.         position_events = []
266.
267.         from prediction_tools.DQN.testing import TradeTracker
268.
269.         initial_price = test_data.iloc[env.start_idx]['Close']
270.         trade_tracker = TradeTracker(
271.             initial_balance=float(config['initial_balance']),
272.             initial_price=initial_price
273.         )
274.         actions_taken = []
275.         balances = []
276.         realized_balances = []
277.
278.         #run episode
279.         while not env.done:
280.             action = agent.select_action(state, training=False)
281.
282.             # env.step() increments current_step, so get the price at the CURRENT step
283.             current_price = test_data.iloc[env.start_idx + env.current_step]['Close']
284.             current_date = test_data.iloc[env.start_idx + env.current_step]['Date']
285.
286.             positions_before = env.position_manager.get_position_count()
287.             next_state, reward, done, info = env.step(action)
288.             positions_after = env.position_manager.get_position_count()
289.
290.             #track buy events
291.             if action >= 2 and positions_after > positions_before:
292.                 new_position = env.position_manager.positions[-1]
293.                 trade_tracker.open_position(new_position.entry_price, new_position.allocation)
294.
295.                 position_events.append({
296.                     'date': current_date,
297.                     'price': current_price,
298.                     'type': 'buy',
299.                     'action': config['action_types'][action]
300.                 })
301.
302.             #track sell events
303.             elif action == 0 and positions_before > 0:
304.                 trade_tracker.close_all_positions(current_price)
305.
306.                 position_events.append({
307.                     'date': current_date,
308.                     'price': current_price,
309.                     'type': 'sell',
310.                     'num_closed': positions_before
311.                 })
312.
313.             actions_taken.append(action)
314.             balances.append(info['balance'])
315.             realized_balances.append(info['realized_balance'])
316.             state = next_state
317.
318.             if info.get('bankruptcy', False):
319.                 break
320.
321.             final_price = test_data.iloc[env.start_idx + env.current_step - 1]['Close']
322.             #close any remaining positions
323.             if trade_tracker.current_entries:
324.                 #use the last price from the episode
325.
326.                 trade_tracker.close_all_positions(final_price)
327.             trade_tracker.final_price = final_price
328.
329.             trade_stats = trade_tracker.get_statistics()
330.

```

```

331.     return position_events, trade_stats, balances, realized_balances, actions_taken
332.
333. def render_console_style(content, height=300):
334.     st.markdown(
335.         f"""
336.         <div style="
337.             background-color:#0e1117;
338.             color:#00FF41;
339.             padding:15px;
340.             border-radius:5px;
341.             height:{height}px;
342.             overflow-y:scroll;
343.             font-family: monospace;
344.             white-space: pre-wrap;
345.             border: 1px solid #00FF41;
346.         ">
347.             {content}
348.         """,
349.         unsafe_allow_html=True
350.     )
351.
352. def render():
353.     st.markdown("### 🔄 Backtesting")
354.     st.divider()
355.
356.     datasets = (
357.         [f.name for f in DATASET_DIR.glob("*.csv")]
358.         if DATASET_DIR.exists()
359.         else []
360.     )
361.
362.     left, right = st.columns([4, 1.7])
363.
364.     with right:
365.         with st.container():
366.             st.markdown("#### ⚙️ Settings")
367.
368.             #evaluation type selector
369.             eval_type = st.selectbox(
370.                 "Evaluation Type",
371.                 ["LightGBM", "DQN"],
372.                 disabled=False
373.             )
374.
375.             st.markdown("---")
376.
377.             #lgbm settings
378.             if eval_type == "LightGBM":
379.                 selected_dataset = st.selectbox(
380.                     "Select Dataset",
381.                     datasets,
382.                     disabled=not datasets
383.                 )
384.
385.                 if selected_dataset:
386.                     try:
387.                         df = pd.read_csv(f"{DATASET_DIR}\\{selected_dataset}")
388.                         df["Date"] = pd.to_datetime(df["Date"])
389.
390.                         # Detect available horizons
391.                         lgbm_cols = [c for c in df.columns if c.startswith("LGBM_")]
392.                         if lgbm_cols:
393.                             horizons = sorted({
394.                                 int(re.search(r"LGBM_(\d+)d", c).group(1))
395.                                 for c in lgbm_cols if re.search(r"LGBM_(\d+)d", c)
396.                             })
397.
398.                             selected_horizon = st.selectbox(
399.                                 "Prediction Horizon",

```

```

400.         horizons,
401.         key="backtest_horizon"
402.     )
403.
404.     st.markdown("---")
405.
406.     st.markdown("**Date Range**")
407.     min_date = df['Date'].min().date()
408.     max_date = df['Date'].max().date()
409.
410.     date_start = st.date_input(
411.         "Start Date",
412.         value=min_date,
413.         min_value=min_date,
414.         max_value=max_date
415.     )
416.
417.     date_end = st.date_input(
418.         "End Date",
419.         value=max_date,
420.         min_value=min_date,
421.         max_value=max_date
422.     )
423.
424.     st.markdown("---")
425.
426.     if st.button("▶ Run Backtest", width='stretch'):
427.         st.session_state['run_backtest'] = True
428.         st.session_state['backtest_type'] = 'lgbm'
429.         st.session_state['lgbm_date_start'] = date_start
430.         st.session_state['lgbm_date_end'] = date_end
431.         st.session_state['lgbm_horizon'] = selected_horizon
432.         st.session_state['lgbm_dataset'] = selected_dataset
433.         st.rerun()
434.     else:
435.         st.warning("No LGBM predictions found in dataset.")
436.         st.stop()
437.
438.     except Exception as e:
439.         st.error(f"Failed to load dataset: {e}")
440.         st.stop()
441. else:
442.     st.info("Select a dataset to begin.")
443.     st.stop()
444.
445. #DQN Settings
446. elif eval_type == "DQN":
447.     available_models = get_available_dqn_models()
448.
449.     if not available_models:
450.         st.warning("No DQN models found in prediction_tools/saved_models/DQN/")
451.         st.info("Train a model first using the DQN trainer.")
452.         st.stop()
453.
454.     selected_model = st.selectbox(
455.         "Select Model",
456.         available_models
457.     )
458.
459.     config, metadata, error = load_dqn_model(selected_model)
460.
461.     if error:
462.         st.error(f"Failed to load model: {error}")
463.         st.stop()
464.
465.
466.     with st.expander("📄 Model Info", expanded=False):
467.         st.text(f"Trained: {metadata.get('timestamp', 'Unknown')}")
468.         st.text(f"Episodes: {config.get('episodes', 'N/A')}")

```

```

469.         st.text(f"Horizons: {config.get('horizons', 'N/A')}")
470.         if 'test_stats' in metadata:
471.             st.text(f"Test Win Rate: {metadata['test_stats'].get('win_rate',
0)*100:.1f}%")
472.
473.         st.markdown("----")
474.
475.         selected_dataset = st.selectbox(
476.             "Select Dataset",
477.             datasets,
478.             disabled=not datasets
479.         )
480.
481.         if selected_dataset:
482.             try:
483.                 df = pd.read_csv(DATASET_DIR / selected_dataset)
484.                 df["Date"] = pd.to_datetime(df["Date"])
485.
486.                 st.markdown("----")
487.
488.                 lookback = config.get('lookback_window', 30)
489.                 dataset_length = len(df)
490.
491.                 st.markdown("***Episode Configuration**")
492.
493.                 default_episode_length = 252
494.                 max_possible_length = dataset_length - lookback - 1
495.
496.                 episode_length = st.number_input(
497.                     "Episode Length (days)",
498.                     min_value=30,
499.                     max_value=max_possible_length,
500.                     value=min(default_episode_length, max_possible_length),
501.                     step=10,
502.                     help=f"Number of trading days to test. Max:
{max_possible_length}"
503.                 )
504.
505.                 min_date = df['Date'].min().date() + timedelta(days=lookback)
506.                 max_date = df['Date'].max().date()
507.
508.                 #adjusted max start date must have enough data for lookback +
episode_length
509.                 adjusted_max_idx = dataset_length - lookback - episode_length - 1
510.                 if adjusted_max_idx < lookback:
511.                     st.error(f"Episode length {episode_length} is too long for this
dataset!")
512.                     st.stop()
513.
514.                 adjusted_max = df['Date'].iloc[adjusted_max_idx].date()
515.
516.                 use_random = st.checkbox(
517.                     "🎲 Use Random Start Date",
518.                     value=True,
519.                     help="Episode will start at a random valid date"
520.                 )
521.
522.                 if use_random:
523.                     col1, col2 = st.columns(2)
524.                     with col1:
525.                         random_start = st.date_input(
526.                             "Minimum Date",
527.                             value=min_date,
528.                             min_value=min_date,
529.                             max_value=adjusted_max
530.                         )
531.                     with col2:
532.                         random_end = st.date_input(
533.                             "Maximum Date",

```

```

534.         value=adjusted_max,
535.         min_value=min_date,
536.         max_value=adjusted_max
537.     )
538.
539.     if random_start > random_end:
540.         st.error("'Random From' must be before or equal to 'Random
541. To'")
542.
543.         st.stop()
544.
545.         st.info(f"🎲 Episode will start at a random date between
546. {random_start} and {random_end}")
547.         date_start = None
548.     else:
549.         date_start = st.date_input(
550.             "Start Date",
551.             value=min_date,
552.             min_value=min_date,
553.             max_value=adjusted_max
554.         )
555.
556.         try:
557.             start_idx = df[df['Date'] ==
558. pd.to_datetime(date_start)].index[0]
559.             end_idx = min(start_idx + episode_length, len(df) - 1)
560.             end_date = df.iloc[end_idx]['Date'].date()
561.             st.success(f"Episode: {date_start} to {end_date}
562. ({episode_length} days)")
563.         except:
564.             st.warning(f"⚠️ Date {date_start} not found in dataset")
565.
566.     st.markdown("---")
567.
568.     if st.button("▶ Run Backtest", width='stretch'):
569.         st.session_state['run_backtest'] = True
570.         st.session_state['backtest_type'] = 'dqn'
571.         st.session_state['dqn_model'] = selected_model
572.         st.session_state['dqn_start_date'] = date_start
573.         if use_random:
574.             st.session_state['dqn_random_range'] = (random_start,
575. random_end)
576.
577.             st.session_state['dqn_dataset'] = selected_dataset
578.             st.session_state['dqn_episode_length'] = episode_length
579.             st.rerun()
580.
581.         except Exception as e:
582.             st.error(f"Failed to load dataset: {e}")
583.             st.stop()
584.     else:
585.         st.info("Select a dataset to begin.")
586.         st.stop()
587.
588. with left:
589.     if not st.session_state.get('run_backtest', False):
590.         st.info("Click 'Run Backtest' in the settings panel to begin.")
591.
592.         #LGBM Results
593.         elif st.session_state.get('backtest_type') == 'lgbm':
594.             with st.spinner("Running backtest..."):
595.                 #fet params from session state
596.                 date_start = st.session_state.get('lgbm_date_start')
597.                 date_end = st.session_state.get('lgbm_date_end')
598.                 selected_horizon = st.session_state.get('lgbm_horizon')
599.                 selected_dataset = st.session_state.get('lgbm_dataset')
600.
601.                 df = pd.read_csv(DATASET_DIR / selected_dataset)
602.                 df["Date"] = pd.to_datetime(df["Date"])
603.

```

```

598.         #calculate metrics
599.         date_range = (pd.to_datetime(date_start), pd.to_datetime(date_end))
600.         metrics, plot_data, qs = calculate_lgbm_metrics(df, selected_horizon,
601. date_range)
602.
603.         if metrics is None:
604.             st.error("No valid data in selected date range.")
605.             st.stop()
606.
607.         st.markdown("#### ☑ Predictions vs Actual")
608.         fig = go.Figure()
609.
610.         # Actual returns
611.         fig.add_trace(go.Scatter(
612.             x=plot_data['dates'],
613.             y=plot_data['actual'],
614.             mode='lines',
615.             name='Actual',
616.             line=dict(color='#00ff41', width=2)
617.         ))
618.
619.         colors = ["#facc15", "#f87171", "#3b82f6", "#10b981", "#8b5cf6"]
620.         for i, q in enumerate(qs):
621.             fig.add_trace(go.Scatter(
622.                 x=plot_data['dates'],
623.                 y=plot_data['predictions'][q],
624.                 mode='lines',
625.                 name=f'q{int(q*100):02d}',
626.                 line=dict(color=colors[i % len(colors)], dash='dash')
627.             ))
628.
629.         fig.update_layout(
630.             title=f"LGBM Predictions vs Actual ({selected_horizon}d Horizon)",
631.             xaxis_title="Date",
632.             yaxis_title="Return",
633.             height=400,
634.             margin=dict(l=10, r=10, t=40, b=10),
635.             plot_bgcolor="#0e1117",
636.             paper_bgcolor="#0e1117",
637.             font=dict(color="#00FF41"),
638.             dragmode="zoom",
639.             xaxis_rangeslider_visible=False,
640.             showlegend=True
641.         )
642.
643.         st.plotly_chart(fig, use_container_width=True)
644.
645.         #metrics console
646.         st.markdown("#### 📊 Evaluation Metrics")
647.
648.         #build console output
649.         console_output = "\n"+"\"*60+\"\n"
650.         console_output += f"LIGHTGBM BACKTEST RESULTS - {selected_horizon}d
651. HORIZON"
652.         console_output += "\n"+"\"*60+\"\n"
653.         console_output += f"OVERALL DIRECTIONAL ACCURACY:
654. {metrics['overall_accuracy']:.2%}"
655.         console_output += "\n"+"\"*60+\"\n"+"QUANTILE COVERAGE\n"+"\"*60+\"\n"
656.
657.         for q in qs:
658.             coverage = metrics['coverage'][q]
659.             ideal = q
660.             diff = coverage - ideal
661.             status = "✓" if abs(diff) < 0.05 else "△"
662.             console_output += f"q{int(q*100):02d}: {coverage:.2%} (ideal
663. {ideal:.0%}) {status} [{diff:+.2%}]\n"

```

Objective 8.5

LightGBM Directional
Accuracy displayed

```

663.         console_output += "CONFIDENCE ANALYSIS (Spread vs Accuracy)\n"
664.         console_output += "\n"*60+"\n"
665.         for item in metrics['spread_bins']:
666.             console_output += f"Bin {item['bin']} | Width: {item['width']:.4f}
| Accuracy: {item['accuracy']:.2%} | n={item['count']}\n"
667.
668.
669.         render_console_style(console_output, height=350)
670.
671.         #finding correlations between spread and accuracy
672.         if metrics['spread_bins']:
673.             st.markdown("#### 📊 Confidence Analysis")
674.
675.             spread_fig = go.Figure()
676.
677.             widths = [item['width'] for item in metrics['spread_bins']]
678.             accuracies = [item['accuracy'] for item in metrics['spread_bins']]
679.             counts = [item['count'] for item in metrics['spread_bins']]
680.
681.             spread_fig.add_trace(go.Scatter(
682.                 x=widths,
683.                 y=accuracies,
684.                 mode='markers+lines',
685.                 marker=dict(
686.                     size=[c/10 for c in counts],
687.                     color='#00ff41',
688.                     line=dict(color='#00ff41', width=2)
689.                 ),
690.                 line=dict(color='#00ff41', width=2),
691.                 name='Accuracy'
692.             ))
693.
694.             #50% reference line
695.             spread_fig.add_hline(
696.                 y=0.5,
697.                 line_dash="dash",
698.                 line_color="#f87171",
699.                 annotation_text="Random (50%)",
700.                 annotation_position="right"
701.             )
702.
703.             spread_fig.update_layout(
704.                 title="Quantile Width vs Directional Accuracy",
705.                 xaxis_title="Quantile Width (Spread)",
706.                 yaxis_title="Directional Accuracy",
707.                 height=350,
708.                 margin=dict(l=10, r=10, t=40, b=10),
709.                 plot_bgcolor="#0e1117",
710.                 paper_bgcolor="#0e1117",
711.                 font=dict(color="#00FF41"),
712.                 showlegend=False,
713.                 yaxis=dict(
714.                     tickformat='.0%',
715.                     range=[0, 1]
716.                 )
717.             )
718.
719.             st.plotly_chart(spread_fig, use_container_width=True)
720.
721.             #interpretation
722.             correlation = np.corrcoef(widths, accuracies)[0, 1] if len(widths) > 1
723.             else 0
724.             if correlation > 0.3:
725.                 st.success(f"✓ Positive correlation detected ({correlation:.3f}):
Wider spreads → Higher confidence")
726.             elif correlation < -0.3:
727.                 st.warning(f"⚠ Negative correlation detected ({correlation:.3f}):
Wider spreads → Lower confidence")
728.             else:

```

```

728.                 st.info(f"→ Weak correlation ({correlation:.3f}): Spread may not
indicate confidence")
729.
730.         #DQN Results
731.         elif st.session_state.get('backtest_type') == 'dqn':
732.             with st.spinner("Running DQN backtest..."):
733.                 model_name = st.session_state.get('dqn_model')
734.                 start_date = st.session_state.get('dqn_start_date')
735.                 selected_dataset = st.session_state.get('dqn_dataset')
736.                 episode_length = st.session_state.get('dqn_episode_length', 252)
737.
738.                 #run backtest
739.                 position_events, trade_stats, balances, realized_balances, actions, error =
run_dqn_backtest(
740.                     model_name,
741.                     DATASET_DIR / selected_dataset,
742.                     start_date,
743.                     episode_length
744.                 )
745.
746.                 if error:
747.                     st.error(f"Backtest failed: {error}")
748.                     st.stop()
749.
750.                 #get ohlc data for the episode
751.                 df = pd.read_csv(DATASET_DIR / selected_dataset)
752.                 df['Date'] = pd.to_datetime(df['Date'])
753.
754.                 #find date range from position events
755.                 if position_events:
756.                     start_date_actual = position_events[0]['date']
757.                     end_date_actual = position_events[-1]['date']
758.
759.                     episode_df = df[(df['Date'] >= start_date_actual) & (df['Date'] <=
end_date_actual)].copy()
760.                 else:
761.                     st.warning("No position events recorded.")
762.                     st.stop()
763.
764.                 st.markdown("#### ☑ Price & Trading Activity")
765.
766.                 fig_ohlc = go.Figure()
767.
768.                 fig_ohlc.add_trace(go.Candlestick(
769.                     x=episode_df['Date'],
770.                     open=episode_df['Open'],
771.                     high=episode_df['High'],
772.                     low=episode_df['Low'],
773.                     close=episode_df['Close'],
774.                     name='Price',
775.                     increasing_line_color='#00ff41',
776.                     decreasing_line_color='#ff3b3b'
777.                 ))
778.
779.                 #buy arrows
780.                 buy_events = [e for e in position_events if e['type'] == 'buy']
781.                 if buy_events:
782.                     buy_dates = [e['date'] for e in buy_events]
783.                     buy_prices = [e['price'] for e in buy_events]
784.
785.                     fig_ohlc.add_trace(go.Scatter(
786.                         x=buy_dates,
787.                         y=buy_prices,
788.                         mode='markers',
789.                         name='Buy',
790.                         marker=dict(
791.                             symbol='triangle-up',
792.                             size=12,
793.                             color='#3b82f6',

```

Objective 8.5

Trading performance for
DQN displayed

```

794.         line=dict(color='#1e40af', width=2)
795.     )
796. ))
797.
798. #sell arrows
799. sell_events = [e for e in position_events if e['type'] == 'sell']
800. if sell_events:
801.     sell_dates = [e['date'] for e in sell_events]
802.     sell_prices = [e['price'] for e in sell_events]
803.
804.     fig_ohlcv.add_trace(go.Scatter(
805.         x=sell_dates,
806.         y=sell_prices,
807.         mode='markers',
808.         name='Sell',
809.         marker=dict(
810.             symbol='triangle-down',
811.             size=12,
812.             color='#f87171',
813.             line=dict(color='#dc2626', width=2)
814.         )
815.     ))
816.
817. fig_ohlcv.update_layout(
818.     title="Price Action with Trade Signals",
819.     xaxis_title="Date",
820.     yaxis_title="Price",
821.     height=400,
822.     margin=dict(l=10, r=10, t=40, b=10),
823.     plot_bgcolor="#0e1117",
824.     paper_bgcolor="#0e1117",
825.     font=dict(color="#00ff41"),
826.     xaxis_rangeslider_visible=False,
827.     showlegend=True
828. )
829.
830. st.plotly_chart(fig_ohlcv, use_container_width=True)
831.
832. st.markdown("#### 💰 Profit/Loss Curve")
833.
834. fig_capital = go.Figure()
835.
836. days = np.arange(len(balances))
837. initial_balance = balances[0]
838.
839. # Show both unrealized and realized
840. fig_capital.add_trace(go.Scatter(
841.     x=days,
842.     y=np.array(balances) - initial_balance,
843.     mode='lines',
844.     name='Total Portfolio (Unrealized)',
845.     line=dict(color='#3b82f6', width=2, dash='dash'),
846.     opacity=0.6
847. ))
848.
849. fig_capital.add_trace(go.Scatter(
850.     x=days,
851.     y=np.array(realized_balances) - initial_balance,
852.     mode='lines',
853.     name='Realized PnL',
854.     line=dict(color='#00ff41', width=3),
855.     fill='tozero',
856.     fillcolor='rgba(0, 255, 65, 0.1)'
857. ))
858. #zero line
859. fig_capital.add_hline(
860.     y=0,
861.     line_dash="dash",
862.     line_color="gray",

```

```

863.         annotation_text="Breakeven",
864.         annotation_position="right"
865.     )
866.
867.     #calculate drawdown from REALIZED balances
868.     peak = np.maximum.accumulate(realized_balances)
869.     drawdown_pct = (np.array(realized_balances) - peak) / peak * 100
870.     max_dd_pct = np.min(drawdown_pct)
871.
872.     fig_capital.update_layout(
873.         title=f"Realized Profit/Loss Over Time (Initial:
874.         ${initial_balance:,.0f}, Max DD: {max_dd_pct:.1f}%)",
875.         xaxis_title="Trading Day",
876.         yaxis_title="Profit/Loss ($)",
877.         height=300,
878.         margin=dict(l=10, r=10, t=40, b=10),
879.         plot_bgcolor="#0e1117",
880.         paper_bgcolor="#0e1117",
881.         font=dict(color="#00FF41"),
882.         showlegend=False
883.     )
884.
885.     st.plotly_chart(fig_capital, use_container_width=True)
886.
887.     st.markdown("#### 📊 Performance Metrics")
888.
889.     final_balance = balances[-1]
890.     final_realized = realized_balances[-1]
891.     total_return = (final_balance - initial_balance) / initial_balance
892.     realized_return = (final_realized - initial_balance) / initial_balance
893.
894.     peak = np.maximum.accumulate(realized_balances)
895.     drawdown_pct = (np.array(realized_balances) - peak) / peak * 100
896.     max_dd = np.min(drawdown_pct)
897.
898.     returns = np.diff(realized_balances) / realized_balances[:-1]
899.     sharpe_ratio = 0
900.     if len(returns) > 1 and np.std(returns) > 0:
901.         sharpe_ratio = np.mean(returns) / np.std(returns) * np.sqrt(252)
902.
903.     console_output = "\n"+"~"*60+"\n"
904.     console_output += f"DQN BACKTEST RESULTS - {model_name}"
905.     console_output += "\n"+"~"*60+"\n"
906.     console_output += "PORTFOLIO METRICS\n"
907.     console_output += "\n"+"~"*60+"\n"
908.     console_output += f"Initial Balance:      ${initial_balance:,.2f}\n"
909.     console_output += f"Final Balance:      ${final_balance:,.2f}\n"
910.     console_output += f"Realized Balance:    ${final_realized:,.2f}\n"
911.     console_output += f"Total Return:        {total_return*100:+.2f}%\n"
912.     console_output += f"Realized Return:     {realized_return*100:+.2f}%\n"
913.     console_output += f"Max Drawdown:        {max_dd:.2f}%\n"
914.     console_output += f"Sharpe Ratio:        {sharpe_ratio:.2f}\n"
915.
916.     console_output += "\n"+"~"*60+"\n"
917.     console_output += "TRADE STATISTICS\n"
918.     console_output += "\n"+"~"*60+"\n"
919.     console_output += f"Total Trades:        {trade_stats['total_trades']}\n"
920.     console_output += f"Winners:              {trade_stats['winners']}"
921.     console_output += f"({trade_stats['win_rate']*100:.1f}%) \n"
922.     console_output += f"Losers:               {trade_stats['losers']} ({(1-
923.     trade_stats['win_rate'])*100:.1f}%) \n"
924.
925.     if trade_stats['total_trades'] > 0:
926.         pf_str = f"{trade_stats['profit_factor']:.2f}" if not
927.         np.isinf(trade_stats['profit_factor']) else "∞"
928.         console_output += f"\nAvg
929.         Win:           ${trade_stats['avg_win']:.2f} ({trade_stats['avg_win_pct']*100:+.2f}%)\n"
930.         console_output += f"Avg Loss:           ${trade_stats['avg_loss']:.2f}
931.         ({trade_stats['avg_loss_pct']*100:.2f}%) \n"

```

```

926.                console_output += f"Profit Factor:        {pf_str}\n"
927.                console_output +=
f"Expectancy:      ${trade_stats['expectancy']:.2f}\n"
928.
929.                if 'buy_hold_return' in trade_stats:
930.                    console_output += "\n"+"\"*60+"\n"
931.                    console_output += "BUY & HOLD COMPARISON\n"
932.                    console_output += "\"*60+"\n"
933.                    console_output += f"B&H Final
Value:      ${trade_stats['buy_hold_final_value']:.2f}\n"
934.                    console_output += f"B&H
Return:      {trade_stats['buy_hold_return']*100:+.2f}%\n"
935.                    console_output += f"B&H
PnL:      ${trade_stats['buy_hold_pnl']:.2f}\n"
936.                    console_output += f"Outperformance:      {(realized_return -
trade_stats['buy_hold_return'])*100:+.2f}%\n"
937.                else:
938.                    console_output += "Error finding buy and hold stats."
939.
940.                console_output += "\n"+"\"*60+"\n"
941.                console_output += "ACTION DISTRIBUTION\n"
942.                console_output += "\"*60+"\n"
943.
944.                #get config for action types
945.                config, _, _ = load_dqn_model(model_name)
946.                action_counts = np.bincount(actions, minlength=len(config['action_types']))
947.                for i, (action_type, count) in enumerate(zip(config['action_types'],
action_counts)):
948.                    console_output += f"{action_type:20s}: {count:4d}
({count/len(actions)*100:5.1f}%)\n"
949.
950.                render_console_style(console_output, height=400)
951.

```

Testing

Functionality

After the completion of the system, individual parts were tested to ensure functionality and completion of all objectives. Considering the scale of this system, it would be impractical to record tests for every single element. While the entire system was thoroughly tested, not all tests are explicitly documented. Instead, a representative sample is shown.

Due to the modular nature of the system, individual scripts can easily be tested simply from running them directly. In some cases, modifications were made to scripts for printing out information necessary to validate tests.

Test Number	Objective(s)	Purpose	Methodology	Pass/Fail
1	1, 3.6, 4.1, 6.3, 7.1, 8.1, 8.2, 8.3, 8.4, 8.5	Verify that the complete ML research environment runs without runtime errors	Input: - Date range - Ticker - Feature parameters - LightGBM parameters - DQN parameters - Backtest period Expected result: - No errors - Visible dataset OHLC - Successfully trained models - Performance metrics - Generated predictions - Predictions for LIVE data Actual result: - All expected results met	Pass
2	2.1	Confirm API URLs are correctly constructed to make specific requests	Input: - Varying date ranges - Different categories for articles Expected result: - Articles only fall within date range - Articles are all within requested categories Actual result: - All expected results met	Pass
3	2.2	Ensure HTTP errors are handled	Input: - Intentionally modify URL constructor to make an invalid request Expected result: - Error 404 message Actual result: - All expected results met	Pass

4	2.3	Ensure relevant article filtering is functional	Input: - Target tag for articles Expected result: - Only articles related to target tag shown Actual result: - All expected results met	Pass
5	2.4	Confirm rate limiting logic slows down API requests by a random amount between a user-defined minimum and maximum	Input: - Minimum and maximum delay Expected result: - Time between new requests falls between this range Actual result: - All expected results met	Pass
6	2.5	Confirm OHLC data is retrieved correctly	Input: - Recent OHLC date range Expected result: - OHLC is accurate within one day of price action Actual result: - OHLC falls within the last day of price action	Pass
7	3.1	Confirm transformer model is correctly integrated	Input: - Sample headline Expected result: - Sentiment label probabilities for the headline Actual result: - Appropriate probabilities returned	Pass
8	3.2	Confirm discrete probabilities convert to continuous polarity score	Input: - Sample headline Expected result: - Continuous score with the correct polarity from discrete label Actual result: - Expected result met	Pass
9	3.3	Verify contextual biasing constrains sentiment interpretation	Input: - Headline with ambiguous sentiment based on context Expected result: - Appropriate score Actual result: - Expected result met	Pass
10	3.4	Confirm temperature scaling adjusts confidence	Input: - Sample headline - Varying temperatures Expected result: - Confidence reduces at higher temperatures Actual result: - Met expectations	Pass
11	3.5	Verify nonlinear scaling reshapes magnitude correctly	Input: - Polarity scores near zero and extremes Expected result:	Pass

			- Amplified near-zero scores and compressed extremes Actual result: - Met expectations	
12	3.6	Confirm parameters can be modified externally	Input: - Modified neutrality scaler in class parameters Expected result: - Higher neutrality scaler increases the effect of compression near extremes Actual result: - Met expectations	Pass
13	4.2	Validate technical indicator calculations	Input: - Known OHLC sample Expected result: - RSI matches manual calculations Actual result: - Met expectations	Pass
14	5.1	Confirm daily mean sentiment is aggregated correctly	Input: - Date range with multiple articles in a day - Manually extracted articles - Individual polarity scores Expected result: - Correct mean Actual result: - Met expectations	Pass
15	5.2	Ensure dataset is complete with all requested features	Input: - Requested features Expected result: - All columns are present Actual result: - Met expectations	Pass
16	5.3	Confirm target labels generated correctly	Input: - Known price percentage movement Expected result: - Correct movement within horizon Actual result: - Met expectations	Pass
17	6.3	Confirm multiple quantiles are predicted	Input: - Request for specific quantiles Expected result: - Lower bound < median < upper bound Actual result: - Met expectations	Pass
18	7.2	Confirm trading environment updates balance correctly	Input: - Known sequence of actions Expected result: - Balance matches manual calculations Actual result:	Pass

			- Python floating point math produced an error of 0.0000399% (negligible), otherwise met expectations	
19	7.3	Verify Sharpe Ratio calculation	Input: - Known returns series Expected result: - Matches manual calculations Actual result: - Met expectations	Pass
20	7.4	Confirm circular replay buffer overwrites oldest entries	Input: - Transitions exceeding the queue capacity Expected result: - Overwritten early entries Actual result: - Met expectations	Pass
21	4.1	Confirm parameters are defined and visible within the UI	Input: - Clicking on the Indicator Calculator expander Expected result: - Configurable parameters pre-defined for calculations Actual results: - Met expectations	Pass
22	6.1	Confirm UI exposes configurable LightGBM parameters	Input: - Clicking on the LightGBM config expander Expected result: - Exposed LightGBM training parameters Actual result: - Met expectations	Pass
23	6.2	Confirm conformal correction can be applied to predictions	Input: - LightGBM parameters with conformal enabled Expected result: - Improved quantile coverage - Potentially lower directional accuracy Actual result: - Conformal predictions were correctly applied - Quantile coverage improved - Directional accuracy significantly reduced	Pass

Test 1

Refer to testing video for evidence of completion. See timestamps below:

Objective	Timestamp
1	0:00-3:50
3.6, 4.1, 6.1	0:28-0:40
7.1	1:17-1:30
8.1	0:10-1:00
8.2	1:57-3:25
8.3	1:00-1:57
8.4	3:25-3:50
8.5	2:12-3:25

Test 2

Below is the testing code used to make the API request:

```
1. if __name__ == "__main__":
2.     start_date = datetime(2025, 1, 1)
3.     end_date = datetime(2025, 2, 1)
4.
5.     scraper = ArticleScraper(
6.         cookies_path= "training_resources\\fetching\\scraper\\cookies.json",
7.         target_tag="80",
8.         page_size=50,
9.         category="market-outlook::gold-and-precious-metals",
10.        min_delay=0,
11.        max_delay=1
12.    )
13.
14.    scraper.fetch_articles(start_date, end_date,
15.    "training_resources\\fetching\\scraper\\articles.csv")
```

As can be seen above, a request is made to pull articles from the start of 2025 up till the 1st of February 2025. The target category is gold and precious metals. The articles are then saved to a csv file, which will be inspected for the purpose of this test.

```
1. Date,Title,Sentiment
2. 2025-01-01T12:30:00-05:00,CFTC CoTs Report: Froth Leaves The Markets In A Healthy Consolidation,none
3. 2025-01-04T02:50:00-05:00,Gold's 2024 Performance Best In 14 Years,none
4. 2025-01-06T11:30:00-05:00,Just How Good Was Gold In 2024?,none
5. 2025-01-07T05:50:00-05:00,Central Bank Gold Statistics: November 2024,none
6. 2025-01-07T08:40:00-05:00,Gold Price Outlook: Bulls To Take Charge? U.S. Services PMI Ahead,none
7. 2025-01-07T08:50:00-05:00,Why Did COMEX And DXY Start Trading Together?,none
8. 2025-01-07T10:20:00-05:00,The Next Phase Of Gold's Bull Market Could Be Starting Soon (Technical Analysis),none
```

```

9. 2025-01-09T04:41:28-05:00,How To Earn 20%+ Annualized Yields On Gold And Silver ETFs In 2025,none
10. 2025-01-09T12:15:00-05:00,Gold Market Commentary: January Jitters,none
11. 2025-01-10T06:29:00-05:00,Gold ETF Flows: December 2024,none
12. 2025-01-14T21:41:00-05:00,Gold: This Is Why Gold Ignores Treasury Yields,none
13. 2025-01-14T23:58:49-05:00,Gold: An Unusual 2024 Unlikely To Be Repeated,none
14. 2025-01-15T12:32:00-05:00,Unusual Situation In Gold And Silver Market May Spark Big Price Moves,none
15. 2025-01-16T09:00:00-05:00,Gold Stocks Seek To Reconnect With Gold In 2025,none
16. 2025-01-16T10:25:00-05:00,Gold Surprisingly Resilient Despite Surging Dollar And Yields,none
17. 2025-01-18T05:30:00-05:00,Are Silver Markets Sleepwalking Into A Supply Shortage?,none
18. 2025-01-18T07:40:00-05:00,Gold's Entering Its Next Bull Market Phase,none
19. 2025-01-18T08:20:00-05:00,China's Gold Market Update: Seasonal Strength In December,none
20. 2025-01-20T07:05:00-05:00,Why Gold Will Continue To Shine In 2025 And Beyond,none
21. 2025-01-20T11:10:00-05:00,"Gold Price Steady On Inauguration Day, The Calm Before The Storm?",none
22. 2025-01-20T12:45:00-05:00,India Gold Market Update: Gold Outpaces Other Asset Classes,none
23. 2025-01-20T23:20:00-05:00,It's A Buyer's Market For Bullion,none
24. 2025-01-21T03:48:31-05:00,Buy Gold And Silver All The Way As Memecoins Abound,none
25. 2025-01-22T01:50:00-05:00,"Gold Gets Boost From Dollar Instability, Faltering Bond Prices",none
26. 2025-01-22T03:00:00-05:00,Trump's Tariff Threat Fuels Ongoing Gold And Silver Market Volatility,none
27. 2025-01-23T12:07:57-05:00,Gold: An Unwinding Of The Yen Carry Trade Could Boost Gold To New Highs,none
28. 2025-01-23T14:08:00-05:00,"Gold-Silver Ratio Over 90-1, Indicating Silver Is Historically Underpriced",none
29. 2025-01-26T10:00:00-05:00,Erosion Of U.S. Confidence And 3 Other Factors May Drive Gold To Record Highs,none
30. 2025-01-27T09:21:00-05:00,"Gold And Silver Technicals: Short-Term Caution, Medium-Term Bullish",none
31. 2025-01-28T05:53:00-05:00,China Continues Making Covert Gold Purchases In London,none
32. 2025-01-28T06:55:00-05:00,Markets Get Deep-Sixed By DeepSeek,none
33. 2025-01-28T10:27:00-05:00,These 3 Catalysts Could Fuel Precious Metals Markets,none
34. 2025-01-28T15:00:00-05:00,Gold Price Analysis: Bullish Bias Amid Tariff Uncertainty And FOMC,none
35. 2025-01-29T10:26:00-05:00,Is Someone Attacking The Comex? January Sees $5.2B In Gold Deliveries,none
36. 2025-01-30T06:40:00-05:00,Gold To Keep Lifting Australian Portfolios In 2025,none
37. 2025-01-30T08:05:00-05:00,Gold As A Strategic Asset: 2025 Edition,none
38. 2025-01-31T11:28:00-05:00,Gold And Silver Are Staging Powerful Breakouts (Technical Analysis),none
39.

```

Above is the resultant csv output. As can be seen above, only articles within the requested date range are pulled, and all articles are related to the requested category. Therefore, test 2 is a pass.

Test 3

For this test, the URL constructor will be intentionally modified to create a 404 error, by attempting to access a site that does not exist. The relevant code is seen below:

```

1.     def build_url(self, start_timestamp, end_timestamp, page, category=None):
2.
3.         #the default category is for gold and precious metals, but you can change it if
necessary
4.         if category is None:
5.             category = self.category
6.
7.         return (

```

```

8.         "https://[REDACTED].com/api/v3/articles?"
9.         "fields[article]=structuredInsights,publishOn,author,commentCount,title,primaryTi
ckers,secondaryTickers,summary,sentiments"
10.        f"&filter[category]={category}INTENTIONAL_ERROR_404"
11.        f"&filter[since]={start_timestamp}"
12.        f"&filter[until]={end_timestamp}"
13.        "&include=author,primaryTickers,secondaryTickers,sentiments"
14.        "&isMounting=false"
15.        f"&page[size]={self.page_size}"
16.        f"&page[number]={page}"
17.    )
18.

```

As seen on line 10, the URL has intentionally been modified to cause an error. While such an error should not occur within the range of normal function, if the API is discontinued, the 404 error will notify the user that the URL is no longer valid.

```

Starting article fetch...
Fetching pages:  0%|                                     | 0/1 [00:00<?, ?page/s]
Error 404: attempted to access a page that does not exist
Fetching pages:  0%|                                     | 0/1 [00:01<?, ?page/s]

```

As is seen in the console output, an error message was displayed as a result of attempting to access a non-existent page. Other HTTP error conditions were tested in a similar fashion. Therefore test 3 is a pass.

Test 4

For this test, the target tag will be modified to look for only articles related to bitcoin, which is tag 581328. Bitcoin was chosen for this test, as the category pulled includes all cryptocurrencies, so the tag filtering is helpful in discarding articles unrelated to bitcoin specifically.

```

1. if __name__ == "__main__":
2.     start_date = datetime(2025, 1, 1)
3.     end_date = datetime(2025, 2, 1)
4.
5.     scraper = ArticleScraper(
6.         cookies_path= "training_resources\\fetching\\scraper\\cookies.json",
7.         target_tag="581328",
8.         page_size=50,
9.         category="market-outlook::cryptocurrency",
10.         min_delay=0,
11.         max_delay=1
12.     )
13.
14.     scraper.fetch_articles(start_date, end_date,
15.         "training_resources\\fetching\\scraper\\articles.csv")

```

The output csv file is seen below:

```

1. Date,Title,Sentiment

```

2. 2025-01-02T06:00:00-05:00,Why The 60/40 Portfolio Needs Bitcoin In Today's Market,none
3. 2025-01-02T08:30:00-05:00,Bitcoin: One Indicator Signals Rocky Start To 2025,none
4. 2025-01-05T08:43:24-05:00,FBTC Offers Superior Bitcoin Exposure Versus Other ETFs,none
5. 2025-01-06T05:27:30-05:00,Bitcoin: Listen To The Market! An Unprecedented Year Ahead,none
6. 2025-01-06T08:46:59-05:00,Using MARA Holdings To Get Long Bitcoin In 2025,none
7. 2025-01-07T05:57:44-05:00,A Call To Be Cautiously Bullish In 2025: Key Concerns Around Bitcoin's Emerging Reversal Pattern,none
8. 2025-01-07T10:15:00-05:00,VanEck Crypto Monthly Recap For December 2024,none
9. 2025-01-07T12:11:12-05:00,Bitcoin: 2025 Will Be A Pivotal Year - Why You Should Be In,none
10. 2025-01-08T03:43:00-05:00,MicroStrategy Is A Coiled Spring About To Jump Higher,none
11. 2025-01-08T10:54:21-05:00,"Bitcoin: Torn Between Bull Runs And Bear Plunges, Extreme Volatility Ahead",none
12. 2025-01-08T23:59:00-05:00,CleanSpark: Stacking 'Digital Gold' The 'Old Way',none
13. 2025-01-09T23:58:37-05:00,BITB: A Way To Fund Much-Needed Development,none
14. 2025-01-10T08:00:00-05:00,One Year Of Bitcoin ETFs - And Why They Matter In 2025,none
15. 2025-01-10T22:58:27-05:00,YBIT: It Is An Avoid For Me,none
16. 2025-01-11T04:32:44-05:00,Metaplanet: Japan's MicroStrategy?,none
17. 2025-01-11T05:05:23-05:00,MARA Holdings: Should Shorts Be Worried? (Rating Upgrade),none
18. 2025-01-13T07:45:00-05:00,"IBIT: Income Strategy, 15% Annualized Yield From Bitcoin Options",none
19. 2025-01-13T07:59:54-05:00,Our Prediction For Bitcoin In 2025,none
20. 2025-01-15T14:40:00-05:00,Bitcoin Vs. Nasdaq: Is The Correlation Affecting Crypto Outflows?,none
21. 2025-01-16T04:40:31-05:00,MARA Holdings: Second-Largest Bitcoin Company Trades At Deep Value,none
22. 2025-01-16T11:30:00-05:00,Bitcoin Continues Breaking Minds And Models,none
23. 2025-01-16T22:57:00-05:00,BITO: On-Chain Signals And Seasonality Say Bitcoin's Rally Isn't Over,none
24. 2025-01-16T23:30:00-05:00,Why I Was Wrong About Ripple (Rating Upgrade),none
25. 2025-01-19T23:34:37-05:00,"Donald Trump Just Launched His 'Meme Coin', What Does It Mean For Bitcoin?",none
26. 2025-01-20T06:15:00-05:00,What Trump's Presidency Will Mean For Bitcoin,none
27. 2025-01-21T05:00:50-05:00,Trump-And-Dump Or Paradigm Shift? Why You Should Closely Watch Crypto,none
28. 2025-01-21T12:02:25-05:00,Bitcoin: Maturing Cycle Is Creating Downside Risk,none
29. 2025-01-22T07:38:53-05:00,The Difference Between Bitcoin And Gold ETF Flows In The First Year Is Incredible,none
30. 2025-01-22T19:30:00-05:00,Cryptocurrencies: Bitcoin Closes At New All-Time High,none
31. 2025-01-23T07:27:17-05:00,Tracking Trump's Bitcoin Promises,none
32. 2025-01-23T07:30:00-05:00,Bitcoin Update: It Still Looks Bullish (Technical Analysis),none
33. 2025-01-23T15:35:00-05:00,Bitcoin Volatility Measures Are Tightening Up - Is It Time For Another Major Move?,none
34. 2025-01-24T12:03:13-05:00,BITW: I'm Not A Fan Of Crypto Diversity,none
35. 2025-01-24T13:52:34-05:00,Breaking Down Trump's Bitcoin Executive Order,none
36. 2025-01-24T15:33:00-05:00,VanEck Mid-January Bitcoin ChainCheck,none
37. 2025-01-25T08:30:00-05:00,Bitcoin And 'Strategic Reserve' Progress,none
38. 2025-01-28T05:32:37-05:00,iShares Bitcoin Trust: Bitcoin Volatility Likely To Go Down Over Long-Term,none
39. 2025-01-29T22:35:00-05:00,Cryptocurrencies: Bitcoin Down 5% This Week,none
40. 2025-01-30T10:00:00-05:00,MicroStrategy: The Preferred Stock Is A Bitcoin-Per-Share Accretion Engine,none
41. 2025-02-01T04:00:00-05:00,Bitcoin In 2025: A Year That Could Redefine Crypto's Role,none
- 42.

As can be seen in the file above, only articles related to bitcoin are saved, therefore test 4 is a pass.

Test 5

For this test, minimum and maximum delay were modified along with the code for fetching articles to get a timestamp at the start and end of the article fetching loop, in order to determine the delay between each page accessed and print out the number to

the console. If the logic works correctly, only delays between `min_delay` and `max_delay` will be seen (which in this case has been set to 5 and 10 respectively). The timestamp start is placed after “`get_page`”, as to not include the time it takes to fetch a page within the delay. The modified code is seen below:

```

1.         while True:
2.             url = self.build_url(since, until, page)
3.             data = self.get_page(url)
4.
5.             #time after the page is pulled
6.             timestamp_start = time.time()
7.
8.             if not data:
9.                 break
10.
11.            articles = data.get("data", [])
12.            if not articles:
13.                break
14.
15.            sentiments = self.relevant_sentiments(data.get("included", []))
16.
17.            new_articles_this_page = 0
18.            page_dates = []
19.
20.            for i in range(len(articles)):
21.                article = articles[i]
22.                if self.has_relevant_tag(article):
23.                    attrs = article.get("attributes", {})
24.                    article_id = article.get("id")
25.                    title = attrs.get("title", "").strip()
26.                    publish_date_str = attrs.get("publishOn")
27.                    publish_date = pd.to_datetime(publish_date_str).date()
28.                    sentiment = sentiments.get(article_id, "none")
29.                    relevant_articles.append((publish_date_str, title, sentiment))
30.                    new_articles_this_page += 1
31.                    page_dates.append(publish_date)
32.
33.            #update dynamic days per page estimate
34.            #figures out the date range you cover with each page pull, and therefore how
many pages will be needed
35.            if page_dates:
36.                earliest = min(page_dates)
37.                latest = max(page_dates)
38.                days_covered = (latest - earliest).days + 1
39.                all_page_days.append(days_covered)
40.                avg_days_per_page = sum(all_page_days) / len(all_page_days)
41.            else:
42.                avg_days_per_page = 1 # fallback
43.
44.            #recalculate estimated total pages
45.            total_days = (end.date() - start.date()).days + 1
46.            estimated_total_pages = max(1, int(total_days / max(1, avg_days_per_page)))
47.            pbar.total = max(page, estimated_total_pages)
48.
49.            self.progress["total"] = max(page, estimated_total_pages)
50.
51.            pbar.refresh()
52.
53.            #advance page and progress bar
54.            self.progress["current"] = page
55.            page += 1
56.            pbar.update(1)
57.
58.            #delay between requests
59.            delay = random.uniform(min_delay, max_delay)
60.            time.sleep(delay)
61.

```

```

62.         timestamp_end = time.time()
63.
64.         total_time = timestamp_end - timestamp_start
65.         print(f"\nDelay: {total_time}")
66.
67.
68.     pbar.close()
69.     self.progress["state"] = "idle"
70.     # Sort articles by date using built-in sort
71.     relevant_articles.sort(key=lambda x: x[0])
72.
73.     if output_file is not None:
74.         with open(output_file, "w", newline="", encoding="utf-8") as f:
75.             writer = csv.writer(f)
76.             writer.writerow(["Date", "Title", "Sentiment"])
77.             for a in relevant_articles:
78.                 writer.writerow(a)
79.             print(f"\nSuccess! Saved {len(relevant_articles)} relevant articles to
{output_file}")
80.     else:
81.         return relevant_articles
82.
83. #for backend testing, this will run if the script is run without being called by another
84. if __name__ == "__main__":
85.     start_date = datetime(2025, 1, 1)
86.     end_date = datetime(2025, 6, 1)
87.
88.     scraper = ArticleScraper(
89.         cookies_path= "training_resources\\fetching\\scraper\\cookies.json",
90.         target_tag="581328",
91.         page_size=50,
92.         category="market-outlook::cryptocurrency",
93.         min_delay=5,
94.         max_delay=10
95.     )
96.
97.     scraper.fetch_articles(start_date, end_date,
"training_resources\\fetching\\scraper\\articles.csv")
98.

```

Below is the output console log:

```

Starting article fetch...
Fetching pages: 11% |██████████| 1/9 [00:01<00:13, 1.70s/page]
Delay: 6.552917718887329
Fetching pages: 25% |██████████| 2/8 [00:09<00:31, 5.28s/page]
Delay: 6.277202844619751
Fetching pages: 43% |██████████| 3/7 [00:17<00:25, 6.35s/page]
Delay: 6.163423776626587
Fetching pages: 57% |██████████| 4/7 [00:25<00:21, 7.27s/page]
Delay: 8.173284530639648
Fetching pages: 71% |██████████| 5/7 [00:36<00:16, 8.39s/page]
Delay: 8.58476448959082
Fetching pages: 86% |██████████| 6/7 [00:45<00:08, 8.87s/page]
Delay: 6.227253675460815
Fetching pages: 88% |██████████| 7/8 [00:53<00:08, 8.36s/page]
Delay: 7.889979839324951
Fetching pages: 100% |██████████| 8/8 [01:02<00:00, 8.59s/page]
Delay: 9.001180648803711
Fetching pages: 100% |██████████| 8/8 [01:12<00:00, 9.09s/page]
Success! Saved 204 relevant articles to training_resources\fetching\scraper\articles.csv

```

As can be seen above, delays only fall between 5 and 10 seconds, therefore a random delay exists between pulling pages, and test 5 is a pass.

Test 6

For this test, the price fetching script was modified to print out the most recent OHLC data. As daily candles are used, this will be yesterday's data, as the current candle will not have closed. The official yahoo finance website will still display today's candle however, as it updates LIVE. The API, however, does not, and updates daily. The relevant code is seen below:

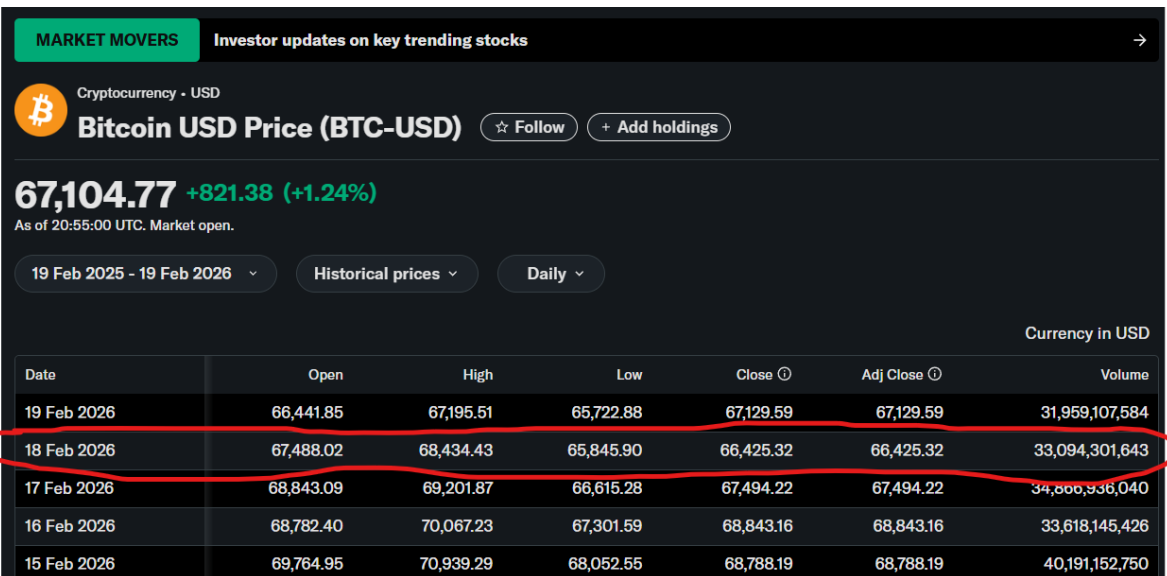
```
1. if __name__ == "__main__":
2.     csv_path = "training_resources\\processing\\indicator_calculator\\ohlcv_data.csv"
3.
4.     try:
5.         data = fetch_prices("BTC-USD", "2016-01-01", "today")
6.         print(data)
7.
8.         print(f"Today's date: {datetime.now().strftime('%Y-%m-%d')}")
9.         print("Below is the most recent completed candle, which will be from YESTERDAY")
10.        print(f"Recent Open: {data['Open'].iloc[-1]}")
11.        print(f"Recent High: {data['High'].iloc[-1]}")
12.        print(f"Recent Low: {data['Low'].iloc[-1]}")
13.        print(f"Recent Close: {data['Close'].iloc[-1]}")
14.
15.        save_csv = input("\nOutput CSV? (y/n): ").lower()
16.        if save_csv == "y":
17.            data.to_csv(csv_path)
18.            print(f"CSV saved at: {os.path.abspath(csv_path)}")
19.
20.    except Exception as e:
21.        print(f"Error: {e}")
22.
```

The console output is seen below:

```
[3702 rows x 5 columns]
Today's date: 2026-02-19
Below is the most recent completed candle, which will be from YESTERDAY
Recent Open: 67488.0234375
Recent High: 68434.4296875
Recent Low: 65845.8984375
Recent Close: 66425.3203125
```

As noted by the console log above, although the date of this report being written is the 19th of February 2026, the most recent entry will be from the 18th of February 2026.

The displayed OHLC data was cross referenced with yahoo finance's website:



Date	Open	High	Low	Close ①	Adj Close ①	Volume
19 Feb 2026	66,441.85	67,195.51	65,722.88	67,129.59	67,129.59	31,959,107,584
18 Feb 2026	67,488.02	68,434.43	65,845.90	66,425.32	66,425.32	33,094,301,643
17 Feb 2026	68,843.09	69,201.87	66,615.28	67,494.22	67,494.22	34,800,936,040
16 Feb 2026	68,782.40	70,067.23	67,301.59	68,843.16	68,843.16	33,618,145,426
15 Feb 2026	69,764.95	70,939.29	68,052.55	68,788.19	68,788.19	40,191,152,750

The OHLC data produced by the code matched yahoo finance, therefore test 6 is a pass.

Test 7

For this test, the sentiment analysis script was modified to print out the raw probabilities before any scaling occurs. The relevant code is seen below:

```

1. if __name__ == "__main__":
2.
3.     print("TESTING SENTIMENT ANALYSIS")
4.
5.     #parameters to mess with :D
6.     analyser = SentimentAnalyser(bias_prompt="Gold market outlook: ",
7.                                   temperature=1.6,
8.                                   neutrality_scaler=7.7,
9.                                   testing=True)
10.
11.     while True:
12.         headline = input("Enter a headline: ")
13.
14.         if headline.lower() == "quit":
15.             break
16.
17.         score, negative, neutral, positive = analyser.analyse_sentiment(headline)
18.         print(f"Score: {score:.4f}, Negative: {negative:.4f}, Neutral: {neutral:.4f},
19.               Positive: {positive:.4f}")

```

To test the sentiment analysis model, an example headline was used for positive, negative, and neutral sentiment.

TESTING SENTIMENT ANALYSIS

Enter a headline: Bullish Market for Gold

Score: 0.8546, Negative: 0.0119, Neutral: 0.0130, Positive: 0.9751

Enter a headline: See our latest report

Score: 0.0542, Negative: 0.0047, Neutral: 0.9787, Positive: 0.0166

Enter a headline: Bearish Market for Gold

Score: -0.8894, Negative: 0.9808, Neutral: 0.0119, Positive: 0.0073

As is seen above, article headlines are analysed, and appropriate probabilities are output for labelling. Therefore, test 7 is a pass.

Test 8

As is seen above in test 7, the final output scores have the same polarity as the combined raw probabilities. Therefore, test 8 is a pass.

Test 9

A sample headline was passed to FinRoBERTa, which could be a negative or positive sentiment depending on what market is being traded. The bias prompt is intended to give the model this context. Below are the logs with different bias prompts:

bias_prompt = "Gold market outlook:"

```
Enter a headline: US dollar declines, investors turn to Gold
Score: 0.3298, Negative: 0.0191, Neutral: 0.7782, Positive: 0.2027
```

bias_prompt = "US economy outlook:"

```
TESTING SENTIMENT ANALYSIS
Enter a headline: US dollar declines in value, investors turn to Gold
Score: -0.7361, Negative: 0.9201, Neutral: 0.0698, Positive: 0.0101
```

As can be seen above, the bias prompt can change the polarity of the sentiment entirely and does so appropriately based on the context. Therefore, test 9 is a pass.

Test 10

For this test, the temperature value was adjusted from 1 to 2, with all other factors remaining the same.

temperature = 1

```
Enter a headline: Bullish market
Score: 0.9870, Negative: 0.0008, Neutral: 0.0011, Positive: 0.9981
```

temperature = 2

```
Enter a headline: Bullish market
Score: 0.7436, Negative: 0.0272, Neutral: 0.0307, Positive: 0.9421
```

As can be seen above, the confidence level was reduced at higher temperatures, therefore test 10 is a pass.

Test 11

For this test, the code was modified to display the raw probability score alongside the scaled score. The relevant code is below:

```
1. if __name__ == "__main__":
2.
3.     print("TESTING SENTIMENT ANALYSIS")
4.
5.     #parameters to mess with :D
6.     analyser = SentimentAnalyser(bias_prompt="Gold market outlook: ",
7.                                   temperature=1.6,
8.                                   neutrality_scaler=7.7,
9.                                   testing=True)
10.
11.     while True:
12.         headline = input("Enter a headline: ")
13.
14.         if headline.lower() == "quit":
15.             break
16.
17.         score, negative, neutral, positive = analyser.analyse_sentiment(headline)
18.         print(f"Raw probability score: {(positive - negative):.4f}")
19.         print(f"Score: {score:.4f}, Negative: {negative:.4f}, Neutral: {neutral:.4f},
20.         Positive: {positive:.4f}")
```

This was then tested with headlines that have neutral, positive, and negative sentiment.

```

TESTING SENTIMENT ANALYSIS
Enter a headline: See our report
Raw probability score: 0.0061
Score: 0.0288, Negative: 0.0056, Neutral: 0.9827, Positive: 0.0117
Enter a headline: Bullish market
Raw probability score: 0.9633
Score: 0.8549, Negative: 0.0116, Neutral: 0.0135, Positive: 0.9749
Enter a headline: Bearish market
Raw probability score: -0.9731
Score: -0.8877, Negative: 0.9803, Neutral: 0.0124, Positive: 0.0072

```

As can be seen above, scores near zero are amplified, and scores approaching -1 or 1 are compressed. Therefore, test 11 is a pass.

Test 12

For this test, different values for the neutrality scaler were tested, with all other factors remaining the same. The parameters were directly modified from the testing section:

```

1. analyser = SentimentAnalyser(bias_prompt="Gold market outlook: ",
2.                               temperature=1.6,
3.                               neutrality_scaler=7.7,
4.                               testing=True)
5.

```

This is where the configuration for sentiment analysis can be managed, and have parameters massed through.

neutrality_scaler = 1

```

TESTING SENTIMENT ANALYSIS
Enter a headline: Bullish market
Raw probability score: 0.9633
Score: 0.9492, Negative: 0.0116, Neutral: 0.0135, Positive: 0.9749

```

neutrality_scaler = 7.7

```

TESTING SENTIMENT ANALYSIS
Enter a headline: Bullish market
Raw probability score: 0.9633
Score: 0.8549, Negative: 0.0116, Neutral: 0.0135, Positive: 0.9749

```

As can be seen above, the external modification of parameters when instantiating a SentimentAnalyser class caused the same headline to have its sentiment score compressed more with a higher neutrality scaler. Therefore, test 12 is a pass.

Test 13

For this test, I will manually verify that technical indicators are correctly calculated. For this test, I will manually calculate RSI according to Wilder's Smoothing, as a representative sample of other technical indicators.

The manual calculation for RSI is as follows:

Let a closing price on timestep "t" be defined as:

$$\mathbf{P_t}$$

Let the price change "delta" be defined as:

$$\Delta_t = \mathbf{P_t} - \mathbf{P_{t-1}}$$

Let gains and losses be defined as:

$$\mathbf{G_t} = \mathbf{max}(\Delta_t, 0)$$

$$\mathbf{L_t} = \mathbf{max}(-\Delta_t, 0)$$

For this test, a length of $\mathbf{n} = 4$ will be used for the brevity of manual calculations.

Then, average gains and losses are computed. Since this calculation requires an average gain and loss to already exist, the first instance will simply be the gain and loss.

$$\mathbf{AvgGain_t} = \alpha \cdot \mathbf{G_t} + (1 - \alpha) \cdot \mathbf{AvgGain_{t-1}}$$

$$\mathbf{AvgLoss_t} = \alpha \cdot \mathbf{L_t} + (1 - \alpha) \cdot \mathbf{AvgLoss_{t-1}}$$

Where α in this case is set to:

$$\alpha = \frac{1}{n}$$

This is equivalent to the exponential smoothing provided by NumPy.

Now, relative strength is calculated as:

$$\mathbf{RS_t} = \frac{\mathbf{AvgGain_t}}{\mathbf{AvgLoss_t}}$$

And finally, RSI is calculated as:

$$\mathbf{RSI_t} = 100 - \frac{100}{1 + \mathbf{RS_t}}$$

Now, I will cross-reference the output on row 5 for my programmatic RSI calculation, which in this case uses GLD closes from the 2nd of January 2020 till the 8th of January 2020 (as non-trading days are excluded). Row 5 is used as this is the point where sufficient historical data exists to correctly calculate RSI with exponential weighting.

The code used is seen below:

```
1. if __name__ == "__main__":
2.     csv_file = "training_resources\\processing\\indicator_calculator\\ohlc_data.csv"
3.     df = pd.read_csv(csv_file, index_col=0, parse_dates=True)
4.
5.     numeric_cols = ["Open", "High", "Low", "Close", "Adj Close"]
6.     for col in numeric_cols:
7.         if col in df.columns:
8.             df[col] = df[col].astype(float)
9.
10.    #testing with length 4
11.    calc = IndicatorCalculator(params={"rsi_length": [4]})
12.
13.    calc.load_dataframe(df)
14.    df_with_indicators = calc.add_rsi()
15.
16.    print(df_with_indicators)
17.
18.    df_with_indicators.to_csv(
19.        "training_resources\\processing\\indicator_calculator\\ohlc_data_with_indicators.csv",
20.        index=True
21.    )
22.    print("Indicators added and saved to 'ohlc_data_with_indicators.csv'")
23.
```

Where the “add RSI” function is defined as:

```
1. @requires_df
2. def add_rsi(self):
3.     for length in self.params["rsi_length"]:
4.         delta = self.df[self.price_col].diff()
5.         gain = delta.clip(lower=0)
6.         loss = -delta.clip(upper=0)
7.
8.         avg_gain = gain.ewm(alpha=1/length, adjust=False).mean()
9.         avg_loss = loss.ewm(alpha=1/length, adjust=False).mean()
10.
11.         rs = avg_gain / avg_loss
12.         self.df[f"RSI_{length}"] = 100 - (100 / (1 + rs))
13.     return self.df
14.
```

The csv output for the 8th of January was:

```

1. Date,Open,High,Low,Close,Adj Close,RSI_4
2. 2020-01-02,143.86000061035156,144.2100067138672,143.39999389648438,143.9499969482422,143.9499969482422,
3. 2020-01-03,145.75,146.32000732421875,145.39999389648438,145.86000061035156,145.86000061035156,100.0
4. 2020-01-06,148.44000244140625,148.47999572753906,146.9499969482422,147.38999938964844,147.38999938964844,
100.0
5. 2020-01-07,147.57000732421875,148.13999938964844,147.42999267578125,147.97000122070312,147.97000122070312,
100.0
6. 2020-01-08,148.49000549316406,148.61000061035156,146.13999938964844,146.86000061035156,146.86000061035156,
80.27982880901632
7.

```

Using columns for Date, Open, High, Low, Close, Adjusted Close, and RSI respectively.

It should be noted that the initial values for RSI are 100 due to the absence of losses, making the Relative Strength infinite, and therefore the RSI will be 100.

As seen above, the RSI calculated for the 8th of January is **80.28**.

This will be verified via manual calculations.

Step 1: Find Close Prices

Row	Date	Close
1	2020-01-02	143.9499969482422
2	2020-01-03	145.86000061035156
3	2020-01-06	147.38999938964844
4	2020-01-07	147.97000122070312
5	2020-01-08	146.86000061035156

Step 2: Compute Δ

Date	Δ
2020-01-02	N/A
2020-01-03	145.86 – 143.95 = 1.91
2020-01-06	147.39 – 145.86 = 1.53
2020-01-07	147.97 – 147.39 = 0.58
2020-01-08	146.86 – 147.97 = -1.11

Step 3: Separate Gains and Losses

Date	Gain	Loss
2020-01-03	1.91	0
2020-01-06	1.53	0
2020-01-07	0.58	0
2020-01-08	0	1.11

Step 4: Exponential Smoothing

As $\alpha = \frac{1}{n}$, we know that $\alpha = 0.25$

To start exponential smoothing, we will use the first gain and loss at row 2 as the seed.

$$AvgGain_2 = 1.91$$

$$AvgLoss_2 = 0$$

Now, the rest of the exponential smoothing can be calculated

Row 3:

$$AvgGain_3 = 0.25 \cdot 1.53 + 0.75 \cdot 1.91 = 0.3825 + 1.4325 = 1.8150$$

$$AvgLoss_3 = 0.25 \cdot 0 + 0.75 \cdot 0 = 0$$

Row 4:

$$AvgGain_4 = 0.25 \cdot 0.58 + 0.75 \cdot 1.815 = 0.145 + 1.36125 = 1.50625$$

$$AvgLoss_4 = 0.25 \cdot 1.11 + 0.75 \cdot 0 = 0.2775$$

Step 5: Calculate Relative Strength

$$RS = \frac{AvgGain}{AvgLoss} = \frac{1.1297}{0.2775} = 4.072$$

Step 6: Calculate RSI

$$RSI = 100 - \frac{100}{1 + RS} = 100 - \frac{100}{5.072} = 80.28$$

As seen above, this matches the programmatic calculation of RSI, therefore test 13 is a pass.

Test 14

For this test, the “compute daily sentiment” function was modified to randomly sample a date with articles and print out their scores, to determine if the mean was calculated correctly. The relevant code is seen below:

```
1.     def compute_daily_sentiment(self):
2.         print("Computing daily sentiment...")
3.
4.         articles_by_date = {}
5.         for date_str, title, true_sentiment in self.articles:
6.             date = pd.to_datetime(date_str).date()
7.             articles_by_date.setdefault(date, []).append((title, true_sentiment))
8.
9.         all_tasks = []
10.        task_dates = []
11.        for date, articles in articles_by_date.items():
12.            for title, ts in articles:
13.                all_tasks.append((title, ts, self.true_sentiment_weight))
14.                task_dates.append(date)
15.
16.        #seems to be a sweet spot for speed from testing
17.        chunksize = 8
18.
19.        self.progress["state"], self.progress["current"], self.progress["total"] =
"Computing sentiment...", 0, len(all_tasks)
20.        scores = []
21.
22.
23.
24.        #parallel processing because this is terribly slow
25.        print(f"Processing {len(all_tasks)} articles across {self.max_workers} workers...")
26.        with Pool(
27.            processes=self.max_workers,
28.            initializer=create_sentiment_worker,
29.            initargs=(self.sentiment_params,)
30.        ) as pool:
31.
32.            for score in tqdm(
33.                pool.imap(compute_single_sentiment, all_tasks, chunksize=chunksize),
34.                total=len(all_tasks),
35.                desc="Sentiment",
36.                unit="article"
37.            ):
38.                scores.append(score)
39.                self.progress["current"] += 1
40.
41.        self.progress["state"] = "idle"
42.
43.        #create a dictionary of scores for each date
44.        score_by_date = {}
45.
46.        #test 14 stuff
47.        debug_articles = {}
48.
49.        for i in range(len(scores)):
50.            date = task_dates[i]
51.            score = scores[i]
52.            title = all_tasks[i][0]
53.
54.            if date not in score_by_date:
55.                score_by_date[date] = []
56.            score_by_date[date].append(score)
57.
```

```

58.         if date not in debug_articles:
59.             debug_articles[date] = []
60.             debug_articles[date].append((title, score))
61.
62.     #end
63.
64.
65.     for i in range(len(task_dates)):
66.         date = task_dates[i]
67.         score = scores[i]
68.
69.         if date not in score_by_date:
70.             score_by_date[date] = []
71.
72.         score_by_date[date].append(score)
73.
74.     all_dates = pd.date_range(self.start_date - timedelta(days=self.warmup_days),
self.end_date)
75.     sentiment_series = pd.Series(index=all_dates, dtype=float)
76.     for date in all_dates:
77.         day_scores = score_by_date.get(date.date(), [])
78.         sentiment_series[date] = np.mean(day_scores) if day_scores else np.nan
79.
80.     #randomly select a date to test
81.     if debug_articles:
82.         sample_date = random.choice(list(debug_articles.keys()))
83.         sample_entries = debug_articles[sample_date]
84.         print(f"\n[TEST 14 DEBUG] Sample date: {sample_date}")
85.         for title, score in sample_entries:
86.             print(f"  Score: {score:.4f} | {title}")
87.
88.
89.     #in case the user inputs dumb stuff from the UI
90.     if hasattr(self, "sentiment_rolling_window") and self.sentiment_rolling_window > 1:
91.         sentiment_series =
sentiment_series.rolling(window=self.sentiment_rolling_window, min_periods=1).mean()
92.
93.         self.daily_sentiment = self.apply_decay(sentiment_series)
94.

```

The result of running this on Gold in early 2020 yielded this random sample:

```

[TEST 14 DEBUG] 2020-02-28
Score: 0.6482 | The Ecstasy Of Gold Continues As Concerns Over Coronavirus Continue To Mount
Score: 0.3311 | A Diversified Approach To Precious Metals Investing
Score: -0.8703 | Gold Set To Test $1,700 Resistance As U.S. Yields Fall To Record Lows
Calculating technical indicators...

```

The mean of this sample is:

$$\frac{0.648 + 0.331 - 0.870}{3} = 0.0363$$

Looking at the csv dataset:

	AP	AQ	AR	AS	AT	AU	AV	AW		
1	Price_Slope_20d	MACD_Slope_20d	MACD_Slope_Div_20d	RSI_Slope_Div_20d	PSY_12	CCI_20	CMO_14	Sentiment	Pi	
2	0.24199981689453126	0.08591771990053303	0.15608209699399822	-1.0091381451435442	75.0	166.52210987709566	81.25964453866443	-0.24885913154882366	0.	
3	0.3470001220703125	0.09424080054727427	0.25275932152303826	-1.2111976221958822	83.33333333333334	193.5833107876121	97.63439609729494	0.10774760619112239	1.	
41	2020-02-28	152.7100067138672	152.77000427246094	147.0500030517578	148.3800048828125	148.3800048828125	45.776522426372026	1.7246059684797217	1.8023792430916676	-0.077773;

	AP	AQ	AR	AS	AT	AU	AV	AW	
22	0.269000244140625	0.029638390922048074	0.23936185321857695	0.5766817871162673	83.33333333333334	156.4130506829516	30.325807827112857	-0.6324960325654227	-0.
23	0.125	0.012419692524912308	0.1125803074750877	1.0504469614011995	75.0	74.58943972070175	32.315433700954934	-0.36232305607624404	1.
24	-0.048000335693359375	-0.01501287222929193	-0.032987463464067446	1.5963076349964982	75.0	-56.03215918285612	7.660351489074016	0.1875264872483908	-0.
25	-0.06800003051757812	-0.03792995278076603	-0.03007007773681209	1.5793170403604335	75.0	-44.1491890124204	0.7787226097730706	0.1809753537393378	-1.
26	0.02699964306640626	-0.047874357012827315	0.07487402131946794	1.0144769035912216	75.0	14.97675491304004	11.413552335149472	0.36041981997439504	-1.
27	0.08799972534179687	-0.05025387845664966	0.13825360379844653	0.6762690300789587	75.0	40.14954356360934	12.512820836502382	0.15844392763684567	-0.
28	0.06299972534179688	-0.053539784106861533	0.1165395094486584	0.6960095328961478	75.0	63.29115321152066	14.458954072501513	0.4319972216903529	0.
29	0.09199981689453125	-0.05319067400402559	0.14519049089855685	0.517562018209156	66.66666666666666	17.483278117335296	8.405880002771639	0.4319972216903529	1.
30	0.09249954223632813	-0.052271129035744934	0.14477067127207305	0.5145555857980462	58.33333333333333	1.90252087259707	4.141981560093719	0.0033281855027455136	0.
31	0.09200057983398438	-0.05099480870766939	0.14299538854165378	0.45692035758320804	66.66666666666666	64.94974756593963	3.9526436649895134	0.009035356924757033	-0.
32	0.1345001220703125	-0.046023428343940506	0.180523550414253	0.2804253289753721	66.66666666666666	110.85579172715735	0.10271818990162578	-0.13768776635962893	0.
33	0.21650009155273436	-0.035062680597759764	0.2515627721504941	0.06413827919162499	66.66666666666666	215.16965991706726	31.522707561885987	-0.17410411069943674	0.
34	0.2524993896484375	-0.023203886582746237	0.27570327623118374	0.002035757676675576	66.66666666666666	245.12660047289742	32.04987370028785	-0.6449778292776112	2.
35	0.28100051879882815	-0.011374782546646145	0.2923753013454743	-0.043375319548698554	75.0	240.79706723067287	35.818104641710754	-0.3951889971311948	2.
36	0.3790000915527344	0.005840290892736277	0.3731598006599981	-0.13025937239469343	83.33333333333334	272.2400781612535	43.201810919036504	0.8609211130059372	2.
37	0.4055000305175781	0.021375283266736746	0.38412474725084134	-0.058332323509019124	83.33333333333334	254.44205146300877	60.1555087699299	0.35675994037804754	2.
38	0.2154998779296875	0.01814627328759997	0.19735360464208754	0.7110404965333579	75.0	134.49161720128308	50.109460333289185	0.009741315830792472	2.
39	0.3154998779296875	0.02338683500437213	0.29211304292531537	0.1911238557826408	75.0	112.31073659054576	51.831001352874615	-0.28679918917650143	0.
40	0.2769996430664064	0.024154929450666884	0.25284473485597375	0.29830897157898306	75.0	111.42755178869051	49.10717934977588	-0.32907193322064054	-0.
41	-0.00449981689453125	0.0020256439272927198	-0.006525460821823969	1.0885238547618954	75.0	-19.931280467307523	3.16021239713393	0.03633333333333333	-1.

As can be seen above, the calculated sentiment matches the saved sentiment in the table, therefore test 14 is a pass.

Test 15

For this test, the features of the dataset will be compared against what has been requested in the code.

The relevant code making the request is seen below:

```

1. if __name__ == "__main__":
2.
3.     generator_config = {
4.         'ticker': 'BTC-USD',
5.         'start_date': datetime(2017,1,1),
6.         'end_date': datetime(2026, 1, 16),
7.         'max_workers': 8,
8.         'pct_change_features': [3, 7, 14, 30, 90, 365],
9.         'pct_change_targets': [3,7, 14, 30, 90, 365],
10.        'return_targets': [3,7, 14, 30, 90, 365],
11.        'vol_targets': [3,7, 14, 30, 90, 365],
12.        'ema_targets': [3,7, 14, 30, 90, 365],
13.        'normalised_return_targets': [3,7,9,14, 30, 90, 365]
14.    }
15.
16.    scraper_config = {
17.        'cookies_path': "training_resources\\fetching\\scraper\\cookies.json",
18.        'target_tag': '581328',
19.        'page_size': 100,
20.        'category': "market-outlook::cryptocurrency",
21.        'min_delay': 0,
22.        'max_delay': 1,
23.    }
24.
25.    sentiment_calculations = {
26.        'sentiment_params': {
27.            'bias_prompt': "Bitcoin market outlook: ",

```

```

28.         'temperature': 1.6,
29.         'neutrality_scaler': 7.7,
30.         'testing': False
31.     },
32.     'true_sentiment_weight': 0.7,
33.     'rolling_window': 3,
34.     'decay': 0.7,
35.     'lags': [3, 7, 14, 28]
36. }
37.
38. generator = DatasetGenerator(
39.     generator_config=generator_config,
40.     scraper_config=scraper_config,
41.     sentiment_calculations=sentiment_calculations,
42.     calculator_params=None
43. )
44.
45. dataset = generator.generate_dataset()
46.
47. #experimented according to the paper for LightGBM assets forecasting (Bisdoulis, 2024),
see References
48. lgbm_config = {
49.     'lgbm': {
50.         'prediction_mode': 'quantile',
51.         'quantiles': [0.1, 0.5, 0.9],
52.         'conformal': False,
53.         'lgbm_params': {
54.             'n_estimators': 250, #paper used 500-2200
55.             'learning_rate': 0.008, #lower LR for better convergence
56.             'max_depth': 12, #paper searched -1 to 30
57.             'num_leaves': 84, #paper searched 10-80
58.             'min_child_samples': 30, #increase for stability
59.             'subsample': 0.85,
60.             'colsample_bytree': 0.7,
61.             'reg_alpha': 0.0005, #L1 regularization
62.             'reg_lambda': 2.0, #L2 regularization (paper used 1e-6 to 10)
63.             'max_bin': 125, #paper searched 125-750
64.             'boosting_type': 'gbdt',
65.             'random_state': 42,
66.             'n_jobs': -1,
67.             'verbose': -1
68.         },
69.     },
70.     'target': 'pct',
71.     'horizons': [365, 90, 30, 14, 7], #in this order for cascading predictions
72.     'train_min': 500,
73.     'train_max': 1000,
74.     'retrain': 100,
75.     'conformal_split': 0.15,
76.     'conformal_retrain': False
77. }
78.
79. generator.add_lgbm_predictions(config=lgbm_config)
80.
81. #save
82. path = r"training_resources\datasets\gen_test.csv"
83. generator.save_dataset(path)
84. print("\nDataset generated and saved.")
85.
86. #plot
87. generator.plot_price_and_sentiment(title="BTC Price and Sentiment")
88.

```

As “None” is passed into the indicator calculator, defaults will be used:

```
1. DEFAULT_PARAMS = {  
2.     "rsi_length": [14],  
3.     "macd": {"fast": 12, "slow": 26, "signal": 9},  
4.     "bollinger": {"length": 20, "std": 2},  
5.     "stochastic_osc": {"k": 14, "d": 3, "smooth_k": 3},  
6.     "atr_length": [14],  
7.     "volatility_windows": [5, 10, 20, 30],  
8.     "ema_lengths": [10, 14, 20, 50],  
9.     "slope_windows": [5, 10, 14, 20],  
10.    "psy_length": [12],  
11.    "cci_length": [20],  
12.    "cmo_length": [14],  
13.    "gap_features": True  
14. }  
15.
```

So based on the code above, the requested columns are:

Price

Open, High, Low, Close, Adj Close

Percentage Change Features

*Pct_Change_3d, Pct_Change_7d, Pct_Change_14d, Pct_Change_30d, Pct_Change_90d,
Pct_Change_365d*

Sentiment Features

*Sentiment, Sentiment_lag_3, Sentiment_lag_7, Sentiment_lag_14, Sentiment_lag_28,
Sentiment_Rolling*

Technical Features

*RSI_14, MACD, MACD_Signal, MACD_Hist, BB_Mid, BB_Upper, BB_Lower, %K, %D, ATR_14,
Vol_5d, Vol_10d, Vol_20d, Vol_30d, Vol_Ratio_5_20, Gap, Gap_EMA_Norm, Gap_2d, PSY_12,
CCI_20, CMO_14, EMA_Ratio_10_14, EMA_Ratio_14_20, EMA_Ratio_20_50,
Price_EMA10_Ratio, ATR_EMA10_Ratio, Price_Slope_5d, MACD_Slope_5d,
MACD_Slope_Div_5d, RSI_Slope_Div_5d, Price_Slope_10d, MACD_Slope_10d,
MACD_Slope_Div_10d, RSI_Slope_Div_10d, Price_Slope_14d, MACD_Slope_14d,
MACD_Slope_Div_14d, RSI_Slope_Div_14d, Price_Slope_20d, MACD_Slope_20d,
MACD_Slope_Div_20d, RSI_Slope_Div_20d*

Targets

*Future_Pct_Change_3d, Future_Pct_Change_7d, Future_Pct_Change_14d,
Future_Pct_Change_30d, Future_Pct_Change_90d, Future_Pct_Change_365d,
Future_Return_3d, Future_Return_7d, Future_Return_14d, Future_Return_30d,
Future_Return_90d, Future_Return_365d, Target_VolAdjusted_3d, Target_VolAdjusted_7d,*

*Target_VolAdjusted_14d, Target_VolAdjusted_30d, Target_VolAdjusted_90d,
 Target_VolAdjusted_365d, Target_EMA_Position_3d, Target_EMA_Position_7d,
 Target_EMA_Position_14d, Target_EMA_Position_30d, Target_EMA_Position_90d,
 Target_EMA_Position_365d, Target_Normalised_Return_3d, Target_Normalised_Return_7d,
 Target_Normalised_Return_9d, Target_Normalised_Return_14d,
 Target_Normalised_Return_30d, Target_Normalised_Return_90d,
 Target_Normalised_Return_365d*

LightGBM Predictions

*LGBM_365d_q10, LGBM_365d_q50, LGBM_365d_q90, LGBM_90d_q10, LGBM_90d_q50,
 LGBM_90d_q90, LGBM_30d_q10, LGBM_30d_q50, LGBM_30d_q90, LGBM_14d_q10,
 LGBM_14d_q50, LGBM_14d_q90, LGBM_7d_q10, LGBM_7d_q50, LGBM_7d_q90*

The real columns in the csv dataset are seen below:

```

1. Date,Open,High,Low,Close,Adj
Close,RSI_14,MACD,MACD_Signal,MACD_Hist,BB_Mid,BB_Upper,BB_Lower,%K,%D,ATR_14,Vol_5d,Vol_10d,Vol
_20d,Vol_30d,Vol_Ratio_5_20,Gap,Gap_EMA_Norm,Gap_2d,EMA_Ratio_10_14,EMA_Ratio_14_20,EMA_Ratio_20
_50,Price_EMA10_Ratio,ATR_EMA10_Ratio,Price_Slope_5d,MACD_Slope_5d,MACD_Slope_Div_5d,RSI_Slope_D
iv_5d,Price_Slope_10d,MACD_Slope_10d,MACD_Slope_Div_10d,RSI_Slope_Div_10d,Price_Slope_14d,MACD_S
lope_14d,MACD_Slope_Div_14d,RSI_Slope_Div_14d,Price_Slope_20d,MACD_Slope_20d,MACD_Slope_Div_20d,
RSI_Slope_Div_20d,PSY_12,CCI_20,CMO_14,Sentiment,Pct_Change_3d,Pct_Change_7d,Pct_Change_14d,Pct
_Change_30d,Pct_Change_90d,Pct_Change_365d,Sentiment_lag_3,Sentiment_lag_7,Sentiment_lag_14,Senti
ment_lag_28,Sentiment_Rolling,Future_Pct_Change_3d,Future_Pct_Change_7d,Future_Pct_Change_14d,Fu
ture_Pct_Change_30d,Future_Pct_Change_90d,Future_Pct_Change_365d,Future_Return_3d,Future_Return
_7d,Future_Return_14d,Future_Return_30d,Future_Return_90d,Future_Return_365d,Target_VolAdjusted_3
d,Target_VolAdjusted_7d,Target_VolAdjusted_14d,Target_VolAdjusted_30d,Target_VolAdjusted_90d,Tar
get_VolAdjusted_365d,Target_EMA_Position_3d,Target_EMA_Position_7d,Target_EMA_Position_14d,Targe
t_EMA_Position_30d,Target_EMA_Position_90d,Target_EMA_Position_365d,Target_Normalised_Return_3d,
Target_Normalised_Return_7d,Target_Normalised_Return_9d,Target_Normalised_Return_14d,Target_Norm
alised_Return_30d,Target_Normalised_Return_90d,Target_Normalised_Return_365d
2.

```

As it would be quite time consuming to manually confirm all these exist, a regex string match was used instead, with an OR operator between every column predicted to show up. To pass this test, everything should be highlighted, and nothing should be left blank.

The following regex was used. Wherever there is a column name that can be found within other column names (i.e. Sentiment and Sentiment_Lag), the match was put at the end of the pattern matching string:

```

1. /Open|High|Low|Close|Adj
Close|Pct_Change_3d|Pct_Change_7d|Pct_Change_14d|Pct_Change_30d|Pct_Change_90d|Pct_Change_365d|S
entiment_lag_3|Sentiment_lag_7|Sentiment_lag_14|Sentiment_lag_28|Sentiment_Rolling|Sentiment|RSI
_14|MACD_Signal|MACD_Hist|BB_Mid|BB_Upper|BB_Lower|%K|%D|ATR_14|Vol_5d|Vol_10d|Vol_20d|Vol_30d|V
ol_Ratio_5_20|Gap_EMA_Norm|Gap_2d|PSY_12|CCI_20|CMO_14|EMA_Ratio_10_14|EMA_Ratio_14_20|EMA_Ratio
_20_50|Price_EMA10_Ratio|ATR_EMA10_Ratio|Price_Slope_5d|MACD_Slope_5d|MACD_Slope_Div_5d|RSI_Slop

```

e_Div_5d|Price_Slope_10d|MACD_Slope_10d|MACD_Slope_Div_10d|RSI_Slope_Div_10d|Price_Slope_14d|MACD_Slope_14d|MACD_Slope_Div_14d|RSI_Slope_Div_14d|Price_Slope_20d|MACD_Slope_20d|MACD_Slope_Div_20d|RSI_Slope_Div_20d|Future_Pct_Change_3d|Future_Pct_Change_7d|Future_Pct_Change_14d|Future_Pct_Change_30d|Future_Pct_Change_90d|Future_Pct_Change_365d|Future_Return_3d|Future_Return_7d|Future_Return_14d|Future_Return_30d|Future_Return_90d|Future_Return_365d|Target_VolAdjusted_3d|Target_VolAdjusted_7d|Target_VolAdjusted_14d|Target_VolAdjusted_30d|Target_VolAdjusted_90d|Target_VolAdjusted_365d|Target_EMA_Position_3d|Target_EMA_Position_7d|Target_EMA_Position_14d|Target_EMA_Position_30d|Target_EMA_Position_90d|Target_EMA_Position_365d|Target_Normalised_Return_3d|Target_Normalised_Return_7d|Target_Normalised_Return_9d|Target_Normalised_Return_14d|Target_Normalised_Return_30d|Target_Normalised_Return_90d|Target_Normalised_Return_365d|LGBM_365d_q10|LGBM_365d_q50|LGBM_365d_q90|LGBM_90d_q10|LGBM_90d_q50|LGBM_90d_q90|LGBM_30d_q10|LGBM_30d_q50|LGBM_30d_q90|LGBM_14d_q10|LGBM_14d_q50|LGBM_14d_q90|LGBM_7d_q10|LGBM_7d_q50|LGBM_7d_q90|MACD|Gap|gm

2.

TEST STRING

Date,Open,High,Low,Close,Adj
Close,RSI_14,MACD,MACD_Signal,MACD_Hist,BB_Mid,BB_Upper,BB_Lower,%K,%D,ATR_14,Vol_5d,Vol_10d,Vol_20d,Vol_30d,Vol_Ratio_5_20,Gap,Gap_EMA_Norm,Gap_2d,EMA_Ratio_10_14,EMA_Ratio_14_20,EMA_Ratio_20_50,Price_EMA10_Ratio,ATR_EMA10_Ratio,Price_Slope_5d,MACD_Slope_5d,MACD_Slope_Div_5d,RSI_Slope_Div_5d,Price_Slope_10d,MACD_Slope_10d,MACD_Slope_Div_10d,RSI_Slope_Div_10d,Price_Slope_14d,MACD_Slope_14d,MACD_Slope_Div_14d,RSI_Slope_Div_14d,Price_Slope_20d,MACD_Slope_20d,MACD_Slope_Div_20d,RSI_Slope_Div_20d,PSY_12,CCI_20,CMO_14,Sentiment,Pct_Change_3d,Pct_Change_7d,Pct_Change_14d,Pct_Change_30d,Pct_Change_90d,Pct_Change_365d,Sentiment_lag_3,Sentiment_lag_7,Sentiment_lag_14,Sentiment_lag_28,Sentiment_Rolling,Future_Pct_Change_3d,Future_Pct_Change_7d,Future_Pct_Change_14d,Future_Pct_Change_30d,Future_Pct_Change_90d,Future_Pct_Change_365d,Future_Return_3d,Future_Return_7d,Future_Return_14d,Future_Return_30d,Future_Return_90d,Future_Return_365d,Target_VolAdjusted_3d,Target_VolAdjusted_7d,Target_VolAdjusted_14d,Target_VolAdjusted_30d,Target_VolAdjusted_90d,Target_VolAdjusted_365d,Target_EMA_Position_3d,Target_EMA_Position_7d,Target_EMA_Position_14d,Target_EMA_Position_30d,Target_EMA_Position_90d,Target_EMA_Position_365d,Target_Normalised_Return_3d,Target_Normalised_Return_7d,Target_Normalised_Return_9d,Target_Normalised_Return_14d,Target_Normalised_Return_30d,Target_Normalised_Return_90d,Target_Normalised_Return_365d

As can be seen above, everything is highlighted apart from the Date index, as this is not a feature. Therefore, test 15 is a pass.

Test 16

For this test, I will compare a target percentage change to the actual percentage change in price for that horizon.

	BE	BF	BG	BH	BI	BJ	BK	BL
1	Sentiment_lag_7	Sentiment_lag_14	Sentiment_lag_28	Sentiment_Rolling	Future_Pct_Change_3d	Future_Pct_Change_7d	Future_Pct_Change_14d	Future_Pct_Change_30d
2	0.00023332205737958134	-0.21530127653925926	-0.4935886171957944	-0.11987264003100982	2.792639359281357	1.2990694099485944	2.202152300208171	3.50816475082911
3	0.00023332205737958134	0.10769428615297232	-0.384129130706297	-0.12999021896884136	0.6855889180141794	-0.1165488607116072	1.4534451585879404	3.46222612822300
4	0.2884112920041693	0.2676062678019067	-0.022634880204648804	0.33885903701949976	-0.9227224479160134	-0.5767054121959103	1.0855594749585196	2.98527302714231
5	0.138100343004617	-0.025356110493009898	-0.022889582159241046	0.20275397473811244	-0.7163597687701029	-1.1218514890956932	-0.20950027440452823	3.00060985657748
6	-0.24885913154882366	-0.8916875593326116	-0.014101306388200749	0.08270202856887025	-0.7081528542902005	-0.1906569373098188	1.0894771189336572	5.33841502472254
7	-0.24885913154882366	0.00023332205737958134	0.2897043591892489	0.018478024535074595	-0.23282636494743603	0.4862060671110946	1.6708912290644973	6.88899379763603
8	0.10774760619112239	0.00023332205737958134	0.18362373127036027	0.05131381845737665	-0.25186193366898424	-0.08168972363010729	1.647265746797788	4.34960127313408
9	0.24380030860257043	0.2884112920041693	0.37740301339745064	-0.13042073819843042	0.33602400889032324	0.8915016648424201	1.741868851018058	5.58907796401595
10	0.004410656241064903	0.138100343004617	-0.14685491470067652	-0.03691933282037916	0.610885698904684	1.5718259645536081	0.5079210803586864	5.70389005377076

As is seen above, for row 2, the predicted change in price between row 2 and 5 is 2.793%. This will be compared to the true %change.

	A	B	C	D	E	F	G	H	I	
1	Date	Open	High	Low	Close	Adj Close	RSI_14	MACD	MACD_Signal	MACD_His
2	2020-01-02	143.86000061035156	144.2100067138672	143.39999389648438	143.9499969482422	143.9499969482422	76.87147924541247	1.122243671987576	0.5956371073727326	0.5266065
3	2020-01-03	145.75	146.32000732421875	145.39999389648438	145.86000061035156	145.86000061035156	81.89705505382537	1.393918152735523	0.7552933164452907	0.6386248
4	2020-01-06	148.44000244140625	148.47999572753906	146.9499969482422	147.38999938964844	147.38999938964844	84.75474015206504	1.7129346069094424	0.9468215745381211	0.7661130
5	2020-01-07	147.57000732421875	148.13999938964844	147.42999267578125	147.97000122070312	147.97000122070312	85.67773606095707	1.9896236033189325	1.1553819802942835	0.8342416
6	2020-01-08	148.49000549316406	148.61000061035156	146.13999938964844	146.86000061035156	146.86000061035156	76.1729128270239	2.0951819010667236	1.3433419644487716	0.7518399
7	2020-01-09	146.07000732421875	146.41000366210938	145.44000244140625	146.02999877929688	146.02999877929688	69.92613443593885	2.087796581997935	1.4922328879586044	0.5955636
8	2020-01-10	146.22000122070312	146.99000549316406	146.1699981689453	146.91000366210938	146.91000366210938	72.50104658423058	2.128417629424831	1.6194698362518498	0.5089477
9	2020-01-13	146.35000610351562	146.3800048828125	145.77999877929688	145.82000732421875	145.82000732421875	65.06956039241612	2.0490364900053635	1.7053831670025528	0.3436533

The true change in price is $147.97 - 143.95 = 4.02$

Expressed as a percentage of 143.95 (the original price), this is:

$$100 \times \frac{4.02}{143.95} = 2.793$$

This matches the reported change.

Therefore, test 16 is a pass.

Test 17

To ensure quantile predictions were correctly added to the dataset, a short script was written to randomly sample quantiles at an index in a generated dataset and confirm that the quantiles are in the correct order. The script is seen below:

```

1. import pandas as pd
2. import random
3.
4. df = pd.read_csv(r"training_resources\datasets\dqn_all_gld.csv")
5.
6. preds_low_q = df["LGBM_7d_q10"]
7. preds_mid_q = df["LGBM_7d_q50"]
8. preds_high_q = df["LGBM_7d_q90"]
9.
10. random_idx = random.randint(0, len(preds_mid_q))
11.
12. print(f"""
13. {"="*50}
14. Q10: {preds_low_q[random_idx]}
15. Q50: {preds_mid_q[random_idx]}
16. Q90: {preds_high_q[random_idx]}
17. {"-"*50}
18. """)
19.
20. if (preds_low_q[random_idx] < preds_mid_q[random_idx]) and (preds_mid_q[random_idx] <
preds_high_q[random_idx]):
21.     print("Quantiles are in the correct order!")
22. else:
23.     print("Quantiles are NOT in the correct order :(")
24.
25. print("\n" + "="*50)
26.

```

This code produced the following output:

```
=====
Q10:    -2.087854326038107
Q50:    -0.9210066103897168
Q90:     1.6418337525195257
-----

Quantiles are in the correct order!

=====
```

As can be seen above, the quantiles all exist and are in the correct order, therefore test 17 is a pass.

Test 18

For this test, a dummy trading environment object (as seen in the technical solution) was initialized and had predefined actions tested over a short time period to match with manual calculations. The script is seen below:

```
1. import sys
2. import os
3. import numpy as np
4. import pandas as pd
5.
6. sys.path.insert(0, os.path.abspath("."))
7.
8. from prediction_tools.DQN.trading_env import TradingEnvironment, PositionManager
9.
10. INITIAL_BALANCE = 10000.0
11. FORCED_START    = 30
12.
13. #is needed to correctly initialise the training environment
14. config = {
15.     'initial_balance':      INITIAL_BALANCE,
16.     'episode_length_days':  5,
17.     'lookback_window':     5,
18.     'horizons':             [7, 14, 30, 90, 365],
19.     'horizon_split_threshold': 30,
20.     'lgbm_lower':           0.1,
21.     'lgbm_upper':           0.9,
22.     'lgbm_confidence_window': 10,
23.     'confidence_horizons':  'all',
24.     'pyramid_sizes':        [0.10, 0.20, 0.50],
25.     'max_positions':        10,
26.     'transaction_costs':     0.001,
27.     'reward_frequency':      'close',
28.     'reward_profit_scale':   0.0,
29.     'punish_loss_scale':     0.0,
30.     'loss_penalty_nonlinear': False,
31.     'reward_sharpe_bonus':   0.0,
32.     'drawdown_penalty_enabled': False,
33.     'indecision_penalty_enabled': False,
34.     'opening_penalty':       0.0,
35.     'reward_clip':           False,
```

```

36.     'noise_std':                0.0,
37. }
38.
39. df = pd.read_csv(r"training_resources\datasets\dqn_all_gld.csv")
40.
41. env = TradingEnvironment(df, config, device='cpu', is_training=True, auto_reset=False)
42.
43. env.start_idx                = FORCED_START
44. env.current_step             = 0
45. env.balance                  = INITIAL_BALANCE
46. env.initial_balance          = INITIAL_BALANCE
47. env.peak_balance             = INITIAL_BALANCE
48. env.done                     = False
49. env.balance_history          = [INITIAL_BALANCE]
50. env.realized_balance_history = [INITIAL_BALANCE]
51. env.cumulative_realized_pnl  = 0.0
52. env.prev_action              = 1
53. env.action_history           = []
54. env.prev_unrealized_pnl      = 0.0
55. env.lgbm_hit_history         = []
56. env.episode_hit_rate         = 0.0
57. env.indecision_steps         = 0
58. env.indecision_penalty_current = 0.0
59. env.position_manager         = PositionManager()
60.
61. print("=" * 60)
62. print("PRICES")
63. action_labels = ['BUY 10%', 'HOLD', 'BUY 10%', 'CLOSE ALL', 'HOLD']
64. for step in range(5):
65.     idx = FORCED_START + step
66.     date = df['Date'].iloc[idx]
67.     price = env.prices[idx]
68.     print(f" Step {step} ({date}): {price:.6f} <- {action_labels[step]}")
69. print("=" * 60)
70.
71. actions = [2, 1, 2, 0, 1]
72. action_names = {0: 'CLOSE ALL', 1: 'HOLD', 2: 'BUY 10%', 3: 'BUY 20%', 4: 'BUY 50%'}
73.
74. print()
75. for step_num, action in enumerate(actions):
76.     price = env.get_current_price()
77.     cash_before = env.balance
78.     pos_before = env.position_manager.get_position_count()
79.     exp_before = env.position_manager.get_total_exposure()
80.
81.     agent_dummy, reward, done, info = env.step(action)
82.
83.     print(f"step {step_num} | {action_names[action]} at {price:.6f}")
84.     print(f" cash before      : {cash_before:.6f}")
85.     print(f" positions before   : {pos_before} (exposure: {exp_before:.4f})")
86.     print(f" cash after       : {info['cash']:.6f}")
87.     print(f" positions after    : {info['num_positions']}")
88.     print(f" position value     : {info['position_value']:.6f}")
89.     print(f" total balance      : {info['balance']:.6f}")
90.     print()
91.
92.     if done:
93.         break
94.
95. print("=" * 60)
96. print(f"FINAL BALANCE : {info['balance']:.6f}")
97. print(f"INITIAL      : {INITIAL_BALANCE:.6f}")
98. print(f"NET PnL       : {info['balance'] - INITIAL_BALANCE:.6f}")
99. print(f"RETURN        : {info['total_return']*100:.4f}%")
100. print("=" * 60)
101.

```

The results from running this code can be seen below:

```

1. =====
2. PRICES
3. Step 0 (2010-07-27): 113.510002 <- BUY 10%
4. Step 1 (2010-07-28): 113.779999 <- HOLD
5. Step 2 (2010-07-29): 114.290001 <- BUY 10%
6. Step 3 (2010-07-30): 115.489998 <- CLOSE ALL
7. Step 4 (2010-08-02): 115.540001 <- HOLD
8. =====
9.
10. step 0 | BUY 10% at 113.510002
11. cash before : 10000.000000
12. positions before : 0 (exposure: 0.0000)
13. cash after : 8999.000000
14. positions after : 1
15. position value : 1000.000000
16. total balance : 9999.000000
17.
18. step 1 | HOLD at 113.779999
19. cash before : 8999.000000
20. positions before : 1 (exposure: 1000.0000)
21. cash after : 8999.000000
22. positions after : 1
23. position value : 1002.378615
24. total balance : 10001.378615
25.
26. step 2 | BUY 10% at 114.290001
27. cash before : 8999.000000
28. positions before : 1 (exposure: 1000.0000)
29. cash after : 8098.200100
30. positions after : 2
31. position value : 1906.771630
32. total balance : 10004.971730
33.
34. step 3 | CLOSE ALL at 115.489998
35. cash before : 8098.200100
36. positions before : 2 (exposure: 1899.9000)
37. cash after : 10023.065239
38. positions after : 0
39. position value : 0.000000
40. total balance : 10023.065239
41.
42. step 4 | HOLD at 115.540001
43. cash before : 10023.065239
44. positions before : 0 (exposure: 0.0000)
45. cash after : 10023.065239
46. positions after : 0
47. position value : 0.000000
48. total balance : 10023.065239
49.
50. cash before : 10023.065239
51. positions before : 0 (exposure: 0.0000)
52. cash after : 10023.065239
53. positions after : 0
54. position value : 0.000000
55. total balance : 10023.065239
56. cash before : 10023.065239
57. positions before : 0 (exposure: 0.0000)
58. cash after : 10023.065239
59. positions after : 0
60. cash before : 10023.065239
61. positions before : 0 (exposure: 0.0000)
62. cash after : 10023.065239
63. positions after : 0
64. cash before : 10023.065239
65. positions before : 0 (exposure: 0.0000)
66. cash after : 10023.065239
67. cash before : 10023.065239
68. positions before : 0 (exposure: 0.0000)
69. cash before : 10023.065239

```

```

70. positions before : 0 (exposure: 0.0000)
71. cash after      : 10023.065239
72. cash before     : 10023.065239
73. positions before : 0 (exposure: 0.0000)
74. cash after      : 10023.065239
75. cash before     : 10023.065239
76. positions before : 0 (exposure: 0.0000)
77. cash before     : 10023.065239
78. positions before : 0 (exposure: 0.0000)
79. cash after      : 10023.065239
80. cash before     : 10023.065239
81. positions before : 0 (exposure: 0.0000)
82. cash after      : 10023.065239
83. positions after  : 0
84. cash before     : 10023.065239
85. positions before : 0 (exposure: 0.0000)
86. cash after      : 10023.065239
87. positions after  : 0
88. cash before     : 10023.065239
89. positions before : 0 (exposure: 0.0000)
90. cash before     : 10023.065239
91. positions before : 0 (exposure: 0.0000)
92. cash after      : 10023.065239
93. positions after  : 0
94. positions before : 0 (exposure: 0.0000)
95. cash after      : 10023.065239
96. positions after  : 0
97. positions after  : 0
98. position value   : 0.000000
99. position value   : 0.000000
100. total balance   : 10023.065239
101. total balance   : 10023.065239
102.
103. =====
104. FINAL BALANCE : 10023.065239
105. INITIAL       : 10000.000000
106. NET PnL      : 23.065239
107. RETURN       : 0.2307%
108. =====
109.

```

For reference, the predefined actions were: **BUY 10%, HOLD, BUY 10%, CLOSE, HOLD**

Step 1: BUY 10% at 113.510002

capital = 10000 x 0.10 = 1000.00

tx_cost = 1000 x 0.001 = 1.00

cash = 10000 – 1001 = 8999.00

shares1 = 1000 / 113.510002 = 8.81142248

Step 2: HOLD at 113.779999

cash = 8999.00

Step 3: BUY 10% at 114.290001

capital = 8999.00 x 0.10 = 899.90

tx_cost = 899.90 x 0.001 = 0.8999

cash = 8999 - 900.7999 = 8098.2001

shares2 = 899.90 / 114.290001 = 7.87356

Step 4: CLOSE ALL at 115.489998

pos1_returned = 1000 + (115.489998 - 113.510002) x 8.81142248 = 1000 + 17.4475 = 1017.4475

pos2_returned = 899.90 + (115.489998 - 114.290001) x 7.87356 = 899.90 + 9.4482 = 909.3482

capital_returned = 1017.4475 + 909.3482 = 1926.7957

tx_cost = 1926.7957 x 0.001 = 1.9268

Step 4: HOLD

cash = 10023.069

This is approximately equal to the code's result of 10023.065, with an error margin of 0.0000399%

Given the small margin of error, this is likely due to python floating point rounding errors, which are unavoidable. Therefore, test 18 is a pass.

Test 19

The same script as before is used to test the existing training environment from the technical solution, with some modifications to display the training environment's reported Sharpe ratio, to compare with manual calculations.

The code is seen on the next page:

```
1. import sys
2. import os
3. import numpy as np
4. import pandas as pd
5.
6. sys.path.insert(0, os.path.abspath("."))
7.
8. from prediction_tools.DQN.trading_env import TradingEnvironment, PositionManager
9.
10. INITIAL_BALANCE = 10000.0
11. FORCED_START    = 30
12.
13. config = {
14.     'initial_balance':          INITIAL_BALANCE,
15.     'episode_length_days':      5,
16.     'lookback_window':         5,
17.     'horizons':                 [7, 14, 30, 90, 365],
18.     'horizon_split_threshold':  30,
19.     'lgbm_lower':               0.1,
20.     'lgbm_upper':               0.9,
21.     'lgbm_confidence_window':   10,
22.     'confidence_horizons':      'all',
23.     'pyramid_sizes':            [0.10, 0.20, 0.50],
24.     'max_positions':            10,
25.     'transaction_costs':        0.001,
26.     'reward_frequency':         'close',
27.     'reward_profit_scale':       0.0,
28.     'punish_loss_scale':         0.0,
29.     'loss_penalty_nonlinear':    False,
30.     'reward_sharpe_bonus':       0.0,
31.     'drawdown_penalty_enabled':  False,
32.     'indecision_penalty_enabled': False,
33.     'opening_penalty':          0.0,
34.     'reward_clip':              False,
35.     'noise_std':                0.0,
36. }
37.
38. df = pd.read_csv(r"training_resources\datasets\dqn_all_gld.csv")
39.
40. env = TradingEnvironment(df, config, device='cpu', is_training=True, auto_reset=False)
41.
42. env.start_idx          = FORCED_START
43. env.current_step       = 0
44. env.balance            = INITIAL_BALANCE
45. env.initial_balance    = INITIAL_BALANCE
46. env.peak_balance       = INITIAL_BALANCE
47. env.done               = False
48. env.balance_history    = [INITIAL_BALANCE]
49. env.realized_balance_history = [INITIAL_BALANCE]
50. env.cumulative_realized_pnl = 0.0
51. env.prev_action        = 1
52. env.action_history     = []
53. env.prev_unrealized_pnl = 0.0
54. env.lgbm_hit_history   = []
55. env.episode_hit_rate   = 0.0
56. env.indecision_steps   = 0
```

```

57. env.indecision_penalty_current = 0.0
58. env.position_manager          = PositionManager()
59.
60. print("=" * 60)
61. print("PRICES")
62. action_labels = ['BUY 20%', 'HOLD', 'CLOSE ALL', 'BUY 10%', 'CLOSE ALL']
63. for step in range(5):
64.     idx = FORCED_START + step
65.     date = df['Date'].iloc[idx]
66.     price = env.prices[idx]
67.     print(f" Step {step} ({date}): {price:.6f} <- {action_labels[step]}")
68. print("=" * 60)
69.
70. actions = [3,1,0,2,0]
71. action_names = {0: 'CLOSE ALL', 1: 'HOLD', 2: 'BUY 10%', 3: 'BUY 20%', 4: 'BUY 50%'}
72.
73. print()
74. for step_num, action in enumerate(actions):
75.     price = env.get_current_price()
76.     cash_before = env.balance
77.     pos_before = env.position_manager.get_position_count()
78.     exp_before = env.position_manager.get_total_exposure()
79.
80.     agent_dummy, reward, done, info = env.step(action)
81.
82.     print(f"step {step_num} | {action_names[action]} at {price:.6f}")
83.     print(f"  cash before      : {cash_before:.6f}")
84.     print(f"  positions before   : {pos_before} (exposure: {exp_before:.4f})")
85.     print(f"  cash after         : {info['cash']:.6f}")
86.     print(f"  positions after    : {info['num_positions']}")
87.     print(f"  position value     : {info['position_value']:.6f}")
88.     print(f"  total balance      : {info['balance']:.6f}")
89.     print()
90.
91.     if done:
92.         break
93.
94. print("=" * 60)
95. print(f"FINAL BALANCE : {info['balance']:.6f}")
96. print(f"INITIAL          : {INITIAL_BALANCE:.6f}")
97. print(f"NET PnL          : {info['balance'] - INITIAL_BALANCE:.6f}")
98. print(f"RETURN           : {info['total_return']*100:.4f}%")
99. print(f"SHARPE (env)     : {info['sharpe_ratio']:.6f}")
100. print("=" * 60)
101.
102. print("\nREALIZED BALANCE HISTORY:")
103. for i, b in enumerate(env.realized_balance_history):
104.     print(f"  [{i}] {b:.6f}")
105.
106. realized_returns = np.diff(env.realized_balance_history) / env.realized_balance_history[:-1]
107. print("\nREALIZED RETURNS SERIES:")
108. for i, r in enumerate(realized_returns):
109.     print(f"  [{i}] {r:.8f}")
110.

```

The relevant output is seen below:

```

1. =====
2. FINAL BALANCE : 10008.160524
3. INITIAL      : 10000.000000
4. NET PnL      : 8.160524
5. RETURN       : 0.0816%
6. SHARPE (env) : 7.453105
7. =====
8.
9. REALIZED BALANCE HISTORY:
10.  [0] 10000.000000
11.  [1] 10000.000000

```

```

12. [2] 10000.000000
13. [3] 10011.729518
14. [4] 10011.729518
15. [5] 10011.161497
16.
17. REALIZED RETURNS SERIES:
18. [0] 0.00000000
19. [1] 0.00000000
20. [2] 0.00117295
21. [3] 0.00000000
22. [4] -0.00005674
23.

```

For this project, the Sharpe ratio is computed over the sequence of per-step realized returns within each episode, annualised by $\sqrt{252}$ (extrapolating as if trading over an entire year). The risk-free rate is omitted as negligible over short episode horizons, and population standard deviation is used. Zero-return steps where no position is closed are included in the series. This metric is used as a relative performance signal for comparing episodes during training. So, while this is indeed a Sharpe ratio included (as stated in the objectives), these assumptions cause it to deviate from a typical Sharpe ratio used in trading. This is intentional by design.

Now, the reported Sharpe ratio of **7.45** will be compared to manual Sharpe calculations, following the same assumptions.

Step 1: Calculate the mean

$$\text{Mean} = (0 + 0 + 0.00117295 + 0 - 0.00005674) / 5 = 0.00111621 / 5 = 0.00022324$$

Step 2: Calculate standard deviation

Deviations from mean: -0.00022324 , -0.00022324 , $+0.00094971$, -0.00022324 , -0.00027998

Deviations squared: 0.0000000498 , 0.0000000498 , 0.0000009019 , 0.0000000498 , 0.0000000784

$$\text{Variance} = (0.0000000498 + 0.0000000498 + 0.0000009019 + 0.0000000498 + 0.0000000784) / 5 = 0.0000002259$$

$$\text{Standard deviation} = \sqrt{0.0000002259} = 0.00047530$$

Step 3: Calculate Sharpe Ratio

$$\text{Sharpe} = (0.00022324 / 0.00047530) \times \sqrt{252} = 0.46969 \times 15.8745 = 7.453$$

This matches the reported Sharpe Ratio, therefore test 19 is a pass.

Test 20

To confirm that the circular replay buffer works as intended, a dummy replay buffer object was initialized with a size of 5, and had 8 transitions pushed onto it, which were rewards corresponding to the transition number being pushed (transition 1 reward = 1, transition 2 reward = 2, etc)

The expected rewards stored in the final set of transitions are [5, 6, 7, 3, 4]

```
1. import sys
2. import os
3. import torch
4. sys.path.insert(0, os.path.abspath("."))
5.
6. from prediction_tools.DQN.agent import ReplayBuffer
7.
8. #tiny buffer that can only hold 5 transitions
9. CAPACITY = 5
10. NUM_FEATURES = 1
11. LOOKBACK = 1
12.
13. buffer = ReplayBuffer(CAPACITY, NUM_FEATURES, LOOKBACK, 'cpu')
14.
15. #dummy state (content doesn't matter, just needs the right shape)
16. dummy_state = torch.zeros(LOOKBACK, NUM_FEATURES)
17.
18. # push 8 transitions (more than the buffer can hold)
19. # reward is the transition number so we can clearly see which ones survive
20. print("Pushing 8 transitions into a buffer of capacity 5...\n")
21. for i in range(8):
22.     buffer.push(dummy_state, action=0, reward=float(i), next_state=dummy_state, done=False)
23.     print(f" Pushed transition {i} (reward={float(i)}) | buffer size: {buffer.size} |
write position: {buffer.position}")
24.
25. print(f"\nBuffer rewards after 8 pushes (capacity={CAPACITY}):")
26. print(f" {buffer.rewards.tolist()}")
27. print(f"\nExpected: transitions 0,1,2 overwritten by 5,6,7")
28. print(f"Expected rewards: [5.0, 6.0, 7.0, 3.0, 4.0]")
29. print(f"\nMatch: {buffer.rewards.tolist() == [5.0, 6.0, 7.0, 3.0, 4.0]}")
30.
```

Pushing 8 transitions into a buffer of capacity 5...

Pushed transition 0	(reward=0.0)	buffer size: 1	write position: 1
Pushed transition 1	(reward=1.0)	buffer size: 2	write position: 2
Pushed transition 2	(reward=2.0)	buffer size: 3	write position: 3
Pushed transition 3	(reward=3.0)	buffer size: 4	write position: 4
Pushed transition 4	(reward=4.0)	buffer size: 5	write position: 0
Pushed transition 5	(reward=5.0)	buffer size: 5	write position: 1
Pushed transition 6	(reward=6.0)	buffer size: 5	write position: 2
Pushed transition 7	(reward=7.0)	buffer size: 5	write position: 3

Buffer rewards after 8 pushes (capacity=5):
[5.0, 6.0, 7.0, 3.0, 4.0]

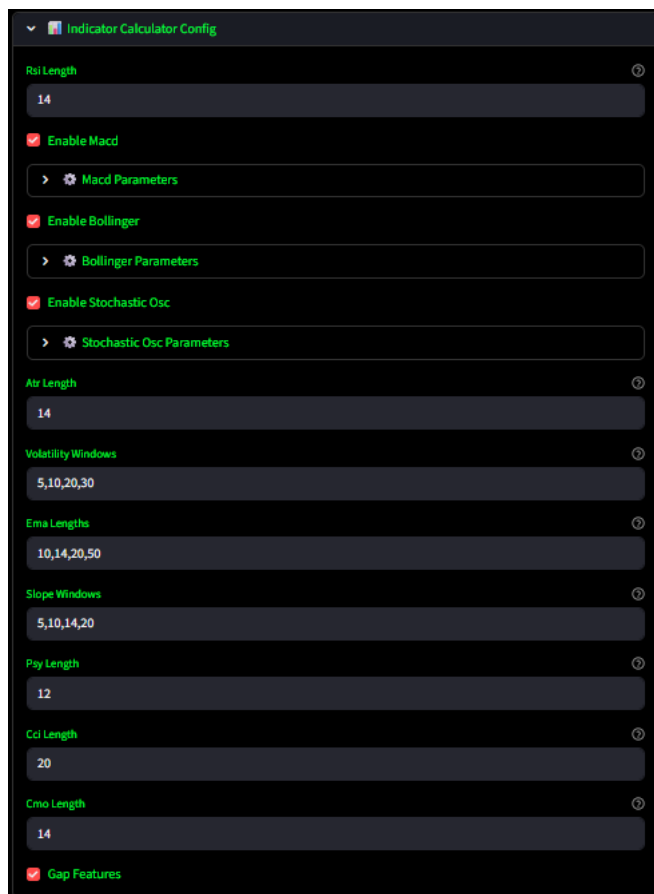
Expected: transitions 0,1,2 overwritten by 5,6,7
Expected rewards: [5.0, 6.0, 7.0, 3.0, 4.0]

Match: True

As can be seen above, the circular buffer correctly replaced the first transitions with the excess transitions. Therefore, test 20 is a pass.

Test 21

This test simply checks if the indicator calculation expander reveals defined values to calculate technical indicators.



The screenshot displays the 'Indicator Calculator Config' window. It features several sections for configuring technical indicators:

- Rsi Length:** A text input field containing the value '14'.
- Enable Macd:** A checked checkbox with a green checkmark.
- Macd Parameters:** A button with a gear icon and a right-pointing arrow.
- Enable Bollinger:** A checked checkbox with a green checkmark.
- Bollinger Parameters:** A button with a gear icon and a right-pointing arrow.
- Enable Stochastic Osc:** A checked checkbox with a green checkmark.
- Stochastic Osc Parameters:** A button with a gear icon and a right-pointing arrow.
- Atr Length:** A text input field containing the value '14'.
- Volatility Windows:** A text input field containing the values '5,10,20,30'.
- Ema Lengths:** A text input field containing the values '10,14,20,50'.
- Slope Windows:** A text input field containing the values '5,10,14,20'.
- Pay Length:** A text input field containing the value '12'.
- Cci Length:** A text input field containing the value '20'.
- Cmo Length:** A text input field containing the value '14'.
- Gap Features:** A checked checkbox with a green checkmark.

As can be seen above, the indicator calculator parameters are defined within streamlit textboxes, making them editable. The unopened expander can also be seen in the testing video, using the same defaults, where the dataset was correctly generated without error. The generated dataset columns are also seen with the indicators in test 15. Therefore, test 21 is a pass.

Test 22

This test simply checks if LightGBM parameters are exposed and within a configurable container. The unopened expander can be seen in the testing video, where it is correctly

used to generate LightGBM predictions, confirming its functionality. The exposed parameters can be seen below:



The screenshot displays the 'LightGBM Config' window with the following parameters and values:

- Predictor Mode:** quantile
- Quantiles:** 0.1,0.5,0.9
- Conformal:** ☐
- LightGBM Params:**
 - N Estimators:** 250
 - Learning Rate:** 0.0080
 - Max Depth:** 12
 - Min Child Samples:** 84
 - Subsample:** 0.85
 - Colsample bytree:** 0.70
 - Reg Alpha:** 0.0005
 - Reg Lambda:** 2.00
 - Max Bin:** 125
 - Boosting Type:** gbm
 - Feature Drop:** 42
 - W John:** -1
 - Verbosity:** -1
 - Target:** pct
 - Importance:** 562,90,30,14,7
 - Train Mtu:** 500
 - Train Mtu:** 1000
 - Numon:** 100
- Conformal Light:** 0.15
- Conformal Release:** ☐

As seen above, all LightGBM parameters are exposed in configurable containers, therefore, test 22 is a pass.

Test 23

As it would require waiting several months to gather a reasonable sample size to evaluate LIVE predictions for LightGBM, the dataset generator's LightGBM predictions will be used in a backtest to demonstrate conformal corrections.

The following parameters were used:

The screenshot shows a configuration window titled "Add LightGBM Predictions" with a sub-header "LightGBM Config". The parameters are as follows:

- Prediction Mode:** quantile
- Quantiles:** 0.1,0.5,0.9
- Conformal:** ☒
- LightGBM Params:**
 - N Estimators:** 250
 - Learning Rate:** 0.0080
 - Max Depth:** 7
 - Num Leaves:** 128
 - Min Child Samples:** 50
 - Subsample:** 0.85
 - Colsample Bytree:** 0.75
 - Reg Alpha:** 0.0005
 - Reg Lambda:** 2.00
 - Max Bin:** 150
 - Boosting Type:** gbdt
 - Random State:** 42
 - N Jobs:** -1
 - Verbose:** -1
 - Target:** pct
 - Horizons:** 365,90,30,14,7
 - Train Min:** 100
 - Train Max:** 252
 - Retain:** 14
- Conformal Split:** 0.15 (slider)
- Conformal Ratios:** ☐

The same dataset was also evaluated without conformal predictions:

Below are the results WITHOUT conformal predictions for the 30-day horizon:

```
1. -----
2. LIGHTGBM BACKTEST RESULTS - 30d HORIZON
3. -----
4. OVERALL DIRECTIONAL ACCURACY: 73.94%
5. -----
6. QUANTILE COVERAGE
7. -----
8. q10: 24.60% (ideal 10%) ⚠ [+14.60%]
9. q50: 45.92% (ideal 50%) ✓ [-4.08%]
10. q90: 67.46% (ideal 90%) ⚠ [-22.54%]
11. -----
12. CONFIDENCE ANALYSIS (Spread vs Accuracy)
13. -----
14. Bin 1 | Width: 5.8378 | Accuracy: 74.24% | n=920
15. Bin 2 | Width: 7.7912 | Accuracy: 74.78% | n=920
16. Bin 3 | Width: 9.4945 | Accuracy: 71.96% | n=920
17. Bin 4 | Width: 11.2590 | Accuracy: 75.54% | n=920
18. Bin 5 | Width: 15.9925 | Accuracy: 78.37% | n=920
19. -----
```

After gathering these results, the backtest was repeated for the 30-day horizon where conformal predictions were applied, according to the aforementioned parameters:

```
1. -----
2. LIGHTGBM BACKTEST RESULTS - 30d HORIZON
3. -----
4. OVERALL DIRECTIONAL ACCURACY: 58.99%
5. -----
6. QUANTILE COVERAGE
7. -----
8. q10: 13.35% (ideal 10%) ✓ [+3.35%]
9. q50: 47.62% (ideal 50%) ✓ [-2.38%]
10. q90: 82.36% (ideal 90%) ⚠ [-7.64%]
11. -----
12. CONFIDENCE ANALYSIS (Spread vs Accuracy)
13. -----
14. Bin 1 | Width: 1.5270 | Accuracy: 54.01% | n=724
15. Bin 2 | Width: 18.6878 | Accuracy: 71.78% | n=723
16. Bin 3 | Width: 28.5330 | Accuracy: 65.56% | n=723
17. Bin 4 | Width: 39.8225 | Accuracy: 64.59% | n=723
18. Bin 5 | Width: 64.5561 | Accuracy: 44.12% | n=723
19. -----
```

An improvement in quantile coverage can be observed in the conformal results. However, directional accuracy was reduced by a significant amount. This makes it less useful in trading. As conformal corrections were still correctly applied, test 23 is a pass.

Investigation

After confirming the system is fully functional, and results are valid, tests were carried out to complete the investigation, as stated in Objective 1.

The following experimental plan was designed to systematically evaluate the predictive and trading effectiveness of Gradient-Boosted Decision Trees (LightGBM) and Deep Q-Networks within the constructed research environment. Each experiment evaluates their impact using consistent datasets, walk-forward validation, and quantitative performance metrics.

To ensure the robustness of experiments, hypothesis tests will be carried out at a 5% significance level. This means that if the test is passed, there is less than a 5% chance that the results occurred by chance and the null hypothesis was incorrectly rejected. Tests will be conducted *out-of-sample (OOS)* to minimise the effects of overfitting, where models might otherwise “memorise” existing patterns rather than generalise. For LightGBM specifically, predictions are generated using rolling windows, where only historical data is used as features to predict future values, ensuring that all predictions are out-of-sample (OOS). The *null hypothesis (H_0)* assumes that the models do not have a predictive edge and behave randomly, while *the alternative hypothesis (H_1)* assumes that the model has a positive predictive edge.

The test statistic Z is calculated as:

$$Z = \frac{\bar{X} - \mu}{\sigma_{\bar{X}}}$$

Where $\bar{X}$ is the sample mean, μ is the population mean, and $\sigma_{\bar{X}}$ is the standard error of the mean. At a 5% significance level, the null hypothesis may be rejected when $Z > 1.645$ (one tailed test)

The standard error of the mean is calculated with the following formula:

$$\sigma_{\bar{X}} = \frac{\sigma}{\sqrt{n}}$$

Where σ is the sample's standard deviation, and n is the total number of independent observations (years).

Experiment Number	Objective(s)	Purpose	Configuration	Metric(s) Evaluated	Hypothesis Test
1. LightGBM forecasting performance	6, 8.2, 8.5	Evaluate predictive performance of LightGBM for time-series forecasting	Complete feature set on the GLD ticker from 2007 to 2025, with predictions added from rolling windows to prevent leakage.	Directional Accuracy	$H_0: p = 0.5$ $H_1: p > 0.5$ <i>Where p is the probability of predicted direction being correct, and n is taken to be the number of predictions</i>
2. DQN trade win rate	7, 8.2, 8.5	Determine if the DQN Agent's trades are individually profitable	OOS test on a DQN agent trained on GLD data from 2007-2021, and tested from 2021-2025	Trades with positive returns	$H_0: p = 0.5$ $H_1: p > 0.5$ <i>Where p is the probability of any given trade being profitable, and n is the number of trades executed</i>
3. DQN profitability	7, 8.2, 8.5	Determine if the DQN Agent will be profitable when trading on any given year	OOS test on a DQN agent trained on GLD data from 2007-2021, and tested on random start date samples with a length of 252 days (trading year) from 2021-2025	Percentage returns after a year of trading	$H_0: \mu = 0$ $H_1: \mu > 0$ <i>Where μ is the population mean return, and n is the number of 252-day samples.</i>

Experiment 1

This hypothesis test will be carried out for the 7-day horizon, as a representative sample of other horizons. This is because it is the shortest horizon, and therefore the most difficult to predict, therefore these results are conservative on the reported predictive power.

Below are the results for LightGBM 7-day horizon predictions on GLD data from 2007 up till 2025, added to the dataset with walk-forward expanding windows to prevent leakage.

```
1. -----
2. LIGHTGBM BACKTEST RESULTS - 7d HORIZON
3. -----
4. OVERALL DIRECTIONAL ACCURACY: 63.42%
5. -----
6. QUANTILE COVERAGE
7. -----
8. q10: 12.55% (ideal 10%) ✓ [+2.55%]
9. q50: 48.10% (ideal 50%) ✓ [-1.90%]
10. q90: 84.10% (ideal 90%) △ [-5.90%]
11. -----
12. CONFIDENCE ANALYSIS (Spread vs Accuracy)
13. -----
14. Bin 1 | Width: 3.5509 | Accuracy: 64.00% | n=925
15. Bin 2 | Width: 4.5386 | Accuracy: 63.74% | n=924
16. Bin 3 | Width: 5.3032 | Accuracy: 64.76% | n=925
17. Bin 4 | Width: 6.1927 | Accuracy: 63.20% | n=924
18. Bin 5 | Width: 8.4806 | Accuracy: 65.08% | n=925
19. -----
```

Now, the hypothesis will be conducted using these results.

Let the observed directional accuracy 0.6342, which is the probability of a prediction being correct, be denoted as $\hat{p}$

The null hypothesis assumes p_0 is 0.5

As any given prediction can only be “right” or “wrong”, each individual prediction is a **Bernoulli trial**. Therefore, the standard deviation can be calculated as follows:

$$\sigma = \sqrt{(p_0(1 - p_0))} = 0.5$$

Given that n (the total count of all predictions) is 4623, the standard error of the mean, as stated in the experimental method is:

$$\sigma_{\bar{x}} = \frac{\sigma}{\sqrt{n}} = 0.007354$$

Now, the Z-Statistic can be calculated:

$$Z = \frac{\bar{X} - \mu}{\sigma_{\bar{X}}} = \frac{\hat{p} - p_0}{\sigma_{\bar{X}}} = 18.25$$

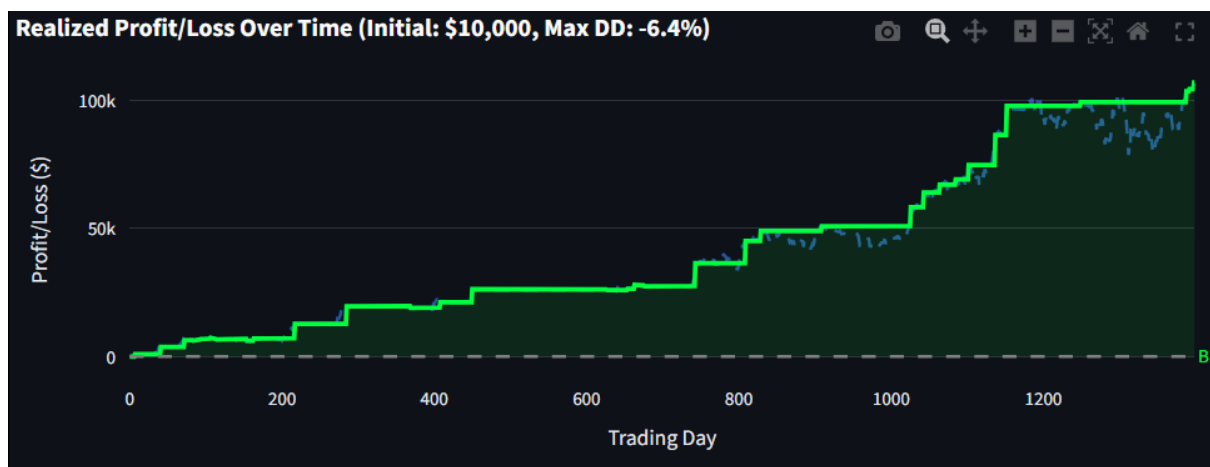
This is exceedingly greater than the critical region at the 5% level (1.645)

There is strong statistical evidence that LightGBM's directional accuracy on GLD over the 7-day horizon is significantly greater than chance, and the null hypothesis can be rejected.

Experiment 2

For this hypothesis test, a DQN Agent will be evaluated on BTC-USD data from 2021 till the start of 2025. This is entirely Out of Sample.

The results can be seen below:



```

1. -----
2. DQN BACKTEST RESULTS - DQN_BTC_2
3. -----
4. PORTFOLIO METRICS
5. -----
6. Initial Balance: 10,000.00
7. Final Balance: 115,480.86
8. Realized Balance: $116,856.10
9. Total Return: +1054.81%
10. Realized Return: +1068.56%
11. Max Realized Drawdown: -6.44%
12. Max Equity Drawdown: -20.35%
13. Sharpe Ratio: 1.60
14. -----
15. TRADE STATISTICS
16. -----
17. Total Trades: 280
18. Winners: 233 (83.2%)
19. Losers: 47 (16.8%)
20. Avg Win: 486.33 (+12.62%)
21. Avg Loss: -105.59 (-3.35%)
22. Profit Factor: 22.83
23. Expectancy: $386.97
24. -----
25. BUY & HOLD COMPARISON
26. -----
27. B&H Final Value: 23,903.73
28. B&H Return: +139.04% B&H
29. PnL: 13,903.73
30. Outperformance: +929.52%
31. -----
32. ACTION DISTRIBUTION
33. -----
34. close : 134 ( 9.6%)
35. hold : 579 ( 41.4%)
36. pyramid_0.1 : 216 ( 15.4%)
37. pyramid_0.25 : 318 ( 22.7%)
38. pyramid_0.5 : 153 ( 10.9%)
39.

```

Now, the hypothesis test can be conducted.

$$H_0: p = 0.5 \text{ (trades are random)}$$

$$H_1: 0 > 0.5 \text{ (individual trades are profitable)}$$

Given that:

$$n = 280$$

$$\bar{X} = 0.8321 \text{ (mean of trades where 0 is loss and 1 is gain)}$$

$$\mu = 0.5$$

$$\sigma = 0.5 \text{ (Bernoulli Trial)}$$

$$\sigma_X = 0.02989$$

The Z-Statistic can be calculated as:

$$Z = \frac{0.8321 - 0.5}{0.02989} = 11.11$$

11.11 is far greater than the critical region of 1.645, therefore the null hypothesis can be rejected.

There is strong statistical evidence that the DQN agent's trades are individually profitable at a rate significantly greater than chance.

Experiment 3

For this experiment, the DQN was run for 20 random episodes within the OOS period. The table below shows the results:

Episode	Total Return (%)	Profitable?
1	+6.77%	✓
2	+36.41%	✓
3	+28.37%	✓
4	+104.59%	✓
5	+15.46%	✓
6	+4.16%	✓
7	+28.19%	✓
8	+7.10%	✓
9	+81.06%	✓
10	+107.00%	✓
11	+80.20%	✓
12	+73.65%	✓
13	+10.07%	✓
14	+26.76%	✓
15	+36.44%	✓
16	+66.01%	✓
17	+5.18%	✓
18	+117.96%	✓
19	+43.15%	✓
20	+25.28%	✓

From the results above, statistical features can be determined:

$$\sigma = 37.78$$

$$\sigma_x = 8.45$$

$$\bar{X} = 45.19$$

As H_0 assumes no returns on average, $\mu = 0$

$$Z = \frac{45.19}{8.45} = 5.35$$

5.35 exceeds the critical region at the 5% level of 1.645, therefore the null hypothesis can be rejected.

There is strong statistical evidence that the DQN agent produces positive returns over a 252-day trading period.

Evaluation

Objective Review

The table below details how each objective was completed. Evidence can be seen in the testing section.

Objective	Description	Evidence	Complete
1	Create a machine learning research environment to investigate the use and effectiveness of Gradient-Boosted Decision Tree models and Deep Q-Networks for forecasting and decision making in financial markets	The testing video in test 1 shows the complete research environment, with complete backtesting systems to show the effectiveness of different models. Specific data is shown in the investigation subsection under the testing section.	✓
2	Build infrastructure to dynamically pull required training data	Tests 2-6 demonstrate the functionality of the individual parts required for the infrastructure to function, and the full dataset can only be constructed where this data exists. As the dataset was successfully constructed in the testing video and in test 15, the infrastructure works as intended.	✓
2.1	Dynamically construct web API request URLs parametrically	Test 2 demonstrates the ability to parametrically construct requests to fetch relevant data.	✓
2.2	Detect and handle HTTP error conditions	Test 3 demonstrates one of many handled error conditions, by catching a 404 error. Other common errors include 403, which is displayed when access is blocked (forbidden)	✓
2.3	Filter articles based on relevance	Test 4 demonstrates the scraper's ability to filter articles based on relevance to the ticker tag.	✓

2.4	Control frequency to respect rate limits	Test 5 demonstrates the random delay used to control frequency	✓
2.5	Retrieve historical OHLC market data via an external finance API	Test 6 confirms the accuracy of OHLC data pulled from yfinance	✓
3	Adapt and calibrate a pre-trained sentiment model for financial new analysis	Tests 7-12 demonstrate the functionality of subcomponents to complete this objective, demonstrating the use of a FinRoBERTa model for label probabilities, and using the probabilities to generate a continuous score for the analysis of financial news articles.	✓
3.1	Integrate a pre-trained transformer model for financial sentiment classification	Test 7 demonstrates the functionality of the integrated FinRoBERTa model	✓
3.2	Convert discrete label probabilities into a continuous polarity score	Test 7 displays a continuous polarity score generated from label probabilities. Full explanation can be seen in test 8.	✓
3.3	Apply contextual biasing to constrain sentiment interpretation	Test 9 shows the effects of contextual bias text appended onto headlines.	✓
3.4	Calibrate confidence via temperature scaling	Test 10 shows the effects of temperature scaling on confidence	✓
3.5	Apply a nonlinear scaling function to reshape sentiment magnitude and neutrality	Test 11 shows the custom scaling function's effect on generated sentiment scores	✓
3.6	Expose configurable parameters for systematic experimentation	Test 12 shows how exposed parameters can be modified, allowing for experimentation. This is also exposed in the UI seen in the testing video.	✓
4	Process OHLC data into technical features for time-series data	Tests 21 and 13 show that the technical feature calculations work as intended, and	✓

		test 15 shows that the features can be added to datasets	
4.1	Define parameters to calculate technical features	Test 21 shows the defined parameters exposed in the UI for configurability of indicators	✓
4.2	Compute features to enhance the training data, including technical indicators	Test 13 confirms that RSI indicator was calculated correctly, as a representative sample of the other correctly calculated technical indicators and statistical features	✓
5	Dynamically construct a unified dataset for training RL and regression models	Tests 14-16 demonstrate the functionality of the steps required to complete this objective. Test 15 specifically shows that all feature columns were generated correctly.	✓
5.1	Include mean sentiment from individual days to produce a single score per day	Test 14 demonstrates the correct calculation of the mean sentiment from several articles in one day.	✓
5.2	Integrate sentiment data, OHLC, and technical features into a single dataset	Test 15 shows all the features contained within the dataset.	✓
5.3	Generate target labels for supervised learning	Test 16 shows that target labels were correctly added to the dataset.	✓
6	Implement a quantile LightGBM forecaster for time series prediction	Tests 17, 22 and 23 demonstrate the functionality of the individual components of the implemented LightGBM forecaster. The testing video also shows the LightGBM time series predictions in backtests, which are investigated in the testing section.	✓

6.1	Configure and expose LightGBM parameters	Test 22 shows the exposed LightGBM parameters, that can be configured	✓
6.2	Include the option to use conformal predictions	Test 23 demonstrates the conformal correction functionality and its effect on predictions	✓
6.3	Predict the target variable for different quantiles	Test 17 shows that multiple quantiles are predicted, and are in the correct order	✓
7	Develop a 1D Convolutional Deep Q-Network with LightGBM informed features	Tests 18-20 demonstrate the completion of the steps required to complete this objective, as well as test 1 which shows that the environment is configurable. The testing video also shows the agent working correctly in backtests, training, and LIVE predictions.	✓
7.1	Allow for customisation of observation space, environment, rewards, and actions	Test 1 shows that the observation space, environment, rewards, and actions are configurable in the streamlit UI expanders, and uses these parameters to train a DQN.	✓
7.2	Use a custom trading environment to track the agent's performance	The testing video shows that the DQN works as intended and therefore must have a working environment. Test 18 confirms that the environment calculates the agent's balance correctly as a representative sample of all the calculations involved with running the environment.	✓
7.3	Calculate performance metrics such as returns and Sharpe ratio during and after training	Test 19 confirms that Sharpe ratio is correctly calculated as a representative sample of other metrics.	✓

		Specific performance metrics are investigated in the testing section.	
7.4	Sample past transitions via an experience replay buffer using a fixed-capacity circular queue	Test 20 shows that the circular queue replay buffer works as intended	✓
8	Create an interactive user interface for training and backtesting	Test 1 showcases the working UI interface for BASLIX terminal	✓
8.1	Dataset management for training environment	See testing video for demonstrated functionality. Additional functionality includes being able to edit datasets and replace LightGBM predictions for DQN training.	✓
8.2	Backtesting for LightGBM and trained DQNs	See testing video for backtests, and the investigation section under testing for detailed backtest reports.	✓
8.3	Training environment with LIVE view of statistics	See testing video where the training environment is demonstrated with a LIVE view of statistics	✓
8.4	Dashboard to show model predictions on current-day data	See testing video, model predictions are shown for current day data. Additional functionality not shown in the video includes a news feed and sentiment analysis for the past month.	✓
8.5	Display Directional Accuracy Metrics for LightGBM, and trading performance metrics for trained DQNs	See testing video for displayed performance metrics, and investigation section under testing for more detailed reports.	✓

Sponsor Feedback

Below is the sponsor's feedback on the complete system:

This project demonstrates a strong level of technical ambition and thoughtful system design. The interface is visually clear and well-structured, with a consistent dark theme and logical grouping of components that supports readability and interpretability of results. Navigation is easy, with clear separation between data ingestion, configuration, modelling and visualisation, allowing the user to follow the analytical workflow with minimal friction.

Functionally, the use of Light Gradient Boosting (LightGBM) is a strong choice and appropriately implemented, with configurable feature windows, prediction horizons, quantile forecasts and model hyperparameters that reflect a solid understanding of gradient boosting methods for time-series prediction.

From a research perspective, the inclusion of out-of-sample prediction visualisations, multiple horizons and uncertainty-aware outputs confirms an appreciation of evaluation beyond point estimates.

The system could be improved by explicitly organising parameters such that their purposes are clear, especially around feature engineering, temporal validation strategy. There was clear discussion of model limitations and robustness. Overall, this is a well-executed and thoughtfully presented project that showcases strong technical skills and developing research maturity.

Overall, the sponsor was impressed by the scale and quality of the project and suggested that the interface should more explicitly give details about features and hyperparameters being used.

Potential Improvements

As stated by the sponsor, the system would benefit from organising parameters in a clearer manner for research and experimentation.

Despite the scale of the system, there is much room for improvement.

One such example is the live trading system, which could benefit from having automatic executions to test models on real-world data to gauge their effectiveness. Evidence of high performance on live data is more useful than backtests, as it proves the model can function in the real world.

More options could also be included for experimentation purposes. For example, different loss functions could be configured for the DQN, such as *Smooth1Loss* as opposed to *MSE*.

Alternative targets are already experimented with for LightGBM, however additional targets suggested by research papers¹⁴ include logarithmic targets, which might perform better in non-stationary financial environments

The research system could also be improved with a Bayesian Optimiser for hyperparameter tuning on LightGBM models, which would allow for automatic refinement of hyperparameters to improve the accuracy of LightGBM predictions. This methodology was also used in research papers¹⁴ into LightGBM for financial predictions, where it was also demonstrated that LightGBM is highly sensitive to hyperparameters.

More tickers could also be added to the system. Currently, support only exists for BTC-USD and GLD, due to the availability of data and range of markets covered by two very different markets. However, it might be worth including tickers for other natural resources such as crude oil, and standard stocks such as TSLA.

¹⁴ (Bisdoulis, 2024)

Conclusion

Many challenges were faced when building this project. A major challenge was learning a lot about how machine learning works, for which online resources such as *3blue1brown* and several YouTube videos from smaller creators have proven extremely useful. More specific challenges faced include data leakage, which was a major problem early in development, as much of the results produced from backtests were unreliable and overly optimistic, however feature leakage in the final system has been completely removed (as per my knowledge). Designing the RL environment for the DQN was also a daunting task as initially the DQN struggled to learn, however after adding meta features and conducting ablation studies as suggested by the sponsor, DQN performance was greatly improved.

This project has improved my OOP skills, as through the development of the system the Object-oriented design made it incredibly easy to make changes without having to rewrite massive sections of code and also allowed for the system to easily be expanded and have more features added. It has also broadened my knowledge in machine learning and quantitative research, allowing me to make better models to analyse data.

This project has largely been a success, as when starting this system, I had no experience in reinforcement learning, with minimal experience in machine learning as a whole. This project surpassed my own expectations of what could be done to forecast financial markets, and has pushed me further into the world of quantitative research, which is a field I plan to enter with the skills I have and will continue to develop.

References

- (n.d.). Retrieved from Investopedia: <https://www.investopedia.com>
- Aswathi, R. (2023, December 21). *Gradient Boosted Decision Tree — Clearly Explained*. Retrieved from medium.com: <https://medium.com/@ruchi.awasthi63/gradient-boosted-decision-tree-clearly-explained-bd1d8c7d9923>
- Barto, R. S. (2018). *Reinforcement Learning: An Introduction (2nd ed.)*. MIT Press.
- Bisdoulis, K.-L. (2024, December 27). *Assets Forecasting with Feature Engineering and Transformation Methods for LightGBM*. Retrieved from [https://arxiv.org/](https://arxiv.org/https://arxiv.org/abs/2501.07580): <https://arxiv.org/abs/2501.07580>
- Bloomberg, T. a. (2023, May 11). *Humans in the Loop: AI & Machine Learning in the Bloomberg Terminal*. Retrieved from bloomberg.com: <https://www.bloomberg.com/company/stories/humans-in-the-loop-ai-machine-learning-in-the-bloomberg-terminal/>
- Chen, Y. (2024, October 22). *Corrected Soft Actor Critic for Continuous Control*. Retrieved from arXiv: <https://arxiv.org/abs/2410.16739>
- LightGBM. (2025). *readthedocs*. Retrieved from lightgbm documentation: <https://lightgbm.readthedocs.io/en/latest/>
- Lunartech. (2025, January 18). *Mastering AdamW Optimizer: Enhancing Deep Learning Models with Superior Regularization*. Retrieved from <https://www.lunartech.ai/>: <https://www.lunartech.ai/blog/mastering-adamw-optimizer-enhancing-deep-learning-models-with-superior-regularization>
- PyTorch. (2025, March 24). *Reinforcement Learning (DQN) Tutorial*. Retrieved from docs.pytorch.org: https://docs.pytorch.org/tutorials/intermediate/reinforcement_q_learning
- Sanderson, G. (2017, October 5). *But what is a Neural Network?* Retrieved from <https://www.3blue1brown.com/>: <https://www.3blue1brown.com/lessons/neural-networks#title>
- Schulman, J. (2017, August 28). *Proximal Policy Optimization Algorithms*. Retrieved from arxiv: <https://arxiv.org/abs/1707.06347>
- scikit documentation. (2025). *scikit-learn*. Retrieved from Supervised Learning: https://scikit-learn.org/stable/supervised_learning.html
- Sibisi, M. (2023, December 18). *C# vs Python for Web Scraping*. Retrieved from Bright Data: <https://brightdata.com/blog/how-tos/c-sharp-vs-python-for-web-scraping>

XGBoost. (2022). *xgboost docs*. Retrieved from xgboost:
<https://xgboost.readthedocs.io/en/stable/index.html>

Yi Yang, M. C. (2020). *FinBERT: A Pretrained Language Model for Financial Communications*. Retrieved from arXiv:
<https://arxiv.org/abs/2006.08097>